KNOWMINER:

KONZEPTION UND ENTWICKLUNG EINES GENERISCHEN WISSENSERSCHLIESSUNGSFRAMEWORKS

Dissertation zur Verleihung des akademischen Grades

Doktor der Technischen Wissenschaften
an der
Technischen Universität Graz

vorgelegt von

Michael Granitzer

Know-Center
Technische Universität Graz
Österreich

15.11.2006

Erster Begutachter: Univ.-Prof. Dr. Klaus Tochtermann

Zweiter Begutachter: Univ.-Prof. Dr. Dr.h.c.mult. Hermann Maurer

Zusammenfassung

Der immer stärkere Zuwachs an Information bedarf neuer Zugangswege und neuer Techniken, um das darin enthaltene Wissen effizient und zielgerecht nutzen zu können. Erschwerend hierfür ist die Unstrukturiertheit, in der Information heutzutage im Allgemeinen vorliegt, die unvollständige Vergabe von Metadaten sowie die Vagheit der menschlichen Sprache. Um trotzdem den uns umgebenden Wissenspool für verschiedenste Benutzergruppen nutzbar zu machen, bedarf es einer Kombination unterschiedlicher Techniken aus verschiedenen Bereichen. Entwicklungen im Bereich des Knowledge Discovery und Information Retrieval bieten Ansätze und Techniken zur semantischen Anreicherung von Information und damit verbunden zur besseren Nutzbarkeit heutzutage teilweise brach liegender Informationsbestände.

Vor diesem Hintergrund präsentiert sich die vorliegende Arbeit, welche die Konzeption und Entwicklung eines integrativen und flexiblen Software Frameworks zur Unterstützung der Integration von Knowledge Discovery und Information Retrieval Diensten beschreibt. Die Zielsetzung liegt dabei auf der Unterstützung der Integration verschiedener Algorithmen und Techniken aus den genannten Bereichen, wobei das Framework den Anspruch der Einsetzbarkeit in unterschiedlichsten Anwendungskontexten erfüllen soll.

Auf Basis der Analyse der Prozesse, Datenflüsse und Anwendungsbereiche der Wissenserschließung erfolgte die Schaffung der konzeptionellen Basis für die Umsetzung des Software-Frameworks. Die Konzeption und Umsetzung untergliederte sich in mehrere Iterationszyklen, orientiert am Spiralmodell der Softwareentwicklung. Das Ende eines Zyklus bildete dabei die Anwendung und Evaluierung des Frameworks in praktischen Szenarien und Projekten. Dies ermöglichte einerseits die Weiterentwicklung des Frameworks und zeigte andererseits die Anwendbarkeit in unterschiedlichen Szenarien und technischen Umfeldern.

Das entwickelte KnowMiner-Framework konnte, wie in dieser Arbeit dargestellt, insgesamt in fünf Großprojekten und mehreren kleineren Projekten erfolgreich eingesetzt und angewendet werden. Die Erfahrungen aus den Projekten zeigen dabei die einfache Einsatzbarkeit des Frameworks in unterschiedlichen Gebieten und damit verbunden die einfache Integration von Wissenserschließungstechniken in praktischen Anwendungen.

Abstract

Steadily increasing information amounts require new ways and techniques for making knowledge efficiently and goal oriented utilisable. The lack of structure in information, the incomplete allocation of metadata, and the vagueness of human language make things even more difficult. For making information utilisable for different user groups, various techniques from different domains must be combined: Knowledge Discovery and Information Retrieval provide approaches and techniques for a semantic enrichment of information and thus for a better utilisation for information which partly lies idle.

Against this background the work at hand deals with the conceptualisation and development of an integrative and flexible software framework. This framework supports the development of knowledge discovery and information retrieval services. The main goal of the work concerns the integration of different algorithms and techniques from the above mentioned domains, whereby the framework should be applicable in different application scenarios.

The analysis of processes, data flows and application areas of Knowledge Discovery provides the conceptual foundation for the realisation of the software framework. The conceptualisation and realisation was divided into several iteration cycles, oriented along the spiral model of software development. The end of each cycle consisted in applying and evaluating the framework in practical scenarios and projects. This, on the one hand allowed for the further development of the framework, and on the other hand showed its applicability in different scenarios and technical areas.

The developed KnowMiner-Framework was successfully realised and applied in five large and a number of smaller projects. As experiences show, the framework can easily be integrated in various application areas and techniques from Knowledge Discovery can easily be applied in practical scenarios

I hereby certify that the work presented in this thesis is my own and that work performed by others is appropriately cited.

Ich versichere hiermit, diese Arbeit selbstständig verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und mich auch sonst keiner unerlaubten Hilfsmittel bedient zu haben.

Danksagung

Die Entwicklung von Software und vor allem von Software-Frameworks ist ein herausforderndes Unterfangen: Ohne die Hilfe und Unterstützung von vielen Personen wäre die vorliegende Arbeit nicht möglich gewesen. Während in den verschiedenen Phasen dieser Arbeit sich der Personenkreis um mich teilweise änderte, so möchte ich an dieser Stelle den größten Dank an das Kernteam bei der Entwicklung und Umsetzung des KnowMiner-Frameworks richten, an Werner Klieber, Vedran Sabol und Wolfgang Kienreich. Ohne Euren Einsatz und Euer Engagement, ohne die unzähligen Diskussionen die wir geführt haben und ohne die ausgebrüteten Ideen wäre diese Arbeit und der KnowMiner nie zustande gekommen. Vielen, vielen Dank! Auch den Anderen an der Entwicklung oder an den mit dem Framework umgesetzten Projekten Beteiligten, namentlich Mathias Lux, Thomas Neidhart, Roman Kern, Mario Zechner, Walter Sarka und Andreas Juffinger, möchte ich meinen Dank für die Unterstützung, Hilfe und vor allem für die Geduld in der Anwendung des KnowMiner's, trotz den zahlreichen aufgetretenen Bugs, aussprechen.

Für die Möglichkeit, überhaupt ein solches Projekt in Angriff nehmen zu können, sowie für die im Umfeld des Know-Center's entstandenen Möglichkeiten für mich, möchte ich von ganzem Herzen meinem Professor und Mentor Klaus Tochtermann danken. Vor allem seinen hilfreichen Hinweisen und Anmerkungen ist die erfolgreiche und rasche Fertigstellung dieser Arbeit zu verdanken, denn ohne diese Hinweise wäre ich mehreren Irrwegen zu lange gefolgt. Ohne seine großartige Unterstützung, sowohl inhaltlicher als auch moralischer Natur, wäre diese Arbeit nie so schnell von der Hand gegangen. Vielen Dank Klaus!

Auch die von ihm geprägte konstruktive und offene Atmosphäre im Know-Center trägt zum erfolgreichen Abschluss dieser und auch vieler anderer Arbeiten bei. An dieser Stelle und in diesem Kontext möchte ich mich bei allen Kollegen am Know-Center, und speziell bei Erwin Duschnig - der immer hilfreiche Worte fand – und Stefanie Lindstaedt bedanken. Ein besonderer Dank gilt Professor Hermann Maurer, der dankenswerterweise die Zweitbegutachtung übernahm und der mir in unseren Diskussionen und Gesprächen, neben inhaltlichen Hinweisen, auch immer wieder den Blick über den Tellerrand zeigte und mein Denken so erweiterte.

Ohne den Rückhalt, den mir meine Lebenspartnerin und zukünftige Ehefrau Gisela gegeben hat, und ohne ihre Ratschläge würde diese Arbeit wahrscheinlich immer noch in der digitalen Welt meines Laptops ruhen. Ich kann nicht mehr, als mich zu bedanken, nicht nur für die Korrektur meiner sprachlichen Unzulänglichkeiten, sondern auch für die moralisch-seelische Unterstützung und ich hoffe, ich kann mich in Zukunft mit mehr Zeit revanchieren.

Die letzten Sätze dieser Danksagung möchte ich meiner Familie widmen. Wenngleich sie mich auf diesem Lebensabschnitt nicht direkt begleitet haben, so gebührt ihnen doch der Dank, mir die Möglichkeit gegeben zu haben, den Weg bis hierher und eventuell noch weiter gehen zu können. Ein Haus kann nur auf einem guten Fundament gebaut werden. Ewigen Dank für dieses Fundament.

„42“

(Douglas Adams, The Hitchhiker's Guide to the Galaxy)[1]

[1] http://www.google.at/search?hl=de&q=answer+to+life+the+universe+and+everything&btnG=Google-Suche&meta=

INHALTSVERZEICHNIS

INHALTSVERZEICHNIS ..... **XIII**

TABELLENVERZEICHNIS ..... **XIX**

1 EINLEITUNG **1-1**

1.1 ZIELSETZUNG DER VORLIEGENDEN ARBEIT 1-6
1.2 METHODIK DER VORLIEGENDEN ARBEIT 1-7
1.3 ZUSAMMENFASSUNG DER ERGEBNISSE 1-8
1.4 AUFBAU DER ARBEIT 1-9

2 EINFÜHRUNG UND ANWENDUNG VON KNOWLEDGE DISCOVERY **2-13**

2.1 KNOWLEDGE DISCOVERY UND DER KNOWLEDGE DISCOVERY PROCESS (KDP) 2-13
2.2 ANWENDUNGEN DES KNOWLEDGE DISCOVERY PROCESS 2-17
2.3 ANFORDERUNGEN 2-23

3 VORVERARBEITUNG UND REPRÄSENTATION UNSTRUKTURIERTER DATEN **3-25**

3.1 TEXTVORVERARBEITUNG 3-25
3.1.1 Formatnormalisierung und lexikalische Analyse *3-25*
3.1.2 Merkmalsanalyse *3-26*
3.2 INFORMATIONSEXTRAKTION 3-28
3.3 MERKMALSGENERIERUNG UND STATISTISCHE ANALYSEN 3-29
3.3.1 Wortanalysen *3-29*
3.3.2 Kollokationen *3-30*
3.3.3 Co-Occurence Analyse *3-31*
3.3.4 Merkmalsgewichtung *3-31*
3.4 VORVERARBEITUNG VON MULTIMEDIALEN INHALTEN 3-33
3.5 OPERATIONEN UND ÄHNLICHKEITSMETRIKEN IN MERKMALSRÄUMEN 3-34
3.6 ANFORDERUNGEN AN DAS KNOWMINER-FRAMEWORK 3-39

4 KD UND IR ALGORITHMEN UND METHODEN **4-41**

4.1 MERKMALSTRANSFORMATION UND ORDINATION 4-41
4.1.1 Latent Semantic Analysis/Indexing *4-42*
4.1.2 Multi Dimensionale Skalierung *4-43*
4.1.3 Sonstige *4-45*
4.2 INFORMATION RETRIEVAL 4-46
4.3 ÜBERWACHTE LERNVERFAHREN 4-50
4.3.1 Beschreibung ausgewählter Methoden *4-51*
4.3.2 Anwendungsgebiete *4-53*
4.4 UNÜBERWACHTE LERNVERFAHREN 4-54
4.4.1 Ausgewählte Verfahren *4-55*
4.4.2 Anwendungsgebiete *4-58*
4.5 INFORMATIONSVISUALISIERUNG UND VISUAL ANALYTICS 4-59
4.6 TEXTBASIERTES LERNEN VON ONTOLOGIEN 4-61
4.7 ANFORDERUNGEN AN DAS KNOWMINER-FRAMEWORK 4-62

5 KD FRAMEWORKS UND STANDARDS **5-65**

5.1 TEXT MINING FRAMEWORKS 5-65
5.1.1 GATE *5-65*
5.1.2 UIMA *5-67*
5.1.3 WEKA *5-69*
5.1.4 Atlas *5-70*
5.2 STANDARDS 5-71
5.2.1 JDM - The Java Data Mining API *5-71*
5.2.2 The Predcitive Model Markup Language - PMML *5-73*
5.2.3 RDF Ressource Description Framework *5-74*
5.3 SERVICE-ORIENTIERTE ARCHITEKTUREN 5-76
5.4 ANFORDERUNGEN 5-80

6 ÜBERBLICK ÜBER DIE ZYKLEN DER ENTWICKLUNG 6-83

6.1 ANALYSE BEREITS EXISTIERENDER APPLIKATIONEN: WEBRAT, INFOSKY UND HIERARCHISCHE DOKUMENTKLASSIFIKATION 6-83
6.1.1 WebRat *6-84*
6.1.2 InfoSky *6-86*
6.1.3 DocClass: Hierarchische Dokumentklassifikation *6-88*
6.1.4 Lessons Learned *6-89*
6.2 KNOWMINER V 1.0: ENTWICKLUNG EINES ALLGEMEINEN KD UND IR FRAMEWORKS 6-91
6.2.1 OnAIR – On APA Intelligent Retrieval *6-93*
6.2.2 MAGICK- Ein Werkzeug für Cross-Media-Clustering und Visualisierung *6-96*
6.2.3 KnowMiner Hierarchische Dokumentklassifikation *6-97*
6.2.4 Lessons Learned *6-98*

7 KONZEPTIONELLE GRUNDLAGEN DES KNOWMINER-FRAMEWORKS 7-101

7.1 DER KNOWLEDGE DISCOVERY PROCESS IM DETAIL 7-102
7.1.1 Akquise und Homogenisierung *7-102*
7.1.2 Vorverarbeitung und Merkmalsgenerierung *7-103*
7.1.3 Merkmalstransformation *7-103*
7.1.4 Mining *7-103*
7.1.5 Aufbereitung und Visualisierung *7-104*
7.2 DATEN UND INFORMATIONSFLÜSSE IM KNOWMINER-FRAMEWORK 7-104
7.3 STRUKTURIERUNG DES INFORMATIONSRAUMES 7-106
7.4 KONZEPTE IM KNOWMINER-FRAMEWORK 7-110
7.4.1 Informationsentität *7-110*
7.4.2 Entitätsrelation *7-111*
7.4.3 Informationsraum *7-112*
7.4.4 Element *7-112*
7.4.5 Eigenschaften von Elementen *7-112*
7.5 TRANSFORMATIONEN IM INFORMATIONSRAUM 7-117
7.5.1 Eigenschaftstransformation *7-118*
7.5.2 Strukturtransformation *7-121*
7.6 KD KOMPONENTEN UND ALGORITHMEN 7-121
7.6.1 Entitäten modifizierende Komponentenklassen *7-123*
7.6.2 Struktur modifizierende Komponentenklassen *7-124*
7.6.3 Nicht-modifizierende Komponentenklassen *7-125*

7.7 ZUSAMMENFASSUNG 7-126

8 IMPLEMENTIERUNG DES KNOWMINER-FRAMEWORKS 8-127

8.1 BASISARCHITEKTUR 8-127
8.1.1 Import Controller 8-128
8.1.2 Generische Interne Repräsentation (GIR) 8-129
8.1.3 Worker 8-130
8.1.4 Persistence Layer 8-131
8.1.5 Output Layer 8-131
8.2 GML GENERIC MARKUP LANGUAGE 8-133
8.3 GENERISCHE INTERNE REPRÄSENTATION 8-136
8.3.1 Repräsentation von Strukturen 8-136
8.3.2 Repräsentation von Eigenschaften von Elementen eines Informationsraumes 8-145
8.4 WORKER KONZEPT UND ALGORITHMEN 8-153
8.4.1 Implementierungsdetails zum Worker Konzept 8-153
8.4.2 ContentProcessor 8-154
8.4.3 Clustering 8-159
8.4.4 Ordination und Merkmalstransformation 8-160
8.4.5 Klassifikation 8-162
8.4.6 Information Retrieval 8-164
8.5 DIE KNOWMINER SERVICE-ORIENTIERTE ARCHITEKTUR 8-167
8.5.1 Basis und Assembly Services 8-168
8.5.2 Implementierungsdetails 8-170
8.5.3 Grafisches Service Frontend 8-175
8.6 KONFIGURATION, MONITORING UND DEBUGGING TOOLS 8-176
8.7 ZUSAMMENFASSUNG 8-179

9 ANWENDUNGEN UND ABGELEITETE ERGEBNISSE 9-181

9.1 XTREND 9-181
9.2 MISTRAL 9-187
9.2.1 Systemüberblick 9-188
9.2.2 Systemarchitektur und Integration des KnowMiner-Frameworks 9-191
9.2.3 Zusammenfassung und Lessons Learned 9-193
9.3 SONSTIGE ANWENDUNGEN UNTER VERWENDUNG DES KNOWMINER-FRAMEWORKS 9-194
9.3.1 DYONIPOS - Dynamic Ontology based Integration Process Optimization 9-194
9.3.2 Semiautomatische Erzeugung von Wissenslandkarten 9-196
9.3.3 Lessons Learned 9-197
9.4 ZUSAMMENFASSUNG 9-197

10 ZUSAMMENFASSUNG UND OFFENE PUNKTE 10-199

10.1 ERGEBNISSE UND VERGLEICH MIT DER ZIELSETZUNG 10-199
10.2 VERGLEICH ZU EXISTIERENDEN SYSTEMEN 10-201
10.3 AUSBLICK 10-202

ABBILDUNGSVERZEICHNIS

ABBILDUNG 1-1: ENTWICKLUNG ÜBER DIE ANZAHL DER REGISTRIERTEN DOMÄNEN SEIT 1992 1-1
ABBILDUNG 1-2: DAS SPIRALMODELL NACH BOEHM (BILDQUELLE: IT-HANDBUCH ISBN 3-14-225042-5) 1-7
ABBILDUNG 1-3: AUFBAU DER VORLIEGENDEN ARBEIT 1-10
ABBILDUNG 2-1: DER KDD PROZESS NACH [FAYYAD ET. AL. 1996] 2-14
ABBILDUNG 2-2: DAS MAURER TOCHTERMANN MODELL (MTM) NACH [MAURER ET. AL. 2002] DEFINIERT 7 VERSCHIEDENE INTERAKTIONEN VON BENUTZERN IM KONTEXT EINES WISSENSMANAGEMENTSYSTEMS. 2-18
ABBILDUNG 2-3: WISSENS-/INFORMATIONSKREISLAUF BEZOGEN AUF DIE INTERAKTIONSGRENZEN IM MAURER TOCHTERMANN MODELL ZWISCHEN BENUTZER UND WMS. 2-21
ABBILDUNG 3-1: ZIPF'S LAW: ZUSAMMENHANG ZWISCHEN DEM VORKOMMNISSEN (=F) UND INFORMATIONSGEHALT VON WÖRTERN.. DIE GRAU SCHATTIERTE FLÄCHE BEINHALTET DIE LAUT ANALYSE SIGNIFIKANTEN WÖRTER (ENTNOMMEN AUS [RIJSBERGEN 1979]) 3-30
ABBILDUNG 3-2: BEISPIEL FÜR DIE REPRÄSENTATION VON DOKUMENTEN IM VECTOR-SPACE-MODEL 3-32
ABBILDUNG 3-3: KERNEL TRANSFORMATION EINES 2D RAUMES IN EINEN 3D RAUM UNTER ANWENDUNG EINES POLYNOMIELLEN KERNEL. 3-38
ABBILDUNG 4-1: SCHEMATISCHE ABBILDUNG DER LSI TRANSFORMATION 4-43
ABBILDUNG 4-2: KONZEPTUELLES MODELL EINES IR SYSTEMS ENTNOMMEN AUS [FUHR 2006] 4-47
ABBILDUNG 4-3: BEISPIEL FÜR EINEN INVERTIERTEN INDEX MIT WORTEN ALS MERKMALEN. LINKS IST DIE SORTIERTE LISTE AN MERKMALEN, WELCHE AUF EINE LISTE VON DOKUMENTEN IN DENEN DIE MERKMALE VORKOMMEN ZEIGT. 4-48
ABBILDUNG 4-4: SCHEMATISCHER ABLAUF EINER SUCHE UNTER BERÜCKSICHTIGUNG DES ITERATIVEN CHARAKTERS . 4-49
ABBILDUNG 4-5: LINEARE KLASSIFIKATIONSHYPOTHESE IM 2D. KREISE STELLEN DIE NEGATIVEN, RECHTECKE DIE POSITIVEN BEISPIELE DAR. DER VEKTOR *W* DEFINIERT DEN NORMALVEKTOR AUF DIE TRENNENDE HYPEREBENE. LINKS IST DIE WAHL DER EBENE BELIEBIG (ES GIBT MEHRERE LÖSUNGEN) WÄHREND RECHTS JENE EBENE GEWÄHLT WIRD, DIE DEN ABSTAND ZWISCHEN POSITIVEN UND NEGATIVEN BEISPIELEN MAXIMIERT. 4-51
ABBILDUNG 4-6: CLUSTERING IM 2D. AUSGEHEND VON EINER MENGE AN AUS 5 GAUSSVERTEILUNGEN GEZOGENEN OBJEKTEN (LINKS) ERFOLGT DAS FINDE DER GRUPPEN (RECHTS). DIE GRUPPE LINKS OBEN IN DER RECHTEN ABBILDUNG KONNTE NICHT KORREKT IDENTIFIZIERT WERDEN. 4-54
ABBILDUNG 4-7: DENDROGRAMM (RECHTS) ERSTELLT AUF BASIS DER EUKLIDSCHEN DISTANZ ZWISCHEN DEN PUNKTEN LINKS 4-56
ABBILDUNG 4-8: BEISPIELE FÜR DIE VIELFALT AN CLUSTERFORMEN BEI DICHTEBASIERTEN VERFAHREN(QUELLE: HTTP://WWW.INF.UNIBZ.IT/DIS/TEACHING/MSC/PROJECT-DBSCAN.HTML) 4-57
ABBILDUNG 4-9: CLUSTERING DER TREFFER DER SUCHE "JAVA" AUF HTTP://WWW.CLUSTY.COM 4-58
ABBILDUNG 5-1: GUI EDITOR ZUR DARSTELLUNG DER IE ERGEBNISSE UND ZUR MANUELLEN ANNOTATION (QUELLE: HTTP://GATE.AC.UK/SALE/ACL02/ACL-MAIN.PDF) 5-66
ABBILDUNG 5-2: ÜBERBLICK ÜBER DIE UIMA KOMPONENTEN UND DIE EINBETTUNG VON APPLIKATIONEN IM UIMA FRAMEWORK AUS SICHT DER COLLECTION PROCESSING ENGINE (QUELLE [IBM 2004, SEITE 2-38]) 5-67
ABBILDUNG 5-3: AUFBAU VON COLLECTION PROCESSING ENGINES IN UIMA (QUELLE [IBM 2004, SEITE 2-35]) 5-68
ABBILDUNG 5-4: AUSSCHNITT AUS DER WEKA GUI. (QUELLE: HTTP://PRDOWNLOADS.SOURCEFORGE.NET/WEKA/WEKA.PPT) 5-69
ABBILDUNG 5-5: ONTOLOGISCHER AUFBAU VON ANNOTATIONEN IM ATLAS SYSTEM (QUELLE: HTTP://WWW.NIST.GOV/SPEECH/ATLAS/DEVELOP/CORE.HTML) 5-70
ABBILDUNG 5-6: VISUALISIERUNG DER ERSTEN EBENE DES PMML XML SCHEMAS 5-73
ABBILDUNG 5-7. BEISPIEL EINES RDF GRAPHEN FÜR DIE SPEZIFIKATION EINES DUBLIN CORE DATENSATZES (QUELLE: [KOKKELINK & SCHWÄNZL 2002]) 5-75

ABBILDUNG 5-8: FOAF BEISPIEL FÜR MICHAEL GRANITZER 5-75
ABBILDUNG 5-9: DAS FIND-BIND-EXECUTE PATTERN IN EINER SERVICE-ORIENTIERTEN ARCHITEKTUR (QUELLE: HTTP://WWW.DEVELOPER.COM/JAVA/WEB/ARTICLE.PHP/10935_2207371_1) 5-77
ABBILDUNG 5-10: SOA PYRAMIDE MIT DEN UNTERSCHIEDLICHEN SERVICE EBENEN 5-79
ABBILDUNG 6-1: LINKS VISUELLE DARSTELLUNG VON SUCHERGEBNISSEN ZUM THEMA KNOWLEDGE MANAGEMENT. RECHTS OBEN: DURCH ZOOMING VERGRÖßERTER AUSSCHNITT, RECHTS UNTEN: QUERY REFINEMENT MIT DEM BEGRIFF INFORMATION 6-84
ABBILDUNG 6-2: UNTERSCHIEDLICHE ANWENDUNGSMÖGLICHKEITEN DER WEBRAT. LINKS ZUM VERGLEICH VON VERSCHIEDENEN SUCHERGEBNISMENGEN UND RECHTS ZUR VISUALISIERUNG DER ZUSAMMENHÄNGE EINES VERZEICHNISSES IM WISSENSMANAGEMENTSYSTEM HYPERWAVE 6-85
ABBILDUNG 6-3: ÜBERBLICK ÜBER DIE EINZELNEN WEBRAT SYSTEMKOMPONENTEN 6-86
ABBILDUNG 6-5: ONAIR BEISPIEL ZUM SUCHBEGRIFF „WWF“ 6-95
ABBILDUNG 6-6: ONAIR: ANALYSE DER METADATEN UND DARSTELLUNG ALS BAUM. DURCH NAVIGATION IM BAUM WIRD AUCH DIE ERGEBNISSMENGE ENTSPRECHEND EINGESCHRÄNKT. 6-95
ABBILDUNG 6-7: ÜBERBLICK ÜBER DEN ABLAUF IN MAGICK (AUS [LUX 2004]) 6-96
ABBILDUNG 6-8: SCREENSHOT AUS MAGICK. GEMEINSAMES CLUSTERING UND VISUALISIERUNG VON BILD-, TEXT- UND AUDIODATEN 6-97
ABBILDUNG 7-1: ABBILDUNG DES KNOWMINER-INFORMATIONSFLUSSES. IN JEDEM SCHRITT WERDEN ANDERE ASPEKTE EINER INFORMATIONSENTITÄT VERARBEITET. KD-ALGORITHMEN KÖNNEN AUF ALLE EIGENSCHAFTEN EINER INFORMATIONSENTITÄT ZUGREIFEN 7-105
ABBILDUNG 7-2: ÜBERBLICK UND ZUSAMMENHANG DER KNOWMINER KONZEPTE 7-107
ABBILDUNG 7-3: TRANSFORMATIONEN ZWISCHEN EIGENSCHAFTEN VON ELEMENTEN EINES INFORMATIONSRAUMES. PUNKTIERTE LINIEN BESCHREIBEN VORWIEGEND INFORMATIONSREDUZIERENDE TRANSFORMATIONEN, DURCHGEZOGENE LINIEN INFORMATIONSERHALTENDE TRANSFORMATIONEN UND STRICHLIERTE LINIEN INFORMATIONSANREICHERNDE TRANSFORMATIONEN 7-118
ABBILDUNG 8-1: KNOWMINER-KOMPONENTENARCHITEKTUR MIT IMPORTCONTROLLER, WORKER, GENERISCHE INTERNE REPRÄSENTATION, PERSISTENCE LAYER, OUTPUT LAYER. 8-128
ABBILDUNG 8-2: XML SCHEMA STRUKTUR VON GML 8-133
ABBILDUNG 8-3: DIE ABSTRAKTEN DATENTYPEN DATA UND METADATA IM DETAIL 8-135
ABBILDUNG 8-4: HAUPTKLASSEN FÜR DIE UMSETZUNG DER STRUKTURVERWALTUNG AUF BASIS EINER ABSTRAHIERTEN RDF SCHNITTSTELLE 8-137
ABBILDUNG 8-5: SEQUENZDIAGRAMM ZUR ERZEUGUNG EINER GIR ERGEBNISMENGE UND ITERATION ÜBER DIE ENTHALTENEN OBJEKTE NACH ERZEUGEN EINES READLOCKS 8-139
ABBILDUNG 8-6: BEISPIELHAFTE ERZEUGUNG VON ENTITÄTEN UND RELATIONEN MITTELS ELEMENTSET. 8-141
ABBILDUNG 8-7: DARSTELLUNG VON ZWEI INFORMATIONSPACEELEMENT’S ALS GRAPH. MERKMALE, ANNOTATIONEN UND METADATEN UNTERSCHIEDLICHER ELEMENTE VERWEISEN DABEI JEWEILS AUF DEN GLEICHEN KONTEXTKNOTEN 8-144
ABBILDUNG 8-8: GRAPH-ABBILDUNG VON ENTITÄTEN UND RELATIONEN AUF BASIS VON INFORMATIONSPACEELEMENT’S, WELCHE WIEDERUM ALS GRAPH ABGEBILDET SIND (SIEHE ABBILDUNG OBEN) 8-145
ABBILDUNG 8-9: KLASSENDIAGRAMM ZUM KNOWMINER MERKMAL-TYPSYSTEM. RECHTS SIND DIE SCHNITTSTELLEN ABGEBILDET, LINKS DIE MERKMALSTYPEN. DIE INSTANZIERUNG VON MERKMALEN, DEREN SCHLÜSSELN UND DEREN MERKMALSWERTEN ERFOLGT ÜBER FACTORY PATTERNS. 8-146
ABBILDUNG 8-10: UML KLASSENDIAGRAMM FÜR DIE VERWALTUNG VON MERKMALSWERTEN IM KNOWMINER-FRAMEWORK. DIE FEATUREVALUE-FACTORY SORGT FÜR INSTANZIERUNG DER UNTERSCHIEDLICHEN WERTEBEREICHE. 8-147
ABBILDUNG 8-11: UML KLASSENDIAGRAMM FÜR DIE VERWALTUNG VON MERKMALSSCHLÜSSEL. WIEDERUM ERFOLGT DIE INSTANZIERUNG DER SCHLÜSSEL ÜBER EIN FACTORY-PATTERN (FEATUREKEYTYPE). 8-148

ABBILDUNG 8-12: UML KLASSENDIAGRAMM FÜR DIE VERWALTUNG VON MERKMALEN. GRUNDLEGEND GREIFT DIE KLASSE FEATURECLASS AUF DIE KLASSEN ZUR ERZEUGUNG VON MERKMALSSCHLÜSSELN UND MERKMALSWERTEN ZURÜCK, UM EIN ENTSPRECHENDES MERKMAL ZU INSTANZIEREN..... 8-148
ABBILDUNG 8-13: UML KLASSENDIAGRAMM FÜR MERKMALSRÄUME UND MERKMALSMENGEN..... 8-150
ABBILDUNG 8-14: UML KLASSENDIAGRAMM BZGL. DEN VERWENDETEN KALKULATOREN. 8-151
ABBILDUNG 8-15: UML KLASSENDIAGRAMM ZUM ANNOTATIONSSYSTEM..... 8-152
ABBILDUNG 8-16: UML DIAGRAMM DER WORKER-KLASSEN. 8-153
ABBILDUNG 8-17: UML KLASSENDIAGRAMM DER PIPELINE ZUR INHALTSNORMALISIERUNG. PIPEDATA KAPSELT DIE DATEN DER PIPELINE UND PIPECOMPONENT DIE EINZELNEN KOMPONENTEN. DIE PIPELINE IST HIERARCHISCH ORGANISIERT SODASS EINE PIPECOMPONENT WIEDERUM ALS KOMPONENTEN EINER ANDEREN PIPECOMPONENT VERWENDET WERDEN KANN. 8-155
ABBILDUNG 8-18: UML DIAGRAMM DER CONTEXT PIPELINE ZUR ANNOTIERUNG UND VORVERARBEITUNG VON INHALTEN. 8-158
ABBILDUNG 8-19: IMPLEMENTIERUNGEN DES CONTEXTCOMPONENT INTERFACE FÜR VERSCHIEDENE IM KNOWMINER VERWENDETE STEMMER 8-159
ABBILDUNG 8-20: IMPLEMENTIERUNGEN DES CONTEXTCOMPONENT INTERFACE FÜR DIE IMPLEMENTIERUNG VON GATE KOMPONENTEN IM KNOWMINER 8-159
ABBILDUNG 8-21: BESCHREIBUNG DER CLUSTER INTERFACES UND DEREN EINBETTUNG IN DIE WORKER INFRASTRUKTUR8-160
ABBILDUNG 8-22: UML DIAGRAMM ZUR IMPLEMENTIERUNG DES K-MEANS CLUSTERING-ALGORITHMUS AUF BASIS DER INTERFACES UND WORKER STRUKTUR 8-160
ABBILDUNG 8-23: ÜBERBLICK ÜBER DIE KOMPONENTEN DER KLASSIFIKATION 8-162
ABBILDUNG 8-24: SCHNITTSTELLEN ZUR IMPLEMENTIERUNG DER KLASSIFIKATIONSHYPOTHESEN /-ALGORITHMEN 8-163
ABBILDUNG 8-25: INTERFACE STRUKTUR DER KNOWMINER RETRIEVAL FUNKTIONALITÄT 8-165
ABBILDUNG 8-26: UML DIAGRAMM ZUR IMPLEMENTIERUNG DER SUCHMASCHINE LUCENE AUF BASIS DER OBEN DARGESTELLTEN INTERFACE STRUKTUR. 8-165
ABBILDUNG 8-27: KNOWMINER SERVICE ARCHITEKTUR. AM KNOWMINER CORE SETZEN DIE BASIS-SERVICES FÜR DIE EINZELNEN FUNKTIONALITÄTEN AUF, WELCHE IM ALS WEB-SERVICE ANSPRECHBAREN TOPLEVELSERVICE KOMBINIERT WERDEN..... 8-167
ABBILDUNG 8-28: DATEN UND PROCESSING INTERFACES DER SERVICE ARCHITEKTUR 8-171
ABBILDUNG 8-29: FINITE STATE MODELL VON PROCESSING UND INCREMENTAL PROCESSING SERVICES (QUELLE [SABOL, KLIEBER, GRANITZER 2006])..... 8-173
ABBILDUNG 8-30: GRAFISCHES SERVICE FRONTEND DES KNOWMINER-FRAMEWORKS..... 8-176
ABBILDUNG 8-31: SCREENSHOT DES KNOWMINER WORKER MONITORS 8-178
ABBILDUNG 8-32: GIR DEBUGGING TOOL. DARSTELLUNG DES GIR GRAPHEN 8-178
ABBILDUNG 9-1: 3-TIER SYSTEMARCHITEKTUR DER X-TREND APPLIKATION 9-184
ABBILDUNG 9-2: ÜBERBLICK ÜBER DIE MISTRAL DIENSTE 9-189
ABBILDUNG 9-3: DATENFLUSS ZWISCHEN DEN MISTRAL KOMPONENTEN 9-192
ABBILDUNG 9-4: ÜBERBLICK ÜBER DEN FEEDBACK-LOOP ZUR ERZEUGUNG VON WISSENSLANDKARTEN 9-196

TABELLENVERZEICHNIS

TABELLE 2-1: ABGRENZUNG KDD/KDT AUF BASIS DER „NEUHEIT“ DER ENTDECKUNG UND DEN UNTERSCHIEDLICHEN EINGANGSDATEN 2-15
TABELLE 2-2: UM STRUKTURELLE DATEN UND MULTIMEDIALE DATEN ERWEITERTES MODELL. 2-16
TABELLE 2-3: FOKUS DES UMZUSETZENDEN FRAMEWORKS BEZOGEN AUF [KROEZE ET. AL. 2003] 2-23
TABELLE 3-1: BEISPIEL FÜR DIE AUSPRÄGUNG VON MERKMALEN FÜR OBJEKTE 3-34
TABELLE 6-1: ERGEBNISSE DES AUF DEM REUTERS CORPUS VOLUME 1 TRAINIERTEN ASSOZIATIONSINDEX (TRAININGSZEIT 5H) 6-94
TABELLE 6-2: KONVERTIERUNG VON INPUTFORMATEN IN MAGICK AUF DUBLIN CORE (AUS [LUX 2004]) 6-96
TABELLE 7-1: ÜBERBLICK ÜBER DIE VERSCHIEDENEN ENTITÄTEN-MODIFIZIERENDEN KD-KOMPONENTENKLASSEN 7-122
TABELLE 7-2: ÜBERBLICK ÜBER DIE VERSCHIEDENEN STRUKTUR-MODIFIZIERENDEN KD-KOMPONENTENKLASSEN 7-122
TABELLE 7-3: ÜBERBLICK ÜBER DIE VERSCHIEDENEN NICHT-MODIFIZIERENDEN KOMPONENTENKLASSEN 7-122
TABELLE 8-1: EIGENSCHAFTEN VON BEISPIELKOMPONENTEN AUS DEM BEREICH TEXTVORVERARBEITUNG 8-157

1 Einleitung

„Data is the next Intel Inside“ ist einer der Leitsprüche in Tim O'Reilly's Artikel „What is Web 2.0“ [O'Reilly 2005], der die Wichtigkeit von Daten und Information für die Applikationen der Zukunft darstellt. Vor allem die von Applikationen bereitgestellten und für den Benutzer aufbereiteten Informationen, sowie die angebotenen Funktionen zum intelligenten Umgang mit Information stellen den Mehrwert für Applikation der Zukunft dar. Betrachtet man jedoch die weltweit vorhandene Informationsmenge und deren Anstieg, so wird schnell klar, wie schwierig die adäquate Bereitstellung und Aufbereitung von Information eigentlich ist. So zeigt eine Studie zur weltweiten Informationsmenge der University of California [Lyman 2003] eindrucksvoll auf, wie viele Informationen pro Jahr neu erzeugt werden. So waren es im Jahr 2002 weltweit 5 Exabyte (=$5x10^{18}$ Byte) - das sind ungefähr 37.000-mal der Dokumentenbestand der berühmten amerikanischen Library of Congress, die immerhin 17 Millionen Bücher zu ihrem Bestand zählt. Von dieser Informationsmenge liegen 92% in digitaler Form vor, und immerhin sind 170 Terrabyte oder 17-mal der Bestand der Library of Congress im Internet verfügbar. Bei diesem Vergleich zwischen Bestand der Library of Congress und digitaler Informationsmenge sei jedoch ausdrücklich betont, dass er sich allein auf die Quantität und auf keinen Fall auf die Qualität der Informationen bezieht! Bei einer angenommenen Weltbevölkerung von 6,3 Milliarden entstehen, so die Studie, pro Jahr 800 Megabyte neue Daten pro Kopf und Jahr.

Ein Vergleich zur äquivalenten Studie aus dem Jahre 1999 zeigt eine Verdreifachung der neu erzeugten Informationsmenge, was in etwa 30% Steigerung pro Jahr entspricht. Betrachtet man vor allem die Entwicklung des World Wide Web's (WWW's) und des Internets, so ist dieses Wachstum leicht nachvollziehbar. So unterstreichen Statistiken über die Anzahl der registrierten Internetdomänen [2], zusammengefasst in Abbildung 1-1, das immense Wachstum des WWW's und damit verbunden auch die zur Verfügung stehende Menge an Information.

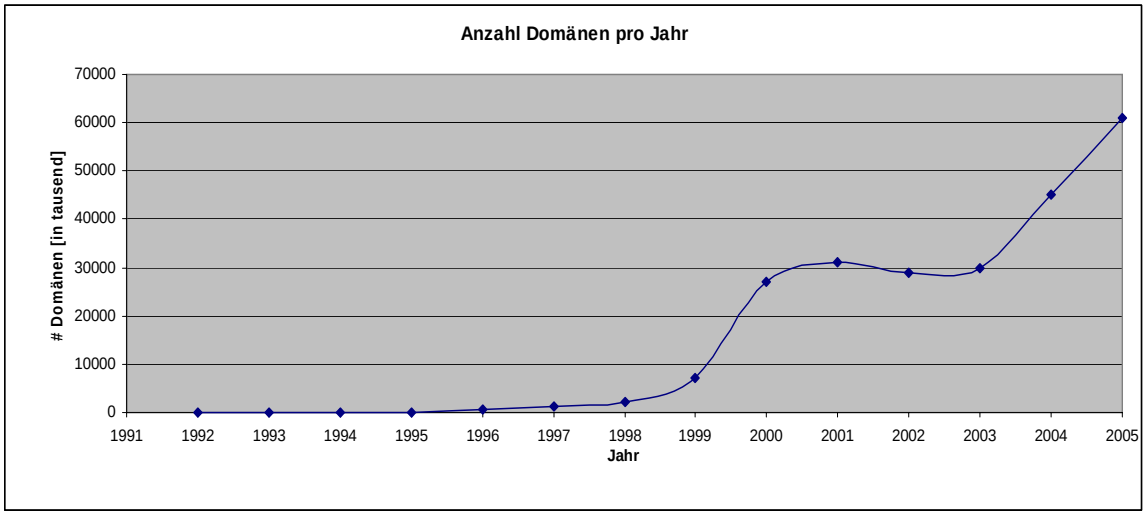


Abbildung 1-1: Entwicklung über die Anzahl der registrierten Domänen seit 1992

[2] http://www.zooknic.com/Domains/counts.html

Dieses Wachstum zeigt auch die zentrale Bedeutung des WWW's und des Internets. Information stellt den wichtigsten Faktor für die Entscheidungsfindung dar und effiziente, einfach bedienbare Techniken zum Finden der benötigten Information formen die Eckpfeiler der heutigen Informationsgesellschaft. Vor allem Textdaten wie Nachrichten, E-Mails, enzyklopädische Artikel, wissenschaftliche Publikationen etc. sind Hauptinformationsquellen für Nutzer, wenn es um das Finden der gewünschten oder benötigten Information geht.

Die Bereitstellung von Wissen und Information geht zurück bis ins alte Ägypten, wo die erste Bibliothek zur Verwaltung von Papyrusrollen entstand. Betrachtet man die Methoden der klassischen Bibliotheken und deren lange Historie im Verwalten und Auffinden von Texten, so sieht man die Notwendigkeit der Kategorisierung und Katalogisierung von Schriftstücken, um ein einfaches Auffinden dieser möglich zu machen. Das Dewy Decimal Classification System (DDC), entwickelt im Jahre 1876 (siehe [Dewey 1876]), ist ein allgemeines Klassifikationssystem zur Kategorisierung von Büchern. Das DDC organisiert dabei das allgemeine Wissen in 10 Hauptklassen, welche hierarchisch weiter unterteilt sind. Die eindeutige Identifikation einer Klasse erfolgt dabei auf Basis eines speziellen, hierarchisch aufgebauten Nummernsystems. In Bibliotheken erstellten Bibliothekare Indizes bzw. Verzeichnisse, in denen Bücher anhand solcher Klassifikationsschemata aufgelistet wurden. Damit konnte das Auffinden, die Kenntnis dieses Klassifizierungssystems vorausgesetzt, immens erleichtert werden und ein schneller Zugriff auf Wissen auch in damaligen großen Bibliotheken ermöglicht werden.

Mit der beginnenden Digitalisierung von Information eröffneten sich neue Möglichkeiten, Information und Wissen in Form von Texten zu verwalten. Schon im Jahre 1945 beschreibt Vannevar Bush in [Bush 1945] seine Vorstellungen zur Verwaltung von Information und Wissen in Form eines einfach zu bedienenden Wissensfindungs- und Wissensverwertungssystems namens Memex (*Mem*ory *Ex*tender):

> *„Der menschliche Geist arbeitet... mittels Assoziation. Kaum hat er sich eine Information beschafft, greift er schon auf die nächste zu, die durch die Gedankenverknüpfung vorgeschlagen wird, entsprechend einem komplizierten Gewebe von Pfaden, das über die Hirnzellen verläuft. ...(Die Memex ist) ein Gerät, in dem ein Individuum alle seine Bücher, Aufzeichnungen und Kommunikation speichert, und das mechanisiert ist, so dass es mit steigender Geschwindigkeit und Flexibilität zu Rate gezogen werden kann. Sie ist ein vergrößerter Anhang seines Gedächtnisses."*

Aus den anhand der Memex beschriebenen Ideen von Vannevar Bush und mit den Möglichkeiten der Digitalisierung entwickelte sich in den 60er Jahren das Gebiet des Information Retrieval (IR) (siehe auch [Lesk 1995] für einen Überblick über die Geschichte des Information Retrieval). In ersten Experimenten mit dem NASA RECON System digitalisierte man die gedruckten Indizes und Klassifikationssysteme und ermöglichte eine mechanisierte Suche in diesen. Während es in gedruckten Indizes nur schwer möglich war, Texte zu mehreren Klassifikationen zu finden, eigneten sich Computer hervorragend dazu, Themengebiete über logische AND Verknüpfungen zu verbinden und so Werke, die beide Themengebiete behandelten, zu finden. Auch die Größe des Index spielte nur mehr eine untergeordnete Rolle. Der Nachteil des arbeitsintensiven, manuellen Indizierens blieb jedoch weiterhin bestehen.

Um das manuelle Indizieren von Texten zu vermeiden, entwickelte sich die Idee der Volltextsuche und damit verbunden der Textanalyse. In Volltextsystemen erfolgte die Erstellung des Index ebenfalls automatisch, indem alle Wörter eines Text-Abstracts als „Klassifikationen" in den Index aufgenommen wurden. Die Formulierung von Suchen erwies sich jedoch als problematisch, da Klassifikationssysteme Texte nicht nur indizierten, sondern auch ein gemeinsames Vokabular zur Beschreibung einer Domäne identifizierten. Im

Jahre 1968 entwickelte Salton [Salton 1968] effiziente Methoden zur Volltextsuche und berücksichtigte über Feedback-Methoden auch die Verfeinerung von Suchanfragen auf Basis von bereits gefundenen Texten. Das von Salton entwickelte Vektorraummodell (siehe auch Abschnitt 4.2) stellt heute noch eine weit verbreitete IR Technik dar.

Durch den Einzug der computergestützten Textverarbeitung und in weiterer Folge des Internets entwickelte sich Information Retrieval zu einem Eckpfeiler der Informationsbereitstellung. In den 80er Jahren wurden die ersten großen Retrievalsysteme im Umfeld digitaler Bibliotheken entwickelt und mit dem Boom des WWW's in den 90ern erhöhte sich auch die Notwendigkeit, schnell in Millionen von Webseiten zu suchen und gleichzeitig mehrere Tausend Benutzer zu unterstützen. Indiziert wurde der gesamte Text von Dokumenten, nicht nur das Abstract. Die Entwicklung neuer Retrievalmodelle und Datenstrukturen sowie die rasante Erhöhung der Leistungsfähigkeit von Computerhardware ermöglichen heutzutage die Suche in über 20 Milliarden Dokumenten mit 90 Millionen Suchen pro Tag[3].

Gleichzeitig mit dem Boom des WWW's entwickelten sich auch neue Methoden zur Analyse von Dokumenten. Techniken aus den Bereichen des Knowledge Discovery (KD) und Text Mining ermöglichen das Erschließen unstrukturierter Information sowie die Ableitung von Konzepten und Zusammenhängen zwischen Dokumenten, um damit die Informationsqualität zu erhöhen.

Knowledge Discovery ist dabei definiert als ([Fayyad et. al. 1996])

"the nontrivial process of identifying valid, novel, potentially useful, and ultimately understandable patterns in data"

und ermöglicht das Finden von Zusammenhängen in unstrukturierten Daten. Dazu bedient sich KD der Techniken aus vielen verschiedenen Gebieten, wie dem maschinellen Lernen, dem Data Mining, dem Information Retrieval und, speziell im Umgang mit Text, dem Text Mining. Hinsichtlich der zu verarbeitenden Information deckt KDD neben textuellen auch numerische (z.B. Umsatzzahlen) und multimediale Daten ab, wodurch sich einerseits ein breites Anwendungsspektrum und andererseits auch hohes Potential ergibt. Während Text im Allgemeinen der Hauptinformationsträger ist, zeigt der Trend auch in Richtung eines Anstiegs multimedialer Inhalte. Laut einer IDC Studie[4][Chute 2006] wächst z.B. die Anzahl der Bilder von 2005 bis 2009 um 24%, und auch in [Lyman 2003] findet man Hinweise auf die zunehmende Multimodalität der Information.

Trotz der verarbeitbaren Informationsvolumina, der effizienten Bereitstellung von Suchmöglichkeiten und der Entwicklung neuer Methoden zur Analyse unstrukturierter Information, stößt der Nutzer bei der Suche nach relevanter Information an Grenzen. Um Information effizient und den Bedürfnissen des Benutzers entsprechend bereitstellen zu können, müssen einerseits die Semantik bei der Formulierung des Informationsbedürfnisses seitens des Benutzers und andererseits die Semantik der abgelegten Information klar spezifiziert sein. Natürliche Sprache ist mitunter durch ihre Mehrdeutigkeit und Vagheit charakterisiert, wodurch es schwer ist, die korrekte Semantik natürlicher Sprache beim Indizieren zu extrahieren. Mit steigender Dokumentanzahl steigen dabei auch die indizierten Mehrdeutigkeiten und der Benutzer findet mehr und mehr irrelevante Treffer.

[3] http://searchenginewatch.com/showPage.html?page=2156461

[4] http://www.idc.com/getdoc.jsp?containerId=prUS20035406

Metadaten - Daten über Daten, die den Inhalt von Dokumenten auf einer abstrakten Ebene beschreiben sollen - können die Semantik sowohl bei der Formulierung des Informationsbedürfnisses als auch beim Indizieren erhöhen. Autor, Datum oder Kategorisierungen nach einem Klassifikationssystem wie in Bibliotheken sind mögliche Metadaten. Untersuchungen zeigen jedoch, dass Metadaten im Umfeld des WWW's, aber auch in Unternehmensnetzwerken kaum an erstellte Texte angehängt werden. O'Neill et. al. bringen dies in [O'Neill et. al. 2003] auf den Punkt:

> *"Finally, examination of metadata usage on the public Web over the five-year span of the Web Characterization Project surveys indicates that little if any progress is being made to effectively describe Web-accessible resources, with predictable results for ease of search and retrieval."*

Zusätzlich existiert eine Vielzahl an verschiedenen, domänenspezifischen Metadatenstandards, welche untereinander schwer kombinierbar und damit wiederum eine Quelle für Mehrdeutigkeiten sind. Zeitbezüge stellen beispielsweise eine wichtige Komponente im Auffinden von Information dar, wie in [Cutrell et. al. 2006] beschrieben. Während Zeitbezüge in den meisten Metadatenstandards explizit ausgewiesen werden können, ist der Kontext des Zeitbezuges meist nicht klar spezifiziert. So kann es sich um das Erzeugungsdatum des Textes, aber auch um den Zeitpunkt der letzten Aktualisierung handeln. Eine einfache Google-Suche mit einer zusätzlichen zeitlichen Einschränkung macht dies deutlich. Die zeitliche Einschränkung bezieht sich dabei auf das in HTML-Ressourcen angegebene Zeit- Metadatum, was für gewisse Seiten das Erzeugungsdatum, für andere das Aktualisierungsdatum darstellt. Durch diese Mehrdeutigkeit ist es beispielsweise schwer, die Nachrichten der letzten drei Monate zu finden, da Seitenaktualisierungen mit einbezogen werden. Durch den Mangel an und die teilweisen Mehrdeutigkeiten von Metadaten und durch die, wie anfangs beschrieben, heutige Informationsflut stoßen bis dato entwickelte Retrievalmethoden und Modelle an ihre Grenzen

Vor dem Hintergrund einer intelligenten Informationsbereitstellung formulierte Tim Berners Lee, Vater des heutigen World Wide Web's, seine Vision vom Semantic Web [Berners-Lee 1998]:

> *"I have a dream for the Web [in which computers] become capable of analyzing all the data on the Web – the content, links, and transactions between people and computers. A 'Semantic Web', which should make this possible, has yet to emerge, but when it does, the day-to-day mechanisms of trade, bureaucracy and our daily lives will be handled by machines talking to machines. The 'intelligent agents' people have touted for ages will finally materialize. (Berners-Lee, 1999)"*

Die Realisierung dieser Vision basiert auf der semantischen Beschreibung von Webressourcen. Die semantische Beschreibung ist dabei eine maschinenlesbare Beschreibung des menschlich lesbaren Inhalts. Zusätzlich erhält jede Ressource einen eindeutigen Identifikator, den so genannten Uniform Ressource Identifier (URI), um Verweise auf gleiche elektronische Ressourcen wie z.B. Publikationen eindeutig erkennen zu können. Als Vokabular zur semantischen Beschreibung der Inhalte kommen Ontologien zum Einsatz, welche die Konzepte einer Domäne sowie ihre Beziehungen untereinander definieren und somit eine eindeutige Beschreibung von Ressourcen ermöglichen. Als Format der Beschreibungen kommen standardisierte Beschreibungsformate wie die Extensible Markup Language (XML), das Ressource Description Framework (RDF) und die Web Ontology Language (OWL) in Frage.

Wie in Tim Berners-Lee's Vision beschrieben, zielen die standardisierten semantischen Beschreibungen auf die Entwicklung von intelligenten Agenten zur Abwicklung von benutzerdefinierten Aufgaben und vor allem zur intelligenten Informationsbereitstellung ab. Neben diesen intelligenten Agenten entwickelt sich im Semantic Web ein neues Paradigma zur Bereitstellung von Anwendungen. Service Oriented Computing

(SOC) wendet sich ab von den gängigen monolithischen Systemen hin zu kleinen, logisch kapselbaren Diensten, die von Service Providern angeboten werden. Dabei haben die Dienste klar definierte, wenn möglich semantisch beschriebene Aufgaben und Schnittstellen und können vom Service Client zur Ausführung von verschiedenen Aufgaben benutzt werden. Im Unterschied zu anderen verteilten Architekturen benötigt der Client im SOC-Paradigma kein Wissen über den Service Provider und die dahinter liegende Implementierung. Über die Kombination von vielen Services kann so flexibel eine Gesamtaufgabe gelöst und weiterführend die Lösung der Gesamtaufgaben wiederum als Service angeboten werden. Im Umfeld des WWW's realisiert sich das SOC-Paradigma in Form von Web-Services, die verschiedene Dienste wie z.B. Kreditkartetransaktionen bereitstellen. Amazon und Google sind hier wohl die bekanntesten Anbieter von Suchdiensten in Form von Web-Services.

Während semantische Beschreibungssprachen auf die Standardisierung und die damit verbundene Maschinenlesbarkeit von Information abzielen, kapseln Web-Services den Aspekt der verteilten und standardisierten Verarbeitung von Information. Wie der Schritt vom Index in Papierform zum mechanisierten Index und der damit verbundenen Verbesserung des Informationszugangs, sind diese semantischen Technologien der zentrale Baustein im Umgang mit der Informationsflut unserer Tage.

Auch im Unternehmenskontext halten semantische Technologien über deren Anwendung in Wissensmanagementsystemen oder Suchsystemen Einzug. Immer öfter erfolgt die Beschreibung unternehmensrelevanter Inhalte über standardisierte Beschreibungssprachen wie OWL und deren Integration in die tagtägliche Arbeit. In Untersuchungen rund um den Semantic Desktop [Sauermann 2005] erfolgt der Versuch der Integration von semantischen Beschreibungen am PC Desktop. Ontologien dienen dabei als Bindeglied zwischen den verschiedenen Desktopanwendungen wie E-Mail, Browser und Textverarbeitung Semantische Beschreibungen erlauben eine einheitliche Sichtweise auf Information. Das von der EU geförderte Integrated Project APOSDLE - Advanced Process-Oriented Self-Directed Learning Environment[5] bedient sich ebenfalls semantischer Technologien und erweitert dabei den Fokus des Semantic Desktop. Neben der Integration von für den Arbeitsprozess wichtiger Information sollen auch Lernressourcen für die Weiterbildung über semantische Beschreibungssprachen gefunden und Mitarbeitern während der Arbeit präsentiert werden. Durch die einheitliche Sichtweise auf Information ist somit die Integration von verschiedenen Informationswelten - wie im Beispiel APOSDLE's der Lern- und der Arbeitswelt - möglich. Auch die Bereitstellung von verschiedenen Desktop-Diensten im Vergleich zur monolithischen Desktop-Anwendung zeigt sich als Trend der letzten Jahre. Google bietet mit dem Google Desktop[6] eine individuell aus Einzeldiensten gestaltbare Desktopumgebung an, die auch auf unterschiedlichen Rechnersystemen gleich bleibt und somit nicht an den „Ort" der Anwendung gebunden ist. Tochtermann beschreibt in [Tochtermann 2006] ebenfalls die Zukunftsvision eines individuell, je nach Präferenz konfigurierbaren Service Based Knowledge Desktops, der den zukünftigen Service orientierten Arbeitsplatz für den Wissensarbeiter darstellen wird.

Wenngleich semantische Technologien zukunftsweisend sind, bleibt jedoch das eingangs erwähnte Problem, dass der Großteil der zu Verfügung stehenden Information nicht in der vom Semantic Web benötigten Informationsqualität vorliegt. Nur ein Bruchteil der Information ist mit semantischen Beschreibungen

[5] http://www.aposdle.tugraz.at/

[6] http://desktop.google.com/

annotiert. Zusätzlich bauen sich gängige Wissensmanagement- und Retrievalsysteme meist als monolithische Architekturen auf, in denen die Integration neuer Techniken und Datenstrukturen wie z.B. RDF nur schwer möglich ist; im Gegensatz zu flexiblen service-orientierten Systemen. Die vorliegende Arbeit adressiert diese Probleme.

1.1 Zielsetzung der vorliegenden Arbeit

Schwerpunkt ist die Bereitstellung von Diensten zum einfachen Umgang mit großen, unstrukturierten Informationsmengen im Inter- bzw. Intranet und zur Erhöhung der Informationsqualität unstrukturierter Daten, ohne dabei auf die Voraussetzungen semantisch annotierter Daten, wie es das Semantic Web verlangt, zurück greifen zu müssen. Die Arbeit beschränkt sich dabei nicht auf Anwendungsszenarien aus dem Bereich des (Semantic) Web Umfelds, sondern berücksichtigt auch Informationssysteme im Unternehmens-Intranet, mit Fokus auf Informationssysteme im Kontext des Wissensmanagements. Dem Benutzer soll dabei – durch die Integration der bereitgestellten Dienste in verschiedene Anwendungen – der Zugang zu großen Informations- bzw. Dokumentbeständen erleichtert werden. Dabei liegt der Fokus hauptsächlich auf der Betrachtung von Dokumenten und Dokumentbeständen als Informationsträger. In Anbetracht der zu bearbeitenden Informationsvolumina zielt die Arbeit auf die Umsetzung von Diensten zur automatischen Analyse unstrukturierter Informationen ab. Manuelle Ansätze wie z.B. das Annotieren semantischer Beschreibungen für Bilder werden vernachlässigt.

Die vorliegende Arbeit bewegt sich in diesem Spannungsfeld und zielt auf

„die Konzeption und Entwicklung eines integrativen und flexiblen Software Frameworks zur Umsetzung von Knowledge Discovery und Information Retrieval Diensten und die Möglichkeit der Einsetzbarkeit dieses Frameworks in unterschiedlichen Anwendungsgebieten."

ab. Dazu positioniert sich die Arbeit im Schnittpunkt des Knowledge Management, Knowledge Discovery, Information Retrieval und Semantic Web, wobei der Fokus auf der Integration von KD- und IR-Techniken und im Kontext von Web- und Wissensmanagementanwendungen liegt.

Das zu entwickelnde Framework soll dabei die Integration von Open Source Technologien berücksichtigen, um die Entwicklungszeit von Algorithmen zu verkürzen und die Stabilität der Dienste zu erhöhen. Heutzutage existiert im Open Source - Bereich ein breites Angebot an frei verfügbaren Textanalyse- und Textmining-Werkzeugen, wobei sich der Fokus dieser Anwendungen meist auf einen engen Spezialbereich richtet. Nichtsdestotrotz bieten diese Werkzeuge stabile und erprobte Verfahren an, und sollten daher im zu entwickelnden Framework berücksichtigt werden. Die Entwicklung neuer Methoden oder Algorithmen in diesen Bereichen ist *kein* Ziel dieser Arbeit. Außen vor gelassen werden auch Anwendungsszenarien im Bereich der statistischen Analyse von numerischen Kennzahlen wie z.B. Umsatzzahlen, da dies über den Zweck der vorliegenden Arbeit hinausgehen würde.

Vor dem Hintergrund der eingangs geführten Diskussion können folgende Kriterien für die Konzeption des zu entwickelnden Frameworks formuliert werden. Diese stellen zugleich Verfeinerungen des Gesamtziels dar:

1. *Unabhängigkeit von den zu verarbeitenden Datenquellen und der zu verarbeitenden Informationsart*
2. *Parallelisierbarkeit von Methoden und Algorithmen zur Unterstützung von Multi-User-Szenarien*
3. *Wiederverwendbarkeit bereits entwickelter Techniken in unterschiedlichen Anwendungsszenarien und auf unterschiedlichen Informationsmengen*

4. *Integration von Open Source - Bibliotheken*
5. *Einfache Erweiterbarkeit*

Unter Berücksichtigung dieser Kriterien werden im Verlauf der Arbeit die Anforderungen – anhand der Analyse von Methoden und Algorithmen aus dem Bereich des Knowledge Discovery sowie der Analyse bereits bestehender Systeme –an das umzusetzende Framework in den folgenden Kapiteln entwickelt.

Der Rest dieses Abschnittes stellt noch die der Arbeit zugrunde liegende Methodik dar und fasst kurz die Ergebnisse zusammen. Den Abschluss bildet ein detaillierter Ausblick auf die weiterführenden Abschnitte dieser Arbeit.

1.2 Methodik der vorliegenden Arbeit

Da für die Erreichung der präsentierten Ziele die softwaretechnische Umsetzung im Zentrum steht, wurde als Methodik für die vorliegende Arbeit das Spiralmodell [Boehm 1986] aus der Softwareentwicklung adaptiert.

Wie in Abbildung 1-2 dargestellt, erfolgt die Entwicklung in mehreren Iterationszyklen, die verschiedene Segmente bzw. Phasen durchlaufen. Beginnend mit der Festlegung der Ziele und Aufgaben, erfolgt die Bewertung von möglichen Entwicklungspfaden und der Entwicklung der Software auf Basis der ausgewählten Entwicklungspfade. Die entwickelte Software dient als Grundlage für die Durchführung von Testläufen und für die Evaluierung anderer Kriterien, auf deren Basis in der letzten Phase ein Soll-Ist-Vergleich und die Festlegung der nächsten Entwicklungsschritte erfolgt. An diesem Modell orientiert sich auch die Vorgehensweise der vorliegenden Arbeit. Ausgangspunkt für die Entwicklung des umzusetzenden Frameworks stellten drei unter Mitwirkung des Autors getrennt entwickelte Forschungsprototypen mit unterschiedlichen Anwendungsausrichtungen, aber überlappenden Algorithmen und Technologien dar.

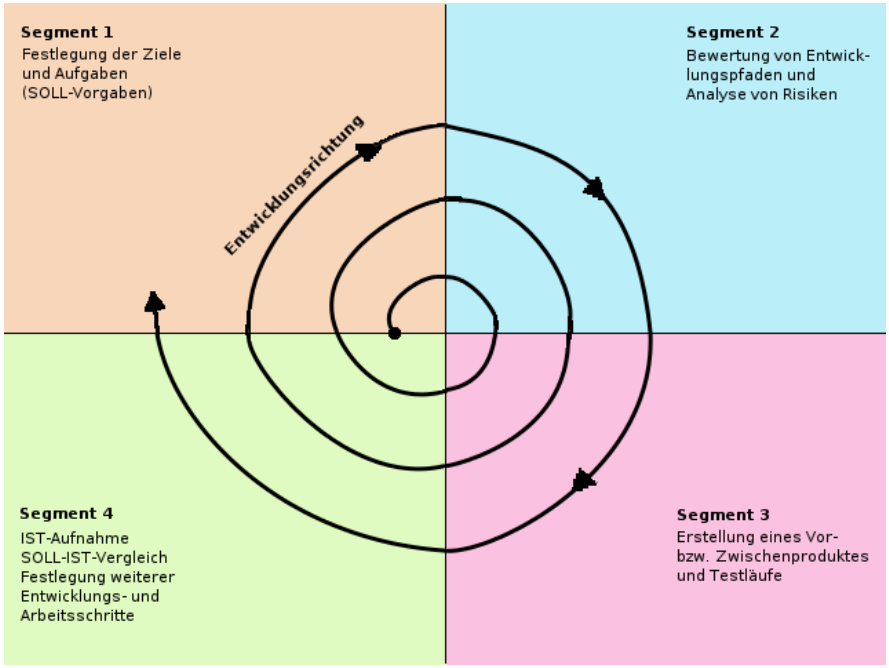


Abbildung 1-2: Das Spiralmodell nach Boehm (Bildquelle: IT-Handbuch ISBN 3-14-225042-5)

Auf Basis der Erfahrungen aus der Entwicklung der drei Projekte erfolgte die Festlegung der Anforderungen an die erste Generation des umzusetzenden Frameworks (Segment 1, Iteration 1) und die Festlegung der Basistechnologien, sowie eine erste Konzeption zur Umsetzung (Segment 2, Iteration 1). Das auf Basis dieser Anforderungen umgesetzte Framework diente als Grundlage für die Umsetzung von drei Projekten, welche auch als Evaluierungsgrundlage herangezogen wurden (Segment 3, Iteration 1). Aus den Lessons-Learned der Projekte definierten sich die Anforderungen bzw. Erweiterungen der nächsten Iteration (Segment 4, Iteration 1).

Es erfolgte, neben der Einarbeitung der Anforderungen (Segment 1, Iteration 2), auch die Berücksichtigung neuer Technologien (Segment 2, Iteration 2) und deren Umsetzung bzw. Integration. Auf Basis dieser zweiten Version konnten wiederum Evaluierungen auf Basis von drei mit dem entwickelten Framework umgesetzten Projekten durchgeführt werden (Segment 3, Iteration 2), um anschließend die aktuell noch offenen weiteren Entwicklungsschritte zu identifizieren.

Auf Basis der Evaluierung erfolgt auch die Beantwortung der Fragestellungen dieser Arbeit. Innerhalb der zwei Iterationszyklen erfolgten mitunter auch kleinere, leicht integrierbare Adaptionen und Anpassungen, was nicht ganz dem Spiralmodell entspricht, sich bei der Entwicklung des Frameworks jedoch als praktikabel erwies.

Während die Entwicklung der drei Ausgangsprojekte für as umzusetzende Framework nicht direkt als voller Zyklus im Spiralmodell angesehen werden kann, so decken sie doch Segment 3 und Segment 4 ab, wodurch von 2,5 Iterationszyklen gesprochen werden kann. Die detaillierte Darstellung der Iterationszyklen und der daraus abgeleiteten Erkenntnisse erfolgt in Abschnitt 6.

1.3 Zusammenfassung der Ergebnisse

Ergebnis dieser Arbeit ist das „KnowMiner-Framework“, welches am Know-Center im Bereich Wissenserschließung unter Leitung des Autors konzipiert und entwickelt wurde. Die Entwicklung des Frameworks umfasst dabei in etwa 10 Personenjahre an Entwicklungszeit, inklusive der auf Basis des Frameworks umgesetzten Projekte und Lösungen. Mit dem KnowMiner-Framework wurden 8 Projekte und werden aktuell 3 Projekte im Bereich anwendungsorientierter Forschung umgesetzt, was einen wichtigen Indikator für die Anwendbarkeit und Nutzbarkeit des entwickelten Frameworks darstellt. In Summe waren 11 Personen an der Entwicklung beteiligt und es wurden 1000 Java Klassen mit insgesamt 183.293 Zeilen Source-Code (exklusive automatisch erzeugter Klassen) geschrieben. Weite Teile des Source Codes, sowie der Konzeption des Frameworks entstammen dabei aus der Feder des Autors. Des Weiteren war der Autor dieser Arbeit an den 8 Projekten teilweise leitend und teilweise auch in der Umsetzung direkt beteiligt. Nichtsdestotrotz wäre eine so umfassende Entwicklung ohne die Unterstützung qualifizierter Dritter unmöglich, wozu der Autor an dieser Stelle nochmals seinen Dank an alle Beteiligten aussprechen möchte.

Mit der Umsetzung des Frameworks konnten folgende Ergebnisse realisiert werden:

1. Ein Framework zur Bereitstellung von Diensten im Knowledge Discovery mit Fokus auf
    a. Clustering
    b. Klassifikation
    c. Multi Dimensionale Skalierung
    d. Information Retrieval

2. Die Entwicklung einer Anwendungsschnittstelle in Form einer Service-orientierten Architektur, welche die verschiedenen KD-Methoden als Dienste anbietet und damit eine einfache Anwendbarkeit der Techniken ermöglicht
3. Die Entwicklung einer Umgebung zur Einbettung konkurrenter, parallel abarbeitbarer Algorithmen
4. Die Entwicklung eines Frameworks zur flexiblen Einbindung unterschiedlicher Datenquellen.
5. Die erfolgreiche Trennung von Datenrepräsentation und Algorithmen und die damit verbundene Wiederverwendbarkeit von Algorithmen auf unterschiedlichen Informations-/Dokumentbeständen
6. Integration semantischer Beschreibungssprachen zur Strukturierung und Verwaltung von Informationsräumen

Insgesamt konnte das Hauptziel der Arbeit erreicht werden, wenngleich noch Punkte hinsichtlich Komposition und Verwaltung der einzelnen Services sowie der semantisch vollständigen Beschreibungen der unterschiedlichen Datentypen und Repräsentationsformen offen sind. In dieser Hinsicht hat die vorliegende Arbeit auch gezeigt, dass die Erstellung von semantischen Beschreibungen ein komplexes, leicht unterschätzbares Gebiet darstellt und mit Sicherheit noch weitere Arbeiten dieser Größenordnung rechtfertigt.

1.4 Aufbau der Arbeit

Die nachstehende Abbildung 1-3 gibt einen Überblick über den Aufbau der vorliegenden Arbeit. Die Arbeit beginnt mit der oben beschriebenen Zielsetzung, womit der Rahmen der Arbeit definiert ist. Entlang der definierten Ziele und der zu berücksichtigenden Einschränkungen erheben die Kapiteln 2 bis 5 die Anforderungen an das zu entwickelnde Framework. Kapitel 6 verfeinert die Anforderungen auf Basis der in den Iterationszyklen gewonnenen Erkenntnisse. Auf Basis der verfeinerten Anforderungen leitet Kapitel 7 die konzeptionellen Grundlagen für das KnowMiner-Framework ab, welches die Voraussetzung für die Implementierung des Frameworks, beschrieben in Kapitel 8, liefert. Kapitel 8 endet mit dem Vergleich des implementierten Frameworks in Bezug auf die zuvor erhobenen Anforderungen. Kapitel 9 gibt Aufschluss über die mit dem Framework umgesetzten Anwendungen und leitet die daraus gewonnenen Erkenntnisse und Ergebnisse ab. Kapitel 10 setzt die Ergebnisse zur getroffenen Zielsetzung in Beziehung, erörtert die noch offenen Punkte und weiterführende Forschungsfragen.

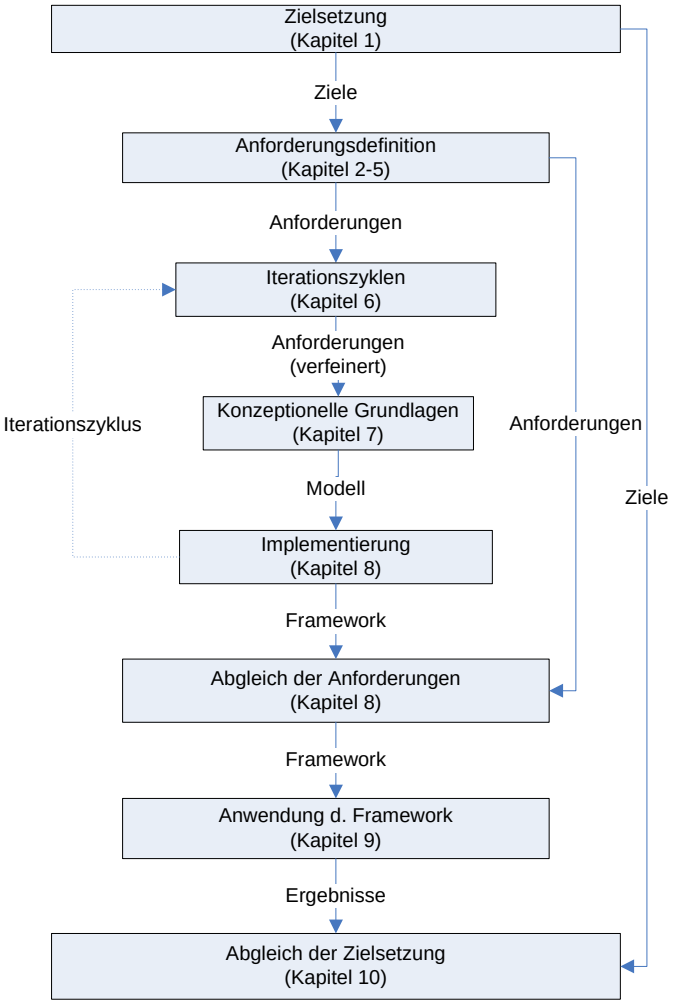


Abbildung 1-3: Aufbau der vorliegenden Arbeit

Im Detail behandeln die Abschnitte der Arbeit folgende Themen:

Kapitel 2 setzt an der Definition von Knowledge Discovery und des Knowledge Discovery - Prozesses an und behandelt diesen im Kontext von Informations- und Wissensmanagementsystemen. Dadurch lassen sich Anforderungen an das zu entwickelnde Framework in Bezug auf verschiedene Anwendungsszenarien und, damit verbunden, an die Erwartungen verschiedener Benutzergruppen ableiten.

In Kapitel 3 erfolgt die Zusammenfassung von, dem aktuellen Stand der Wissenschaft entsprechenden, Techniken zur Vorverarbeitung von unstrukturierten Daten und die Überführung dieser in für Algorithmen anwendbare Formen. Hauptaugenmerk liegt dabei auf der Verarbeitung von unstrukturierten Texten (z.B. Shallow Text Parsing und Informationsextraktion), da diese den Großteil der heutzutage verfügbaren und verwertbaren Information darstellen. Ein Umstand, der auch durch Umfragen auf KDNuggets[7] belegt wird. Umfragen aus den Jahren 2005[8], 2004[9] und 2002[10] zeigen deutlich, dass die Anwendung von Mining-

[7] http://www.kdnuggets.com/

[8] http://www.kdnuggets.com/polls/2005/data_types.htm

[9] http://www.kdnuggets.com/polls/2004/types_data_analyzed.htm

Techniken auf textuellen Daten vor anderen Modalitäten wie Bild, Video und Audio liegt. Ein Umstand, der sich mit der leichteren Verwertbarkeit von Text begründen lässt. Vorverarbeitungsmethoden für diese Modalitäten werden nur in aller Kürze und ohne Anspruch auf Vollständigkeit umrissen.

Nach Betrachtung der Vorverarbeitung unstrukturierter Daten, zielt Kapitel 4 auf die Darstellung von relevanten KD-Algorithmen und -Techniken, wiederum mit Fokus auf das Medium Text, ab. Betrachtet werden Techniken zur Transformation von aus der Vorverarbeitung extrahierten Merkmalen und die Anwendung von überwachten und unüberwachten Lernverfahren zur Identifikation von Zusammenhängen. Zusätzlich gibt dieser Abschnitt einen Überblick über Information Retrieval Techniken und illustriert die Anwendung von Informationsvisualisierung zum Suchen, Finden und Darstellen von Information. Der Abschnitt schließt mit einem Überblick über Techniken aus dem Bereich Ontology Learning, in denen die vorher illustrierten Methoden kombiniert zum Einsatz kommen.

Ziel von Kapitel 3 und 4 ist es, dem Leser einen Überblick über den aktuellen Stand der Algorithmen und Methoden im Bereich KD zu geben und damit die Anforderungen für das umzusetzende Framework aus diesen beiden Bereichen abzuleiten. Die angeführten Techniken entsprechen auch denen, die laut einer Umfrage am häufigsten angewandt werden[11].

Kapitel 5 gibt einen Überblick über andere dem KnowMiner-Framework ähnliche Frameworks und zeigt den Stand der Standardisierung von Algorithmen und Methoden im Bereich Data Mining, Knowledge Discovery und im Umfeld der Wissensrepräsentation auf. Hintergrund ist die Positionierung und Abgrenzung des umzusetzenden Frameworks im Vergleich zu bereits bestehenden Frameworks und Technologien sowie wiederum die Ableitung von Anforderungen an das umzusetzende Framework

Wie bereits zuvor erwähnt, beschreibt Kapitel 6 den ersten Iterationszyklus in der Entwicklung des KnowMiner-Frameworks. Kapitel 7 leitet aus den Erkenntnissen und aus der im Detail dargestellten Analyse des Knowledge Discovery - Prozesses die konzeptionellen Grundlagen für die Umsetzung des Frameworks ab. Die Details zur Umsetzung des Modells stellt Kapitel 8 dar.

Die Darstellung der im Iterationszyklus 2 umgesetzten Anwendungen und der final abgeleiteten Erkenntnisse findet sich in Kapitel 9. Kapitel 10 schließt mit der Zusammenfassung der Gesamtergebnisse und einem Ausblick auf noch offene Fragestellungen und Erweiterungen.

---

[10] http://www.kdnuggets.com/polls/2002/data_types_analyzed_in_2001.htm

[11] Siehe http://www.kdnuggets.com/polls/2006/text_mining_tasks.htm

2 Einführung und Anwendung von Knowledge Discovery

Während die Zielsetzung und Positionierung der Arbeit ein breites Gebiet abdeckt, soll dieser Abschnitt die Anwendungsmöglichkeiten von Knowledge Discovery und Information Retrieval in Bezug auf Informations- und Wissensmanagementsysteme untersuchen. Aus diesen Betrachtungen leiten sich allgemeine Anforderungen für das umzusetzende Framework ab, welche den Rahmen für das weitere vorgehen definieren. So ist zu klären, welche Interaktionen zwischen Benutzer und Informationssystem stattfinden und welches Unterstützungspotential seitens Knowledge Discovery und Information Retrieval für diese Interaktionen verfügbar sind. Die gefundenen Ansatzpunkte ermöglichen in weiterer Folge die detaillierte Analyse der notwendigen Methoden und Techniken zur Unterstützung der identifizierten Interaktionen.

Abschnitt 2.1 beschäftigt sich dabei mit der Definition von Knowledge Discovery und dem dahinter liegenden Prozess. Im Anschluss betrachtet Abschnitt 2.2 mögliche Anwendungsszenarien im Umfeld von Informations- und Wissensmanagementsystemen. Aus beiden Abschnitten erfolgt die Ableitung von Anforderungen an das umzusetzende Framework, mit welchen dieses Kapitel schließt.

2.1 Knowledge Discovery und der Knowledge Discovery Process (KDP)

Der Wurzeln von "Knowledge Discovery" oder zu Deutsch Wissenserschließung lassen sich auf den KDD (Knowledge Discovery in Databases) Workshop aus dem Jahre 1989 zurück führen, in dem der Begriff KDD geprägt wurde (siehe auch den Workshop-Report aus dem Jahre 1991 [Piatetsky-Shapiro 1991]) und als Ausgangsbasis für die rasche Entwicklung sowohl von Data Mining, als auch Knowledge Discovery als eigenständiges Wissenschaftsfeld anzusehen ist.

Die erste Definition von Knowledge Discovery in Databases als

> *"The extraction of implicit, previously unknown, and potentially useful information from data"*

wurde das erste Mal von Frawley, Piatetsky-Shapiro & Matheus in [Frawley et. al. 1991] gegeben und später von Fayyad, Piatetsky-Shapiro, & Smyth, in [Fayyad et. al. 1996] als

> *"the nontrivial process of identifying valid, novel, potentially useful, and ultimately understandable patterns in data"*

verfeinert.

In beiden Definitionen steht die Transformation von Daten als Mengen von Fakten, welche zumeist in Datenbanken vorliegen, in eine höherwertige, nützliche Form im Vordergrund. Fayyad bezeichnet diese als „patterns" und beschreibt sie als in einer beliebigen Sprache formulierte Regeln welche die darunter liegenden Daten beschreiben.

Während die Extraktion dieser Mustern bzw. Regeln aus einer gegebenen Datenmenge schon Ziel in anderen Feldern wie z.B. der Datenanalyse oder in der Statistik ist, zeichnet sich KDD dadurch aus, dass das Endprodukt aus diesem Prozess Wissen ist bzw. dass neue, vorher nicht bekannte aber im aktuellen Kontext nützliche Zusammenhänge entdeckt werden. Auch steht der Prozess der Transformation im Mittelpunkt, wodurch sich KDD von Data Mining, welches sich lediglich mit der Identifikation der Muster beschäftigt,

abhebt. Im Speziellen stellt Data Mining nur einen Teilschritt von KDD dar, kann also als Submenge von KDD betrachtet werden, obwohl in der Literatur die beiden Begriffe oft synonym verwendet werden.

Während die Definition von Frawley relativ allgemein gehalten ist, fokussierten Fayyad, Piatetsky-Shapiro und Smyth mit ihrer Definition von Daten als Fakten und Mustern als Regeln auf die strukturierte Datenbankwelt und weniger auf unstrukturierte Texte wie sie heute im WWW vorliegen. KDD fokussiert dabei nicht nur auf Algorithmen zur Extraktion von Mustern und Regeln, sondern definiert auch den dazugehörigen Prozess ([Fayyad et. al. 1996]), der von der Aufbereitung der Daten bis hin zur Analyse durch den Benutzer die einzelnen KDD Schritte definiert. Data Mining definiert dabei nur einen, wenn auch wichtigen Teilschritt in diesem Prozess.

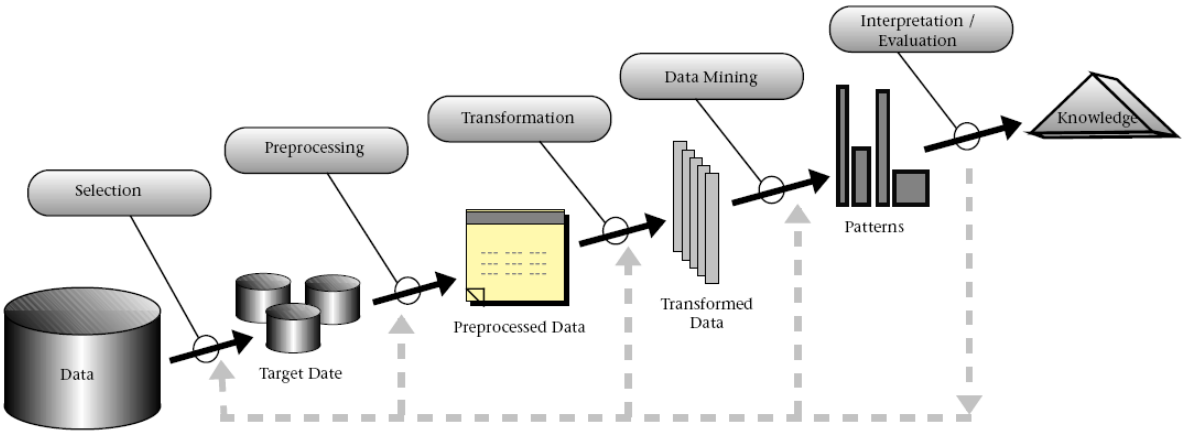


Abbildung 2-1: Der KDD Prozess nach [Fayyad et. al. 1996]

Abbildung 2-1 illustriert den KDD Prozess (KDP) nach Fayyad, wobei der Prozess im Allgemeinen des Öfteren durchlaufen wird und Interaktionen mit dem Benutzer beinhaltet. Im originalen Prozessmodell Fayyad's sind neben technischen Prozessschritten ebenfalls projektbezogene Schritte, wie das Verstehen der Anwendungsdomäne und das Überprüfen der Zielhypothese inkludiert, welche jedoch in dieser Betrachtung aus Gründen der Übersichtlichkeit weg gelassen werden

Die ersten technischen Schritte bestehen in der Selektion, Aufbereitung und Transformation der Daten. *Selection*, *Preprocessing* und *Transformation* beinhaltet meist das Anbinden unterschiedlicher Datenquellen, das Abrufen dieser und die Aufbereitung dieser für den nachfolgenden Data Mining Schritt. Im Data Mining erfolgt die eigentliche Muster- und Regel-Extraktion auf Basis der vorverarbeiteten und transformierten Daten. Anschließend werden die ermittelten Muster für den Benutzer aufbereitet und evaluiert. Je nach Ergebnis kann der Prozess in der nächsten Iteration verfeinert und angepasst werden.

Durch die enge Anlehnung an Data Mining, welches vorwiegend auf numerischen bzw. symbolischen Daten als Grundlage für den Analyseprozess fokussiert, zielen die meisten KDD Anwendung auf die Analyse von in relationalen Datenbanken enthaltener Information ab. Die Hauptanwendungsgebiete von KDD sind unter anderem die Analyse von Kundenverhalten, Betrugserkennung im Finanzsektor und die Analyse von Fehlerquellen in den Produktionsbetrieben. Ein weiteres Merkmal von KDD ist dessen Interdisziplinarität. Zur erfolgreichen Extraktion von Regeln aus Daten werden Techniken aus unterschiedlichsten Wissenschaftsbereichen wie z.B. dem Maschinellen Lernen, der Datenbanken, dem High-Performance

Computing, der Wissensrepräsentation, der Künstliche Intelligenz und der Datenvisualisierung miteinander kombiniert mit dem vereinheitlichtem Ziel der Extraktion von Wissen aus Datenbanken.

Der Analyse von unstrukturierten Texten wird dabei wenig Stellenwert zugewiesen, was sich auch an der Einführung von „Knowledge Discovery in Text" (KDT) durch Feldman und Dagan im Jahre 1995 [Feldman et. al. 1995] zeigt, die den Fokus stärker in Richtung unstrukturierte Textbestände lenkt. Analog der Anlehnung von KDD an Data Mining, erfolgt eine Anlehnung von KDT an Text Mining, definiert durch Hearst [Hearst 1999] als

„ the discovery by computer of new, previously unknown information, by automatically extracting information from different written resources"

KDT stellt wiederum den Discovery Prozess in den Mittelpunkt, welcher sowohl die Datenaufbereitung als auch die Interpretation und Evaluierung der entdeckten Information aus unstrukturierten Textbeständen beinhaltet und sich stark and den KDD Prozess anlehnt.

Während die Unterschiede in der Definition der beiden Gebiete eher als gering zu betrachten sind, unterscheiden sich die angewendeten Algorithmen und auch die Anwendungsgebiete stark voneinander. Zur Analyse von natürlichsprachlichen Texten ist mitunter eine detaillierte Analyse der menschlichen Sprache notwendig, um korrekte und aussagekräftige Regeln ableiten zu können.

Wie auch KDD, ist KDT ein interdisziplinäres Feld mit ähnlichen Verbindungen in die oben genannten Wissenschaftsdisziplinen. Durch die Fokussierung auf natürliche Sprache und das geschriebene Wort ist zusätzlich eine starke Anknüpfung an Information Retrieval, Natural Language Processing und sogar die Computerlinguistik vorhanden. Als Abgrenzung zu diesen Feldern kann wiederum die Extraktion von nützlicher, vorher unbekannter Information herangezogen werden, wobei meist die praktische Anwendung im Vordergrund steht. So haben sich z.B. aus dem Bereich des Natural Language Processing die so genannten Shallow Text Parsing Techniken (c.f. [Crysmann 2002]) durchgesetzt, welche im Gegensatz zum Deep Language Processing nicht auf das Verständnis von Sprache, sondern auf die Anwendung von Heuristiken zur Extraktion relevanter Teile des Textes setzen.

	Non-novel investigation	**Semi-Novel Investigation-Knowledge Discovery**	**Novel investigation-Knowledge Creation**
Text Metadata	Metadata based Information Retrieval (e.g. digital libraries)	KDD/KDT/ Metadata Mining (e.g. correlation between classification categories)	Intelligent Metadata Mining (e.g. interpret the correlation between classification categories)
Textual data	Full text Information Retrieval	KDT (e.g. group text document based on content)	Intelligent Text Mining (e.g. textual summarization of a query)
Numerical data	Database Queries (e.g. SQL)	KDD/Data Mining (e.g. OLAP)	Intelligent Data Mining (e.g. derive business decision)

Tabelle 2-1: Abgrenzung KDD/KDT auf Basis der „Neuheit" der Entdeckung und den unterschiedlichen Eingangsdaten

Zur Unterscheidung von KDD und KDT resp. Data Mining und Text Mining sowie zur Abgrenzung zu anderen Disziplinen traf Hearst [Hearst 1999] eine Unterscheidung der Techniken auf Basis der Neuheit der entdeckten Information. Ein Modell, welches durch Kroeze, Matthee und Bothma [Kroeze et. al. 2003] noch verfeinert wurde. Tabelle 2-1 zeigt einen Überblick über die Kategorisierung, welche einerseits die Art der zu verarbeitenden Daten und die Neuheit der Entdeckung definiert. Kroeze resp. Hearst unterscheiden dabei zwischen numerischen Daten, textuellen Daten und textbasierten Metadaten. Die Unterscheidung zwischen

den einzelnen Datenarten richtet sich grundlegend nach Strukturierungsgrad. Textuelle Daten sind i.A. unstrukturiert, während numerische Daten stark strukturiert sind. Die Unterscheidung zwischen Text und Text-Metadaten wird ebenfalls auf Basis des Strukturierungsgrades getroffen. Beispiele für Text Metadaten sind unter anderem bibliographische Einträge wie Autor, Schlüsselwörter als auch das Erstellungsdatum.

Entsprechend zu [Hearst 1999] definieren sich *non-novel investigations* auf das Finden relevanter Information aus wertvoller, aber im aktuellen Kontext irrelevanter Information. In Abhängigkeit von der Datenbasis sind Methoden dafür z.B. Datenbankabfragen oder Information Retrieval.

Semi novel investigations definieren sich als das Finden von Mustern/Trends, welche innerhalb der Daten bereits existieren, bis jetzt jedoch nicht explizit sind und deren Entdeckung daher neu ist. Offensichtlich entspricht diese Definition der Definition von KDD resp. KDT und beispielhaft wäre das Clustering von Textdokumenten zu nennen, welches ähnliche Dokumente gruppiert und somit zuvor noch nicht bekannte Beziehungen zwischen Dokumenten herstellt.

Die letzte Kategorie entspricht den *novel investigation*, welche neues Wissen aus den vorliegenden Daten extrahieren, Wissen, welches zuvor nicht existierte bzw. nicht mittels den beiden anderen Techniken gefunden werden konnte. Offensichtlich sind die Ziele mit dieser letzten Kategorie sehr hoch gesteckt, da sie in etwa den Prozess der menschlichen Wissensfindung kapseln und somit auch Techniken aus dem Bereich der künstlichen Intelligenz und des Sprachverstehens implizieren. Eine beispielhafte Anwendung wäre das sprachliche Zusammenfassen der Einzelergebnisse einer Suchanfrage, welche die Inhalte der Ergebnismenge natürlichsprachlich beschreibt.

	Non-novel investigation	**Semi-Novel Investigation- Knowledge Discovery**	**Novel investigation- Knowledge Creation**
Text Metadata	Metadata based Information Retrieval (e.g. digital libraries)	KDD/KDT/ Metadata Mining (e.g. correlation between classification categories)	Intelligent Metadata Mining (e.g. interpret the correlation between classification categories)
Textual data	Full text Information Retrieval	KDT (e.g. group text document based on content)	Intelligent Text Mining (e.g. textual summarization of a query)
Graph Data (e.g. Concept Graphs, Folksonomies)	Structural Retrieval (e.g. RDQL[12], SPARQL[13])	Link/Graph Mining (e.g. discovering of common descriptions)	Intelligent Graph Mining (e.g. Automatic Ontology Construction)
Multimedia data	Multimedia Retrieval	Multimedia Analysis (e.g. Detection of Persons in Videos)	Intelligent Multimedia Analysis (e.g. Automatic Video Summarization)
Numerical data	Database Queries (e.g. SQL[14])	KDD/Data Mining (e.g. OLAP[15])	Intelligent Data Mining (e.g. derive business decision)

Tabelle 2-2: Um strukturelle Daten und multimediale Daten erweitertes Modell.

[12] RDF Description Query Langage

[13] SPARQL

[14] Standard Query Language

[15] Online Analytical Processing

Während die Aufteilung nach Neuigkeit der Entdeckung im Modell von Kroeze für die Ableitung der Anforderung an das zu entwickelnde Framework vollständig ist, fehlt bei der Unterscheidung der Datenbasis noch Granularität. Speziell bei Dokumenten als Informationsträger fallen meist auch multimediale Daten an, welche im obigen Modell nicht berücksichtigt wurden. Neben multimedialen Daten sind auch strukturell höherwertige Daten wie z.B. Graphen für den Discovery Process von Interesse, welche nicht notwendigerweise unter den numerischen Daten subsumiert werden können. Graphen basierte Repräsentationen spielen vor allem im Semantic Web zunehmend eine entscheidende Rolle (c.f. [Lux et. al. 2006]), in dem strukturierte Beschreibungen wie semantische Metadaten zur näheren inhaltlichen Beschreibung von Information, meist in Form von Graphen heran gezogen werden. Ebenso zentral stellen sich Folksonomies (c.f. [Wal 2005]) im Bereich Web 2.0 dar, um Benutzer mit Ressourcen und deren Sichtweise auf diese in Beziehung zu setzen. Folksonomies Formen wiederum einen Graph, der aus Trippeln der Form Benutzer ⇔ Ressource ⇔ Beschreibung besteht.

Zur Berücksichtigung dieser Anforderungen wird das Modell von Kroeze um multimediale und strukturelle Daten erweitert. Für nicht neue Untersuchungen, also dem Finden von relevanter Information in irrelevanter Information, gibt es im Bereich Multimedia das bereits etablierte Feld des Multimedia-Retrievals als Teilgebiet des Information Retrieval. Retrieval auf strukturierten Daten erfolgt z.B. über die RDF Query Language, welche in Kroezes Aufzählung überhaupt nicht berücksichtigt wurde. Speziell im Bereich des Semantic Web ist die strukturierte Abfrage von Information essentiell. Aktuell gibt es keine Definition zu Knowledge Discovery im Bereich strukturierten bzw. multimedialen Daten, außer man subsumiert diese im Bereich KDD. Da es außerhalb des Fokus dieser Arbeit liegt, eine Definition für KD in diesen Bereichen zu geben, aber zur Ableitung der Anforderungen an das umzusetzende Framework die Unterscheidung zwischen KDD und multimedialen Daten bzw. strukturellen Daten notwendig ist, werden hier Beispieltechniken zu Umsetzung von Wissenserschließung auf diesen beiden Datenarten angegeben.

Für graphbasierte Daten wäre dies Graph bzw. Link Mining, ein stark wachsendes Feld welches sich mit der Analyse von graphbasierten Daten beschäftigt (c.f. [Bunke 2000]). Die Analyse multimedialer Daten ist schon seit langem ein gut etabliertes Feld, welches sich mit dem Identifizieren von Objekten in z.B. Bildern beschäftigt (siehe [Schmitt 2002) als Übersicht). Als Wissenserzeugung für strukturelle Daten sei hier als Beispiel das automatische Erstellen von Ontologien aus strukturellen Daten bzw. schon vorhandenen Ontologien angeführt und für multimediale Daten wäre dies beispielhaft das automatische Zusammenfassen und Beschreiben von Videos.

2.2 Anwendungen des Knowledge Discovery Process

Der vorliegende Abschnitt erarbeitet Anwendungen für die Einbettung des KDP im Kontext von Informations- und Wissensmanagementsystemen, um einen Überblick über die notwendigen und möglichen Funktionalitäten und Dienste zu geben. Die Erarbeitung der Anwendungen erfolgt dabei auf abstrakter Ebene, da eine Darstellung mittels Anwendungsszenarien zu umfangreich und unvollständig ist.

Zur Analyse von Interaktionen zwischen Benutzer und Informations-/Wissensmanagementsystemen bietet das Maurer Tochtermann Modells (MTM) [Maurer et. al. 2002] einen guten Ansatzpunkt. Das MTM betrachtet dabei Interaktionen zwischen Benutzern und Informationssystemen vor dem Hintergrund des Transfers von Information und hebt dabei speziell Funktionen von Wissensmanagementsystemen (WMS) als Erweiterung von Informationssystemen hervor. Ergänzend zum MTM erfolgt die Betrachtung der einzelnen Schritte einer Interaktion im Kontext eines aus [Staab et. al. 2001] adaptierten Wissensprozessmodell, wobei

mögliche Anwendung von KD-Methoden innerhalb des Prozesses beleuchtet werden. Daher kann eher von einem Informationsprozess als von einem Wissensprozess gesprochen werden. Nichtsdestotrotz demonstriert der adaptierte Wissens- bzw. Informationsprozess in Kombination mit dem Maurer Tochtermann Modell notwendige und mögliche Funktionen und Dienste für das umzusetzende Framework.

Das Maurer Tochtermann Modell definiert die verschiedenen Interaktionen zum Wissenstransfer zwischen Benutzer und Wissensmanagementsystem (siehe Abbildung 2-2), wovon 6 Interaktionen den Transfer zum oder vom Wissensmanagementsystem beschreiben und eine Interaktion direkt zwischen Benutzern definiert.

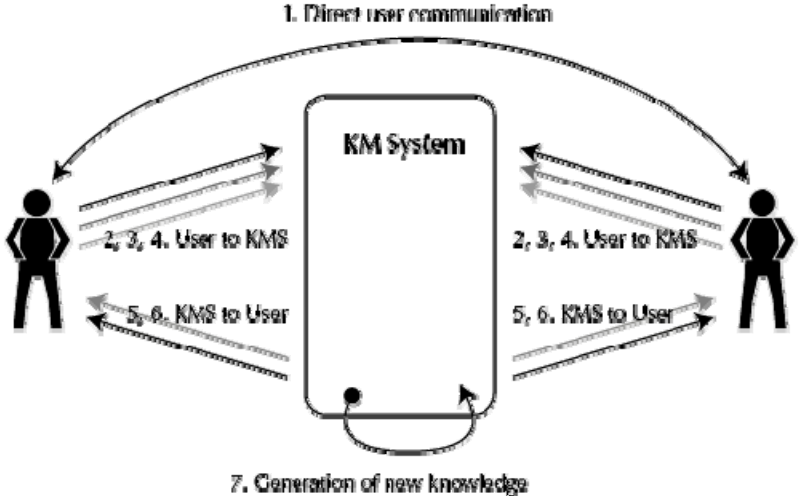


Abbildung 2-2: Das Maurer Tochtermann Modell (MTM) nach [Maurer et. al. 2002] definiert 7 verschiedene Interaktionen von Benutzern im Kontext eines Wissensmanagementsystems.

Folgende Interaktionen sind definiert:

1. Direkte Kommunikation zwischen Benutzern
   Information bzw. Wissen, welches auf diese Art transferiert wird, ist grundlegend vom WM System nicht erfassbar bzw. verwertbar. In speziellen Szenarien ist jedoch eine, momentan noch sehr rudimentäre Erfassung möglich. Im Spezialfall von Sitzungen zum Beispiel erfolgt im Forschungsprojekt MISTRAL (Measurable Intelligent and Reliable Semantic Extraction and Retrieval of Multimedia Data)[16] die Aufzeichnung und Auswertung von Sitzungen. MISTRAL wertet dabei die behandelten Themen und Interaktionen zwischen Sitzungsteilnehmern automatisch aus und ermöglicht somit die teilweise Erfassung des transferierten Wissens. Im Allgemeinen ist die Erfassung dieser Kommunikationsart nicht möglich.

2. Erfassung von direkten Benutzereingaben
   Die einfachste Interaktion zwischen Benutzer und WMS stellt die explizite, direkte Eingabe von Information in das WMS dar. Dies kann z.B. dem Ablegen von Dokumenten entsprechen, dem Schreiben von Beiträgen in einem News-Forum, aber auch der Formulierung des Informationsbedürfnisses beim Absetzen von Suchanfragen.

[16] http://www.mistral-project.at

3. Erfassung von impliziten Benutzereingaben
   Neben explizit formulierten Benutzereingaben können auch implizite Benutzereingaben erfasst und berücksichtigt werden. Zum Beispiel kann das Versenden von E-Mails erfasst und der versendete Inhalt entsprechend im Wissensmanagementsystem, ohne explizite Bestätigung des Benutzers, abgelegt werden. Ein solcher automatischer Ablagemechanismus wurde z.B. im Projekt DocClass (siehe Abschnitt 6.1.3) prototypisch umgesetzt.

4. Erfassung des Benutzerverhaltens
   Durch das Aufzeichnen und Analysieren von impliziten und expliziten Interaktionen kann auch das Benutzerverhalten erfasst und daraus der aktuelle Kontext des Benutzers abgeleitet werden. Aktuelle Forschungsprojekte wie z.B. DYONIPOS[17] [Tochtermann et. al. 2006], bei dessen Antragstellung der Autor maßgeblich beteiligt war, versuchen z.B. über solche Analysen neben der Ableitung des aktuellen Kontextes und der damit verbundenen Möglichkeit gezielt kontextspezifische Information anzubieten, sowie automatisch Geschäftsprozesse und optimierte Prozessmuster abzuleiten.

5. Information Pull
   Erfassung von Wissen und Information bedingt auch eine Nutzung dieser. Information Pull, die explizit gewünschte Bereitstellung von Information, zeigt sich als zentraler Aspekt im Umgang mit Wissensmanagementsystemen. Studien von z.B. Microsoft [Cutrell et. al. 2006] zeigen das der Zugriff auf Information immer weniger über benutzerdefinierte Ablagestrukturen erfolgt, sondern viel mehr der Trend hin zur Suche von Information, dem Information Retrieval erfolgt. Diese Interaktion des Maurer-Tochtermann-Modells umfasst alle Arten der Informationsabfrage, sowohl über Browsing von Ablagestrukturen als auch über Methoden des Information Retrievals.

6. Information Push
   Information Push bezeichnet die automatische Bereitstellung von Information zum aktuellen Benutzerkontext. Just-in-TimeRetrieval ist hierbei ein Schlagwort, welches in verschiedenen Systemen wie z.B. der REMEMBRANCE Machine [Rhodes 2000] vom MIT Media Labratory oder dem IQ Aufsatz des neuen Microsoft Search Tools ‚Stuff I've Seen' [Dumais et. al. 2004] umgesetzt wurde. Beim Erstellen von Texten wird dem Benutzer dabei automatisch weiterführende Information in unaufdringlicher Weise angeboten, um ihn so mit Hintergrundwissen zu versorgen. Essentiell hierbei ist die Identifikation des Kontextes eines Benutzers um zielgerichtet relevante Information zu identifizieren und die Ergebnisse in unaufdringlicher Art und Weise zu präsentieren. Einen ähnlichen Ansatz verfolgt auch das Forschungsprojekt DYONIPOS [Tochtermann et. al. 2006]„ bei welchem der Benutzer abhängig von seinem aktuellen Arbeitskontext mit Vorlagen, weiterführender Information, Experten auf dem Gebiet, sowie mit möglichen nachfolgenden Tätigkeiten versorgt wird.

7. Informationsanalyse und Aufbereitung von Wissen
   Die bisher betrachteten Interaktionen inkludierten immer den Benutzer als involvierte Partei. Die letzte im MTM beschriebene Interaktion behandelt die rein maschinellen Analyse von im WMS vorhandener Information, das In-Beziehung-Setzen dieser und die Aufbereitung von Wissen, welches dem Benutzer zur Validierung präsentiert wird. Dabei können z.B. Techniken wie das automatische

[17] http://www.dyonipos.at

Verlinken von Dokumenten mit ähnlichen Inhalten (siehe Abschnitt 4.4) oder etwa das automatische Lernen von Ontologien auf Basis der im WMS abgelegten Inhalte (siehe Abschnitt 4.6) zum Einsatz kommen. Grundlegen ist das Ziel dieses Schrittes die Erhöhung der Semantik der im WMS abgeleiteten Information.

Das MTM definiert 7 verschiedene Interaktionsarten unter Berücksichtigung von Benutzern und WMS als Beteiligte. Betrachtet man jeweils die Interaktionsgrenzen, also den Übergang vom Benutzer zum WMS und vize versa, und berücksichtigt dabei die Wirkung auf Wissen bzw. Information, so kann man auf Basis des in [Staab et. al. 2001] präsentierten Wissensprozesses den in Abbildung 2-2 gezeigten Wissensprozess ableiten. Bezogen auf die Interaktionsarten des MTM und des dargestellten Wissensprozesses können nun Anwendungsgebiete der Wissenserschließung abgeleitet werden, welche sich vor allem auf die in Abbildung 2-2 eingefärbten Schritte beziehen und im Folgenden bei der Erläuterung des Wissensprozesses umrissen werden. Die hier abgeleiteten Anwendungsgebiete stellen auch die zentralen Anwendungsgebiete des umzusetzenden Frameworks dar, welche den Rest der Arbeit begleiten.

Beginnend auf Seite des Anwenders, erfolgt die *Schaffung und Erzeugung* von Wissen durch den Benutzer. Wie oben erwähnt, gliedert sich Wissen in implizites und explizites, kodifizierbares Wissen, welches der Benutzer als Information weiter geben kann.

Erfassen, speichern, bestätigen in Abbildung 2-2 repräsentiert die Schnittstelle zur Erfassung des informativ transportierten Wissens und kann anhand des MTM als explizit transferierte Information wie z.B. bei Dokumenten oder Dokumentteilen (2 im MTM) oder durch Analyse der impliziten Benutzerinteraktionen (3 im MTM) bzw. durch Analyse des Benutzerverhaltens (4 im MTM) erfasst und gespeichert werden. Neben dem Erfassen und Speichern spielt auch das Bestätigen von vom System identifizierten Zusammenhängen eine wichtige Rolle. Dies kommt zum Tragen, wenn vom System nach Durchlauf der technologischen Schritte automatisch extrahierte Zusammenhänge vom Benutzer validiert und verifiziert bzw. falsifiziert wurden, somit also nur noch bestätigt oder verworfen werden müssen. Generell können innerhalb des Erfassens und Speichern verschiedene KD-Methoden angewandt werden. Da sich dieser Schritt aber an der Schnittstelle zwischen Technologie und Benutzer befindet, und somit im Allgemeinen interaktive Komponenten umfasst, müssen KD-Anwendungen in Echtzeit ausführbar sein, um die Benutzbarkeit zu gewährleisten.

Prominenteste Vertreter zur Unterstützung dieses Prozessschrittes sind hier sicherlich Klassifikationsansätze, welche den Benutzer beim Ablegen von Dokumenten in eine unternehmensweite Ablagesystematik unterstützen, indem sie automatische Zuordnungen treffen oder dem Benutzer Zuordnungen vorschlagen. Hierunter fallen auch Spam Filter oder andere Ablageautomatiken für E-Mails. Aber auch Methoden zur automatischen Zusammenfassung von Dokumenten bzw. zur Extraktion von Metadaten können, sofern sie obiges Echtzeitkriterium erfüllen, angewandt werden. Für explizite Interaktionen gilt dabei, dass semiautomatische Methoden möglich sind, während bei impliziten Interaktionen dies nicht der Fall ist. Hierbei zählt vor allem, den Benutzer nicht im aktuellen Arbeitsfluss durch scheinbar willkürliche Zwischenfragen zu unterbrechen und damit zu stören, wie dies auch von Just-in-TimeRetrieval Systemen (JITR) berücksichtigt wird.

Der Interaktion 4 kommt in Bezug auf Anwendung von KD-Methoden im Schritt Erfassen, Speichern, Bestätigen eine spezielle Stellung zu, da die Ermittlung des Benutzerverhalten im Allgemeinen die Anwendung komplexer Methoden und Techniken impliziert. Im einfachsten Fall kann beispielsweise der in der aktuellen Applikation angezeigte Text als Kontext verstanden werden oder, komplexer, der Ablauf von verschiedenen

Interaktionen des Benutzers mit Applikationen. Ostensives Retrieval [Campbell 2000] setzt hierbei speziell auf letzteren Ansatz und versucht den Kontext einer Suche über die Analyse der vorangegangenen Suchen zu ermitteln. Im Speziellen bei der Analyse verschiedener Abläufe und Interaktion zur Bestimmung des Kontextes kommt es oft zum Einsatz von KD-Techniken und Methoden, wie z.B. die Analyse von Zeitsequenzen und auch Graph Mining Methoden, welche z.B. im Projekt DYONIPOS angewendet werden, um die Aufgabe/Tätigkeit zu bestimmen in der sich ein Benutzer befindet.

Nach der Erfassung bzw. dem Speichern der eingegebenen Information erfolgen die Ablage und das *Organisieren* dieser. Dabei können unterschiedliche Organisationsformen wie z.B. Thesauri, Verschlagwortung oder aber auch eine ontologische Repräsentation gewählt werden.

Zentral ist die Anwendung von KD-Methoden jedoch bei der *Analyse* der entsprechend organisierten Inhalte, der *Extraktion* neuer Zusammenhänge und der semantischen Anreicherung der Inhalte. Im Gegensatz zum Erfassungsschritt kann die Analyse und Extraktionszeit deutlich höher sein und auch mehrere Tage in Anspruch nehmen, wie dies z.B. im System WebLyzard [Scharl 2004] der Fall ist, welches Medienbestände automatisch semantisch erschließt. Im MTM kann dieser Prozessschritt mit Interaktion Nr. 7 gleichgesetzt werden, in der neues Wissen erzeugt wird. Bezüglich der KD-Methoden erfolgt hier hauptsächlich die Anwendung der Informationsextraktion, dem Clustering von Dokumenten, der Extraktion von Assoziationsregeln und den verschiedenen Methoden zur automatischen Extraktion von Ontologien aus textuellen Daten (siehe Abschnitte 3.2, 3.3 und 3.1 für einen Überblick). Beispielsweise erfolgt in diesem Schritt die Extraktion von Personen und Ortsbezügen und das In-Bezug-Setzen von Inhalten zu Geo-Referenzen. Methoden, welche vor allem in letzter Zeit mit der Etablierung verschiedener Geo-Browsing Plattformen immer stärker an Bedeutung gewinnen. Zusätzlich ermöglicht die Extraktion von Personen z.B. die Ermittlung von sozialen Netzwerken und Identifikation von Kompetenzträgern zu gewissen Themen.

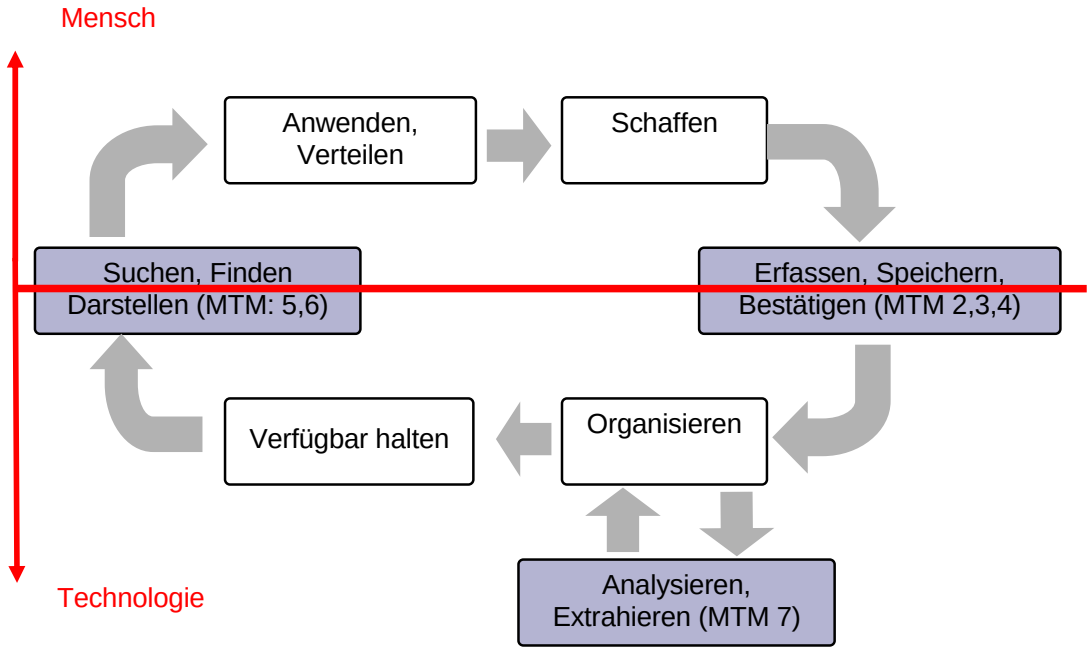


Abbildung 2-3: Wissens-/Informationskreislauf bezogen auf die Interaktionsgrenzen im Maurer Tochtermann Modell zwischen Benutzer und WMS.

Der nächste Prozessschritt beschäftigt sich mit dem Verfügbarhalten der teilweise über KD-Methoden erschlossenen Inhalte. Technologisch gesehen handelt es sich hierbei um die Anwendung von verschiedenen Persistierungsmechanismen die bezogen auf das Anwendungsszenario unterschiedliche Kriterien wie zum Beispiel Zuverlässigkeit oder Geschwindigkeit erfüllen müssen. So reicht die Skala der Technologie von einfachen invertierten Suchindizes im Fall von Retrievalanwendungen bis hin zu auf verschiedenen Systemen verteilten relationalen Datenbanken. Außer der unterschiedlich Anbindungen an unterschiedliche Datenhaltungssysteme weist dieser Schritt keine weiteren Anforderungen an KD-Methoden auf.

Beim *Suchen, Finden und Darstellen* können wiederum KD-Methoden und Anwendungen den Prozessschritt unterstützen. Suche in dem hier dargestellten Sinne umfasst alle Arten von Retrieval, nicht nur das schlüsselwortbasierte Suchen. Wie bereits beim MTM Modell erwähnt, stellt die inhaltsbasierte Suche, unter der Voraussetzung, dass verschiedenste Attribute von Objekten suchbar sind, die wichtigste Zugriffsart dar. Neben den Such- und Retrievalmethoden an sich, welche in Abschnitt 4.2 näher beschrieben werden, ermöglichen vor allem verschiedene Arten der Wissens- und Informationsvisualisierung Benutzern das leichtere Aufnehmen der präsentierten Zusammenhänge und damit eine verbesserte Interaktion. In [Cutrell et. al. 2006] konnte beispielsweise gezeigt werden, dass eine zeitliche Aufbereitung von Suchergebnissen sowie das Hinzufügen von persönlichen Landmarks den Finde-Prozess von Benutzern beschleunigt und verbessert. Ähnlich auch in [Kienreich et. al. 2005], wo unterschiedliche Möglichkeiten zur Präsentation von Suchergebnissen dargestellt wurden. Details zum Thema Informationsvisualisierung gibt Abschnitt 4.5. Im Zusammenhang mit dem MTM stellt dieser Prozessschritt die Interaktionen 5 und 6.

Der Kreislauf schließt sich mit der *Anwendung und Verteilung* des aus dem vorhergehenden Prozessschritt abgeleiteten Wissens. Hier zeigt sich auch die Grenze der beiden Modelle, sowohl auf Benutzer- als auch auf Technologieseite. In beiden Modellen stellt sich die Kommunikation und der damit verbundene Austausch von Information/Wissen von Benutzer zu Benutzer bzw. von WMS zu WMS nur implizit dar. Neueste Trends wie das Web 2.0 [O'Reilly 2005] und das Semantic Web [Berners-Lee et. al. 2001] zeigen deutliche Richtungen hinsichtlich Kollaboration zwischen Benutzern und Austausch von semantisch hochwertigen Inhalten zwischen WMS. Speziell der Austausch zwischen WMS existiert nicht explizit im MTM und könnte als achte Interaktion dargestellt werden. Unter der Annahme, dass Interaktion Nr. 1 kollaborative Interaktionen abgedeckt, würde Interaktion Nr. 8 das MTM vervollständigen. Sowohl kollaborative Aspekte als auch der Austausch zwischen WMS erschließen zusätzliche Anwendungsgebiete im KD. Die Analyse von so genannten Folksonomies, kollaborativ erstellten Beschreibungen von digitalen Ressourcen, ermöglicht zum Beispiel die Ableitung von Kontexten bzw. Sichtweisen von Communities auf Inhalte und erhöht damit die Semantik der digitalen Ressourcen. Auch die Überführung der Repräsentation von Inhalten eines WMS in ein anderes, im Bereich der Metadaten „Standard Crosswalking" [McCray et. al. 2001] und im Bereich von Ontologien „Ontologie Alignment" (c.f. [Mayfield 2002]) genannt, stellt mögliche Anwendungsgebiete von z.B. überwachten Lernverfahren (siehe Abschnitt 4.3) dar.

Vor allem Funktionen wie Suche, Clustering, Klassifikation und Metadatenextraktion, als auch das Entdecken und Visualisieren von komplexen Zusammenhängen sind essentielle Funktionen zur Erhöhung der Informationsqualität und dem verbesserten Umgang mit großen Informationsmengen.

2.3 Anforderungen

Die aus diesem Kapitel ableitbaren Anforderungen lassen sich in die zwei Sichtweisen einteilen; in die zu behandelnden Informationsarten und in die zu unterstützenden Funktionen.

Die Anforderungen bezüglich der zu **berücksichtigenden Informationsarten** können unter Anwendung des in Abschnitt 2.1 erweiterten Modell von Kroeze und unter Berücksichtigung der in Abschnitt 2.2 erarbeiteten Anwendungsszenarien, wie in Tabelle 2-3 dargestellt, abgeleitet werden.

<table>
<tr><th></th><th>Non-novel investigation</th><th>Semi-Novel Investigation-Knowledge Discovery</th><th>Novel investigation-Knowledge Creation</th></tr>
<tr><td>Text Metadata</td><td colspan="2" rowspan="2">Hauptfokus</td><td rowspan="4">Zukunft</td></tr>
<tr><td>Textual data</td></tr>
<tr><td>Graph Data</td><td colspan="2" rowspan="2">Berücksichtigt</td></tr>
<tr><td>Multimedia data</td></tr>
<tr><td>Numerical data</td><td colspan="3">Unberücksichtigt</td></tr>
</table>

Tabelle 2-3: Fokus des umzusetzenden Frameworks bezogen auf [Kroeze et. al. 2003]

Der Hauptfokus des zu entwickelnden Frameworks liegt dabei im Bereich **Text und Text-Metadaten** hinsichtlich non-novel und semi novel investigations (wie in Abschnitt 2.1 definiert). Daraus leitet sich auch der Fokus in Richtung Textanalysemethoden und Information Retrieval ab. Unter Berücksichtigung der in Abschnitt 2.2 dargestellten Anwendungsmöglichkeiten, soll das zu entwickelnde Framework folgende **Funktionsblöcke** durch Bereitstellung von Knowledge Discovery und Information Retrieval Funktionalität unterstützen:

1. Funktionen zum Erfassen und Speichern von Information

2. Funktionen zum Analysieren von Information und zur Extraktion von Zusammenhängen

3. Funktionen zum Suchen, Finden und Darstellen von Information

Umsetzungen auf graphbasierten und multimedialen Daten sollen im umzusetzenden Framework wegen der in Abschnitt 2.1 dargestellten aktuellen Trends und bzgl. der Schaffung eines möglichst vollständigen Frameworks berücksichtigt werden, jedoch liegt die Bereitstellung von Diensten und Umsetzung von Algorithmen im Umfeld dieser Informationsarten außerhalb des Fokus dieser Arbeit.

Aufgrund der Komplexität der Aufgabe und da es derzeit noch kaum Techniken auf diesem Gebiet gibt, wird die Wissenserzeugung (novel investigations) im umzusetzenden Framework nicht abgedeckt; dieser Punkt stellt jedoch mögliche zukünftige Weiterentwicklungen auf den unterschiedlichen Datenbasen dar.

Numerische Daten bzw. Data Mining bleiben, bezogen auf die in Abschnitt 2.2 dargestellten Anwendungsmöglichkeiten, bei der Entwicklung und Umsetzung des umzusetzenden Frameworks unberücksichtigt.

Die in diesem Kapitel erhobenen Anforderungen können sozusagen als Grundgerüst für die weitere Vorgehensweise angesehen werden und stellen eine erste Verfeinerung der Zieldefinition der Arbeit dar, indem sie die Punkte der umzusetzenden Dienste und die zu betrachtenden Informationsarten näher spezifizieren. In den nächsten beiden Kapiteln erfolgt die Detailbetrachtung der Techniken und Algorithmen und die Detaillierung und Vervollständigung der hier erhobenen Anforderungen.

3 Vorverarbeitung und Repräsentation unstrukturierter Daten

Ziel dieses Abschnittes ist die Darstellung der wichtigsten Techniken zur Vorverarbeitung von unstrukturierten Daten, im Speziellen Texten. Die Vorverarbeitung multimedialer Inhalte wird kurz umrissen, um zu zeigen, dass ähnliche Merkmale wie aus Texten zur Weiterverarbeitung extrahiert werden können und somit die in Abschnitt 4 illustrierten Algorithmen anwendbar sind.

Der Abschnitt beginnt mit der Vorverarbeitung von Texten und illustriert dabei die einzelnen Schritte, beginnend von der Normalisierung unterschiedlicher Formate bis hin zur Generierung von Merkmalen. Nach einem kurzen Ausflug in die Vorverarbeitung multimedialer Inhalte schließt der Abschnitt inhaltlich mit einer Betrachtung der auf Basis der extrahierten Merkmale ableitbaren Ähnlichkeitsmetriken. Um den Kreis zur vorliegenden Arbeit zu schließen, leitet Abschnitt 3.6 die aus den Vorverarbeitungstechniken ableitbaren Anforderungen an das KnowMiner-Framework ab.

Der Großteil der Texte stammt aus dem vom Autor verfassten Buchkapitel [Granitzer 2006] über statistische Textanalyseverfahren. Die Inhalte wurden entsprechend dem für die Ableitung der Anforderungen notwendigen Detailgrad ergänzt.

3.1 Textvorverarbeitung

Als Mensch fällt es uns im Gegensatz zum PC leicht, mit natürlich sprachlicher Information umzugehen, diese zu verstehen, anzuwenden und weiter zu verarbeiten. Natürliche Sprache beinhaltet für das maschinelle Verstehen von Sprache erschwerende Eigenschaften wie Mehrdeutigkeit, konkurrierende semantisch äquivalente Ausdrucksmöglichkeiten und Vagheit. Bis dato gibt es noch kein maschinelles System, welches Sprache versteht, womit auch eine vollständige automatische Überführung von natürlichsprachlichen Texten in semantische Strukturen immer nur als Approximation zu sehen ist.

Nichtsdestotrotz existieren gut funktionierende Heuristiken und Approximationen, basierend auf fundierten Theorien aus verschiedenen Wissenschaftsdisziplinen, welche das Arbeiten und den Umgang mit natürlichsprachlicher Information erleichtern. Statistische Analysen ermöglichen zusätzlich empirische Aussagen über quantitative Eigenschaften von Sprache wie z.B. die Worthäufigkeiten. Die Durchführung solcher Analysen und die Anwendung von Heuristiken bedarf einer Zerlegung von Text in atomare Teile, so genannte Merkmale. Beginnend bei der Extraktion eines Merkmals erfolgt die schrittweise Anreicherung von Eigenschaften der Merkmale, welche weiteren Analysen und statistischen Evaluierungen unterzogen werden können. In diesem ersten Schritt wird lediglich ein einzelnes Dokument betrachtet und nicht eine Menge von Dokumenten.

Dieser Abschnitt betrachtet die wichtigsten Schritte der Vorverarbeitung von Textdokumenten und bildet die Grundlage für die weiteren Techniken dieses Kapitels.

3.1.1 Formatnormalisierung und lexikalische Analyse

Texte liegen meist in unterschiedlichen Formaten wie Word, HTLM und SGML vor. Neben dem Text selbst kapseln diese Formate zusätzlich Layout-Eigenschaften wie z.B. Überschriften, Absatzstrukturen und Metadaten wie z.B. Autor oder Erzeugungsdatum.

Der erste Schritt in der Vorverarbeitungskette besteht nun darin, Layout und Metadaten beschreibende Elemente zu eliminieren. Ergebnis dieses Prozesses ist eine den Inhalt eines Textes beschreibende Zeichenkette, wobei vorgegebene Metadaten natürlich für die weitere Verarbeitung berücksichtigt werden können. Dieser Aspekt wird jedoch hier aus Komplexitätsgründen außen vor gelassen.

In der lexikalischen Analyse erfolgt die Zerlegung der Zeichenkette in kleinere Einheiten, so genannte Merkmale (engl. Tokens).

(1) Eine intuitive Form der Zerlegung ist die Aufteilung nach Wörtern (Word-Grams), d.h. jedes Merkmal entspricht einem Wort. Im ersten Moment sieht eine solche Vorgehensweise relativ einfach aus, jedoch zeigen sich bei näherer Betrachtung die ersten Probleme bei der Bestimmung dessen, was ein Wort darstellt. Vor allem Satzzeichen wie Punkte bereiten hier Probleme, da diese z.B. das Ende eines Satzes anzeigen können oder aber Abkürzungen (z.B. „Dr."). Im Allgemeinen werden daher in diesem Schritt Satzzeichen und gegebenenfalls Leerzeichen ebenfalls als Merkmale aufgefasst, um diese in einer nachfolgenden, erweiterten Analyse berücksichtigen zu können.

(2) Charakter n-grams sind eine weitere Möglichkeit der Zerlegung von Zeichenketten, die auf Shannon [Shannon 1951, Cavnar et. al. 1994] zurückgeht. Dabei entspricht ein Merkmal einer Zeichenkette der Länge n. Die Erzeugung dieser Merkmale kann man sich so vorstellen, dass über den Text ein Fenster der Länge n bewegt wird und nach jedem Vorrücken um *n* definierte Zeichen der Inhalt dieses Fensters als Merkmal aufgenommen wird.

Beispiel: *„<h2>Formatnormalisierung und lexikalische Analyse:</h2>"*[18]

Formatnormalisierung:
„Formatnormalisierung und lexikalische Analyse:"

Lexikalische Analyse (Word-Grams)*:*
"Formatnormalisierung", „_", „und", „_", "lexikalische", „_" "Analyse","**:**"[19]

Lexikalische Analyse (Charakter 4-gram)
"Form", "orma", "rmat", "matn",…."rung","ung_","ng_u"...

3.1.2 Merkmalsanalyse

Zuvor erzeugte Merkmale zeichnen sich nicht nur durch ihre Zeichenkette, sondern auch durch andere Eigenschaften wie z.B. die Wortform aus. Ziel der Merkmalsanalyse bzw. Tokenanalyse ist die Extraktion von zusätzlichen Eigenschaften, welche in der weiteren Verarbeitung benötigt werden. Die Merkmalsanalyse ist nicht zwingend notwendig, erhöht jedoch in den meisten Fällen die Qualität der Ergebnisse beträchtlich. Natürlich hängen die hier möglichen Verarbeitungsschritte auch von der Art der extrahierten Merkmale ab, wobei die hier vorgestellten Verfahren auf einer wortbasierten Zerlegung aufsetzen.

[18] <h2>…</h2> kennzeichnet in HTML eine Überschrift für Gliederungen der 2. Ebene

[19] „_" dient als Platzhalter für das Leerzeichen im obigen Beispiel

3.1.2.1 Lemmatisierung

Bei der lexikalischen Analyse werden Merkmale rein aufgrund ihrer beschreibenden Zeichenkette unterschieden, was im Allgemeinen nicht auf die semantische Unterscheidung schließen lässt. Im Schritt der Lemmatisierung wird nun versucht, Merkmale auf ihre gemeinsame Form bzw. ihren gemeinsamen Merkmalstyp zurück zu führen. Im Falle von Wörtern wäre dies z.B. die Reduktion jedes Wortes auf seinen Wortstamm, auch unter dem Begriff Stemming bekannt. Dabei wird grundlegend zwischen Stripping und „echten" Stemming unterschieden.

Beim Stripping erfolgt die Rückführung auf den Stamm anhand simpler Regeln zum Abschneiden von Vor- bzw. Nachsilben eines Wortes. Im deutschen wäre eine solche Regel das Abschneiden von Suffixes wie „lich" am Wortende (e.g. „sportlich"➔"sport"). Stripping basierte Verfahren arbeiten rasch und sind im Allgemeinen unabhängig von externen Ressourcen wie z.B. Wörterbüchern, haben jedoch den Nachteil, dass irreguläre Formen (z.B. „gehen", „ging", „gegangen") nicht berücksichtigt werden können.

Echte Stemming Verfahren hingegen ermöglichen die Auflösung von solchen irregulären Formen, wobei meist Wörterbücher zum Einsatz kommen. Die Methode ist simpel: Ein Wort wird mit einem Eintrag in einem Wörterbuch verglichen, welches auch den entsprechenden Wortstamm beinhaltet, der das Merkmal bzw. Wort ersetzt.

Sowohl Stripping als auch Stemming haben den Nachteil relativ fehlerintolerant gegenüber falsch geschriebenen Wörtern zu sein, da mitunter die Regeln nicht mehr greifen bzw. das Nachschlagen im Wörterbuch versagt. Mögliche Lösungen dazu sind fehlertolerante Wortvergleiche wie z.B. die Edit Distance oder auch Charakter n-grams, welche Ähnlichkeiten zwischen Wörtern abbilden (siehe auch [Cavnar et. al. 1994, Weiss et. al. 2005]).

3.1.2.2 Parsing und Part-of-Speech-Tagging

Ist für die weiteren Schritte die Notwendigkeit einer tieferen linguistischen Analyse gegeben, so kann als nächster Schritt eine detaillierte grammatikalische Analyse durchgeführt werden. Dabei erfolgt die Zuordnung der entsprechenden Wortformen (z.B. Substantiv, Verb, Adverb etc.) zu den zuvor ermittelten Merkmalen durch so genannte Part-of-Speech Tagger.

In den einfachsten Fällen erfolgt die Zuordnung von Wortformen unter Verwendung von Wörterbüchern, welche die jeweiligen Wörter und deren korrespondierende Wortform enthalten. Die Unvollständigkeit dieses Ansatzes ist dadurch bestimmt, dass grundlegend keine vollständige Abbildung aller Wörter und Wortformen in einem Wörterbuch möglich ist. Zusätzlich entstehen Probleme bei der Auflösung von Mehrdeutigkeiten, da abhängig von der grammatikalischen Struktur eines Satzes Wörter unterschiedlich gebraucht werden können.

Die meisten Part-of-Speech Tagger ermöglichen jedoch eine vollständige syntaktische Zerlegung eines Satzes nicht nur im Sinne einer Zuordnung der Wortform zu einem Merkmal, sondern auch im Sinne der Herstellung der sprachlichen Beziehungen eines Merkmals zu allen anderen Merkmalen eines Satzes. Durch vollständige syntaktische Zerlegung können zusätzliche Mehrdeutigkeiten aufgelöst werden, aufgrund der Komplexität der natürlichen Sprache ist jedoch eine eindeutige Bestimmung der korrekten syntaktischen Zerlegung nicht immer möglich. Deshalb muss zwischen mehreren Zerlegungsmöglichkeiten gewählt werden. Diese Mehrfachmöglichkeiten stellen derzeit eines der größten Probleme bei tiefen linguistischen Analysen dar und verhindern ein vollständiges Sprachverstehen. Zusätzliche Nachteile von Part-of-Speech Tagging sind

die Sprachabhängigkeit, die Verfügbarkeit von Taggern für diverse Sprachen sowie die Robustheit in Bezug auf grammatikalisch nicht korrekte Sätze.

Nichtsdestotrotz kann schon eine einfache Unterscheidung zwischen Substantiven und Nicht-Substantiven hilfreich für verschiedene nachfolgende Schritte wie z.B. Information Extraction oder Kollokationserkennung sein [Bontcheva et. al. 2003]. Diese Unterscheidung bewirkt eine deutliche Genauigkeitssteigerung.

3.2 Informationsextraktion

Bis jetzt wurden verschiedene Konzepte zur Vorverarbeitung von Text vorgestellt. Ergebnis dieser Vorverarbeitung ist eine Liste von Merkmalen und deren sprachliche Eigenschaften. Ziel der Informationsextraktion (IE) ist nun die Überführung dieser noch unstrukturierten Merkmale in eine tabellen- bzw. datenbankartige Struktur. Ein Beispiel dafür ist die Extraktion einer Personenkartei mit Vorname, Nachname, Geburtsdatum etc. aus Lebensläufen von Mitarbeitern eines Unternehmens. Im Umfeld der Message Understanding Konferenzreihe (MUC) entwickelten sich folgende Teilbereiche der Informationsextraktion [Grishman et. al. 1996]:

- Erkennen benannter Entitäten (Named Entity Recognition, NE)
  Ziel ist es, ausgezeichnete Entitäten wie z.B. Personennamen, Orte, Datum, Geldbeträge, Titel etc. aus unstrukturierten Texten zu ermitteln.

- Auflösen von Koreferenzen (Co-Reference Resolution, CO)
  Natürlich sprachliche Referenzen, wie z.B. durch persönliche Pronomen wie „Er“, „Sie“ etc. angezeigte, sollen in diesem Schritt aufgelöst und ausgezeichnet werden.

- Füllen von Schablonen-Elemente (Template Element Filling, TE)
  Schablonen-Elemente beschreiben zusätzliche Elemente, die zu einer Entität noch extrahiert werden können. Beispielsweise können für Personen noch Geburtsdatum oder Titel extrahiert werden und gemeinsam mit dem Personennamen als Schablone für eine Person ausgewiesen werden.

- Schablonen-Relationen (Template Relations, TR)
  Dieser Bereich beschäftigt sich mit dem Extrahieren von Relationen zwischen einzelnen Schablonen, wie z.B. des Bezugs eines Angestellten (=Person) zu einer Firma (=Organisation).

- Szenario-Schablonen (Scenario Templates, ST)
  Ziel ist es hier, Szenarien und Ereignisse aus freiem Text zu identifizieren, wobei auf die zuvor extrahierten Schablonen-Elemente und Schablonen-Relationen zurückgegriffen wird.

Zur Durchführung obiger Aufgaben kann prinzipiell zwischen maschinellen Lernverfahren und linguistischen Analysen bzw. grammatikbasierten Ansätzen unterschieden werden.

Bei linguistischen Analysen erfolgt die Untersuchung von Texten mittels Regeln, welche durch Experten von Hand erstellt werden und Eigenschaften einer Sprache ausnützen. Im einfachsten Fall kommen Gazeteers bzw. Lookup-Listen zum Einsatz, welche die zu erkennenden Entitäten (z.B. Ortsnamen oder Personennamen) enthalten. Eine Liste von Ortsnamen beispielsweise ermöglicht das Finden von Merkmalen durch einen einfachen Textvergleich. Stimmt jedes Zeichen im Merkmal mit einem Listeneintrag überein, erfolgt eine Kennzeichnung des Merkmals als benannte Entität bzw. als Ort. Wenn auch dieser Ansatz die Vorteile der Sprachunabhängigkeit und einer hohen Geschwindigkeit besitzt, so ist eine manuelle Erstellung

und Wartung von Listen problematisch und eine Aufzählung aller möglichen Entitäten unmöglich. Zusätzlich können unterschiedliche Schreibweisen und entstehende Mehrdeutigkeiten nicht aufgelöst werden.

Abhilfe bieten hier „Shallow Parsing" - Ansätze. Im Gegensatz zu vollständigen linguistischen Analysen unter Verwendung von Parsing-Bäumen erfolgt die Erkennung von Entitäten hier mittels Heuristiken, welche interne Strukturen von Entitäten ausnutzen. Eine Regel zur Detektion von Orten wäre z.B. „im Ortsgebiet von <Substantiv>" welche darauf hindeutet, dass <Substantiv> eine Ortsbezeichnung ist. Natürlich können vollständige linguistische Satzanalysen die Genauigkeit erhöhen, doch erreicht man meist mit Shallow Parsing - Ansätzen ähnliche Ergebnisse ohne die Notwendigkeit von komplexen, vollständigen linguistischen Analysen. Die Effizienz solcher Ansätze spiegelt sich auch im Open Source Tool GATE wider, auf welches an dieser Stelle verwiesen sei. GATE bietet eine vollständige Umgebung zur Analyse von Texten und Tools zur Informationsextraktion[20].

In der Message Understanding Conference (MUC) - Reihe erfolgt eine regelmäßige Evaluierung der verschiedenen Informationsextraktionsaufgaben. Auf Zeitungskorpora wird für NE ca. 97%, CO ca. 60-70%, TE ca. 80%, TR ca. 75-80%, ST ca. 60% Genauigkeit erzielt [9]. Nichtsdestotrotz sind die Genauigkeit von Informationsextraktion und die Anwendbarkeit stark von der jeweiligen Domäne abhängig. Während Shallow Parsing - Ansätze bei Zeitungstexten aufgrund stilistisch hochwertiger Texte gut funktionieren, trifft dies im Bereich der Analyse von BLOG's oder E-Mails nicht mehr zu.

3.3 Merkmalsgenerierung und statistische Analysen

Die bisherigen Betrachtungen erstreckten sich jeweils auf die Anwendung von verschiedenen Methoden auf ein einzelnes Dokument. Wie in der Einleitung beschrieben, bewegt man sich aber meist in einem Informationsraum, der aus einer großen Menge von domänenspezifischen Dokumenten besteht. Unter Zuhilfenahme statistischer Methoden können für den Informationsraum inhärente Strukturen und Konzepte sowie wichtige Merkmale identifiziert werden. Im Folgenden sollen die wichtigsten Techniken zur Extraktion von Merkmalen des Informationsraums, auf Basis der Merkmale der Einzeldokumente, vorgestellt werden [Manning et. al. 1999]. Im speziellen werden Häufigkeitsanalysen, Kollokationen und Co-Occurrence Analysen betrachtet, welche die Selektion der wichtigsten Merkmale eines Informationsraums ermöglichen. Abschließend erfolgen eine kurze Betrachtung der Vektorgenerierung aus den statistisch gewichteten Merkmalen und der damit verbundenen Möglichkeit des Ähnlichkeitsvergleichs von Dokumenten und Wortgruppen. Die Vektorgenerierung ermöglicht unter anderem den Einsatz der in Abschnitt 4 vorgestellten maschinellen Lernverfahren ermöglicht wird.

3.3.1 Wortanalysen

Wortanalysen sind die einfachste Form der statistischen Analyse und beziehen sich auf die Untersuchung der Häufigkeiten von Merkmalen, wobei meist nur Worte als Merkmale betrachtet werden. Auf Basis dieser Analyse formulierte der Linguist George Kingsley Zipf Zipf's Gesetz, welches den Zusammenhang zwischen Häufigkeit und Rang eines Wortes darstellt [Zipf 1949]. Sortiert man die untersuchten Worte nach deren Häufigkeit, so besagt Zipf's Gesetz, dass der Rang eines Wortes multipliziert mit seiner Häufigkeit in etwa konstant ist. Dies bedeutet, dass z.B. das zehnt-häufigste Wort zweimal öfters vorkommt als das zwanzig-

[20] http://gate.ac.uk/

häufigste Wort. Es zeigt auch, dass in einem Dokumentkorpus wenige Wörter sehr häufig vorkommen und dass viele Wörter wenig oft vorkommen.

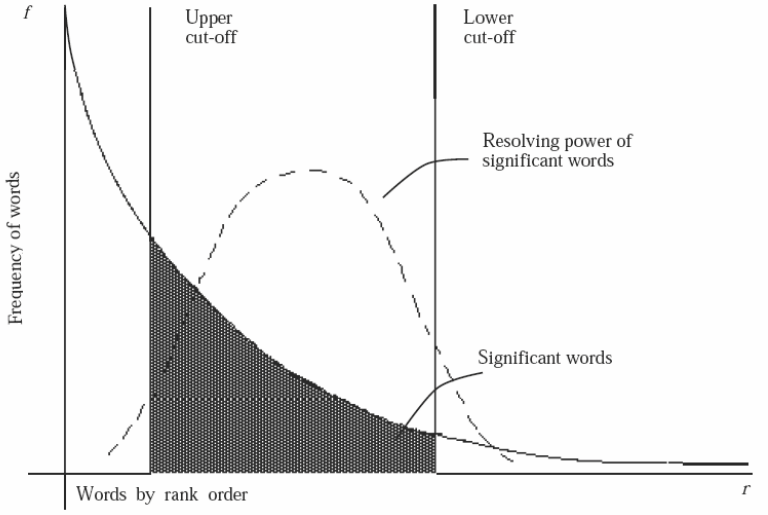


Abbildung 3-1: Zipf's Law: Zusammenhang zwischen dem Vorkommnissen (=f) und Informationsgehalt von Wörtern.. Die grau schattierte Fläche beinhaltet die laut Analyse signifikanten Wörter (entnommen aus [Rijsbergen 1979])

Im Information Retrieval wird dieser Umstand ausgenützt, um die zu durchsuchende Menge von Begriffen einzuschränken, da sowohl häufig als auch selten vorkommende Wörter wenig Informationsgehalt innerhalb eines Informationsraumes haben und daher zum Auffinden von Dokumenten somit ignoriert werden können. Abbildung 3-1 illustriert Zipf's Law und den beschriebenen Zusammenhang zum Informationsgehalt.

Dieser Umstand wird auch oft in der Vorverarbeitung durch maschinelle Lernverfahren zur Dimensionsreduktion (i.e. Reduzierung der verwendeten Merkmale) eingesetzt, womit meist eine Erhöhung der Genauigkeit und Geschwindigkeit verbunden ist. Neben der Betrachtung von Einzelwörtern ist auch die Betrachtung von Wortgruppen bzw. Kollokationen ein wichtiger Schritt in der Analyse.

3.3.2 Kollokationen

Kollokationen definieren eine zusammengehörige Gruppe von Wörtern, welche statistisch signifikant öfters miteinander vorkommen als per Zufall möglich. Eigennamen wie z.B. „New York" oder auch verschiedene Phrasen wie „statistische Analyse" sind Beispiele für Kollokationen. Basis für die Ermittlung von Kollokationen ist die Analyse der Häufigkeit des Vorkommens solcher Wortgruppen. Im einfachsten Fall wird dies erreicht, indem bei der Merkmalsgenerierung Wortgruppen der Länge n erzeugt werden und diese, wie im Falle der Wortanalyse, einer Häufigkeitsanalyse unterzogen werden. Ohne weiteres Post- bzw. Preprocessing ergibt sich jedoch das Problem, irrelevante Kollokationen wie z.B. „in dem" oder „bei der" zu finden. Abhilfe schafft hier eine syntaktische Vorgruppierung der statistisch zu untersuchenden Kollokationen. Betrachtet man z.B. nur Substantivgruppen, so können semantisch sinnvolle Kollokationen relativ einfach und zuverlässig erkannt werden. Auf Basis der betrachteten Häufigkeiten können statistische Hypothesentests wie z.B. der t-Test oder χ^2-Test oder Maße wie der Informationsgehalt zum Ermitteln von statistisch signifikanten Kollokationen angewendet werden. In diesem Zusammenhang sei hier auf [Manning et. al. 1999] verwiesen.

3.3.3 Co-Occurence Analyse

Die Co-Occurence Analyse kann als Spezialfall einer Kollokation der Länge 2 betrachtet werden, ist jedoch prinzipiell auch auf Merkmale, welche nicht einem Wort entsprechen, anwendbar. Ziel der Co-Occurence Analyse ist es, Affinitäten zwischen Merkmalen zu finden. Untersucht wird, welche Merkmalspaare statistisch signifikant oft in einem definierten Kontext vorkommen, wobei der Kontext im Allgemeinen einen Satz, Absatz oder ein gesamtes Dokument darstellt. Neben diesem strukturellen Kontext kann auch ein bestimmtes Textfenster mit einer festen Anzahl von Worten betrachtet werden. Wiederum wird gezählt, wie oft Merkmal A (z.B. „Fußball") gemeinsam mit anderen Merkmalen bzw. Wörtern vorkommt. Über verschiedene Maßzahlen wie z.B. dem Informationsgehalt erfolgt die Bestimmung der wichtigsten Merkmale oder gemeinsamen Vorkommnisse.

Co-Occurrence Analysen ermöglichen so das Finden von möglichen Synonymen oder fast synonymen Begriffen und erlauben die Definition eines Merkmals anhand anderer assoziierter Merkmale. Durch Anwendung maschineller Lernverfahren ergibt sich dann z.B. die Möglichkeit, automatisch hierarchische Beziehungen zwischen Merkmalen zu extrahieren und Begriffshierarchien aufzubauen. Im Information Retrieval können unter Anwendung solcher Methoden automatisch Suchbegriffe um ähnliche Begriffe ergänzt und somit die Breite der Suche vergrößert werden.

3.3.4 Merkmalsgewichtung

Bisher wurde hauptsächlich auf die Genierung von Merkmalen auf Dokumentbasis und die Selektion von Merkmalen mittels statistischer Methoden eingegangen. Um Dokumente innerhalb eines Informationsraumes miteinander vergleichbar zu machen, erfolgt im Allgemeinen die Transformation der Merkmale eines Dokumentes in einen numerischen Vektor, wobei zwischen binären oder reellwertigen Vektoren unterschieden wird. Die binäre Repräsentation definiert dabei lediglich, ob ein Merkmal in einem Dokument vorhanden ist oder nicht, während die reellwertige Repräsentation zusätzlich eine Gewichtung bzw. Wertigkeit eines Merkmales zulässt.

Im Vorfeld zur Vektorgenerierung erfolgt die Extraktion aller relevanten Merkmale aus allen Dokumenten des Informationsraumes. Alle unterschiedlichen Merkmale (auch Terme genannt) bilden die Achsen bzw. Dimensionen des so genannten Merkmals- od. Vektorraums, in dem jedes Dokument als Merkmalsvektor dargestellt werden kann. Abbildung 3-2 illustriert für einen dreidimensionalen Vektorraum die Repräsentation von drei enthaltenen Dokumente (Dok 1={Term 1, Term 2}, Dok 2=={Term 3, Term 2} und Dok 3={Term 1, Term 3}).

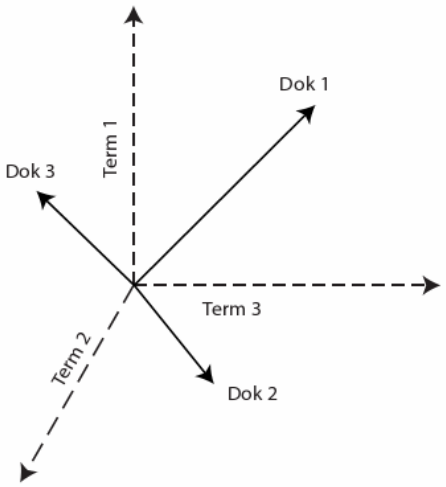


Abbildung 3-2: Beispiel für die Repräsentation von Dokumenten im Vector-Space-Model

Neben der binären Gewichtung, in der ein Merkmal im Vektor vorhanden ist (Wert von 1) oder nicht (Wert von 0), existieren auch reellwertige Gewichtungsschemata, welche sich im Allgemeinen aus den drei Größen Termfrequenz, Dokumentfrequenz und Kollektionsfrequenz zusammensetzen.

- Die Termfrequenz entspricht der Anzahl der Vorkommnisse eines Merkmals/Terms in einem Dokument und ist pro Merkmal und Dokument festgelegt.
- Die Dokumentfrequenz errechnet sich aus der Anzahl der Dokumente, in denen ein Merkmal vorkommt und ist somit pro Merkmal festgelegt.
- Die Kollektionsfrequenz entspricht der Anzahl der Vorkommnisse eines Merkmals/Terms im gesamten Dokumentbestand und ist pro Merkmal festgelegt.

Unter Berücksichtigung einer Ablagesystematik der Dokumente kann zusätzlich noch die Kollektionsfrequenz (Anzahl der Vorkommnisse eines Terms in einer Klasse) pro Klasse mit berücksichtigt werden (pro Term und Klasse festgelegt), was vor allem im Bereich der Textklassifikation eingesetzt wird (siehe [Sebastiani 2002] und [Weiss et. al. 2005).

Bei reellwertigen Vektoren ist das so genannte TFIDF - Term-Frequency Inverse Document Frequency Schema [Salton et. al. 1988] ein häufig gebrauchtes Gewichtungsschema, welches sich aus der Termfrequenz und der inversen Dokumentfrequenz zusammensetzt. Die Idee der TFIDF Gewichtung ist, dass ein Merkmal umso relevanter für die Beschreibung eines Dokumentes ist, umso größer seine Termfrequenz. Im Gegenzug ist ein Merkmal umso diskriminierender zwischen Dokumenten, je niedriger seine Dokumentfrequenz, ein Umstand der unter anderem mit Zipf's Law korreliert. Da das TFIDF Gewichtungsschema einfach zu errechnen ist, aber trotzdem eine gute Basis zur Ähnlichkeitsbestimmung zwischen Dokumenten darstellt, wird es in gängigen Information Retrieval Systemen eingesetzt.

Auf Basis der Merkmalsgewichtung erfolgt auch häufig die Selektion von Merkmalen. Merkmale unter einer gewissen Schwellengröße werden aus der Repräsentation eines Dokumentes entfernt. Hintergrund ist die Annahme, dass Merkmale mit geringem, Gewicht wenig Aussagekraft und meist auch wenig Einfluss auf die weitere Verarbeitung haben. Die zielgerichtete Merkmalsselektion kann mitunter entscheidend zur Qualität der Ergebnisse beitragen und ist ein essentieller Vorverarbeitungsschritt.

3.4 Vorverarbeitung von multimedialen Inhalten

Die Vorverarbeitung multimedialer Daten im Rahmen von KD und IR Anwendungen wurde unter anderem durch die Entwicklung von Multimedia Datenbanksystemen getrieben, welche ihren Beginn mit dem im Jahre 1987 entwickelten ORION System nahmen. Multimedia Datenbanksysteme speichern und indizieren multimediale Inhalte und erlauben deren Suchen bzw. einen Zugriff auf diese, wobei die meisten heute gängigen Systeme lediglich auf so genannten content based retrieval (CBR) Methoden aufsetzen, welche vorrangig auf Basis so genannter low-level Merkmale operieren (siehe auch [Kosch 2003] und [Schmitt 2002] für einen Überblick).

Multimedia Systeme gehen von multi-modalen Dokumenten, also Dokumenten, die aus mehreren Modalitäten wie Video, Bild, Audio und auch Text bestehen können, aus. Video wird hierbei als eigene Modalität betrachtet, obwohl eine Auftrennung in Audio und Bildsignal möglich ist. Zur Vorverarbeitung dieser erfolgt im ersten Schritt die *Zerlegung* eines multi-modalen Dokumentes in die einzelnen Modalitäten (z.B. Video, Audio, Bild) und die *Normalisierung* dieser im darauf folgenden zweiten Schritt. Dabei erfolgt die Überführung der einzelnen Modalitäten in eine entsprechende Normalform und die damit verbundene Unterdrückung von Störungen, wie z.B. des Rauschens in einem Audiosignal.

Nach Normalisierung der Daten erfolgt die Segmentierung der Modalität, bei der die Inhalte in semantisch sinnvolle Teile zerlegt werden (ähnlich der Lemmatisierung). Bei der Bildsegmentierung geht es dabei hauptsächlich um die Extraktion von zusammenhängenden Regionen, bei denen Pixel einem entsprechenden Homogenitätskriterium entsprechen. So werden z.B. Regionen mit gleicher Textur, durch Kanten umschlossene Regionen oder Kanten an sich extrahiert. Die Audio-Segmentierung versucht Signale, die entsprechend einem Kriterium zusammenhängen, zu extrahieren. Dies kann z.B. die Identifikation der Frequenzbänder bzw. des Signals von gesprochener Sprache oder aber eines Geräusches sein. Für Video erfolgt meist eine zeitliche Segmentierung, wobei versucht wird sinnvolle, zeitliche Video-Abschnitte mit den entsprechenden Schlüsselszenen (Key Frames) zu identifizieren. So kann z.B. das Video einer Nachrichtensendung automatisch in die einzelnen Beiträge überführt werden.

Für die im vorigen Schritt ermittelten Segmente erfolgt anschließend die Extraktion von Merkmalen, welche, äquivalent zur Vorverarbeitung von Text, Eigenschaften von Dokumenten bzw. Segmenten darstellen. Die Merkmale stellen wiederum die Grundlage für die Anwendung verschiedener KD-Algorithmen und Techniken sowie die Berechnung von Ähnlichkeiten zwischen Objekten dar. Die extrahierten Merkmale sind stark vom Medientyp abhängig, wobei eine Verallgemeinerung auf vektorbasierte und graphbasierte Darstellung im Allgemeinen möglich ist. Für Bildinhalte bzw. für die relevanten Segmente können z.B. Farbhistogramme oder statistische Momente von Texturmerkmalen extrahiert und in entsprechenden Datenstrukturen verwaltet werden. Die Beschreibung von Video-Daten erfolgt beispielsweise anhand von Bewegungsvektoren, welche Bewegung und Geschwindigkeit von Einzelbildsegmenten beschreiben.

Während sich die notwendigen Techniken zur Extraktion von Merkmalen aus multimedialen Inhalten maßgeblich von den Techniken der Textvorverarbeitung unterscheiden, so werden auch diese Inhalte wiederum auf Merkmale zurückgeführt bzw. in einen Merkmalsraum transformiert, auf dem operiert wird. Auf Basis dieses Merkmalsraums erfolgt dann wiederum die Anwendung der unterschiedlichen KD-Techniken, welche auch auf aus Textdaten extrahierten Merkmalen operieren. Unterschiede können hierbei in der zu verwendenden Datenstruktur auftreten, da die Verteilung der Merkmale zwischen den einzelnen Modalitäten sehr unterschiedlich ist.

3.5 Operationen und Ähnlichkeitsmetriken in Merkmalsräumen

Wie in den Abschnitten zuvor erörtert, überführen Vorverarbeitungsmethoden Inhalte der verschiedenen Modalitäten in Merkmale bzw. Merkmalsräume, auf welchen unterschiedliche KD-Techniken angewendet werden können. Bevor nun die einzelnen Techniken näher untersucht werden, sollen zum besseren Verständnis in diesem Abschnitt ein Auszug aus möglichen Repräsentationsformen gegeben und Methoden zur Ermittlung von Distanzen oder Ähnlichkeiten zwischen Inhalten auf Basis der Merkmale erörtert werden. Der Begriff der Ähnlichkeit respektive der Distanz zwischen zwei Objekten ist ein zentraler Aspekt im KD. Die meisten Algorithmen greifen darauf zurück.

Betrachtet man die einzelnen Merkmale, so kann zwischen folgenden Merkmalstypen in Bezug auf den Wertebereich, den sie annehmen können, unterschieden werden ([Duda et. al. 2000, Witten et. al. 2005]):

1. Numerische Merkmale
   Numerische Merkmale haben einen numerischen Wertebereich und weisen somit eine Ordnung in Bezug auf die Merkmale auf. Die grundlegenden algebraischen Operationen sind auf ihnen definiert:
   a. Kontinuierliche Merkmale
      Merkmale dieser Art decken einen kontinuierlichen, meist reellwertigen Wertebereich ab. Beispiele für diese Merkmale sind z.B. die aus dem TFIDF Gewichtungsschema ermittelten Gewichte.
   b. Diskrete Merkmale
      Diskrete Merkmale haben einen diskreten, meist ganzzahligen Wertebereich. RGB Farbwerte eines Bildes können z.B. als solche verstanden werden.
2. Nominale Merkmale
   Nominale Merkmale nehmen Werte aus einer vorgegebenen Menge an möglichen Ausprägungen an. Beispiel hierfür wäre die Zuordnung von Objekten zu n möglichen Kategorien, wie es in einem Klassifikationsschema der Fall ist, oder auch die Repräsentation von binären Merkmalen (z.B. ein Merkmal ist für ein Objekt vorhanden oder nicht). Nominale Merkmale können in diskrete Merkmale überführt werden, wobei jedoch das Ordnungskriterium der diskreten Merkmale und die mathematischen Operationen keine Aussagekraft besitzen.
3. Ordinale Merkmale
   Ordinale Merkmale leiten sich aus nominalen Merkmalen ab, haben also kategoriale Ausprägungen, wobei jedoch eine Ordnungsrelation erhalten bleibt. Ordinale Merkmale können in diskrete Merkmale überführt werden, wobei jedoch die mathematischen Operationen keine Aussagekraft besitzen.

Objekt	Farbe (nominal)	Länge (kontinuierlich)	Testpunkte (diskret)
VW	Rot	5,2	50
Audi	Blau	6,3	40
Mein Auto	Hellblau	5,6	99

Tabelle 3-1: Beispiel für die Ausprägung von Merkmalen für Objekte

Wie bereits oben erwähnt werden die verschiedenen Merkmale in einem Merkmalsraum zusammengefasst. Die Dimensionen des Merkmalsraumes entsprechen dabei den unterschiedlichen Merkmalen und jedes zu

beschreibende Objekt ist in diesem Merkmalsraum eingebettet, indem es Merkmale der entsprechenden Dimension mit der entsprechenden Ausprägung definiert. Tabelle 3-1 zeigt Beispiele für die unterschiedlichen Merkmale von Objekten (in dem Fall Pkws) und spannt dabei den Merkmalsraum Farbe, Länge und Testpunkte auf.

Der Merkmalsraum kann nun unterschiedlich strukturiert sein. Mögliche Ausprägungen von Merkmalsräumen sind Mengen von Vektoren, Sequenzen, Mengen, aber auch Graphen, wobei das Wort Raum hier etwas missleitend sein kann, aber in der Literatur so verwendet wird. Mengen sind wiederum als binäre Vektoren repräsentierbar und werden daher im Folgenden nicht explizit betrachtet. Im Gegensatz zu Graphen werden die einzelnen Merkmale in Vektorräumen als voneinander unabhängig betrachtet. KD-Algorithmen operieren meist auf diesen Räumen, wobei die möglichen Operationen natürlich von der Repräsentationsform abhängen, d.h. Operationen definieren sich als Mengen-, Vektor- oder Graphoperationen (siehe auch [Duda et. al. 2000, Weiss et. al. 2005, Witten et. al. 2005] für nähere Details). Die nachfolgende Nomenklatur wird im Rest des Abschnitts angewendet:

(i) O_i definiert das i-te Objekt im Merkmalsraum

(ii) M definiert den Merkmalsraum und m_j die j-te Dimension

(iii) $o_{i,j}$ definiert das j-te Merkmal des i-ten Element

Die meisten Algorithmen im Bereich KD operieren auf Vektoren, wobei in letzter Zeit auf Graphen basierende Repräsentationsformen immer stärker in den Vordergrund rücken. Etwa im Bereich des Semantic Web, des Web 2.0 und der multimedialen Informationssysteme gewinnen graphenbasierte semantische Beschreibungen digitaler Inhalte und deren Auswertung immer mehr an Bedeutung (siehe z.B. [Lux et. al. 2006], [Beretti et. al. 2004]). Vor diesem Hintergrund wird im Folgenden vor allem auf die Anwendung von durch Graphen und Vektoren repräsentierte Merkmalsräume eingegangen.

Die Operationen zwischen Objekten O_i definieren sich wie oben erwähnt auf Basis der jeweiligen Repräsentationsform. Für Vektorräume sind dies, in Abhängigkeit von den enthaltenen Merkmalen und deren Wertebereich (siehe unten), grundlegend Vektoroperationen oder auch weiterführend Methoden aus dem Bereich der linearen Algebra (e.g. Eigenwertzerlegung, Orthonormalisierung etc.). Für graphenbasierte Repräsentationen sind zusätzlich Graphenoperationen wie z.B. das Finden des minimalen Spannbaums oder aller nicht verbundenen Sub-Graphen zu berücksichtigen. Während die manipulativen Operationen variieren, ist die Ermittlung von Ähnlichkeiten bzw. Distanzen zwischen Objekten über z.B. Clustering oder Klassifikation von zentraler Bedeutung für die meisten KD-Algorithmen. Die Wahl der verschiedenen Ähnlichkeiten ist grundlegend problemabhängig. Durch die Manipulation dieser kann das Ergebnis von KD-Algorithmen beeinflusst und so den Bedürfnissen des Benutzers angepasst werden (siehe dazu auch [Granitzer et. al. 2006]).

Vor dem Hintergrund der zentralen Bedeutung von Ähnlichkeiten respektive Distanzen zwischen Objekten werden im Folgenden die Begriffe Ähnlichkeit und Distanz eingeführt sowie unterschiedliche Metriken im Hinblick auf die verschiedenen Merkmalsarten diskutiert. Für Details zu diesem Thema sei auf [Xu et. al. 2005, Duda et. al. 2000, Weiss et. al. 2005] verwiesen.

Distanzfunktionen bzw. Unähnlichkeiten D sind durch folgende Eigenschaften definiert:

1. Symmetrie: $D(O_i,O_j) = D(O_j,O_i)$
2. Positivität: $D(O_i,O_j)>0$ für alle O_i *und* O_j

Man spricht von einer Metrik, wenn zusätzlich die Dreiecksungleichung und Reflexivität erfüllt sind:

3. Dreiecksungleichung: $D(O_i,O_j) \leq D(O_i,O_k) + (O_k,O_j)$ für alle O_i,O_j *und* O_k
4. Reflexivität: $D(O_i,O_j)=0$ genau dann wenn $O_i=O_j$

Für Ähnlichkeiten S muss ebenfalls Symmetrie und Positivität gewährleistet sein. Zusätzlich ist der Wertebereich von $S(O_i,O_j)$ im Intervall [0,1] beschränkt. Man spricht von einer Ähnlichkeitsmetrik, wenn zusätzlich folgende Bedingungen erfüllt sind:

1. $S(O_i,O_j)\ S(O_i,O_k) \leq [S(O_i,O_j) + S(O_i,O_k)]\ S(O_i,O_k)$ für alle O_i,O_j *und* O_k
2. $D(O_i,O_j)=1$ genau dann wenn $O_i=O_j$

Draus folgt, dass Ähnlichkeiten unter Verwendung der Beziehung $D(O_i,O_j)= 1- S(O_i,O_j)$ relativ einfach in Distanzen überführt werden können. Unter Berücksichtigung der unterschiedlichen Ausprägungen von Merkmalen in einem Merkmalsraum können folgende Merkmalsräume und dazugehörige Metriken unterschieden werden:

Homogene Merkmalsräume enthalten alle Merkmale aus dem gleichen Wertebereich. Sind diese z.B. reellwertiger (kontinuierlicher) Natur, spricht man von einem reellwertigen Vektorraum. Im Allgemeinen ist diese Art von Merkmalsräumen einfach zu bearbeiten, da Operationen für alle Merkmale gleich sind.
Numerische Merkmale sind meist in einem metrischen Raum eingebettet, wobei als Metrik meist die so genannte Minkowski Metrik

$$D_p(O_i,O_j) = \left(\sum_{k=1}^{|M|} \left| O_{i,k} - O_{j,k} \right|^{1/p} \right)^p$$

Anwendung findet. Der Wert p definiert dabei die Art der Metrik wobei $p=1$ der Eins-Metrik, $p=2$ der Euklidschen Metrik und $p=\infty$ der Maximums-Metrik entspricht. Neben diesen "einfachen" Metriken können Ähnlichkeit und Distanz auch über komplexere Zusammenhänge ermittelt werden. Zu diesen zählen z.B. der Winkel zwischen zwei Vektoren - das so genannten Kosinusmaß, die Mahalanobis Distanz, welche Co-Varianzen zwischen den Vektoren berücksichtigt, oder auf informationstheoretischen Maßen basierende Zusammenhänge. Das Kosinusmaß ist vor allem im Umfeld des IR sehr bekannt, da es in Zusammenspiel mit der TFIDF Gewichtung gute Ähnlichkeiten zwischen Textdokumenten liefert und doch einfach zu berechnen ist.

Nominale Merkmale werden meist in Form von binären Vektoren dargestellt (eine Kategorie ist vorhanden oder nicht) und unter Anwendung von binären Metriken wie z.B. der Tanimoto- oder der Hamming-Distanz verglichen (siehe auch [Duda00]). Die Metriken leiten sich grundsätzlich aus binären Kontingenztabellen ab und kombinieren jeweils unterschiedliche Aspekte wie z.B. die Übereinstimmung von Merkmalen oder das Vorhandensein von Merkmalen. Während bei diesen Metriken keine Gewichtung der einzelnen Merkmale möglich ist, ermöglicht die Value Difference Metric (VDM) (siehe [Stanfill et. al. 1986]) die Gewichtung der einzelnen binären Merkmale in Abhängigkeit ihrer Verteilung, wodurch die Ähnlichkeit zwischen Objekten an die Wahrscheinlichkeit des Auftretens der Merkmale angepasst wird.

Ordinale Merkmale können über relativ simple Strategien in binäre Merkmalsvektoren überführt und dann anhand der oben eingeführten binären Metriken miteinander verglichen werden. Neben der Anwendung

binärer Metriken ist auch die Anwendung von kontinuierlichen Metriken möglich (siehe auch [Kaufman et. al. 1990]).

Heterogene Merkmalsräume enthalten Merkmale aus unterschiedlichen Wertebereichen. Tabelle 3-1 zeigt ein Beispiel dafür. Operationen lassen sich hier schwerer definieren als im homogenen Fall, da die unterschiedlichen Eigenschaften von Merkmalen berücksichtigt werden müssen. Im einfachsten Fall erfolgt die Kombination der Merkmale in einen metrischen Raum, was grundsätzlich ohne Informationsverlust möglich ist, und die Anwendung einer einfachen Metrik wie z.B. der euklidschen Metrik (im Beispiel für multi-modales Clustering siehe [Heer et. al. 2001]). Während kein Informationsverlust auftritt, ist die Aussagekraft von z.B. nominalen Merkmalen in einem metrischen Raum bzw. in Kombination mit reellwertigen Merkmalen unklar und semantisch nicht wohl definiert. Die Adaption der Metrik unter diesen Gesichtspunkten stellt sich i. A. als schwierig heraus.

Eine Möglichkeit heterogene Merkmalsräume zu bearbeiten, besteht in der Transformation der unterschiedlichen Merkmale auf eine Merkmalsart. So können z.B. numerische bzw. kontinuierliche Merkmale mittels Diskretisierung in nominale Merkmale überführt und anschließend mittels der entsprechenden Metrik verglichen werden. Mit der Diskretisierung ist natürlich ein Informationsverlust verbunden.

Einen anderen Ansatz bietet die so genannte Heterogeneous Euclidean Overlap Metric (HEOM) [Aha et. al. 1991]. Bei der HEOM erfolgt die Ermittlung der Distanz zwischen nominalen, ordinalen und numerischen Merkmalen unter Berücksichtigung der für die einzelnen Merkmale entsprechenden Metriken (z.B. Hamming Metrik für binären und Euklidsche Metrik für numerische Merkmale) und deren Kombination über eine einfache Linearkombination. Um unterschiedliche Gewichtungen der Merkmale berücksichtigen zu können, erfolgte in [Wilson et. al. 1997] die Erweiterung der HEOM durch die VDM zur Gewichtung der binären Merkmale.

Sequenz basierte Repräsentationen nehmen einen besonderen Stellenwert im Bereich der Textanalyse ein, da Texte bzw. Worte als Sequenzen von Zeichen verstandenen werden können. Im Falle numerischer Merkmale können Sequenzen auch in Vektoren überführt bzw. über Methoden der Signalverarbeitung wie z.B. einer Kreuzkorrelation miteinander verglichen werden. Vor allem im Bereich der Zeichenkettenvergleiche, welche Sequenzen von nominalen Merkmalen (=Zeichen) darstellen, nehmen sequenzbasierte Ähnlichkeitsmetriken einen wichtigen Platz ein. So können z.B. falsch geschriebene Wörter über so genannte Edit Distanzen (siehe auch [Duda00]) verglichen werden, wodurch sich die Möglichkeit der Korrektur von Suchbegriffen über ein dahinter liegendes Wörterbuch ergibt. Die Edit Distanz berechnet dabei die Ähnlichkeit zwischen zwei Zeichenketten über die Anzahl der Transformationen, die notwendig sind, um eine Zeichenkette in eine andere umzuwandeln.

Graphenbasierte Repräsentationen gewinnen mit dem Einzug des Semantic Web und Web 2.0 immer stärker an Bedeutung. Folksonomien, Ontologien und soziale Netzwerke sind grundsätzlich als Graphen repräsentiert. Knoten eines Graphen können dabei als Merkmale verstanden werden, welche über Kanten mit anderen Merkmalen verbunden sind. Zum Vergleich von Graphen existieren verschiedene Ansätze wie z.B. die Graph Edit Distanz, welche der Edit Distanz von Zeichenketten nachempfunden ist (siehe [Bunke 2000, Beretti et. al. 2004]). Die Distanz zwischen zwei Graphen wird durch die minimale Anzahl an nötigen Transformationen zur Überführung des einen Graphen in den anderen Graphen definiert. Weitere Metriken sind z.B. der kleinste oder größte übereinstimmende Subgraph oder die Überführung von Graphen in eine

Vektorrepräsentation, welche sich vor allem für das Retrieval von Graphenstrukturen eignet (siehe dazu [Bunke 2000, Lux et. al. 2006])

Neben der Berücksichtigung der unterschiedlichen Struktureigenschaften bieten die so genannten Kernel-Methoden für viele Algorithmen wie z.B. Support Vector Maschinen (siehe Abschnitt 4.3.1.1) einen effizienten Weg unterschiedliche Repräsentationsarten zu vereinheitlichen. Kernel-Methoden gehen dabei von einem so genannten Hilbert Raum aus, einem Vektorraum, in dem zusätzlich das innere Produkt zwischen Vektoren definiert ist. Das innere Produkt zweier Vektoren entspricht dabei der Ähnlichkeit zwischen den Vektoren. Im normierten Raum eines Vektorraummodells würde damit die Ähnlichkeit z.B. dem Kosinusmaß entsprechen. Da meist die Auswertung linearer Zusammenhänge nicht ausreicht, können Vektorräume transformiert werden. Eine effiziente Möglichkeit dazu bieten Kernel-Methoden, da hierbei die Transformation nicht explizit berechnet werden muss. Ein Kernel ist definiert als

$$K(O_i, O_j) = < \Phi(O_i), \Phi(O_j) >$$

wobei Φ die Transformation und $<\cdot,\cdot>$ das innere Produkt darstellt. Abbildung 3-3 zeigt eine solche Transformation eines 2D-Raumes in einen 3D-Raum unter Anwendung eines polynomiellen Kernel.

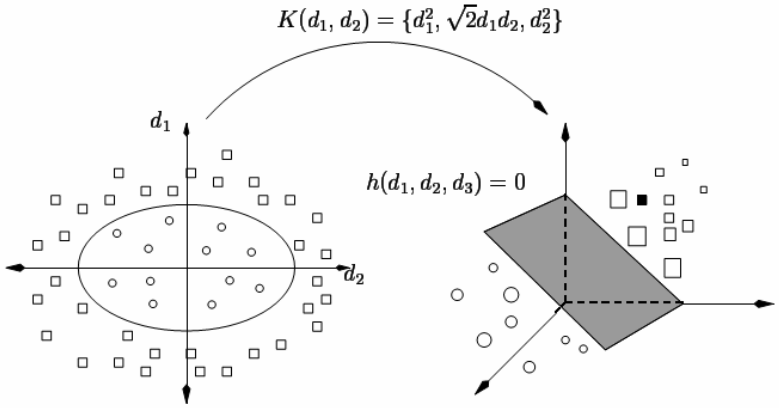


Abbildung 3-3: Kernel Transformation eines 2D Raumes in einen 3D Raum unter Anwendung eines polynomiellen Kernel.

Wesentlich daran ist, dass statt der expliziten Durchführung der Transformation von Objekten die Kernel Funktion zur Berechnung des inneren Produktes im transformierten Merkmalsraum heran gezogen werden kann (für eine Einführung siehe z.B. [Müller et. al. 2001]). Die Kernel-Funktion muss gewisse Bedingungen, die so genannten Mercer-Eigenschaften inklusive der Eigenschaften für Ähnlichkeitsmetriken erfüllen, kann aber ansonsten frei gewählt werden. Eine weitere daraus resultierende Eigenschaft von Kernel-Funktionen ist, dass diese in verschiedenen Fällen durch das innere Produkt im Ursprungsraum ausgedrückt werden können. Ein polynomineller Kernel der Ordnung *d* lässt sich durch den Ursprungskernel beispielsweise wie folgt ausdrücken

$$K_{poly}(O_i, O_j) = K(O_i, O_j)^d$$

und ist damit lediglich durch den Ursprungskernel beschreibbar. Die fast freie Wählbarkeit der Kernelfunktionen zeigt auch die Mächtigkeit dieses Konzeptes in Bezug auf eine flexible Gestaltung von Ähnlichkeitsmetriken. So wurden in den vergangenen Jahren Kernels für den Vergleich von Sequenzen,

Bäumen und auch Graphen entwickelt (siehe dazu [Gärtner et. al. 2003]) und erfolgreich angewandt. Damit decken Kernel-Funktionen einen Großteil der möglichen Datenstrukturen und Algorithmen ab und dienen daher auch als zentrales Element im Einsatz in KD-Anwendungen.

3.6 Anforderungen an das KnowMiner-Framework

Aus den Betrachtungen zur Vorverarbeitung von unstrukturierten Daten lassen sich verschiedene Anforderungen an das KnowMiner-Framework ableiten.

Die Anbindung unterschiedlicher Informationsarten soll die Möglichkeit der Verarbeitung verschiedener Modalitäten und deren Kombination bieten. Da die Vorverarbeitung der einzelnen Modalitäten die gleichen Ziele, nämlich die Extraktion von Merkmalen verfolgen, kann eine Vereinheitlichung der unterschiedlichen Modalitäten geschaffen werden.

Flexible und Austauschbare Vorverarbeitungsmodule für die Verarbeitung der unterschiedlichen Modalitäten werden benötigt. Da die verschiedenen Modalitäten von Objekten grundsätzlich durch Merkmale beschrieben werden, muss dieser Transformationsschritt aus austauschbaren Modulen bestehen, welche flexibel miteinander kombiniert werden können. Pipeline-Architekturen eignen sich hierfür hervorragend, da für die Vorverarbeitung von Modalitäten lediglich Einzelobjekte berücksichtigt werden müssen.

Flexible Datenstrukturen zur Abbildung von Merkmalen von Objekten. Aus den unterschiedlichen Modalitäten leiten sich unterschiedliche Anforderungen an die Verwaltung der Merkmale ab. So benötigten beispielsweise Textdaten effiziente Möglichkeiten, um „sparse" Vektoren zu verwalten, während Bilddaten meist als „dense" Vektoren abgelegt werden.

Abstraktion von Ähnlichkeiten und Operationen auf Merkmalen: Wie Abschnitt 3.5 gezeigt hat, ist die Abstraktion von Operationen, insbesondere von Ähnlichkeits- und Distanzberechnungen, essentiell für die Gewährleistung der Austauschbarkeit von Algorithmen und Daten. Durch Abstraktion kann somit die Wiederverwendbarkeit und Robustheit erhöht werden.

4 KD und IR Algorithmen und Methoden

Dieses Kapitel umreißt für Anwendungen im Bereich Wissensmanagement die wichtigsten Methoden und Algorithmen aus den Bereichen Knowledge Discovery und Information Retrieval, welche im KnowMiner-Framework umgesetzt bzw. angedacht wurden. Die Auswahl erfolgte einerseits nach der Notwendigkeit der Anwendungsszenarien und andererseits nach den heutzutage wichtigsten Techniken in diesem Bereich[21].

Der Abschnitt beleuchtet dabei auf einem relativ abstrakten Niveau den Stand der Wissenschaft und leitet daraus Anforderungen für das KnowMiner-Framework ab. Während hier die Techniken relativ unabhängig voneinander betrachtet werden, erfolgt in der Anwendung meist eine Kombination der verschiedenen Verfahren. Zum Beispiel können Information Retrieval-Techniken zum schnellen Finden von Informationen dienen, während Clustering-Techniken aus den gefundenen Ergebnissen die wichtigsten Themen extrahieren.

Die hier vorgestellten Techniken operieren meist auf einer Menge von Objekten, für die mittels der in Abschnitt 3 vorgestellten Techniken Merkmale extrahiert wurden. Teile der Techniken können auch zur Entwicklung von Vorverarbeitungsalgorithmen angewendet werden. Im speziellen die beschriebenen überwachten Lernverfahren eignen sich z.B. zum Extrahieren von benannten Entitäten aus Texten. In den entsprechenden Kapiteln wird auf diesen Umstand explizit hingewiesen.

Vorgestellt werden Techniken aus den Bereichen der Merkmalstransformation, des Information Retrieval, der überwachten und unüberwachten Lerntechniken sowie der Informationsvisualisierung. Bevor die daraus abgeleiteten Anorderungen beschrieben werden, wird noch kurz ein Blick in Richtung Ontology Learning geworfen, welches die meisten der hier vorgestellten Techniken zur semantischen Anreicherung von Inhalten kombiniert.

4.1 Merkmalstransformation und Ordination

Wie in Abschnitt 3.3 beschrieben extrahiert die Vorverarbeitung Merkmale aus den jeweiligen Modalitäten, welche gewichtet und entsprechend der Gewichte selektiert werden. Merkmalsselektion ist notwendig, um die Dimensionalität der betrachteten Problemstellung sinnvoll einzuschränken und fehlerhafte, nicht informative und verrauschte Merkmale zu entfernen. Dadurch erhöhen sich bei den meisten Algorithmen die Genauigkeit und auch die Verarbeitungsgeschwindigkeit. Grund dafür ist, dass einerseits Berechnungen in höherdimensionalen Räumen länger dauern und andererseits höherdimensionale Räume mehr Freiheitsgrade und damit auch mehr Beispiele benötigen, um statistisch signifikante Aussagen treffen zu können. Dieser Umstand ist auch als Curse of Dimensionality bekannt (siehe [Duda et. al. 2000]). Merkmalstransformationen werden oft auch als Dimensionsreduktionsmethoden (engl. Dimensionality Reduction) bezeichnet.

Die Merkmalsselektion schneidet, wie oben beschrieben, lediglich Merkmale anhand eines spezifizierten Kriteriums ab und berücksichtigt dabei keine Korrelationen zwischen Merkmalen. Insbesondere im Textbereich entspricht dies nicht der Realität.. Worte formen meistens die Merkmale von Textobjekten und diese als unabhängig voneinander zu betrachten entspricht nicht der Realität.

[21] http://www.kdnuggets.com/polls/2006/text_mining_tasks.htm

Merkmalstransformationen bilden nun eine Gruppe von Methoden, welche Merkmale aus einem Merkmalsraum in einen anderen Merkmalsraum transformieren, wobei die Dimensionen des neu entstandenen Raumes meist aus einer Linearkombination der Methoden aus dem Ursprungsraum bestehen. Da meist Transformationen von Merkmalsvektoren betrachtet werden, beschränken sich die betrachteten Methoden auch auf diese Repräsentationsform. Es sind aber auch durchaus Ansätze denkbar, welche Vektorräume in Graphen umwandeln. Über Co-Occurrence basierte Verfahren wäre dies z.B. möglich, indem jene Merkmale miteinander verbunden werden, welche auf Basis einer statistischen Kenzahl am häufigsten miteinander vorkommen.

Techniken zur Ordination von Merkmalen können als Untergruppe von Merkmalstransformationen mit dem Ziel der Projektion in einen zwei- bzw. dreidimensionalen Raum zur Visualisierung von hochdimensionalen Merkmalsvektoren angesehen werden. Ordinationstechniken, im speziellen die multidimensionale Skalierung, dienen als Grundlage für Visualisierungstechniken und zur Darstellung von hochdimensionalen Zusammenhängen in visualisierungsfähigen Dimensionen.

Einen detaillierten Überblick über die hier vorgestellten Methoden gibt [Fodor 2002].

4.1.1 Latent Semantic Analysis/Indexing

Latent Semantic Analysis bzw. die synonyme Bezeichnung Latent Semantic Indexing ([Deerwester et. al. 1990]) bezeichnet eine Technik zur Transformation von Merkmalsvektoren einer Menge von Objekten in einen orthogonalen Unterraum. Wie bereits oben erwähnt, stimmt die Annahme von voneinander unabhängigen Merkmalen, wie sie z.B. im Vektorraummodell getroffen wird, nicht. LSA findet nun mittels linearer Algebra voneinander linear unabhängige Dimensionen, welche sich als Linearkombination aus den ursprünglichen Merkmalen bilden. Ausgehend von der Objekt-Merkmalsmatrix $\boldsymbol{O}$ (im Retrieval auch als Dokument-Term Matrix bezeichnet), in der die Merkmalsvektoren der Objekte jeweils einer Zeile entsprechen, erfolgt im ersten Schritt eine Singulärwertzerlegung

$$O_{n\times m} = S_{nxl} V_{lxl} D_{l\times m}$$

Wobei l dem Rang der Matrix $\boldsymbol{O}$ entspricht. Die Diagonalmatrix $\boldsymbol{V}$ enthält dabei die Eigenwerte der Objekt-Merkmalsmatrix $\boldsymbol{O}$ in ihrer Hauptdiagonale und $\boldsymbol{S}$ bzw. $\boldsymbol{D}$ sind orthonormale Matrizen, welche die Links- bzw. Rechtssingulärvektoren enthalten. Im Falle das $n=m$ ist entspricht die Singulärwertzerlegung einer Eigenwertzerlegung der Matrix $\boldsymbol{O}$.

Im Anschluss an die Singulärwertzerlegung erfolgt die Reduktion der Matrizen $\boldsymbol{V}$ auf die k größten Eigenvektoren, da diese den Dimensionen mit der größten Varianz und damit dem höchsten Informationsgehalt entsprechen. Die Matrizen $\boldsymbol{S}$ und $\boldsymbol{D}$ werden entsprechend beschnitten, wobei einfach die den gelöschten Eigenwerten entsprechenden Spalten und Zeilen entfernt werden. Über die Transformation

$$O^{reduced} = V_{kxk} D_{k\times m} O_{nxm}{}^{T}$$

ergibt sich der transformierte Vektorraum mit Dimensionalität k bzw. die transformierte Objekt-Merkmal Matrix. LSI bzw. LSA können auch über Kernel Funktion, so genannte Latent Semantic Kernels (LSK, siehe [Cristianini et. al. 2002]) realisiert werden. Erweiterungen von LSI ist z.B. die so genannten Independent Component Analysis, welche auch statistische Momente höherer Ordnung berücksichtigt (siehe [Duda et. al. 2000]). Abbildung 4-1 zeigt schematisch die beschriebene SVD Zerlegung und Reduktion.

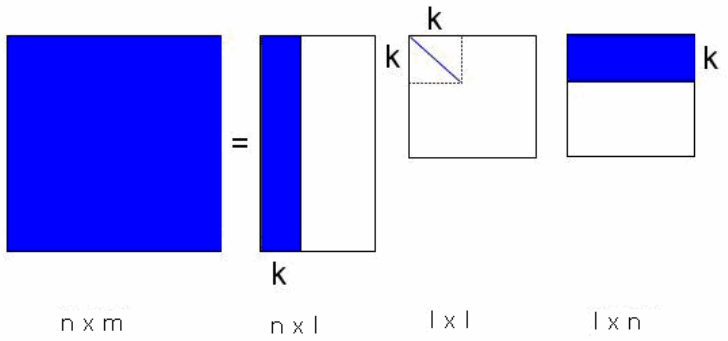


Abbildung 4-1: Schematische Abbildung der LSI Transformation

Angewendet werden LSA bzw. LSI um die in einem Dokumentkorpus enthaltenen impliziten Konzepte auf Basis der häufig in Dokumenten gemeinsam vorkommenden Merkmale zu extrahieren. Bei der Anwendung in Textkorpora können so synonyme und polyseme Begriffe behandelt werden. So können z.B. Linux und Windows über das Konzept als Betriebssysteme erkannt werden, obwohl Linux und Windows nie gemeinsam miteinander in einem Dokument vorkommen. Zusätzlich reduziert LSI die Größe der Objekt-Merkmalmatrix beträchtlich. Experimente haben gezeigt, dass eine Dimensionsreduktion von 10.000 auf 200-300 unter Steigerung der Qualität möglich ist.

Während die Qualität der Repräsentation von Objekten unter Anwendung von LSI im Allgemeinen steigt und fehlerhafte, rauschbehaftete Merkmale eliminiert werden, so ist die Anwendung von LSI rechenintensiv und es ist nicht immer garantiert, dass eine sinnvolle Reduktion gefunden werden kann. Auch muss beim Retrieval von Information berücksichtigt werden, dass es sich dabei um eine Ähnlichkeitssuche handelt, die eine lineare Suchzeit hat. Der Umstand resultiert daraus, dass LSI eine gepackte (engl. Dense) Vektorrepräsentation erzeugt und somit Zugriffe nicht mehr über invertierte Indizes möglich sind.

4.1.2 Multi Dimensionale Skalierung

Multidimensional Scaling (MDS) transformiert Objekte auf Basis ihrer Ähnlichkeit oder Distanz im hochdimensionalen Raum. Die Transformation erfolgt unter dem Kriterium, dass die Differenz zwischen den Ähnlichkeiten respektive Distanzen im niedrig(er) dimensionalen Zielraum und jenen im Ursprungsraum minimiert wird. Da eine Beibehaltung der Ähnlichkeiten im Allgemeinen nicht möglich ist, muss die so genannte Stress-Funktion die Güte der Transformation und damit verbunden ein mögliches Abbruchkriterium für die Transformation angeben. Der Stress zur Minimierung des Fehlers der Distanzabweichung wird als quadratischer Fehler der Distanzen zwischen niedrigdimensionalen und hochdimensionalen Distanzen wie folgt angegeben:

$$stress^2 = \frac{\sum_{i,j} (D_{i,j}^{high} - D_{i,j}^{low})^2}{\sum_{i,j} D_{i,j}^{high\,2}}$$

Dabei gibt $D_{i,j}$ die Distanz zwischen den Objekten i und j an und das Superskript high/low steht für den Ursprungs-/Zielraum.

Mit MDS können jegliche Objektmengen, welche über ein Ähnlichkeits- oder Distanzmaß miteinander vergleichbar sind, analysiert und bei Transformationen in 2D und 3D visuell dargestellt werden. Durch die Anpassung des Ähnlichkeits- bzw. Distanzmaßes ist ein Zuschneiden der Transformation auch auf die Bedürfnisse des Benutzers möglich. Da die Transformation stark von den Maßen abhängt, ist oft keine einfache Interpretation der Ergebnisse möglich. MDS zeigt eine hohe Zeitkomplexität, wobei kürzlich entwickelte Ansätze über verschiedene nichtlineare Methoden Möglichkeiten zur Transformation von großen Datenmengen aufzeigen. Auch Sampling-Methoden ermöglichen eine effizientere Berechnung der Transformation auf Kosten der Genauigkeit.

Im Rest dieses Abschnitts erfolgt die Darstellung von zwei wichtigen Methoden zur Berechnung der Transformation. Force-Directed-Placement ist ein iteratives Verfahren, welches auf Basis eines an die Physik angelehnten Modells von Kräften und Gegenkräften, die den Ähnlichkeiten resp. Distanzen entsprechen, die MDS ausführt. FastMap [Faloutsos et. al. 1995] ermittelt über eine Heuristik die Dimensionen des Zielraumes und projiziert den Ursprungsraum über räumliche Transformationen direkt in den Zielraum. Während FastMap linear zur Anzahl der Dimensionen des Zielraumes und zu transformierenden Objekte ist, liefert es schlechtere Ergebnisse als der FDP Algorithmus. Ein Vergleich der verschiedenen MDS Verfahren präsentiert sich in [Platt 2005].

4.1.2.1 Force Directed Placement

Force Directed Placement (kurz *FDP*) stellt eine iterative Methode zur Transformation dar. Über die Ähnlichkeiten/Distanzen zwischen Objekten im Ursprungsraum und Zielraum ermittelt das FDP-Verfahren anstoßende und abstoßende Kräfte zwischen den Objekten, so dass sich ähnliche Objekte dabei anziehen und unähnliche Objekte abstoßen. Beginnend mit einer zufälligen Positionierung der Objekte im niedrig dimensionalen Raum erfolgt in jeder Iteration die Verschiebung der Objekte anhand der berechneten Kräfte bis das Abbruchskriterium, meist die Minimierung der Stressfunktion, erreicht ist (siehe auch [Sabol et. al. 2001, Andrews et. al. 2002, Granitzer & Auer 2005). Eine einfache Berechnung der Kräfte zwischen zwei Objekten zeigt die nachstehende Formel:

$$force(O_i, O_j) = D_{low}(O_i, O_j) - D_{high}(O_i, O_j)$$

Dabei gibt D_{low} die Distanz zwischen zwei Objekten im niedrigdimensionalen und D_{high} die Distanz im hochdimensionalen an. Dies ist ein einfaches Kraftmodell, welches verschiedene Nachteile wie z.B. das Zusammenhalten sehr unähnlicher Objekte und wenig Parametrisierbarkeit hat. Unter Verwendung anderer Kraftmodelle können diese Nachteile jedoch ausgemerzt werden (siehe dazu [Andrews et. al. 2002]). Die Laufzeitkomplexität von FDP ist i.A. $O(n^3)$, mit n der Anzahl der Objekte, wobei mittels Sampling und Anwendung nichtlinearer Methoden eine Laufzeitkomplexität von $O(n \log n)$ erreicht werden kann.

4.1.2.2 FastMap

FastMap [Faloutsos et. al. 1995] basiert im Gegensatz zum FDP auf einer relativ strikten und simplen Heuristik zur direkten Projektion von Objekten aus einem n-dimensionalen Ursprungsraum in einen k-dimensionalen Zielraum ($k<n$). Während die Heuristik kaum beeinflussbar ist, liegt der Vorteil von FastMap in der schnellen Laufzeitkomplexität von $O(kn)$, also linear in der Anzahl der Zieldimensionen und Objekte.

Der FastMap-Algorithmus startet mit der Suche nach zwei Pivot-Objekten, welche im Ursprungsraum soweit wie möglich voneinander entfernt sind. Da diese Suche $O(n^2)$ dauert, verwendet FastMap in diesem Schritt eine Heuristik. Zuerst erfolgt die Wahl eines zufälligen Objektes O_0, zu welchem das am weitesten entfernte

Objekt O_a gesucht wird. Ausgehend vom Objekt O_a erfolgt die Suche des zu O_a am weitesten entfernten Objekts O_b. O_a und O_b definieren die Pivot-Objekte für den nächsten Schritt.

Über Anwendung des Kosinussatzes und unter Berücksichtigung der von den Pivot-Elementen definierten n-dimensionalen Geraden G erfolgt die Projektion der verbleibenden Objekte auf die Gerade G. Bei einer Einbettung in einen k-dimensionalen Raum wird dies für jede Dimension einmal durchgeführt, also insgesamt k mal, wodurch sich die Laufzeitschranke $O(nk)$ ergibt.

FastMap ist damit eines der schnellsten Verfahren zur Dimensionsreduktion. Durch die verwendeten Heuristiken verschlechtern sich die Ergebnisse von FastMap hinsichtlich Qualität und eine Adaption des Layouts ist ebenfalls nur schwer möglich.

4.1.3 Sonstige

Dieser Abschnitt geht der Vollständigkeit halber kurz auf weitere wichtige Transformationsverfahren ein und illustriert diese.

4.1.3.1 Principal Component Analysis

Die Vorgehensweise bei der Principal Component Analysis (PCA) ist äquivalent zu LSI, lediglich die Ausgangsmatrix, und damit verbunden die anwendbaren linearen Algebra-Techniken, unterscheiden sich. Ausgangspunkt der PCA ist die Kovarianzmatrix der Merkmale des zu untersuchenden Merkmalsraums, wobei diese über

$$K = O^T O / n$$

ermittelt wird. $\boldsymbol{O}$ stellt wiederum die Objekt-Merkmal-Matrix dar. Durch Berechnung der Eigenwerte und der zu den Eigenwerten gehörigen Eigenvektoren, welche wiederum durch Kombination der Ursprungsvariablen gebildet werden, erfolgt die Ermittlung der orthogonalen Dimensionen des transformierten Raumes, von dem wiederum nur die k-besten, entsprechend der größten Eigenwerte, behalten werden. Die größten Eigenwerte entsprechen wiederum den Richtungen mit der größten Varianz und damit den größten Eigenwerten. Bei einer Verteilung der Merkmale mit annähernd ähnlicher Varianz liefert die PCA daher keine brauchbaren Ergebnisse.

Durch die PCA lässt sich der Merkmalsraum wiederum reduzieren und anhand der ermittelten Dimensionen repräsentieren. Voneinander abhängige Dimensionen werden unter Erhaltung des größtmöglichen Informationsgehaltes (im Sinne des quadratischen Fehlers) in unabhängige Dimensionen transferiert und können so analysiert und ggf. in 2D bzw. 3D visualisiert werden.

4.1.3.2 Factor Analysis (FA):

Faktor Analysis ist eine lineare multivariate Explorationstechnik, welche Momente zweiter Ordnung berücksichtigt. Eingesetzt wird sie einerseits zur Reduktion von Variablen oder zum Entdecken von Relationen zwischen Merkmalen. Im Gegensatz zu den anderen Methoden sind die ermittelten Faktoren meist interpretierbar. FA geht bei der Analyse von verstecken Merkmalen aus, von denen die messbaren Merkmale abhängen. Ziel der FA ist es nun, über die versteckten Merkmale Beziehungen zu finden und somit die Dimension des Datensatzes zu reduzieren.

4.1.3.3 Self-Organizing Maps (SOMs)

SOMs basieren auf unüberwachten Neuronalen Netzwerken und dienen dazu hochdimensionale Zusammenhänge darzustellen. Während der iterativen Lernphase modifiziert der SOM-Algorithmus die Gewichtsvektoren des Zielraumes (Output Layer) auf Basis der Vektoren im Ursprungsraum (Input Layer), wodurch niedrigdimensionale (meist 2 dimensional) Räume erzeugt werden. SOMs bieten einen viel versprechenden Ansatz zur Organisation von großen Dokumentbeständen, wobei jedoch die nur schwer mögliche Adaption der Metrik, das fehlende theoretische Grundgerüst sowie die meist schwierig einzustellende Lernrate Probleme in der Anwendung bereiten. Es existiert auch bis dato kein Konvergenzbeweis für SOM's und das von SOM's approximierte Modell entspricht auch keiner Wahrscheinlichkeits-Dichtefunktion.

4.1.3.4 Random Projections (RP)

Random Projections (RP) basieren auf dem Johnson and Lindenstrauss Lemma, welches besagt, dass jede Menge von n Punkten im d-dimensionalen Euklidschen Raum über eine beliebige lineare Transformation in einen q-dimensionalen Raum übertragen werden kann, wobei q in etwa logarithmisch von n abhängt und die Distanzen ungefähr erhalten bleiben. Die Konstruktion der linearen Transformation erfolgt zufällig über Multiplikation der Objekt-Merkmalsmatrix des Ursprungsraumes mit q d-dimensionalen Zufallsvektoren. Die Zeitkomplexität von RP's ist im Vergleich zu den anderen Transformationsmethoden gering, da RP's linear in der Anzahl der Beispiele und der Dimension operieren ($O(nk)$), während z.B. PCA eine Zeitkomplexität von $O(n*p^2)+O(p^3)$ hat (n entspricht der Anzahl der Beispiele). Wie z.B. in [Tang et. al. 2005] gezeigt wurde, ist jedoch die Genauigkeit der Projektion geringer bzw. muss die Dimension des Zielraumes q größer gewählt werden als z.B. bei LSI.

4.1.3.5 Independent Component Analysis (ICA)

ICA ist eine Methode, welche Projektionen ähnlich wie LSI durchführt. Im Gegensatz zu LSI sind die Dimensionen des Zielraums nicht notwendigerweise orthogonal, sondern statistisch voneinander unabhängig. Statistische Unabhängigkeit ist dabei eine stärkere Bedingung als es orthogonale Dimensionen sind und daher ist ICA auch eine Methode höherer Ordnung. Wenngleich die Ergebnisse von ICA im Allgemeinen qualitativ besser als jene von LSI sind (siehe auch [Tang et. al. 2005]), so ist die Zeitkomplexität doch höher und daher fällt die Anwendung auf großen Datenmengen schwer.

4.2 Information Retrieval

Information Retrieval stellt im heutigen Zeitalter eine zentrale Technologie speziell im Umgang mit textueller Information dar. Während IR nicht direkt als KD-Technik angesehen wird, so ist IR doch unabkömmlich, wenn es um die Verwaltung großer Datenmengen geht. Dieser Abschnitt beleuchtet kurz die zentralen Techniken und stellt auch die Unterschiede zum Datenretrieval heraus, wobei auf vieles des schon Gesagten zurückgegriffen wird. IR umfasst normalerweise auch noch angrenzende Gebiete wie Clustering oder Klassifikation, welche jedoch in der folgenden Betrachtung außen vor gelassen werden und im Detail in den Abschnitten 4.3 und 4.4 ausgeführt werden.

Die ursprüngliche Definition von Information Retrieval geht auf Salton 1968 zurück und definiert Information Retrieval als

> *"…field concerned with the structure, analysis, organization, storage, searching, and retrieval of information [*Salton 1968*]. "*

Eine etwas detailliertere und weniger generische Definition von der Fachgruppe für Information Retrieval der Deutschen Gesellschaft für Informatik lautet:

> *"Im Information Retrieval (IR) werden Informationssysteme in Bezug auf ihre Rolle im Prozess des Wissenstransfers vom menschlichen Wissensproduzenten zum Informations-Nachfragenden betrachtet. Die Fachgruppe "Information Retrieval" in der Gesellschaft für Informatik beschäftigt sich dabei schwerpunktmäßig mit jenen Fragestellungen, die im Zusammenhang mit vagen Anfragen und unsicherem Wissen entstehen. Vage Anfragen sind dadurch gekennzeichnet, dass die Antwort a priori nicht eindeutig definiert ist. Hierzu zählen neben Fragen mit unscharfen Kriterien insbesondere auch solche, die nur im Dialog iterativ durch Umformulierung (in Abhängigkeit von den bisherigen Systemantworten) beantwortet werden können; häufig müssen zudem mehrere Datenbasen zur Beantwortung einer einzelnen Anfrage durchsucht werden. Die Darstellungsform des in einem IR-System gespeicherten Wissens ist im Prinzip nicht beschränkt (z.B. Texte, multimediale Dokumente, Fakten, Regeln, semantische Netze). Die Unsicherheit (oder die Unvollständigkeit) dieses Wissens resultiert meist aus der begrenzten Repräsentation von dessen Semantik (z.B. bei Texten oder multimedialen Dokumenten); darüber hinaus werden auch solche Anwendungen betrachtet, bei denen die gespeicherten Daten selbst unsicher oder unvollständig sind (wie z.B. bei vielen technisch-wissenschaftlichen Datensammlungen). Aus dieser Problematik ergibt sich die Notwendigkeit zur Bewertung der Qualität der Antworten eines Informationssystems, wobei in einem weiteren Sinne die Effektivität des Systems in Bezug auf die Unterstützung des Benutzers bei der Lösung seines Anwendungsproblems beurteilt werden sollte."*

Zentral im Information Retrieval ist der Umgang mit Vagheit, Unsicherheit und Unvollständigkeit, sowohl auf Benutzer- als auch auf Technologieseite. Abbildung 4-2 zeigt das konzeptionelle Modell für ein IR-System nach [Fuhr 2006]. Fuhr definiert dabei drei Sichten auf Dokumente resp. Objekte, nämlich

1. die Layout-Sicht, welche die Darstellung eines Dokumentes beschreibt;
2. die logische Sicht, welche die logische Struktur und den Inhalt eines Dokumentes umfasst;
3. die semantische Sicht, welche die Semantik des Inhaltes eines Dokumentes beschreibt.

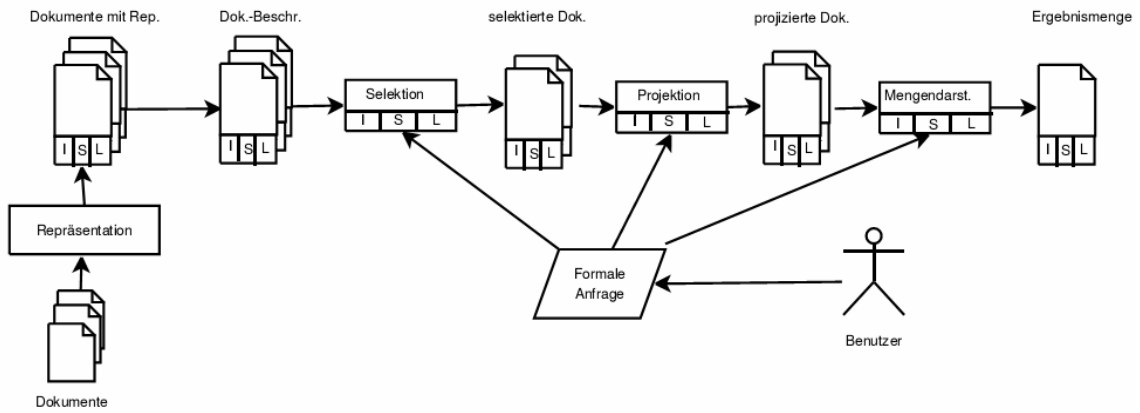


Abbildung 4-2: Konzeptuelles Modell eines IR Systems entnommen aus [Fuhr 2006]

Diese Sicht kann auch auf die in Abschnitt 3.1 eingeführten Schritte zur Vorverarbeitung von Dokumenten angewandt werden, wobei im folgenden Abschnitt Dokumente synonym zu Objekten in den vorigen Abschnitten betrachtet werden. Aus der Formatnormalisierung erhält man die logische Sicht der Dokumente, aus der Merkmalsanalyse und Merkmalsgenerierung die semantische Sicht. Diese Schritte der Vorverarbeitung

zeigt auch Abbildung 4-2 links. Dabei erfolgen die Extraktion der verschiedenen Sichtweisen aus den Dokumenten und die Überführung in eine interne Repräsentation. Die Dokument-Beschreibung verdichtet die Repräsentation in Hinblick auf das Anwendungsszenario bzw. die folgenden Suchanfragen. Beispielsweise sind Layout-Eigenschaften in den meisten IR-Anwendungen zu vernachlässigen und müssen daher nicht notwendigerweise in der Dokument Beschreibung enthalten sein. Die Dokument-Beschreibung muss in hoch optimierten Datenstrukturen, je nach Sichtweise und Repräsentation der Sichtweise, gehalten werden, um ein schnelles Suchen und Selektieren von Dokumenten zu ermöglichen. Für aus Textdokumenten extrahierte Merkmale erfolgt meist die Anwendung eines invertierten Index. Konzeptuell kann man sich einen invertierten Index wie eine Liste von, meist sortierten, Merkmalen vorstellen, in der jedes Merkmal auf eine Liste von Dokumenten zeigt, in welchen es enthalten ist (siehe Abbildung 4-3).

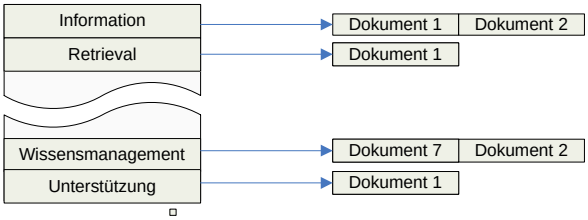


Abbildung 4-3: Beispiel für einen invertierten Index mit Worten als Merkmalen. Links ist die sortierte Liste an Merkmalen, welche auf eine Liste von Dokumenten in denen die Merkmale vorkommen zeigt.

Die Suche in einer sortierten Merkmalsliste ist in O(log m), mit m die Anzahl der Merkmale, möglich. Hat man die Merkmale gefunden, so können die zugehörigen Dokumente über den Zeiger auf die Dokumentliste leicht ausgelesen und präsentiert werden.

Die Dokument-Beschreibung stellt die Grundlage für die Suche dar. Benutzer formulieren ihren Informationswunsch in einer formalen Anfrage (z.B. durch Eingabe von Wörtern, die in einem Dokument enthalten sein sollen), welche zur Selektion der Dokument-Beschreibungen heran gezogen wird.

Im nächsten Schritt erfolgt die Projektion der selektierten Dokumente, welche dem Benutzer in der Ergebnisdarstellung präsentiert wird. Die Projektion kann dabei wiederum aus den verschiedenen Sichtweisen erzeugt werden. Neben inhaltlichen Zusammenfassungen von Treffern, welche aus der logischen und semantischen Sicht extrahiert werden, erfolgt z.B. die Generation von Thumbnails aus der Layoutsicht. Vagheit, Unsicherheit und Unvollständigkeit entstehen nun auf verschiedenen Ebenen in diesem Modell.

Auf Seiten des Benutzers ist meist das Informationsbedürfnis des Benutzers vage formuliert, da in den meisten Fällen (i) der Benutzer nicht exakt weiß, wonach er sucht und (ii) nicht weiß, wie er es formulieren soll. Die formale Anfrage beinhaltet daher immer einen Grad an Unsicherheit aus (ii) und muss auch die Möglichkeit zur Formulierung von Vagheit, z.B. über Gewichtung der Suchterme, bieten.

Die Überführung der Dokumente in die interne Repräsentation und in die verschiedenen Sichtweisen beinhaltet immer einen hohen Grad an Unsicherheit, vor allem bei der Verarbeitung von Text- und Multimediadokumenten. Zum Beispiel ist die Auflösung von Synonymen immer mit einem Fehler behaftet und da im Vorhinein die möglichen Anfragen nicht bekannt sind, ist eine Wahl der ideale Repräsentation nicht möglich.

Zentral für den Benutzer ist daher ein iterativer Suchprozess, in dem das spezifizierte Informationsbedürfnis auf Basis der gefundenen Dokumente iterativ verfeinert wird (siehe auch [Rijsbergen 2004]).

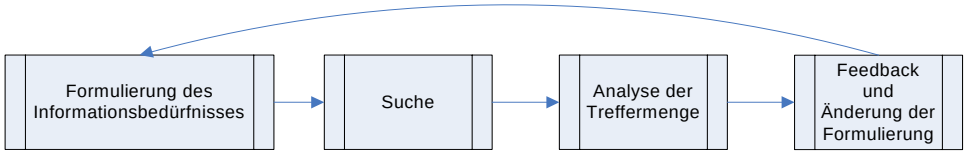


Abbildung 4-4: Schematischer Ablauf einer Suche unter Berücksichtigung des iterativen Charakters

Dazu kann auch die Analyse der Treffermenge über Methoden des KD, wie z.B. über das Clustering von Textdokumenten und die Visualisierung von Zusammenhängen erfolgen. Einen solchen Ansatz verfolgt z.B. das Projekt onAIR, welches in Abschnitt 6.2.1 beschrieben ist und auf Basis des KnowMiner-Frameworks umgesetzt wurde. Zur Unterstützung der Formulierung des Informationsbedürfnisses wurden dabei über Co-Occurence Analyse weiterführende Suchbegriffe ermittelt, die dem Benutzer zur Einschränkung angeboten wurden. Beispielweise ermöglicht der im KnowMiner-Framework entwickelte Assoziationsindex das kontextuelle Vorschlagen von zusätzlichen Suchbegriffen und damit die Einschränkung von inhaltlichen Domänen schon vor der Suche (siehe Abschnitt 6.2.1)

Die Analyse der Treffermenge zerlegt diese im onAIR Projekt mittels Clustering in inhaltliche Themen und extrahiert Personen und Orte mittels Informationsextraktion. Die Zusammenhänge werden visualisiert und der Benutzer kann über die Darstellung der Suchergebnisse weiter einschränken. Dieser Aspekt der Faceted Metadaten wird auch stark in der ‚Stuff i've seen' Suchlösung von Microsoft [Cutrell et. al. 2006], sowie in der in Abschnitt 6.2.1 beschriebenen Suchlösung OnAIR forciert.

Auf Basis der aufbereiteten Suchergebnisse muss es dem Benutzer über Feedback-Mechanismen möglich sein, sein Informationsbedürfnis zu verfeinern. Vor allem im Bereich des Multimedia-Retrieval ist dies unerlässlich, da hier das Informationsbedürfnis sehr schwer spezifizierbar ist. Standard-Feedback-Methoden basieren auf der Bewertung von Suchergebnissen durch den Benutzer, der die für ihn guten und schlechten Treffer spezifiziert.

Heutzutage existieren schon etliche Anbieter von kommerziellen Suchlösungen wie Autonomy[22], Convera[23] oder FAST[24]. Zusätzlich zu diesen kommerziellen Suchlösungen gibt es auch Retrieval-Backends aus dem Umfeld der Open Source Gemeinde, die einerseits kommerziell genutzt werden können, wie z.B. Lucene[25] aus dem Apache Jakarta Umfeld, oder andererseits rein für akademische Zwecke eingesetzt werden, wie z.B. das TERRIER Framework der Universität Glasgow[26].

Zusätzlich zeigt sich auch ein Trend in Richtung Information Push. Information Push bezeichnet die automatische Bereitstellung von Information zum aktuellen Benutzerkontext. Just-in-TimeRetrieval bezeichnet hierbei ein Schlagwort, welches in verschiedenen Systemen wie z.B. der REMEMBRANCE Machine [Rhodes 2000] vom MIT Media Labratory oder dem IQ Aufsatz des neuen Microsoft Search Tools ‚Stuff I've Seen' [Dumais et. al. 2004] umgesetzt wurde. Beim Erstellen von Texten wird dem Benutzer dabei

[22] http://www.autonomy.com/

[23] http://www.convera.com/

[24] http://www.fastsearch.com

[25] http://lucene.apache.org/

[26] http://ir.dcs.gla.ac.uk/terrier/

automatisch weiterführende Information in unaufdringlicher Weise angeboten, um ihn so mit Hintergrundwissen zu versorgen. Essentiell hierbei ist die Identifikation des Kontextes eines Benutzers um zielgerichtet relevante Information zu identifizieren und die Ergebnisse in unaufdringlicher Art und Weise zu präsentieren. Einen ähnlichen Ansatz verfolgt auch das Forschungsprojekt DYONIPOS [Tochtermann et. al. 2006],, bei welchem der Benutzer abhängig von seinem aktuellen Arbeitskontext mit Vorlagen, weiterführender Information, Experten auf dem Gebiet, sowie mit möglichen nachfolgenden Tätigkeiten versorgt wird.

IR ist eine sehr umfassende und gut etablierte wissenschaftliche Disziplin. Der hier gegebene Auszug schneidet dem Zweck der Arbeit entsprechend nur die wichtigsten Aspekte an. Für einen umfassenderen Überblick sei auf [Rijsbergen 1979], [Baeza-Yates et. al 1999], [Hatcher et. al. 2004], [Rijsbergen 2004], [Fuhr 2006] sowie auf die ACM Special Interest Group on Information Retrieval (SIGIR)[27] verwiesen.

4.3 Überwachte Lernverfahren

Bei überwachten Lernverfahren (engl. supervised learning) geht man von einer Menge an vorgegebenen Beispielen und einer Zuordnung dieser Beispiele zu entsprechenden Klassen aus. Durch Analyse der Beispiele können Klassifikationshypothesen erstellt werden, welche die automatische Zuordnung von neuen Beispielen zu den entsprechenden Klassen gestatten. Aufgrund der beispielorientierten Vorgehensweise werden diese Verfahren auch induktive Lernmethoden genannt.

Der Bereich der Computational Learning Theory (COLT) [Kearns & Vazirani 1994]hat sich in den letzten Jahren mit der Untersuchung der Grundlagen von überwachten Lernverfahren beschäftigt, woraus interessante Erkenntnisse ableitbar sind. Da bei der Anwendung von überwachten, induktiven Lernverfahren Beispiele vorgegeben werden müssen, stellt sich natürlich die interessante Frage nach der Güte des Generalisierungsfehlers, d.h. der Genauigkeit der Klassifizierungshypothese auf noch nicht gesehene, unbekannte Beispiele und der zu Erreichung des Generalisierungsfehler notwendigen Beispiele. Wie aus dem Bereich der Neuronalen Netze bekannt, ist die Wahl der richtigen Anzahl der Trainingsbeispiele kritisch. So kann es bei zu vielen Dokumenten und zu mächtigen Hypothesenklassen zum so genannten „Overfitting“ kommen, d.h. obwohl ein kleiner Trainingsfehler – der Fehler, den der Algorithmus auf den Beispieldaten erreicht – erreicht wird, bleibt der Generalisierungsfehler hoch. Grund ist meist, dass aufgrund der mächtigen Hypothesenklasse die Beispiele so zu sagen auswendig gelernt werden und keine Generalisierungsfähigkeit entsteht.

Um mitunter dies zu vermeiden, prüft man empirisch den Generalisierungsfehler durch Unterteilung der Trainingsdaten in einen Trainingsteil und in einen Testteil. Wie der Name schon sagt, erfolgt das Lernen der Klassifizierungshypothese auf den Trainingsdaten und der Generalisierungsfehler wird über die Testdaten approximiert. Verschiedene Auswahlstrategien, wie z.B. Cross-Validation, ermöglichen dabei die Anzahl der Trainings-/Testdaten virtuell zu vergrößern bzw. mehrere unterschiedliche und unabhängige Testläufe durch zu führen (für Details siehe [Mitchell 1997]).

Die COLT gibt auch theoretische Einblicke in das Zusammenspiel zwischen Generalisierungsfehler und Anzahl der Trainingsbeispiele. Oberflächlich betrachtet, werden mit steigender Komplexität der Hypothesenklasse und mit steigender Anzahl der Dimensionalität des Merkmalsraumes mehr

[27] http://www.sigir.org

Trainingsbeispiele benötigt. Die Abhängigkeit von der Dimensionalität bezeichnet man auch als „Curse of Dimensionality“, da umso mehr manuell zu erstellende Trainingsbeispiele notwendig sind, je mehr Merkmale ein Objekt beschreiben. Die COLT hat jedoch gezeigt, dass die Dimensionalität bei gewissen Lernverfahren bzw. bei gewissen Hypothesenklassen eine untergeordnete Rolle spielt und z.B. logarithmisch in die Anzahl der Trainingsbeispiele eingeht. Dies ist z.B. bei Support Vector Maschinen (siehe 4.3.1.1) der Fall, welche auch aus diesem Grund auf hochdimensionalen Textdaten und wenigen Beispielen eine gute Genauigkeit zeigen (siehe dazu [Joachims 1998]).

Im Umfeld der überwachten Lernverfahren gibt es eine breite Anzahl an verschiedenen Algorithmen. Diese erstrecken sich von dem menschlichen Gehirn nachempfundenen Neuronalen Netzwerken über der Evolution nachempfundene genetische Algorithmen bis hin zu statistischen und informationstheoretischen Methoden wie z.B. Support Vektor Maschinen. Einen guten Überblick über die unterschiedlichen Methoden geben [Mitchell 1997] und [Duda et. al. 2000]. Speziell im Bereich der Textdokumente sind statistische und informationstheoretische Verfahren zu bevorzugen, da diese heutzutage die besten Ergebnisse liefern (siehe auch [Sebastiani 2002] für einen Überblick). Ausgewählte Algorithmen aus diesem Bereich, welche relevant für das KnowMiner-Framework sind, werden im Folgenden dargestellt. Der Abschnitt über überwachte Lernverfahren schließt mit einem kurzen Ausflug in die unterschiedlichen Anwendungsgebiete.

4.3.1 Beschreibung ausgewählter Methoden

Ausgehend von den in [Sebastiani 2002] und in [Granitzer 2004a] vorgestellten Ergebnissen werden in diesem Abschnitt die wichtigsten Lernverfahren betrachtet.

4.3.1.1 Lineare Klassifikatoren und Kernel-Methoden

Wie der Name schon sagt, lernen lineare Klassifikatoren Hypothesen aus der Hypothesenklasse der Ebenen bzw. Hyperebenen. Ausgehend von einer Menge an positiven und negativen Beispielen versuchen lineare Klassifikatoren diese Menge linear zu trennen. Im zweidimensionalen Raum geschieht dies durch Einziehen einer Ebene (siehe Abbildung 4-5). Im hochdimensionalen spricht man von einer Hyperebene.

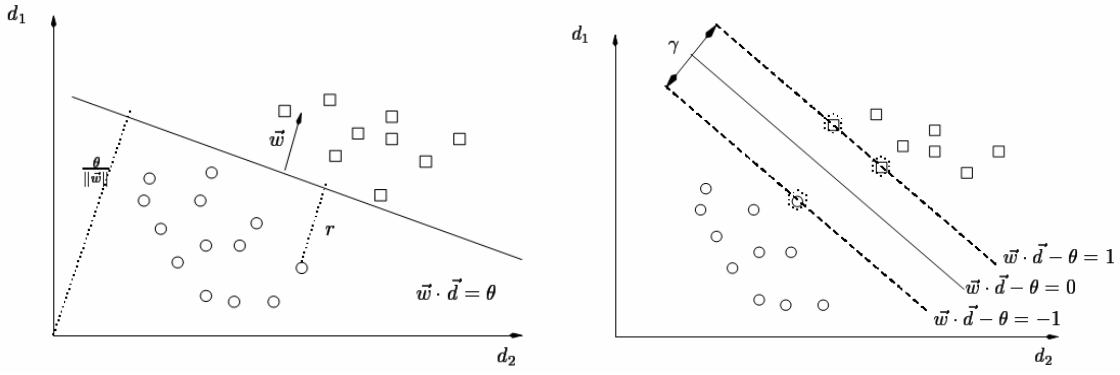


Abbildung 4-5: Lineare Klassifikationshypothese im 2D. Kreise stellen die negativen, Rechtecke die positiven Beispiele dar. Der Vektor *w* definiert den Normalvektor auf die trennende Hyperebene. Links ist die Wahl der Ebene beliebig (es gibt mehrere Lösungen) während rechts jene Ebene gewählt wird, die den Abstand zwischen positiven und negativen Beispielen maximiert.

Wie aus Abbildung 4-5 links ersichtlich, gibt es nun mehrere Möglichkeiten eine solche trennende (Hyper)Ebene zu definieren. Unter der Annahme, dass die beste trennende Hyperebene jene ist, die am

weitesten von den negativen und positiven Beispielen entfernt liegt, also zu beiden Gruppen den größten Abstand (γ in Abbildung 4-5 rechts) besitzt, lernen Support Vektor Maschinen (SVM's) oder genereller Large Margin Classifiers lineare Hypothesenklassen. Durch diese Einschränkung der Hypothesenklasse benötigen diese Klassifikatoren für die meisten praktischen Problemstellungen weniger Trainingsbeispiele zur Erreichung eines guten Generalisierungsfehlers als lineare Klassifikatoren, die den maximalen Abstand nicht berücksichtigen (siehe dazu z.B.: [Joachims 1998]).

Während lineare Hypothesenklassen sehr restriktiv wirken, bieten Support Vektor Maschinen die Möglichkeit Kernel-Methoden (siehe Abschnitt 3.5) einzusetzen, um den Merkmalsraum zu transformieren und um dann im höherdimensionalen Raum eine lineare Trennung zu erreichen. Der Einsatz von Kernel-Techniken ist vor allem durch die Formulierung der Klassifikationshypothese in Form von inneren Produkten zwischen Trainingsbeispielen effizient und einfach möglich. Im Speziellen kann die trennende Hyperebene genau aus jenen positiven und negativen Beispielen berechnet werden, die den minimalen Abstand zur trennenden Ebene aufweisen (gepunktete Beispiele in Abbildung 4-5 rechts). Diese Beispiele werden auch Support-Vektoren genannt.

Support Vektor Maschinen und Kernel-Methoden stellen heutzutage die meistens besten Techniken zum Einsatz in praktischen Problemen dar und fußen zusätzlich auf einem gut verstandenen, theoretischen Fundament. Vor allem im Bereich textueller Daten ist ihre Skalierungsfähigkeit in Bezug auf hochdimensionale Problemstellungen essentiell. Auch existieren schnelle Lernverfahren, welche mit großen Mengen an Trainingsdaten umgehen können. Details zu Kernel-Maschinen und SVM's, im speziellen im Textbereich können in [Granitzer 2004a, Joachims 1998] oder auf der Kernel-Maschinen Community Seite[28] nachgelesen werden.

4.3.1.2 K-Nearest Neighbour Klassifikatoren (k-NN)

K-Nearest Neighbour Verfahren gehören zur Gruppe der Lazy Learner, welche keine Klassifikationshypothese lernen, sondern diese erst zur Klassifikationszeit auswerten. Auf Kosten der Zeit zum Auswerten der Klassifikation reduziert sich dadurch der Aufwand beim Lernen des Klassifikators.

Bei der Zuordnung eines neuen Dokumentes sucht sich dieses Verfahren im ersten Schritt die k ähnlichsten Dokumente, welche bereits dem Klassifikationsschema zugeordnet sind. Dabei kann ein beliebiges Distanz- bzw. Ähnlichkeitsmaß (siehe Abschnitt 3.5) angewendet werden. Im zweiten Schritt werden die häufigsten Klassenzuordnungen der k ähnlichsten Dokumente bestimmt und dem neuen Dokument zugeordnet. Die Zuordnung erfolgt sozusagen auf Basis einer Mehrheitsentscheidung von inhaltlich ähnlichen Dokumenten.

Problematisch bei k-NN Verfahren ist lediglich, das eine exakte Nächsten-Nachbarn-Suche nicht schneller als O(n), mit n als Anzahl der Objekte, funktionieren kann. Um daher k-NN Klassifikatoren auf großen Datenmengen skalierbar zu machen, muss mit Approximationen und Heuristiken gearbeitet werden (e.g. über Hashing, siehe [Gionis et. al. 1999]), wodurch wiederum die Qualität sinkt. Nichtsdestotrotz erweisen sich auch k-NN Klassifikatoren als gute Ansätze im Bereich der Klassifikation textueller Daten.

[28] http://www.kernel-machines.org

4.3.1.3 Classifier Committees

Die Idee hinter Classifier Committees ist, dass nicht einen einzelne Klassifikationshypothese, sondern Mengen von unterschiedlichen Expertenhypothesen gelernt und das die Expertenhypothesen bei der Klassifikation miteinander kombiniert werden.

Boosting, einer der erfolgreichsten Vertreter dieser Klassifikatoren, beschränkt sich auf das Lernen von mehreren unterschiedlichen Hypothesen der gleichen Hypothesenklasse. Typischerweise lernt Boosting in jeder Iteration eine so genannte schwache Klassifikationshypothese und verändert auf Basis dieser Hypothese die Wichtigkeit der Beispiele. Beispiele, welche richtig klassifiziert sind, erhalten eine geringere Wichtigkeit (da sie ja schon gut gelernt wurden), während falsch klassifizierte Beispiele eine höhere Wichtigkeit erhalten, damit sich die schwache Hypothese auf diese falsch klassifizierten Beispiele „konzentrieren" kann. Jede schwache Hypothese kann also als Experte für unterschiedliche Beispiele verstanden werden. Die Gesamthypothese, anhand derer neue Beispiele klassifiziert werden, ergibt sich aus der Kombination der schwachen Hypothesen, wobei den Hypothesen, die besser auf den Trainingsbeispielen abgeschnitten haben, eine höhere Bedeutung zukommt. Einer der bekanntesten Boosting Ansätze ist AdaBoost (siehe [Freund et. al. 1996]).

Boosting gehört ebenfalls zur Gruppe der Large Margin Classifier und liefert ähnliche Ergebnisse wie SVM's im Bereich der Textklassifikation, wenngleich die Trainingszeit etwas höher ist (siehe [Schapire et. al. 2000]). Aufgrund der austauschbaren schwachen Hypothesen kann Boosting einfach an verschiedene Problemstellungen adaptiert werden (siehe [Granitzer 2004a] für ein Beispiel einer solche Adaption).

4.3.2 Anwendungsgebiete

Im Folgenden sollen kurz mögliche Anwendungsgebiete für überwachte Lernverfahren dargestellt werden, wobei kein Anspruch auf Vollständigkeit erhoben wird. Ausgehend vom Anwendungsgebiet Wissensmanagement kann zwischen direkt anwendbaren Gebieten, wie z.B. der Textklassifikation, und unterstützenden Gebieten, wie z.B. dem Part-of-Speech Tagging, unterschieden werden. Diese Gebiete sind nachfolgend kurz beschrieben.

Auf Dokumentebene erfolgt die Anwendung von überwachten Lernverfahren, z.B. zur Wartung großer Klassifikationsschemata. Solche Klassifikationsschemata strukturieren inhaltlich eine bestimmte Domäne, wobei Dokumente ein oder mehreren Klassen zugeordnet werden können. Auf Basis der zugeordneten Dokumente erfolgt die Berechnung der Klassifikationshypothese, um neue, bisher noch nicht gesehene Dokumente den korrekten Klassen innerhalb des Klassifikationsschemas zuzuordnen. Da diese Ablagesystematiken meist hierarchisch organisiert sind, kann diese hierarchische Struktur auch zur Erhöhung der Genauigkeit, der Lern- und Klassifikationszeit angewendet werden (unter bestimmten Voraussetzungen hinsichtlich Aufbau der Hierarchie, siehe dazu [Granitzer & Auer 2005]).

Neben der Zuordnung von Dokumenten zu Ablagesystemen kann die Textklassifikation auch z.B. zur automatischen Weiterleitung von Dokumenten zu bestimmten, am Dokument interessierten Personen oder zum Filtern von Spam Mails eingesetzt werden.

Ein weiteres Anwendungsgebiet für induktive Lernverfahren ist die Extraktion von Entitäten aus Texten oder auch das Part-of-Speech Tagging (i.e. Sequential Tagging, siehe Abschnitt 3.1.2.2). In diesem Fall sind die Beispiele nicht Dokumente, sondern Sätze bzw. Wortgruppen und deren zugeordnete Rolle in einer Entität.

Folgende Tabelle zeigt ein mögliches vorgegebenes Beispiel für einen Sequential Tagging Algorithmus, wobei PER für personenbezogenen Entitäten, ORT für ortsbezogenen Entitäten und O für keine Zuordnung steht.

John	F.	Kennedy	war	Präsident	der	USA
PER	PER	PER	O	O	O	ORT

Auf Basis derart annotierter Beispiele ermitteln maschinelle Lernverfahren Regeln zur automatischen Extraktion von Entitäten, d.h. Regeln zur automatischen Zuordnung der PER- und ORT-Annotationen zu Wörtern (siehe [Weiss et. al. 2005]) auf noch nicht gesehenen Sätzen.

Grundsätzlich können überwachte Lernverfahren überall da eingesetzt werden, wo Trainingsbeispiele zur Verfügung stehen und neue Beispiele den gleichen Kategorien wie die Trainingsbeispiele zugeordnet werden müssen. Die Vorstellung aller Anwendungsgebiete würde den Rahmen diese Arbeit sprengen. Für einen Überblick sei der Leser auf [Weiss et. al. 2005] verwiesen.

4.4 Unüberwachte Lernverfahren

Im Gegensatz zu überwachten Lernverfahren benötigen unüberwachte Verfahren keine vorkategorisierte Menge an Trainingsdokumenten. Das Ziel ist, Strukturen und Gemeinsamkeiten zwischen einzelnen Beispielen zu finden und diese in Gruppen von gemeinsamen Beispielen zu unterteilen, d.h. ein Clustering vorzunehmen. Clustering spielt eine wichtige Rolle in vielen Anwendungen im Textbereich, da es das Finden von Strukturen innerhalb einer Menge von Dokumenten ermöglicht.

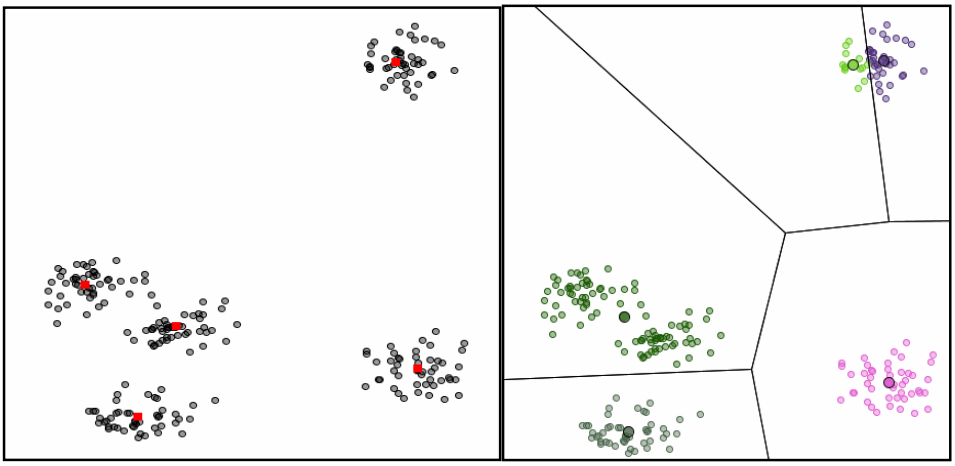


Abbildung 4-6: Clustering im 2D. Ausgehend von einer Menge an aus 5 Gaussverteilungen gezogenen Objekten (links) erfolgt das Finde der Gruppen (rechts). Die Gruppe links oben in der rechten Abbildung konnte nicht korrekt identifiziert werden.

Formaler betrachtet zerlegt Clustering eine Menge von *m* Dokumenten in *n* Gruppen von Dokumenten, mit der Anforderung, dass alle Dokumente innerhalb einer Gruppe maximal ähnlich und alle Dokumente zwischen Gruppen maximal unähnlich zueinander sind. Die Vereinigung der *n* Gruppen ergibt wiederum die gesamte Menge an Dokumenten *m* mit möglichen Ausnahmen von „Outliern". Von hartem Clustering spricht man, wenn jedes Dokument einem Cluster zugeordnet wird oder nicht (binäre Entscheidung). Bei einer

Zuordnung entsprechend dem Grad der Zugehörigkeit (meist zwischen 0 und 1) wird von Fuzzy-Clustering gesprochen.

Die gefundenen Cluster werden durch die Menge an Dokumenten, die ihnen zugeordnet sind, repräsentiert. Eine weitere Möglichkeit zur Repräsentation eines Clusters bieten so genannte Zentroiden, d.h. Repräsentanten für den Cluster, die aus der Menge der zugeordneten Dokumente berechnet wurden. Meistens erfolgt hierbei die Mittelwertbildung über alle Dokumentvektoren. Für den Zentroiden können des Weiteren die wichtigsten Merkmale ermittelt und dem Benutzer als Zusammenfassung des Clusters präsentiert werden. Dadurch erhält man bei Textmerkmalen z.B. Schlagwörter als Beschreibung für den Cluster (siehe auch Anwendung WebRat, Abschnitt 6.1.1). Ist aufgrund der Merkmalsstruktur keine Berechnung des Zentroiden möglich, kann als prototypische Beschreibung auch das am besten passendste Objekt des Clusters, der Medoid, heran gezogen werden. Dieser Ansatz kommt oft in Multimediaanwendungen zum Einsatz.

Weitere wichtige Charakterisierungsmerkmale von Clusteringalgorithmen sind (siehe auch [Xu et. al. 2005, Berkhin 2002]):

- deterministisch vs. stochastisch
  Deterministische Algorithmen liefern bei gleicher Objektmenge immer das gleiche Ergebnis, während dies bei stochastischen Verfahren nicht der Fall ist.
- Inkrementell vs. Batch
  Inkrementellen Clusteringalgorithmen können sukzessive neue Beispiele gegebenen werden, ohne das diese den gesamten Vorgang wiederholen müssen.
- Reihefolgeabhängig vs. reihenfolgeunabhängig
  Die Reihenfolge der Abarbeitung der Objekte beeinflusst die Ergebnisse des Clusteringprozess.
- Monothetisch vs. polythetisch
  Ein Objekt wird jeweils einen (mono) oder mehreren (poly) Clustern zugeordnet.

Clustering ist eine wichtige Methode zur Strukturierung unstrukturierter Objektmengen. Die meisten Clusteringverfahren sind sensitiv gegenüber der verwendeten Metrik und können über Parametrisierung des Ähnlichkeits- bzw. Distanzmaßes beeinflusst werden. Im Folgenden wird ein kurzer Überblick über ausgewählte Methoden gegeben, wobei kein Anspruch auf Vollständigkeit erhoben wird. Für einen guten Überblick sei auf [Xu et. al. 2005, Berkhin 2002] verwiesen.

4.4.1 Ausgewählte Verfahren

Hierarchische Clustering (HAC) Methoden gehören zur bekanntesten Gruppe an Clusteringverfahren. Beim Clustering erfolgt der Aufbau einer Cluster-Hierarchie, des so genannten Dendrogramms. Beim agglomerativen HAC erfolgt die Erzeugung dieser Hierarchie bottom up. Der Clusteringalgorithmus startet mit Clustern, die genau ein Objekt beinhalten (bzw. jedes Objekt entspricht einem Cluster). In jedem darauf folgenden Rekursionsschritt erfolgt die Verschmelzung der Cluster aufgrund eines gegebenen Ähnlichkeitsmaßes bis ein Abbruchkriterium erreicht ist. Das Abbruchkriterium errechnet sich meist aus der durchschnittlichen Ähnlichkeit der Objekte innerhalb der Cluster und der durchschnittlichen Unähnlichkeit zwischen den Clustern. Abbildung 4-7 zeigt beispielhaft ein solches Dendrogramm.

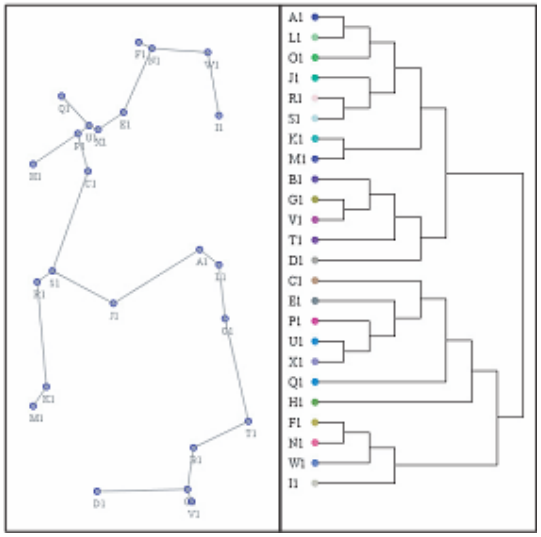


Abbildung 4-7: Dendrogramm (rechts) erstellt auf Basis der euklidschen Distanz zwischen den Punkten links

Divise HAC Methoden gehen top-down vor. Startpunkt ist ein Cluster, welches alle Dokumente umfasst und der rekursiv, wiederum auf Basis eines Ähnlichkeitsmaßes, unterteilt wird. Das Dendrogramm wird also von oben nach unten aufgebaut.

Partitioning Relocation Clustering Methoden optimieren beim Start initialisierte Cluster iterativ durch Umverteilung ihrer zugeordneten Objekte. Zu Beginn erfolgen die Festlegung einer Menge von k-Clustern und die entsprechende Zuordnung von Objekten. Auf Basis eines Ähnlichkeitsmaßes erfolgt die iterative Umverteilung von Objekten zu Clustern bis die optimale Gruppierung gefunden ist. Abhängig vom Modell bzw. der Metrik kann zwischen probabilistischen Methoden, welche ein probabilistisches Modell optimieren, und k-means/k-medoid Methoden, welche ein Ähnlichkeitsmaß optimieren, unterschieden werden.

Während hierarchische Methoden die Anzahl der Cluster auf Basis eines definierten Kriteriums selbst ermitteln, muss bei Partition Relocation Clustering die Anzahl der Cluster meist vorgegeben werden. Da hierarchische Methoden eine höhere Laufzeitkomplexität aufweisen, erfolgt meist in einem ersten Schritt die Ermittlung der Anzahl der Cluster unter Anwendung von HAC auf einem Teilausschnitt der Daten und anschließend die Fertigstellung des Clustering mittels k-means/k-medoid.

Density-based Partitioning Algorithms definieren Cluster über eine verbundene, dichte Region, welche sich nach allen Seiten hin ausbreiten kann, so lange ein Dichtekriterium erfüllt ist. Dichte bemisst sich dabei über die lokale Verteilung, d.h. darüber, wie viele Objekte sich in einer ε-Umgebung um ein Objekt befinden, und hängt damit stark von der Wahl der Dichtefunktion ab.

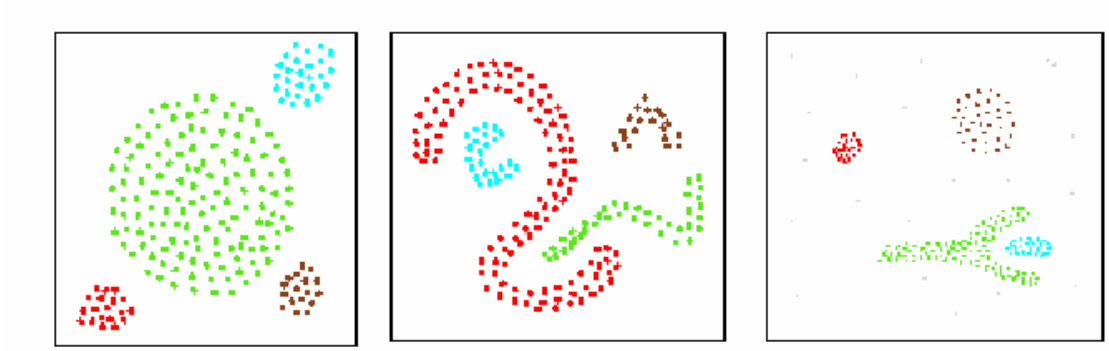


Abbildung 4-8: Beispiele für die Vielfalt an Clusterformen bei dichtebasierten Verfahren(Quelle: http://www.inf.unibz.it/dis/teaching/msc/project-dbscan.html)

Der Vorteil dichtebasierter Verfahren ist die Form der Cluster. Während HAC und Relocation Methoden meist konvexe Gruppen erzeugen, ermöglichen dichtebasierte Methoden Cluster beliebiger Form. Abbildung 4-8 zeigt die mögliche Vielfalt an Clusterformen bei dichtebasierten Verfahren.

Grid Based Clustering Methoden partitionieren den Merkmalsraum in Segmente wie z.B. Kuben, Zellen oder Regionen. Die Gruppierung der Dokumente erfolgt auf Basis der Zugehörigkeit der Merkmale zu den Segmenten. Während Grid-basierte Clusteringmethoden schwerer auf hochdimensionalen Daten skalierbar sind, so können sie mit unterschiedlichen Arten von Merkmalen und auch heterogenen Merkmalsräumen umgehen.

Neben den hier erwähnten Standardmethoden gibt es noch Techniken, die sich an zuvor eingeführten Methoden wie z.B. Kernels oder Classifier Committees anlehnen oder auf Repräsentationsformen wie z.B. Sequenzen operieren. Für Details und weiterführende Information sei auf [Xu et. al. 2005, Berkhin 2002] verwiesen.

Problematisch für viele Clusteringmethoden erweist sich vor allem die Dimensionalität der Daten, einerseits hinsichtlich der Laufzeit und andererseits hinsichtlich der Aussagekraft der verwendeten Metriken. Subspace Clusteringmethoden und die Reduktion der Dimension über Merkmalstransformationen (siehe Abschnitt 4.1) adressieren diesen Umstand.

Ein weiteres Problem ist der Umgang mit großen Datenmengen, da gute Ergebnisse meist eine mindestens quadratische Laufzeit bedingen. Sampling oder Data Squashing Methoden adressieren dieses Problem. Beim Sampling erfolgt dabei die Selektion einer relevanten Untermenge der Daten und dem Clustering dieser. Ähnlich wie bei der oben genannten Kombination von HAC und k-means erfolgen die Anwendung von Techniken mit hoher Laufzeitkomplexität auf den Untermengen der Daten und die anschließende Verfeinerung über Methoden mit geringerer Laufzeitkomplexität. Squashing erzeugt im Gegensatz eine komprimierte Repräsentationsform, um die gesamten Objekte im Hauptspeicher oder in für Sekundärspeicher optimierten Datenstrukturen halten zu können. Hashing-Verfahren in Kombination mit approximativen Ähnlichkeitssuchen ermöglichen so z.B. das Clustern von mehr als 30 Millionen Dokumenten und eine Reduktion der Daten von 150 GB auf 3 GB (siehe [Broder et. al. 1997]

4.4.2 Anwendungsgebiete

Clusteringmethoden lassen sich in unterschiedlichsten Gebiete, in denen es um die Strukturierung unstrukturierter Daten geht, einsetzen. Im Folgenden werden ein paar für den Bereich Wissensmanagement relevante Szenarien aufgezeigt.

Ein Anwendungsszenario für Clustering ist das automatische Gruppieren von Suchergebnissen wie es von manchen Suchmaschinen bereits angeboten wird. Für jeden Treffer einer Suche (z.B. Suche nach „Paris“) erfolgt dabei die oben erwähnte Vektorisierung und unter Berücksichtigung der Ähnlichkeit zwischen Dokumenten erfolgt die Einteilung der Trefferliste in unterschiedliche Themengruppen. Um nun eine gesamte Themengruppe für den Benutzer zusammen zu fassen, werden über statistische Analysen die wichtigsten Merkmale einer Gruppe extrahiert und als Beschriftung angeboten (z.B. „Eifelturm“, “Sagenheld“, „Tourismus“). Hier gilt es zu berücksichtigen, dass meist eine suboptimale Beschriftung erreicht wird, da nur Merkmale zur Beschriftung verwendet werden können, welche auch in den einzelnen Treffern vorkommen. Als Ergänzung könnten überwachte Lernverfahren eingesetzt werden. Da Gruppen von Treffern wiederum als Vektor darstellbar sind, könnten Gruppen von Dokumenten automatisch Klassifikationsschemata zugeordnet und somit auf einer semantisch höheren Ebene beschrieben und kategorisiert werden [Stein et. al. 2004, Kienreich et. al. 2005]. Abbildung 4-9 zeigt ein beispielhaftes Clustering der Suche nach „Java“ mit der Suchmaschine http://www.clusty.com.



Abbildung 4-9: Clustering der Treffer der Suche "Java" auf http://www.clusty.com

Clustering kann auch als Möglichkeit zur Merkmalstransformation bzw. zur Dimensionsreduktion eingesetzt werden. In [Karypis et. al. 2000] erfolgt die Anwendung von spezialisierten Clustering Methoden zur Gruppierung von Textdokumenten. Die so extrahierten Cluster dienen als Dimensionen des Zielraumes. Über die Ermittlung der Ähnlichkeit der Objekte zu den Clustern im Ursprungsraum ergibt sich die Ausprägung der dem Cluster entsprechenden Dimension im Zielraum. Diese Methode, Concept Indexing

(CI), ist hinsichtlich Genauigkeit mit LSI vergleichbar, wobei unter Berücksichtigung eines Schwellenwertes der Ähnlichkeit eine sparse Vektorrepräsentation für das Retrieval erhalt bleibt.

Clustering muss sich nicht nur auf Dokumente beziehen, sondern kann auch auf Worte angewendet werden. Dies geschieht z.B. beim Lernen von Ontologien, wie es in Abschnitt 4.6 beschrieben wird.

4.5 Informationsvisualisierung und Visual Analytics

Beim Umgang mit großen Informationsräumen und Datenmengen, wie sie in heute gängigen Wissensmanagementsystemen auftreten, besteht ein zentraler Aspekt für die Anwendung darin, Benutzern einen einfachen, intuitiven Zugang zu diesen Datenmengen zu bieten. Während die Erschließung der Inhalte und ihrer Zusammenhänge über die in diesem Abschnitt beschriebenen Methoden erfolgt, müssen diese in geeigneter Form bereitgestellt und zugänglich gemacht werden. Über Erkenntnisse aus dem Bereich der Human Computer Interaction (HCI) lassen sich im Allgemeinen angepasste und hoch optimierte Benutzerschnittstellen entwickeln. Gerade aber im Bereich der Wissenserschließung sind der Umgang mit großen Datenmengen und der damit verbundene explorative Charakter von Bedeutung. Informationsvisualisierung (IV) setzt hier an und bietet Methoden und Techniken an, die dem Benutzer diesen Zugang erleichtern. Wie in [Andrews 2002] beschrieben, kann IV als

> *"The visual presentation of abstract information spaces and structures to facilitate their rapid assimilation and understanding"*

zusammen gefasst werden.

Abhängig von den zu visualisierenden Daten kann laut [Andrews 2002] folgende Einteilung getroffen werden:

1. Visualisierung von Hierarchien
2. Visualisierung von Netzwerken und Graphen
3. Visualisierung von mutli-dimensionalen Daten
4. Visualisierung von Textdaten

Bezogen auf die Anwendung der Wissenserschließung im Kontext von Wissensmanagement sind vor allem Visualisierungsarten von Bedeutung. Hierarchien sind zentrale Strukturen zur Organisation von Wissensmanagementsystemen (siehe Abschnitt 6.1.2) und multidimensionale Daten entsprechen den Merkmalsräumen von Objekten.

Als Grundlage, oder besser gesagt als Leitfaden für die Entwicklung von IV Techniken, formulierte Ben Shneiderman [Shneiderman 1996] das so genannte Information Seeking Mantra

> *"Overview first, zoom and filter, details on demand.".*

Darüber sind die folgenden Aufgaben, die ein Benutzer durchführt:, definiert:

- Overview
  Einen Überblick über den gesamten Datenbestand gewinnen
- Zoom
  Hinzoomen zu interessanten Informationen

- Filter
  Filtern uninteressanter Information
- Details on Demand
  Auswählen einer Gruppe von Objekten und anzeigen von Details zu diesen
- Relate
  Anzeigen von Beziehungen zwischen Objekten
- History
  Unterstützung in der Abfolge von Interaktionen wie Undo, Redo etc.
- Extract
  Suche nach einer Untermenge von Objekten
- Organise
  Manuelle Organisation verschiedener Objekte

Ein typisches IV System sollte alle bzw. den Großteil der hier geforderten Funktionen auf den jeweiligen Daten erfüllen.

Ein Beispiel für IV Systeme ist z.B. das Spire System [Thomas et. al. 2001]. Hierbei werden über eine Galaxy-Metapher Dokumente in einer flachen, unstrukturierten Form als Sterne darstellt, wobei ähnliche Dokumente auf einer 2D-Fläche räumlich näher zueinander liegen. Über Interaktionen kann man sich in der 2D-Galaxy bewegen und navigieren. Einen ähnlichen Ansatz verfolgt auch das Projekt InfoSky (siehe Abschnitt 6.1.2), wobei dieses jedoch zusätzlich zu dem Ähnlichkeitslayout Strukturinformation einer Ablagestruktur integriert.

Aufbauend auf den Erkenntnissen aus dem Bereich Informationsvisualisierung entwickelte sich das Gebiet der Visual Analytics (VA), welches aktuell in den USA stark gefördert wird. VA definiert sich als Disziplin zum analytischen Schließen unter Zuhilfenahme von interaktiven visuellen Benutzerschnittstellen. Analytische visuelle Techniken sollen dabei helfen, große, dynamisch veränderliche, mehrdeutige und manchmal auch gegensätzliche Datensätze zu beurteilen und zu verstehen.

Der Unterschied zur IV zeigt sich über das in [Keim 2005] definierte VA Mantra. Im Kontext der Analyse von großen, dynamischen und sehr komplexen Datensätzen ist es nahezu unmöglich, dem IV Mantra von Shneiderman zu folgen. Es ist schwer, einen sinnvollen Überblick über die Daten zu geben und Zooming und Filtering erreichen aufgrund der Komplexität der Datensätze bald ihre Grenzen. Aus diesem Grund erfolgte die Definition des VA Mantra's

"analyse first - show the important – zoom and filter - details on demand - analyse further (iteratively)"

welches den analytischen Prozess ins Zentrum stellt und sich daher gut in den KD-Prozess einbetten lässt. Der Benutzer untersucht die Zusammenhänge iterativ und kann zu jeder Zeit weitere Analysen, je nach gefundenem Zusammenhang, anstoßen. Während IV immer als Frontend zur durchgeführten KD-Analyse wie z.B. zum Clustering und MDS angesehen wird, kann VA als integraler Bestandteil aufgefasst werden, welcher unterschiedliche KD-Methoden auf Basis von gefundenen Zusammenhängen anstößt.

Anwendungsgebiete für Informationsvisualisierung gibt es viele. Im Projekt onAIR z.B. erfolgt nach inhaltlicher, personenbezogener und ortsbezogener Analyse die visuelle Darstellung der Zusammenhänge. So

wurden z.B. die Themenverläufe über die Zeit oder die Zusammenhänge zwischen Themen im Zwei- und Dreidimensionalen dargestellt.

Im Umfeld der Wissenserschließung sind Techniken aus den Bereichen IV und VA i.A. unumgänglich, wenn dem Benutzer die Ergebnisse aus dem KD-Prozess einfach zugänglich gemacht werden sollen.

4.6 Textbasiertes Lernen von Ontologien

Die bisher eingeführten Techniken und Methoden werden in diesem Abschnitt zusammengefasst, um beispielhaft die Anwendung des Lernens von Ontologien zu demonstrieren. Ontologien definieren sich als gemeinsames Verständnis der Konzepte einer Domäne, sowie den Beziehungen zwischen Konzepten. Eine einfache Form einer Ontologie wäre beispielsweise eine Taxonomie, in der Konzepte hierarchisch über IS-A Beziehungen, d.h. ein Konzept ist Teil eines übergeordneten Konzepts, verbunden sind. Während Ontologien auch Axiome und darauf aufbauend Inferenzmechanismen beinhalten können, reicht für die im Folgenden umrissenen Techniken die Betrachtung von Ontologien als Menge an Konzepten und Beziehungen. Für Details zum Thema Ontologien sei auf [Maedche & Staab 2001] verwiesen.

Das Lernen von Ontologien beginnt bei der Frage danach, welche Konzepte enthalten sein sollen. Ausgehend von einer gegebenen Domäne und entsprechenden domänenrelevanten Texten erfolgt über Vorverarbeitung und statistische Analysen die Extraktion von relevanten Merkmalen für eine Domäne. Mögliche Kandidaten für Konzepte sind z.B. statistisch signifikant oft vorkommende Merkmale, aber auch durch Clustering erzeugte Gruppierungen von Textteilen können als Konzepte betrachtet werden. Zusätzlich ermöglicht z.B. eine Co-Occurrence-Analyse die Extraktion von synonymen Begriffen zu einem Konzept.

Nach dem semiotischen Dreieck [Ogden & Richards 1923, Brugger 1976] definiert sich ein Konzept aus seiner Intension - der informellen Definition eines Konzepts, und seiner Extension - der Realisierung des Konzeptes in verschiedenen Instanzen. Am Beispiel des Konzeptes „Krankheit“ wäre eine mögliche Intension „anormale Funktionen des Körpers“ und „Grippe“ oder „Kopfschmerzen“ stellen mögliche Extensionen dar. Zusätzlich wird noch die Realisierung eines Konzepts berücksichtigt. Im Speziellen sind hiermit das identifizierende Literal wie z.B. „Krankheit“ und dementsprechende (multilinguale) Synonyme wie z.B. „disease“ oder “illness“ gemeint. Betrachtet man z.B. mittels Clustering erzeugte Konzepte, so kann die Intension eines Konzepts mit den wichtigsten Merkmalen eines Clusters (i.e. Beschriftung) gleichgesetzt werden und die Extension als die im Cluster enthaltenen Objekte verstanden werden. Hier ist zu bemerken, dass ein Objekt nicht notwendigerweise ein gesamtes Dokument darstellen muss, sondern auch aus einzelnen Sätzen bzw. Satzteilen bestehen kann, wie z.B. in [Bisson et. al. 2000], wo Clustering auf Phrasen inklusive deren syntaktischen Elementen basiert. So erfolgt z.B. in [Dhillon et. al. 2002] das Clustering von Wörtern auf Basis der Dokumente, in denen diese als Merkmale enthalten sind. Dies entspricht einer Invertierung der Objekt-Merkmalsmatrix, wobei die Merkmale zu Objekten und die Objekte zu Merkmalen werden. Unter der Annahme, dass zusammen gehörende Wörter in den gleichen Dokumenten vorkommen, ergeben sich semantisch zusammenhängende Wortgruppen, welche als Extension des so identifizierten Konzepts betrachtet werden können.

Weitere Möglichkeiten zur Zuordnung von Objekten/Dokumenten zu Klassen bieten überwachte Lernverfahren und Informationsextraktion, in denen jeweils Instanzen eines speziellen Typs einem Konzept zugeordnet werden (z.B. Dokumente in einem Klassifikationsschema oder Extraktion von „John Smith“ zum Konzept Person).

Ein weiterer wichtiger Bestandteil in Ontologien sind Relationen zwischen Konzepten, wobei hier der Fokus auf Taxonomiebeziehungen gelegt wird. Grundlegend kann zwischen statistischen und linguistischen Ansätzen unterschieden werden. Linguistische Ansätze berücksichtigen syntaktische Textstrukturen. So zeigt z.B. „<Substantiv 1> wie z.B. <Substantiv 2> etc." eine IS-A Beziehung zwischen Substantiven an. Über dieses Muster (auch Hearst Pattern genannt [Hearst 1992]) kann beispielsweise eine Beziehung zwischen Methoden und Clustering aus dem Satz „Methoden wie z.B. Clustering" extrahiert werden. Da die Erstellung solcher Regeln i. A. aufwändig und fehleranfällig ist, können wiederum maschinelle Lernverfahren eingesetzt werden. Möglichkeiten zur Extraktion solcher Beziehungen basieren auf Clustering, Dokumenten-Subsumption bzw. tieferen linguistischen Analysen, wobei die Genauigkeit all dieser Verfahren meist bei 10-20% liegt[29]. Ein viel versprechender neuer Ansatz zur Steigerung der Genauigkeit ist die Anwendung von maschinellen Lernverfahren zur optimalen Kombination der verschiedenen Verfahren [Cimiano et. al. 2005]. Damit kann die Genauigkeit um etwa 10% erhöht werden.

Die Betrachtungen hinsichtlich der Genauigkeit von automatischen Verfahren im Bereich Ontology-Learning machen deutlich, dass diese lediglich als Startpunkt für das Erstellen einer Ontologie dienen können und nicht die automatische Erzeugung einer wohl definierten, formal richtigen Ontologie ermöglichen. Nichtsdestotrotz erlauben die hier angeführten Techniken eine semantische Anreicherung von Textdokumenten und damit verbunden eine verbesserte Auswertung und Präsentation für den Nutzer von WM Systemen.

4.7 Anforderungen an das KnowMiner-Framework

Die hier vorgestellten Techniken stellen so zu sagen die Grundpfeiler von KD- und IR-Anwendungen dar und sind essentiell im intelligenten Umgang mit textuellen Datenbeständen. Im Gegensatz zu den Vorverarbeitungsmethoden ist für diese Techniken die Anwendung auf einer Menge von Objekten bezeichnend.

Insofern können Klassifikation, Clustering, Information Retrieval und Informationsvisualisierung als Eckpfeiler für die Ableitung der im KnowMiner-Framework nötigen Transformationen von Objektmengen herangezogen werden. Vor diesem Hintergrund werden im Folgenden die wichtigen Anforderungen an das KM Framework dargestellt.

Abstraktion zur Repräsentation verschiedener Zusammenhänge zwischen Objekten: Die eingeführten Techniken und Methoden zeigen, dass in Abhängigkeit vom Anwendungsfeld unterschiedliche Strukturierungsmechanismen für Mengen an Objekten berücksichtigt werden müssen. Während Clustering von einer unstrukturierten Menge an Objekten ausgeht, benötigt die Klassifikation schon die Zuordnung zu einer Menge an Klassen bzw., in komplexeren Fällen, zu einer gesamten Ablagestruktur.

Eine Anforderung an das KnowMiner-Framework ist daher, mit unterschiedlichen Strukturierungen und Strukturierungsmöglichkeiten umgehen zu können und diese in den KD-Prozess einfließen zu lassen.

Schnittstellen und Modelle der einzelnen Methoden: Für die unterschiedlichen Methoden sind unterschiedliche Algorithmen mit verschiedenen Eigenschaften zu berücksichtigen. Um diese nun flexibel anwendbar zu gestalten, müssen Abstraktionen in Form von Schnittstellen für die einzelnen Methoden geschaffen werden, auch im Hinblick auf verschiedene Eigenschaften von Algorithmen. So gestaltet sich die

[29] i.e. der F_1 Wert

Benutzbarkeit von inkrementellen Clusteringalgorithmen anders als jene von nicht inkrementellen oder lassen sich z.B. einzelne Klassifikationstechniken mit unterschiedlichen Klassifikationsmodellen (flach vs. Hierarchisch) kombinieren. Um hier einfache Möglichkeiten zur Nutzung der unterschiedlichen Methoden und Algorithmen zu schaffen, ist eine gute Spezifikation der Schnittstellen essentiell.

Komposition der einzelnen Methoden: Da die einzelnen Methoden abgeschlossene Module bilden und in den meisten KD-Anwendungen nicht alleine verwendet werden, ist eine einfache Komposition der Methoden notwendig, um komplexe Anwendungen zu realisieren. Dadurch können Anwendungen, wie das Suchen von Dokumenten mit anschließendem Clustering und Visualisierung der Zusammenhänge, durch einfache Zusammensetzung der notwendigen Methoden bewerkstelligt werden. Dafür sind einerseits klar definierten Schnittstellen der Module und andererseits auch eine exakte Definition der für die Ausführung von Modulen notwendigen Voraussetzungen erforderlich.

5 KD Frameworks und Standards

Nach Diskussion der Vorverarbeitungsmethoden für unstrukturierte Daten und Darstellung der Methoden im Bereich der Wissenserschließung schließt dieser Abschnitt die Betrachtung des für die Arbeit relevanten wissenschaftlichen Umfelds mit der Analyse von Systemen, welche Aufgaben im Bereich der bisher vorgestellten Systeme erfüllen. Ziel dabei ist es, aus bereits existierenden Systemen Anforderungen an das zu entwickelnde Framework abzuleiten.

Der Abschnitt beginnt mit der Untersuchung der wichtigsten Text Mining - Frameworks, wobei der Fokus auf frei verfügbaren Open Source - Produkten liegt. Ziel ist die kurze Darstellung der Systeme unter Angabe weiter führender Literatur sowie die Erarbeitung der in den Systemen verwirklichten wichtigsten Konzepte.

Abschnitt 5.2 diskutiert Standardisierungsbestrebungen und daraus resultierende Standards im KD- und Data Mining Umfeld und leitet wiederum für das umzusetzende Framework wichtige Rahmenbedingungen ab. Wie in Abschnitt 4.7 angeführt, stellt die Strukturierung eine wichtige Anforderung dar. Der Abschnitt schließt mit der Diskussion von Service-orientierten Architekturen (SOA) und fasst im Anschluss die abgeleiteten Anforderungen an das umzusetzende Framework zusammen.

5.1 Text Mining Frameworks

5.1.1 GATE

GATE bezeichnet die General Architecture for Text Engineering und ist ein Java-basiertes Software Framework und Entwicklungsumgebung für die Verarbeitung natürlichsprachlicher Texte. Bontcheva et. al. [Bontcheva et. al. 2003] beschreiben die Eigenschaften von GATE hinsichtlich der Integration von Sprachverarbeitung als

- ein System, welches zwischen low-level Aufgaben wie Datenhaltung, Datenvisualisierung, Komponentenverwaltung und –ausführung und Aufgaben zur Verarbeitung von natürlicher Sprache unterscheidet
- ein System zur automatischen Auswertung der Genauigkeit von Sprachverarbeitungskomponenten
- ein System, welches den Overhead zur Integration von Sprachtechnologie durch die Verwendung von Standardkomponenten reduziert
- ein System, welches grundlegende Sprachkomponenten zur Verfügung stellt, die beliebig erweitert und ersetzt werden können

Wie aus der Beschreibung hervor geht, fokussiert GATE auf die natürliche Sprachverarbeitung und bietet dazu Funktion zur Verwaltung von:

- Language Resources (LR) – präsentieren Komponenten zur Verwaltung von Ressourcen wie Wörterbücher, Dokumentkorpora oder Ontologien.
- Processing Resources (PR) – präsentieren die algorithmischen Komponenten zur Verarbeitung von Dokumenten wie z.B. Parser, Part-of-Speech Tagger etc.

- Visual Resources (VR) – präsentieren visuelle Komponenten zur Darstellung der Ergebnisse oder zum Editieren von Ressourcen.

Gemeinhin werden die drei Komponenten in einer Pipe-Architektur zusammengefasst und ausgeführt, wobei auch eine Remote-Verarbeitung über HTTP unterstützt wird. Alle Komponenten sind dem Framework als CREOLE-Ressourcen (Collection of REusable Objects for Language Engineering) bekannt, wodurch die Verwaltung und Parametrisierung über XML erfolgen kann. Über eine in GATE enthaltene grafische Oberfläche können die Komponenten vom Benutzer zu Applikationen zusammengefasst und bearbeitet werden. Zusätzlich unterstützt die Oberfläche das manuelle Annotieren von Textdaten und das Evaluieren der Komponenten. Abbildung 5-1 zeigt die Darstellung von Information Extraction Ergebnissen in der GATE Oberfläche

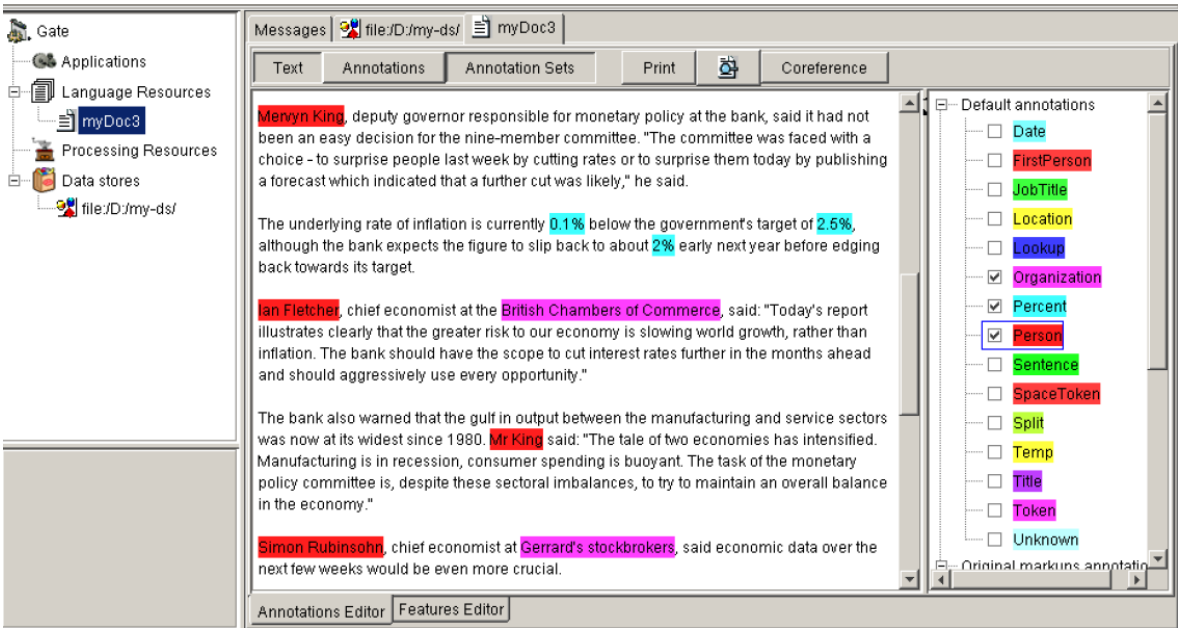


Abbildung 5-1: GUI Editor zur Darstellung der IE Ergebnisse und zur manuellen Annotation (Quelle: http://gate.ac.uk/sale/acl02/acl-main.pdf)

ANNIE - A Nearly New IE System - wird standardmäßig mit GATE ausgeliefert und bietet eine Vielzahl an unterschiedlichen NLP-Komponenten zur Durchführung von Informationsextraktion wie z.B. Satzgrenzenerkennung, Part-of-Speech Tagging und Named Entity Erkennung. Die Standard-ANNIE Pipeline besteht dabei aus einem Tokeniser, einem Sentence Splitter, einem POS Tagger, Gazeteer-Komponenten für das Nachschlagen in Listen und Transducern (Finite State Maschinen zur Abbildung von Grammatiken). ANNIE ist ein auf Grammatiken basierendes IE-System und bietet derzeit Grammatiken für verschiedene westliche Sprachen an.

Neben ANNIE existieren auch noch Komponenten zum Erkennen von Phrasen, verschiedene Techniken aus dem Bereich des Maschinellen Lernens, eine Google API und Plug-Ins zur Integration von Ontologien. GATE unterstützt verschiedene Formate wie XML, RDF, SGML, HTML und Text, welche in einem GATE internen Annotationsformat vereinheitlicht werden. Das Annotationsformat ist dabei flexibel aufgebaut und kompatibel mit dem ATLAS Format (siehe Abschnitt 5.1.4), wobei die in ATLAS definierten Annotationen semantisch detaillierter spezifiziert sind.

5.1.2 UIMA

IBM's Unstructured Information Management Architecture, kurz UIMA, ist eine Architektur und ein Softwareframework zur Erzeugung, Komposition und Verwaltung von multi-modalen Analysetechniken und deren Integration mit Suchtechnologien [IBM 2004]. UIMA selbst stellt keine Methoden und Algorithmen, sondern nur eine Java-basierte Infrastruktur zur Verwaltung dieser bereit. Zusätzlich unterstützt UIMA die Entwicklung von verschiedenen Softwarekomponenten durch dezentrale Expertenteams und erhöht dadurch die Wiederverwendbarkeit der entwickelten Algorithmen. Diese Aspekte unterscheiden UIMA schon klar von den anderen hier vorgestellten Technologien.

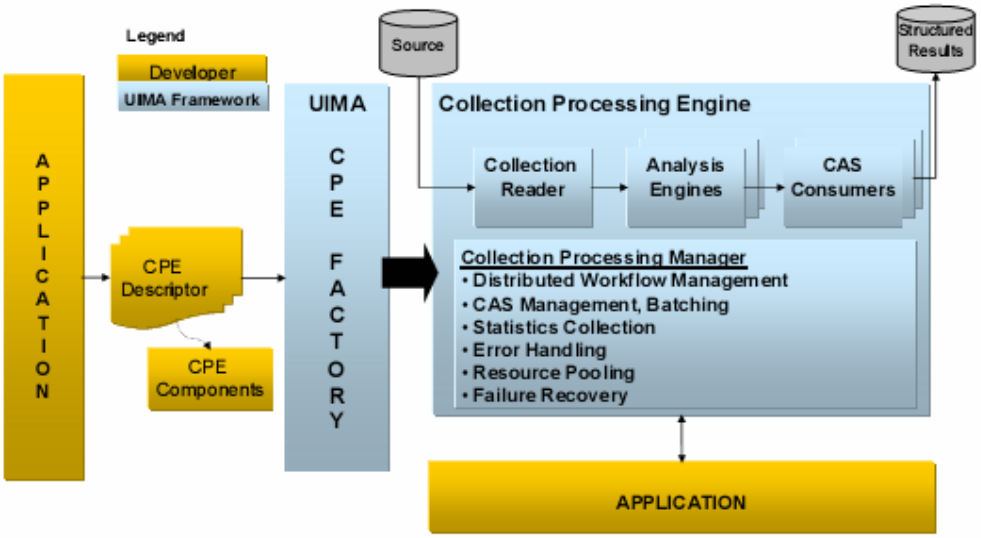


Abbildung 5-2: Überblick über die UIMA Komponenten und die Einbettung von Applikationen im UIMA Framework aus Sicht der Collection Processing Engine (Quelle [IBM 2004, Seite 2-38])

Abbildung 5-2 gibt einen Überblick über die UIMA Komponenten und die Anwendung dieser. Das Herz von UIMA bildet die Common Analysis Structure (CAS), ein mittels XML konfigurierbares Typsystem für die aus unstrukturierten Objekten extrahierten Merkmale. Das Typsystem ist flexibel gehalten und kann beliebig adaptiert werden.

Die verschiedenen Analysemethoden, Analysis Engines (AE) genannt, definieren sich durch das Extrahieren von im CAS definierten Merkmalen aus Objekten. Über XML wird spezifiziert, welche Merkmale dabei von der AE benötigt und welche geliefert werden, wodurch AEs einfach in das UIMA Framework integriert und in einer Pipeline zusammen geschalten werden können. Beispielsweise kann ein Entwicklerteam einen Stemmer für Deutsch und einen für Englisch entwickeln und getrennt über die XML Beschreibung in das UIMA Framework integrieren. Applikationsentwickler wissen nun aufgrund der semantischen Beschreibung der Komponente, welche Merkmale zur Anwendung benötigt werden und welche von der Komponente geliefert werden, wodurch eine einfache Integration möglich ist. UIMA unterstützt momentan lediglich lineare Sequenzschaltungen von AEs (Pipeline), ist aber diesbezüglich erweiterbar.

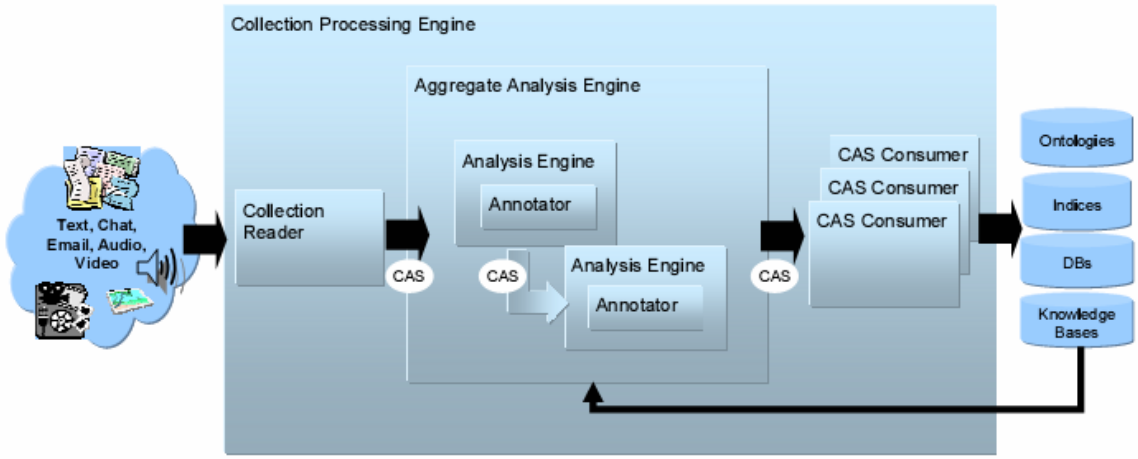


Abbildung 5-3: Aufbau von Collection Processing Engines in UIMA (Quelle [IBM 2004, Seite 2-35])

Charakteristisch für AEs ist, dass sie lediglich ein Objekt nach dem anderen verarbeiten und nicht auf Mengen von Objekten operieren. Diese Aufgabe obliegt den Collection Processing Engines (CPE), welche sich aus einem Collection Reader, AE Pipelines und CAS Consumer zusammensetzen (siehe Abbildung 5-3). Der Collection Reader liest dabei die zu verarbeitenden Daten, entweder von externen Datenquellen oder von anderen CPEs, und speist damit die AE Pipeline, welche die entsprechenden CAS Typen hinzufügt. Im letzten Schritt der CPE greifen CAS Consumer Komponenten auf die Objekte bzw. auf deren Merkmale zu und speisen damit externe Wissensbasen oder Suchsysteme, um so semantische Suchen zu ermöglichen (siehe [Ferrucci & Lally 2004;])

Wiederum bietet sich aus Entwicklersicht die Möglichkeit, unterschiedliche CPE's zu entwickeln, semantisch zu beschreiben und ins UIMA Framework zu integrieren (siehe CPE Factory in Abbildung 5-2). Applikationsentwickler können nun auf ein reichhaltiges Angebot an semantisch beschriebenen AEs und CPEs zur Entwicklung von Applikationen zurückgreifen. Zusätzlich bietet UIMA noch die Möglichkeit CPEs oder AEs als Web-Services zu Verfügung zu stellen, um damit ein verteiltes Rechnen zu ermöglichen.

Die Abgrenzung von UIMA zu anderen Systemen stellt sich wie folgt dar (aus [Ferrucci & Lally 2004]):

> *"UIMA has many features in common with other software architectures for language engineering such as GATE and ATLAS. Each of these systems isolates the core algorithms that perform language processing from system services such as storing of data, communication between components, and visualization of results. However, UIMAs emphasis on transferring UIM technologies to products has led to a richer architecture that allows integrating applications with a host of enterprise products (e.g., WebSphere* Portal Server, Lotus* Workplace) and a variety of middleware and platform options."*

UIMA verfügt über keine detaillierte Strukturierung hinsichtlich der verschiedenen KD-Methoden wie Clustering, Klassifikation sowie die Definition der Schnittstellen zu diesen. UIMA unterstützt als Framework lediglich Prozessoren für einzelne und für mehrere Objekte, definiert aber nicht die Art der Prozessierung. Dadurch existieren bei UIMA auch keine definierten Mechanismen zur Unterstützung unterschiedlicher Strukturierungen des Informationsraumes.

Weiterführende Information ist über [IBM 2004, Ferrucci & Lally 2004, Perna et. al. 2004[30]] verfügbar.

5.1.3 WEKA

http://www.cs.waikato.ac.nz/~nzdl/textmining/

WEKA (Waikato Environment for Knowledge Analysis) ist eine Kollektion von auf Data Mining ausgerichteten Java Machine Learning Algorithmen. Es beinhaltet Komponenten zur Vorverarbeitung von Daten, zur Klassifikation und Regression, zum Clustering, zur Extraktion von Association Rules und zur Visualisierung von unterschiedlichen Datensätzen. WEKA erlaubt nicht nur die Anwendung dieser Algorithmen, sondern bietet auch die Möglichkeit zur Auswertung und Evaluierung sowie zum Vergleich von unterschiedlichen Algorithmen. WEKA erfreut sich großer Beliebtheit aufgrund der einfachen Möglichkeit zur Integration neuer und zur Erweiterung bestehender Algorithmen. Damit verfügt WEKA über eine große Anzahl an zur Verfügung stehenden Algorithmen. Zusätzlich integriert WEKA eine einfach zu bedienende grafische Oberfläche, über die Experimente durchgeführt und Algorithmen analysiert werden können. Abbildung 5-4 zeigt einen Ausschnitt aus der grafischen Oberfläche.

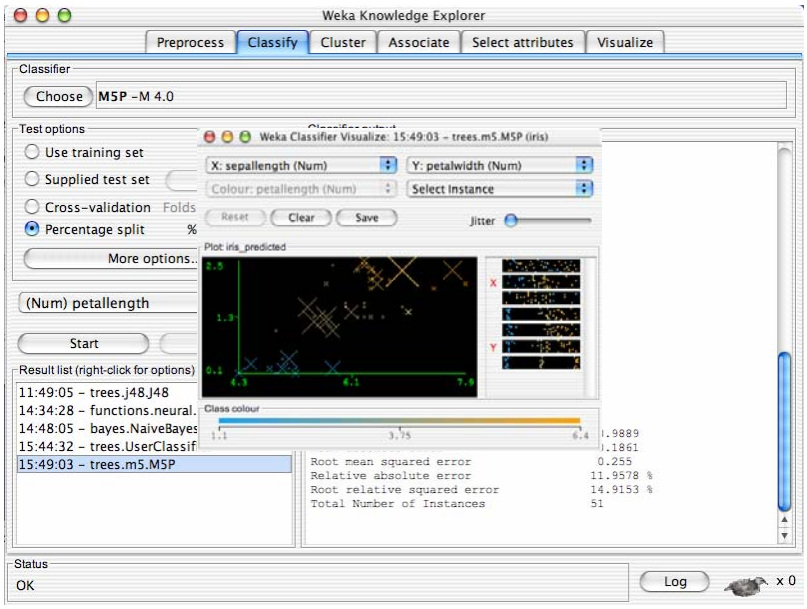


Abbildung 5-4: Ausschnitt aus der WEKA GUI. (Quelle: http://prdownloads.sourceforge.net/weka/weka.ppt)

Die zu verarbeitenden Datensätze müssen als „Flat File" im WEKA eigenen ARFF-Format vorliegen. Das ARFF-Format unterstützt dabei numerische und nominale Merkmale und erlaubt auch sparse Merkmalsrepräsentationen, wenngleich die meisten Algorithmen nicht auf eine sparse Repräsentation optimiert sind.

WEKA erlaubt keine flexiblere Anbindung an bestehende Datenquellen wie es GATE und UIMA erlauben und bietet auch keine Möglichkeit, Merkmale semantisch zu beschreiben bzw. eigene Typsysteme zu verwenden. WEKA zielt hauptsächlich auf die schnelle und einfache Entwicklung und Evaluierung neuer

[30] http://www.research.ibm.com/journal/sj43-3.html

Algorithmen und den Austausch von Standardtestdaten ab. Ein weiterer Grund für den hohen Beliebtheitsgrad in der wissenschaftlichen Gemeinschaft.

5.1.4 Atlas

Atlas ist die Abkürzung für Architecture and Tools for Linguistic Analysis Systems und bietet, wie der Name schon sagt, eine Architektur zur Annotation von Sprachsignalen. Atlas bietet im Vergleich zu den anderen Systemen keine Abstraktion von Algorithmen, sondern lediglich die Möglichkeit zur semantisch hochwertigen Annotation von Signalen. Dazu werden auch verschiedene API's, unter anderem in Java, angeboten.

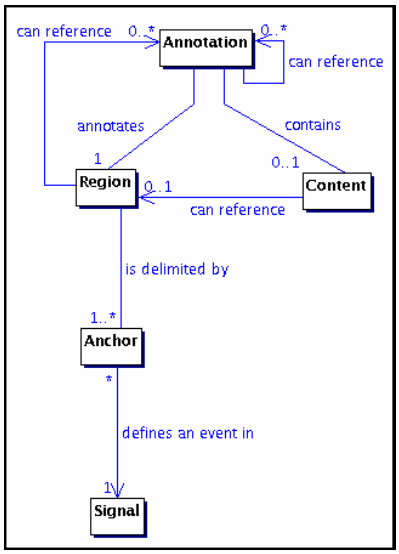


Abbildung 5-5: Ontologischer Aufbau von Annotationen im ATLAS System (Quelle: http://www.nist.gov/speech/atlas/develop/core.html)

Abbildung 5-5 zeigt den Aufbau einer Annotation im Atlas-System. Eine Annotation besteht dabei aus der Region des Inhaltes, den sie annotiert, und dem zugehörigen Inhalt. Die Region selbst wird durch Anker begrenzt, welche ein Ereignis im Signal spezifizieren. Das Signal kann dabei z.B. eine Abfolge von Zeichen, also ein Text sein. Ein Wort, als Region verstanden, definiert sich über einen Anker zu Beginn und einen Anker am Ende des Wortes, identifiziert z.B. dadurch, am wievielten Zeichen, gemessen vom Beginn des Dokuments, sich der Anker befindet. Die einzelnen Elemente wie Annotation, Anchor und Regionen sind typisiert und können daher unterschiedliche Ausprägungen annehmen. Ein Satz wäre beispielsweise ein möglicher Annotierungstyp, eine Person als Named Entity ein anderer. Signale können z.B. vom Typ Audio oder vom Typ Text sein. Die Modellierung erfolgt über die so genannte MAIA Meta-Annotation Infrastructure for ATLAS Ontologie, welche in Form eines DTDs vorliegt.

Während ATLAS viel Wert auf die semantische Annotierung legt, bietet es momentan kein Framework zur Entwicklung von KD-Applikationen und deckt mit dem Fokus auf Annotation auch nur einen kleinen Teilbereich im KD ab. Nichtsdestotrotz stellt die hochwertige semantische Beschreibung von Annotationen eine mögliche Erweiterung des Typsystems von UIMA und GATE dar.

Details zu ATLAS sind auf der ATLAS-Homepage[31] oder in der Publikation [Laprun 2002] enthalten.

5.2 Standards

Nach dem Überblick über dem zu entwickelnden Framework ähnliche Frameworks und den in den Frameworks verwendeten Ansätzen sollen im Folgenden zwei Standards im Bereich Data Mining und KD untersucht werden. Die hier vorgestellte Java Data Mining API (JDM) und die Predictive Model Markup Language (PMML) sind Standardisierungsansätze für Schnittstellen, Methoden und Modelle im Data Mining, um diese zwischen verschiedenen Herstellern und Produkten austauschbar zu machen.

Die JDM fokussiert dabei auf die Schnittstellendefinition für Data Mining Algorithmen und Techniken, um eine einfache Anbindung von Data Mining Plattformen verschiedener Hersteller zu ermöglichen. Orthogonal dazu definiert PMML einen Standard zum Austausch von Data Mining Modellen wie z.B. Klassifikationshypothesen.

5.2.1 JDM - The Java Data Mining API

Im Jahre 2000 startete eine Industriegruppe unter der Leitung von Mark Hornick, Mitarbeiter von Oracle den Java Specification Request Nr. 73 [JSR 73, 2000], kurz JSR 23, mit dem Ziel, eine standardisierte Java API für Data Mining Applikationen zu schaffen. Der Standardisierungsprozess dauerte 4 Jahre und endete mit der Veröffentlichung der Version 1 im Jahre 2004 (siehe [Hornick 2004] für einen Überblick).

Der JSR-73 fokussierte vor allem auf die Abstraktion von Data Mining Funktionalität und ließ dabei die Verarbeitung von unstrukturierten Daten wie Text außen vor. Der derzeit noch in Entwicklung befindliche JSR-247 [JSR 247, 2004] startete im Jahre 2004 und erweiterte JSR-73 in Bezug auf die Verarbeitung unstrukturierter Daten.

Die Motivation hinter JDM liegt vor allem im B2B-Bereich. Kunden soll die Auswahl von Lösungsanbietern im Bereich Data Mining erleichtert werden, indem einfach verschiedene Lösungen über die standardisierte Schnittstelle angesprochen werden können. Dadurch ist die Integration von DM-Funktionalitäten im jeweiligen Anwendungskontext auf Basis der standardisierten Schnittstelle möglich, ohne sich explizit für einen Hersteller entscheiden zu müssen. Der Wechsel zwischen Anbietern sowie die Integration von verschiedenen Produkten ist ebenfalls einfach realisierbar. JDM unterstützt Client/Server-Systeme auf Basis-Service-orientierter Architekturen (SOA, siehe Abschnitt 5.3) und ermöglicht so verteiltes Rechnen.

Konzeptuell unterscheidet die JDM-Spezifikation zwischen DM-Funktionen, DM-Tasks und Objekt- bzw. Modellrepräsentationen, welche im Folgenden kurz beschrieben werden.

Data Mining Funktionen unterscheiden sich im Hinblick darauf, ob es sich um ein beschreibendes (deskriptives) oder voraussagendes (prädiktives) Modell handelt und ob eine überwachte oder unüberwachte Lernmethode die Grundlage bilden. Deskriptive Modelle beschreiben einen Datensatz und fassen die wichtigsten Aspekte zusammen. Prädiktive Modelle ziehen induktive Inferenzen auf Basis des zugrunde liegenden Datensatzes, um auf neuen, nicht gesehenen Daten Voraussagen treffen zu können. Zur Erstellung von prädiktiven und deskriptiven Modellen können jeweils überwachte oder unüberwachte Lernmethoden

[31] http://www.nist.gov/speech/atlas/overview.html

eingesetzt werden, wobei überwachte Methoden stärker hin zur Erzeugung prädiktiver Modelle und unüberwachte Lernmethoden stärker in Richtung Erzeugung deskriptiver Modelle tendieren.

Die folgenden DM-Funktionen beschreibt JDM (JSR-73):

- Klassifikation (siehe Abschnitt 4.3)
- Regression
  Regression ist von der Funktionalität her ähnlich wie die Klassifikation. Es werden lediglich numerische Merkmale vorhergesagt anstatt Kategorien (i.e. nominale Merkmale). Die meisten Klassifikationsmethoden erlauben auch die Berechnung einer Regression.
- Merkmalsselektion und Gewichtung (siehe Abschnitt 3.3)
- Clustering (siehe Abschnitt 4.4)
- Association Rules
  Assoziationsregeln sind Regeln der Form "wenn A, dann B" und werden auf Basis des gegebenen Datensatzes automatisch ermittelt.

Für den JSR-247 sind weitere Funktionalitäten wie Erkennung von Anomalitäten, Umgang mit Zeitserien sowie die Extraktion von Sequenzen aus Daten geplant.

Data Mining Tasks: Aufbauend auf den angeführten DM-Funktionen definiert nun JDM die Möglichkeit Data Mining Tasks zu definieren und auszuführen. DM-Tasks können im Groben als Sammelaufgabe (i.e. Batch Job) zur Abarbeitung der spezifizierten Funktionalität am spezifizierten Datensatz angesehen werden. Es lassen sich dabei Tasks zum Erzeugen eines Modells und Tasks zur Evaluierung eines Modells unterscheiden.

Daten werden einem Task über eine so genannte Build Data Datenstruktur übergeben, welche die für die Abarbeitung benötigten Merkmale bereitstellt. Während JSR-73 nur Sammelaufgaben erlaubt, definiert der JSR-247 ebenfalls inkrementelle Aufgaben und die Auswertung von Modellen während der Erzeugung, für den Fall, dass dies vom ausgeführten Algorithmus unterstützt wird.

Objekt und Datenrepräsentation: JDM unterstützt verschiedene Datenstrukturen zur Bereitstellung von Daten und zur Erzeugung von Modellen. Dabei werden folgende Objekte unterschieden:

- Worfklow-Objekte definieren Tasks, Verbindungen zu verschiedenen Servern/Datenquellen, sie handhaben die Ausführung von Aufgaben und bieten Möglichkeiten zur Statusabfrage und Workflow-Kontrolle.
- Datenobjekte beinhalten die Datensätze an sich, URIs sowie die logische Sichtweise auf die physikalischen Daten. Zusätzlich bieten sie Funktionen zum Import und Export an.
- Parameterobjekte definieren Parameter für Tasks und Funktionen.
- Evaluierungsobjekte beinhalten Statistiken sowie Maßzahlen, die bei der Auswertung von Modellen entstehen und die Bewertung dieser erlauben.
- Strukturierungs-Objekte ermöglichen das Einbinden von Kategorien und Taxonomien.

Alle Objekte können über Web-Services verwaltet und angesprochen werden und sind damit über SOAP serialisierbar. JDM lt. JSR-247 definiert zusätzlich Funktionen zum Import und Export von PMML Modellen, wobei die Implementierung natürlich vom Anbieter abhängt.

JDM bietet eine exzellente Abstraktion für Data Mining Aufgaben und einige der hier vorgestellten Konzepte, wie z.B. der Workflow können als Grundlage zur Erarbeitung des Designs für das umzusetzende Framework heran gezogen werden. Fraglich ist ob im umzusetzenden Framework die volle Abstraktion von JDM übernommen werden soll, da es doch einen erheblich Mehraufwand zur Implementierung darstellt.

5.2.2 The Predcitive Model Markup Language - PMML

Die PMML[32] – Predictive Model Markup Language - ist ein XML-Standard zur Repräsentation von Data Mining und statistischen Modellen. Der Standard wurde von dem unter Führung der Industrie gegründeten Konsortium, der „Data Mining Group", 1998 entwickelt. Mitglieder sind unter anderem IBM und Oracle (siehe auch [PMML 2004] für einen Überblick).

PMML zielt auf die Etablierung einer Infrastruktur zum einfachen und wohl definierten Austausch von DM-Modellen (z.B. Klassifikationshypothesen, vorverarbeiteten Daten etc.) zwischen unterschiedlichen Herstellern ab. Der Fokus liegt dabei auf prädiktiven Modellen und Ensembles von Modellen. Wie bereits bei der JDM erwähnt, dienen prädiktive Modelle zur Vorhersage von neuen Merkmalen auf noch nicht gesehenen Daten, welche meist induktiv über überwachte Lernverfahren erstellt werden.

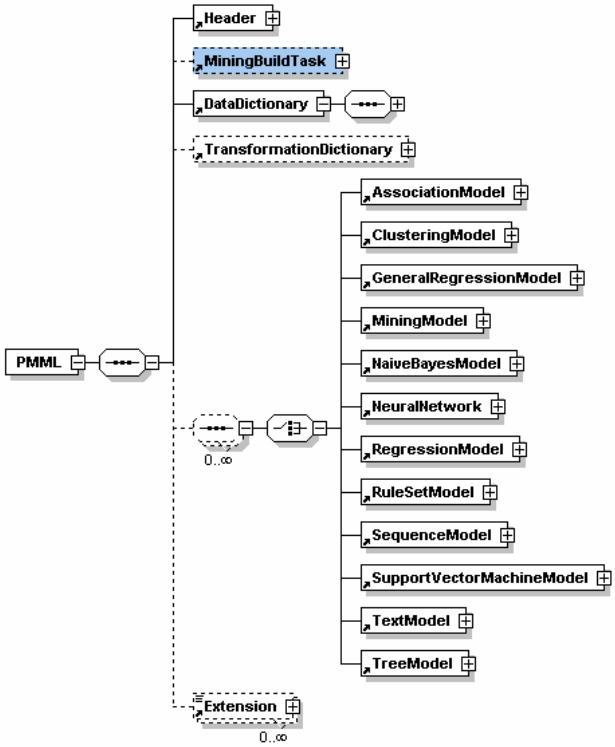


Abbildung 5-6: Visualisierung der ersten Ebene des PMML XML Schemas

[32] http://sourceforge.net/projects/pmml

Im Dezember 2005 erfolgte die Verabschiedung der Version 3.1 der PMML1, welche über 6 Jahre hinweg entwickelt wurde. PMML 3.1 besteht aus folgenden Komponenten und inkludiert im Vergleich zu dem Vorgänger auch Möglichkeiten zum Umgang mit Textdaten:

- Data Dictionary: Das DD definiert die Repräsentation der Daten und deren Merkmale, welche als Eingabe für das Modell vorliegen, und ermöglicht dabei auch die Definition von Taxonomien.
- Mining Schema: Jedem Modell ist ein Schema zugeordnet, welches die spezifischen Eigenschaften für das Schema und die aus dem DD zugeordneten Felder beinhaltet. Für jedes neue Modell muss ein entsprechendes Schema integriert werden. Beispielsweise spezifiziert ein Mining-Schema, welche Merkmale als Eingabe benutzt werden und welche Merkmale vom Modell vorher gesagt werden.
- Transformation Dictionary: Das TD definiert aus dem DD abgeleitete Felder und definiert dabei einfache Merkmalstransformationen. So kann z.B. spezifiziert werden, welche Merkmale normalisiert, diskretisiert, zusammengefasst oder einfach aggregiert werden.
- Model Statistic: Die Model Statistic beinhaltet einfache, univariate statistische Modelle wie z.B. Minimum, Maximum, Mittelwert etc. über das Modell bzw. die für die Erzeugung des Modells verwendeten Daten.
- Modell Parameter: Model Parameter definieren die Parameter, unter denen ein Modell erzeugt wurde.

Aktuell unterstützt PMML Assoziationsmodelle, Clusteringmodelle, SVMs, Data Mining Modelle, Textmodelle, Neuronale Netzwerke, Naive Bayes Modelle, Rule Sets, Sequenzmodelle und Tree Modelle (siehe auch Abbildung 5-6).

5.2.3 RDF Ressource Description Framework

Das Ressource Description Framework (RDF) ist ein vom World Wide Web Konsortuim definierter Standard zur allgemeinen Beschreibung von Metadaten und Ressourcen [Lassila & Swick 1998]. Während der Fokus bei der Definition von RDF auf der Schaffung eines allgemeinen, auf XML basierenden Metadaten Modell lag, wird RDF heutzutage auch häufig zur Repräsentation von Wissen eingesetzt, wobei die Formate nicht auf XML beschränkt sind.

RDF liegt die Idee zugrunde, Aussagen über Ressourcen in Form von Subjekt-Prädikat-Objekt Tripeln zu führen. Beispielsweise weist die Aussage „Autor, ist, Michael Granitzer“ dem Subjekt „Autor“ das Objekt „Michael Granitzer“ über das Prädikat „ist“ zu. Durch Verknüpfung mehrerer solcher Tripel erhält man einen gelabelten Pseudographen, der die Abbildung komplexer Aussagen ermöglicht. Beispielsweise kann durch das voranstellen des Tripels „ Dieses Dokument hat Autor“ und aus dem vorherigen Tripel „Autor ist Michael Granitzer“ beschrieben werden, wer der Autor dieses Dokumentes ist. Subjekte sind dabei über eine URI (Uniform Resource Identifier) identifizierte Ressourcen, während Objekte eines Tripels entweder Ressourcen (wiederum identifiziert mittels einer URI) oder Literale (einfache Strings) sein können. Abbildung 5-7 zeigt einen Beispielgraphen für die Abbildung von Dublin Core. Die Anwendungen von RDF sind vielschichtig und aufgrund seiner generellen Natur auch flexibel einsetzbar.

So haben sich in den letzten Jahren auch Standards zur Wissensrepräsentation unter Anwendung von RDF entwickelt. Während ein kompletter Exkurs hier den Rahmen sprengen würde, soll noch in aller Kürze RDFS (RDF Schema) erwähnt werden. RDFS setzt auf RDF auf und spezifiziert dabei ein erweiterbares Typsystem in RDF. Ziel ist es, semantisch eindeutigere RDF Beschreibungen zu erzeugen. Beispielsweise kann der RDF

Graph aus Abbildung 5-7 auch anders dargestellt werden, um Dublin Core abzubilden; Karl Mustermann ist ein Konzept der Klasse Person, welches jedoch in dem vorliegenden Graphen nicht eindeutig spezifiziert ist.

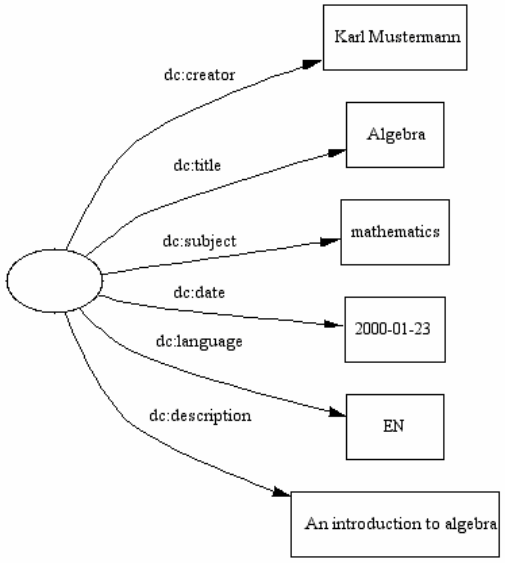


Abbildung 5-7. Beispiel eines RDF Graphen für die Spezifikation eines Dublin Core Datensatzes (Quelle: [Kokkelink & Schwänzl 2002])

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#" xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
xmlns:foaf="http://xmlns.com/foaf/0.1/" xmlns:admin="http://webns.net/mvcb/">
        <foaf:Person>
                <foaf:name>Michael Granitzer</foaf:name>
                <foaf:title>DI</foaf:title>
                <foaf:givenname>Michael</foaf:givenname>
                <foaf:family_name>Granitzer</foaf:family_name>
                <foaf:nick>Grani</foaf:nick>
                <foaf:workplaceHomepage rdf:resource="http://www.know-center.at"/>
                <foaf:knows>
                        <foaf:Person>
                                <foaf:name>Armin Ulbrich</foaf:name>
                                <foaf:mbox rdf:resource="mailto:aulbrich@know-center.at"/>
                                <rdfs:seeAlso rdf:resource="http://www.know-center.at"/>
                        </foaf:Person>
                        <foaf:Person>
                                <foaf:name>Peter Scheir</foaf:name>
                                <foaf:mbox rdf:resource="mailto:pscheir@know-center.at"/>
                                <rdfs:seeAlso rdf:resource="http://www.know-center.at"/>
                        </foaf:Person>
                </foaf:knows>
        </foaf:Person>
</rdf:RDF>
```

Abbildung 5-8: FOAF Beispiel für Michael Granitzer

RDFS ermöglicht nun die Spezifikation von Klassen bzw. Konzepten für Ressourcen sowie die Möglichkeit, diese hierarchisch in Beziehungen zu setzen. Zusätzlich erlaubt RDFS auch die Zuordnung von Klassen zu Prädikaten und die Angabe von hierarchischen Beziehungen für Prädikate. Mit diesen, und noch weiteren Ausdrucksmöglichkeiten, erlaubt RDF die detaillierte Spezifikation einer Domäne durch Konzepte und

Beziehungen zwischen Konzepten. Durch den hohen Grad an Strukturierung können Ressourcen formal und damit maschinenverstehbar beschrieben werden. Für weitere Details sei auf die W3C[33] Homepage verwiesen.

Eine weit verbreitete Anwendung von RDFS ist das FOAF – Friend of a Friend – Schema. FOAF erlaubt die Spezifikation von Visitenkarten und das In-Beziehung-Setzen dieser zu anderen FOAF Beschreibungen. Damit ist beispielsweise der Aufbau von sozialen Netzwerken möglich.

Abbildung 5-8 zeigt ein Beispiel für einen FOAF[34] Eintrag von Michael Granitzer. foaf:Person ist dabei eine RDFS Klasse welche durch Name, Titel etc. beschrieben werden kann. Eine foaf:Person hat die Möglichkeit auf weitere foaf:Personen zu verweisen, wodurch der Aufbau sozialer Netzwerke möglich ist.

Zur Speicherung von RDF Daten kommen meist relationale Datenbanksysteme zum Einsatz, oft als Tripel Stores bezeichnet. Relationale Datenbanksysteme eignen sich gut, um mit der Menge an anfallenden Tripel umzugehen. RDF Frameworks wie Jena [Carroll et. al. 2004] und Sesame [Broekstra et. al. 2002] bieten dabei umfassende Funktionalität im Umgang mit RDF/RDFS an und erlauben auch das Anbinden unterschiedlichster relationaler Datenbanksysteme.

RDF stellt einen wichtigen Standard zur Strukturierung von Information bereit und besitzt die Mächtigkeit zur Vereinheitlichung unterschiedlicher Metadatenstandards. Ein Umstand, der im Bereich der Wissenserschließung zu berücksichtigen ist.

5.3 Service-orientierte Architekturen

In SOA-Architekturen sind Services eigenständige, wieder verwendbare, lose gekoppelte Komponenten, die logisch zusammengehörende Funktonalitäten anbieten und von anderen Services oder Applikationen einfach verwendet werden können. Services sind dabei über ein von der Implementierung des Services getrenntes Interface spezifiziert. Anwender von Services bedienen sich dieses Interfaces, ohne die dahinter liegende Implementierung zu kennen, d.h. es erfolgt eine strikte Trennung des „Was für Funktionen hat ein Service" vom „Wie führt ein Service Funktionen aus".

Laut [Bieber 2001] charakterisieren sich Service-orientierte Architekturen über nachfolgende Eigenschaften, von denen die wichtigsten anschließend im Detail besprochen werden.:

1. Entdeckbarkeit und dynamische Bindung
2. Abgeschlossenheit und Modularität
3. Interoperabilität
4. Lose Kopplung
5. Adressierbare Netzwerkschnittstelle
6. Grobe Schnittstellen mit größeren Funktionsblöcken
7. Ortsunabhängigkeit
8. Zusammensetzbar- und Wiederverwendbarkeit

[33] http://www.w3.org/2001/sw/RDFCore/diffs/20030123-20031010/schema.html

[34] http://www.foaf-project.org/

9. Unterstützung von Selbstheilung

Entdeckbarkeit und dynamische Bindung: In einer Service-orientierten Architektur gibt es grundlegend drei beteiligte Parteien: den Service Provider, den Service Consumer und die Registry (siehe Abbildung 5-9). Service Provider stellen dabei Services, die dazugehörige Schnittstelle, die dahinter liegende Implementierung sowie ein exakte Beschreibung der Services zu Verfügung. Die zur Verfügung gestellten Services publiziert der Service Provider in einer Service Agency oder Service Registrierung. Service Clients benutzen diese Service Agency um für den jeweiligen Anwendungszweck die richtigen Dienste zu finden und damit verbunden auch den Service Provider, der diesen Dienst anbietet. Das gewünschte Service kann nun vom Service Provider angefordert und an den Client gebunden werden, der unabhängig von der Implementierung das Service und damit die gewünschte Funktion ausführt. Diese Vorgehensweise wird im SOA Umfeld auch Publish-Find-Bind [Champion et. al. 2002] oder Find-Bind-Execute [Potts 2003] Pattern genannt.

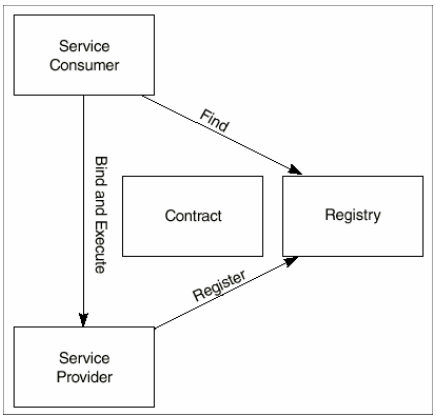


Abbildung 5-9: Das Find-Bind-Execute Pattern in einer Service-orientierten Architektur (Quelle: http://www.developer.com/java/web/article.php/10935_2207371_1)

Der Unterschied zu herkömmlichen Softwarearchitekturen ist, dass in SOA's der Client die vorhandenen Services bis zum Zeitpunkt der Suche nicht kennt und somit Service Clients entkoppelt von Service Providern existieren. Lediglich das Interface und die Spezifikation des Services ist über den Service Contract bekannt. Der Contract spezifiziert dabei das Format der Anfragen und der retournierten Ergebnisse sowie Pre-/Postconditions und die Quality-of-Service Beschreibung. Preconditions geben dabei an, welche Voraussetzungen für die Ausführung eines Services gegeben sein müssen. Beispielsweise könnte ein Service für Kreditkartenbezahlung die Nummer der Kreditkarte als Precondition benötigen. Postconditions im Gegensatz definieren das Ergebnis nach Abarbeitung eines Services. Die Quality-of-Service Spezifikation gibt die nicht funktionalen Aspekte eines Services, wie z.B. benötigte Zeit zur Ausführung, an.

Abgeschlossenheit und Modularität: Einer der wichtigsten Aspekte einer Service-orientierten Architektur ist die Modularität der Services. Services sollten dabei möglichst abgeschlossene funktionale Einheiten kapseln, welche gleichzeitig aber einfach und flexibel miteinander kombinierbar sein sollten. Je feingranularere Funktionalitäten ein Dienst bereitgestellt, umso größer ist zwar die Flexibilität diesen Dienst mit anderen zu kombinieren, aber umso schwieriger ist auch die Kombination der Dienste auf dieser feingranularen Ebene. Die Kombination von Diensten erweist sich im Falle von grobgranularen Diensten als leichter, jedoch fehlt hier die Flexibilität.

Service-orientierte Architekturen zielen auf die Wiederverwendbarkeit von Modulen ab, um nicht nur in einem Anwendungsszenario einsetzbar zu sein. Daraus folgt, dass ein Service eventuell in Anwendungen eingesetzt wird, die bei der Entwicklung nie bedacht wurden. Je grobgranularer Services sind, umso schwieriger ist die Einbettung in unterschiedlichen Anwendungsszenarien. Kriterien, die für die vorliegende Arbeit ebenfalls von Wichtigkeit sind. Für eine detaillierte Diskussion über die Modularität von Softwarekomponenten sei auf [Meyer 1997] verwiesen.

Interoperabilität: Service-orientierte Architekturen sollten im Allgemeinen einen hohen Grad an Interoperabilität besitzen. Unabhängig von der Plattform des Service Consumers oder des Service Providers, oder der jeweils verwendeten Programmiersprache soll die Kommunikation zwischen beiden möglich sein. Interoperabilität ist aufgrund vieler proprietärer Herstellerformate schwer zu erreichen, wenn gleich aktuelle Bestrebungen wie Web Services und XML Standards dem entgegen wirken.

Lose Kopplung: Eine lose Kopplung entsteht, wenn zwischen Services wenige, dafür gut bekannte Abhängigkeiten existieren. Je mehr Abhängigkeiten existieren, umso enger ist die Kopplung der Services. Das Ziel wenig Abhängigkeiten zu schaffen, geht einher mit dem Ziel einer hohen Erweiterbarkeit und Änderbarkeit eines Systems. Existieren viele Abhängigkeiten so kann die Änderung eines Services die Änderung vieler Services nach sich ziehen, und somit eher eine Auswirkung auf bereits bestehende Applikationen haben. In Service-orientierten Architekturen wird die lose Kopplung auch über das Find-Bind-Execute Pattern erreicht, da der Service Consumer erst zum Zeitpunkt des Bindens eine Abhängigkeit zum jeweiligen Service Provider schafft. Trotzdem ist es oft schwer eine lose Kopplung zu erreichen. Vor allem bei datenintensiven Services ist meist eine enge Kopplung, zur Reduzierung des Datentransfers, nötig.

Adressierbare Netzwerkschnittstelle & Ortsunabhängigkeit: Diese Eigenschaft charakterisiert einfach die Notwendigkeit, dass auf Services über eine eindeutige Schnittstelle zugegriffen werden kann. Die Art der Adressierung hängt natürlich vom jeweiligen Netztypus ab. Eine weitere in diesem Zusammenhang wichtige Eigenschaft ist die Ortsunabhängigkeit. Darunter fällt die Voraussetzung, dass der Ort eines Service für den Service Consumer transparent ist. Dadurch kann das Service an verschiedenen Orten oder sogar auf verschiedene Orte verteilt ausgeführt werden.

Unterstützung von Selbstheilung: Die Unterstützung von Selbstheilung behandelt den Aspekt, dass sich eine Applikation nach Auftreten eines Fehlers ohne menschliches Zutun erholt und weiter arbeitet. Mit der Größe einer Applikation steigt die Wichtigkeit dieses Aspektes. Die Zuverlässigkeit der Services fällt auch in diese Kategorie, denn bei besserer „Selbstheilung" steigt auch die Verfügbarkeit eines Services. In Service-orientierten Architekturen kann dies beispielsweise über das Finden und Ausführen eines anderen Services erfolgen, für den Fall, dass das vorherige Service fehlerhaft terminiert. Nichtsdestotrotz steckt die Technik hier noch in den Kinderschuhen, wenn gleich mittels Selbstheilung ein Schritt in die richtige Richtung zur Erhöhung der Verfügbarkeit von Systemen unternommen wird.

Zusammensetzbar- und Wiederverwendbarkeit: Zusammensetzbarkeit und Wiederverwendbarkeit ist die letzte hier betrachtete Eigenschaft Service-orientierter Architekturen und ist neben der Modularität der wichtigste Aspekt von Service-orientierten Architekturen. Wie oben erwähnt ist bei der Erstellung von Services nicht genau spezifiziert, in welchen Anwendungsszenarien diese eingesetzt werden können. Je autarker ein Service ist bzw. je modularer und unabhängiger die Funktionalität definiert ist, umso leichter kann dieses wieder verwendet werden. Dies gilt, wie oben erwähnt, vor allem bei relativ feingranularen

Diensten. Problematisch hierbei erweist sich meist die Zusammensetzbarkeit, da durch die Anzahl der Dienste schnell die Übersichtlichkeit und damit verbunden die Anwendbarkeit verloren gehen.

Aus dieser Problemstellung ergibt sich die in Abbildung 5-10 dargestellte SOA Pyramide, welche verschiedene Abstraktionsebenen von Diensten spezifiziert (für Details siehe [Papazoglou & Georgakopoulos 2003]).

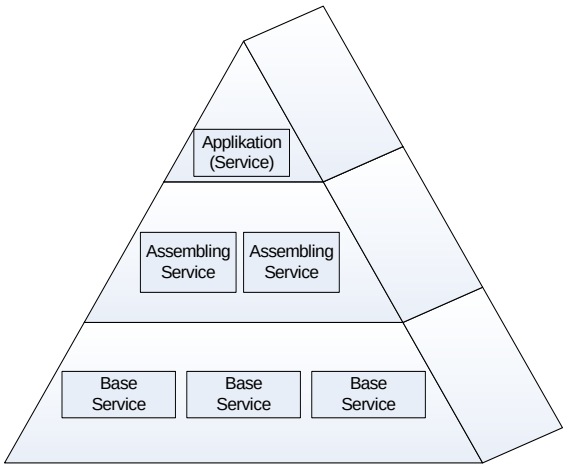


Abbildung 5-10: SOA Pyramide mit den unterschiedlichen Service Ebenen

Grundlegend kann zwischen Basis-Services, Assembling oder Composite Services und Application Services unterschieden werden. Basis-Services spezifizieren dabei die kleinsten Service Einheiten, so zusagen die grundlegenden feingranularen Bausteine. Auf diese setzen die Assembling Services auf und kombinieren Basis-Services zu grobgranulareren Services, wobei in die Kombination natürlich bereits erzeugte Assembling Services einfließen können. Zu berücksichtigen ist jedoch, dass mit dem Steigen der Anzahl der Assembling Services sich auch die Abhängigkeit zwischen den Services erhöht. Schlussendlich erfolgt die Kombination von Assembling und Basis-Services im Application Service, welches Funktionen für bestimmte Applikationen bereitstellt.

Für alle Zusammenschaltungen von Services gilt dabei, dass diese nicht notwendigerweise programmatisch hart verdrahtet sein müssen, sondern auch dynamisch bei der Abarbeitung erfolgen können. Die Ausführung einer einzelnen Transaktion unter Anwendung mehrerer Services, die im Allgemeinen einen Geschäftsprozess beschreibt, nennt man Service Orchestration. Dabei gilt es insbesondere auf die Behandlung von Fehlern und Transaktionen zu achten, denn falls ein Service in der Servicekette fehlerbehaftet terminiert, muss die Wiederherstellung des Ausgangszustandes erfolgen. Eine Voraussetzung, welche nicht einfach zu realisieren ist.

Ein weiterer wichtiger Punkt in der Erhöhung der Flexibilität bei der Orchestrierung von Services ist die maschinenlesbare Beschreibung von Services. Während die Orchestrierung natürlich auch hart verdrahtete (i.e. programmatisch) erfolgen kann, ist es das Ziel, eine dynamische, an die aktuellen Bedürfnisse des Service Consumers angepasste Verschachtelung von Diensten zu erreichen. Dazu müssen die Preconditions, Postconditions und Quality-of-Service Aspekte von Services klar spezifiziert sein. In eine solche Richtung gehen aktuell Entwicklungen im Bereich der Semantic Web Services (SWS), bei denen semantische Beschreibungssprachen wie RDF Services maschinenlesbar beschreiben. Diese Beschreibung ermöglicht nun die dynamische Orchestrierung je nach Anwendungsfall. SWS fokussieren, wie der Name schon sagt, auf Web

Services als Service-orientierte Architekturen. Wichtig dabei ist, dass Web Services momentan nicht alle Eigenschaften von Service-orientierten Architekturen, wie z.B. das Find-Bind-Execute Pattern aus Ermangelung einer zentralen Registrierung, unterstützen.

Nichtsdestotrotz stellen Web Services einen wichtigen Trend im Umgang mit dezentralen Ressourcen im WWW dar und über die semantische Erweiterung ist man der Realisierung von Tim Berners Lee's Vision von unabhängigen Agenten einen Schritt näher. Für die vorliegende Arbeit spezifizieren Service-orientierte Architekturen einen hervorragenden Rahmen zur Bereitstellung von Knowledge Discovery Funktionalität.

5.4 Anforderungen

Unter Betrachtung der Text-Mining Frameworks lassen sich wichtige Anforderungen für das umzusetzende Framework ableiten. Die Anforderungen sind dabei fett im Text hervorgehoben.

GATE und UIMA vollziehen eine klare **Trennung zwischen Repräsentation der Daten und Algorithmen,** wodurch eine getrennte Entwicklung dieser beiden Komponenten und eine anwendungsspezifische Kombination möglich sind. GATE stellt dabei die Annotation von Textkorpora in das Zentrum und weist auf die Wichtigkeit der **Möglichkeit zur Annotation von Objekten** hin; es zeigt die daraus resultierende Mächtigkeit eines Frameworks im Umfeld KD. Die Stärke von UIMA und ATLAS ist jeweils in der **klaren Beschreibung der Merkmals- und Annotationstypen** zu sehen, wodurch der Umgang mit verschiedenen Merkmals- resp. Annotationsarten klar spezifiziert und abstrahiert wird. Der Funktionsumfang von WEKA und GATE, sowie deren Verfügbarkeit als Open Source Frameworks deutet auch auf eine I**ntegration von WEKA und GATE** zur Erhöhung und Verbesserung der Funktionalität sowie der Stabilität der Algorithmen hin

Analysiert man die KD-Standards, so ist für das umzusetzende Framework vor allem die **Definition der einzelnen Modelle/Algorithmen** ableitbar. Durch die Definition des Workflows in JDM sowie der notwendigen Eingangsgrößen und durch die beschriebenen Merkmale im PMML ist eine semantisch höherwertige Definition von KD-Methoden und –Techniken möglich.

Aus der Analyse von möglichen Repräsentationsformen von Wissen lassen sich Anforderungen an die Ausdrucksmöglichkeiten der Datenstrukturen ableiten. Ein weiterer wichtiger Punkt ist die Strukturierung des Informationsbestandes. Zur semantisch hochwertigen und standardisierten Abbildung von Informations-/Dokumentbeständen sollte RDF bzw. RDFS bei der Strukturierung des Informationsraumes berücksichtigt werden. Allgemeiner gesprochen sollte die **Datenstruktur des umzusetzenden Frameworks semantische Beschreibungssprachen berücksichtigen** und **Transformation von Datenquellen in Standards der Wissensrepräsentation technische ermöglichen**.

Hinsichtlich Service-orientierter Architekturen zeigt die Analyse in Abschnitt 5.3 einen klaren Trend in Richtung verteilter, lose gekoppelter Systeme. Auch UIMA verfolgt diesen Ansatz und zeigt dessen Anwendbarkeit auf. Im umzusetzenden Framework sollte daher eine **Service-orientierte Architektur zur Bereitstellung von KD-Diensten** berücksichtigt werden. Die Entwicklung einer SOA beinhaltet auch die **gleichzeitige Abarbeitung unterschiedlicher Algorithmen** und damit verbunden die Unterstützung **konkurrenter Zugriffe auf den Informationsraum**.

Die bisher erarbeiteten Anforderungen deuten auf ein breites Funktionsspektrum für das umzusetzende Framework hin.

6 Überblick über die Zyklen der Entwicklung

Im folgenden Abschnitt werden die einzelnen Iterationsschritte, welche zur Entwicklung der konzeptionellen Grundlagen des KnowMiner-Frameworks (siehe Kapitel 7) und der Implementierung des Modells (siehe Kapitel 8) geführt haben, beleuchtet und die aus den Iterationsschritten abgeleiteten Schlüsse und Erkenntnisse betrachtet. Die Entwicklung erfolgte grundlegend in zwei Iterationen, wobei die erste Iteration bereits auf Erkenntnissen aus Projekten, an denen der Autor in hohem Umfang beteiligt war, aufsetzen konnte.

Auf Basis dieser Erkenntnisse und auf Basis der Analyse des KD-Prozesses (siehe Abschnitt 7.1) erfolgte eine erste Konzeption und Implementierung des KnowMiner-Frameworks, in der versucht wurde, die unterschiedlichen Aspekte aus den Projekten WebRat, InfoSky und der hierarchischen Dokumentklassifikation (Abschnitte 6.1.1, 6.1.2 und 6.1.3) zu integrieren und Abstraktionsmechanismen zwischen Daten und Algorithmen zu schaffen. Diese Version des KnowMiner-Frameworks diente als Grundlage zur Umsetzung der Anwendungen MAGICK und onAIR (On APA Intelligent Retrieval, Abschnitte 6.2.1, 6.2.2). Bei MAGICK wurde die WebRat Funktionalität auf multimediale Datenbestände erweitert, in onAIR erfolgte die Umsetzung und Anwendung der Retrievalfunktionalität aus InfoSky sowie der unterschiedlichen Textvorverarbeitungsmethoden, Clustering und Ordinationsverfahren. Um strukturierte Dokumentmengen berücksichtigen zu können, erfolgte des weiteren die Umsetzung der hierarchischen Textklassifikation im KnowMiner-Framework, welche in einer Diplomarbeit zum Thema Focused Crawling angewendet (siehe [Juffinger 2005]) und evaluiert wurde.

Aus den Erkenntnissen der einzelnen Projekte konnten weitere Anforderungen und Verfeinerungen der KnowMiner-Architektur abgeleitet werden, welche in die Entwicklung des für diese Arbeit finalen KnowMiner-Frameworks einflossen. Das für diese Arbeit finale KnowMiner-Framework wird in Kapitel 8 beschrieben, Details zur den konzeptionellen Grundlagen dazu liefert Kapitel 7.

6.1 Analyse bereits existierender Applikationen: WebRat, InfoSky und hierarchische Dokumentklassifikation

Der erste Iterationsschritt, welcher auch die Grundlage für die Entwicklung des KnowMiner-Frameworks darstellt, besteht aus den zwei Applikationen WebRat und InfoSky, die im Folgenden kurz illustriert und analysiert werden. Hintergrund für die Entwicklung beider Applikationen war das Bedürfnis textuelle Datenbestände zu analysieren und Zusammenhänge zwischen einzelnen Textdokumenten darstellen zu können.

InfoSky fokussiert dabei auf die Analyse und Visualisierung großer strukturierter Textdatenbestände, wie sie in verschiedenen Wissensmanagementsystemen mit vorgegebener Ablagestruktur vorhanden sind. Ziel von InfoSky ist es dabei, einen visuellen Überblick über den gesamten Dokumentbestand einer solchen Ablagestruktur zu geben und eine Alternative zum Baum-basierten Browsing solcher Strukturen zu bieten.

Die WebRat fokussiert auf die Online-Analyse und Visualisierung von Ergebnismengen von Suchmaschinen, wobei nur die von der Suchmaschine zurück gelieferte Trefferumgebung als Analysegrundlage dient.

Beide Applikationen teilen sich ein gemeinsames Repräsentationsmodell für Dokumente, einen ähnlichen Visualisierungsansatz und führen ähnliche Berechnungen durch. Im Folgenden werden beide Applikationen

kurz beschrieben, um im Anschluss die daraus abgeleiteten Erkenntnisse darzustellen. Für Details zu den beiden Systemen sei der Leser auf [Sabol et. al. 2002, Andrews et. al. 2002] verwiesen.

Es sei auch angemerkt, dass Teile der Beschreibungen direkt aus den Veröffentlichungen bzw. aus internen technischen Know-Center Berichten, an denen der Autor mitwirkte, entnommen sind bzw. übersetzt wurden.

6.1.1 WebRat

Ziel der WebRat ist die Unterstützung der Identifikation von thematischen Gruppen in Suchresultaten durch dynamisches, inkrementelles Clustering sowie die automatische Generierung von Schlüsselwörtern und Zusammenfassungen. Als Resultat einer mit WebRat durchgeführten Suche wird eine Themenkarte generiert, die laufend modifiziert wird während neue Ergebnisse von den angegebenen Datenquellen eintreffen. In der Grundversion als Web-Metasuchmaschine beginnt ein Suchprozess in der WebRat mit der Eingabe einiger, meist allgemeiner Suchbegriffe durch den Anwender. Diese Suchbegriffe werden vom System an alle definierten Datenquellen wie etwa Suchmaschinen oder Datenbanken geschickt; die zurück gelieferten Resultate werden verarbeitet und als Themenkarte angezeigt. In dieser Themenkarte kann der Anwender navigieren, um einen Eindruck vom Aufbau des Suchresultats zu erhalten.

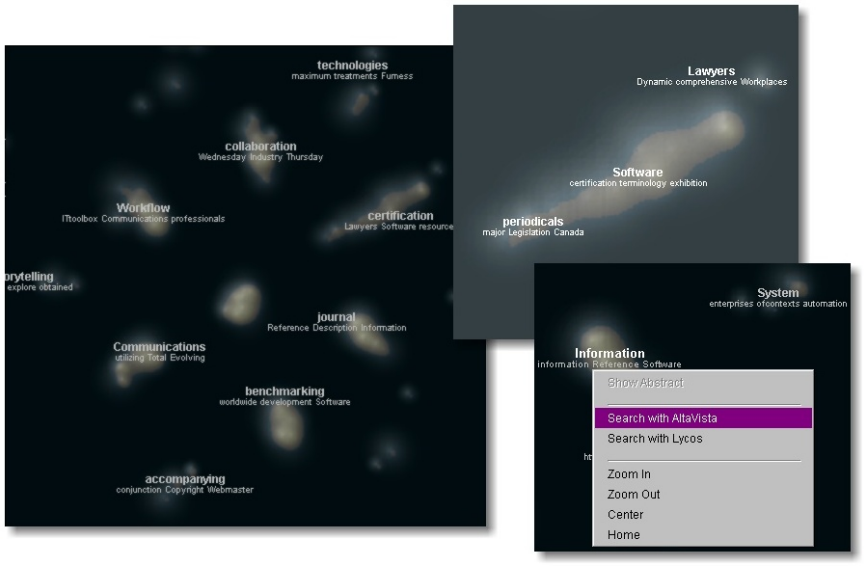


Abbildung 6-1: Links Visuelle Darstellung von Suchergebnissen zum Thema Knowledge Management. Rechts Oben: durch zooming vergrößerter Ausschnitt, Rechts Unten: Query Refinement mit dem Begriff Information

Durch einfache Interaktionen kann eine erneute Suche mit Einschränkung auf eine gewählte Suchmaschine und einen gewählten Themenbereich erfolgen, womit WebRat zu einem Hilfsmittel zur schnellen, themenbasierten Verfeinerung von Suchresultaten wird. Abbildung 6-1 zeigt die Darstellung von solchen Suchergebnissen. Das WebRat-Framework eignet sich zur Erstellung und Visualisierung von Übersichten über Suchresultate oder Bestände von Wissensobjekten von einigen hundert bis einigen tausend Wissensobjekten, von denen jeweils eine kurze Zusammenfassung oder der Volltext bekannt sind. Benutzer erhalten somit einen raschen Überblick über ein bestimmtes Gebiet bzw. eine bestimmte Dokumentmenge.

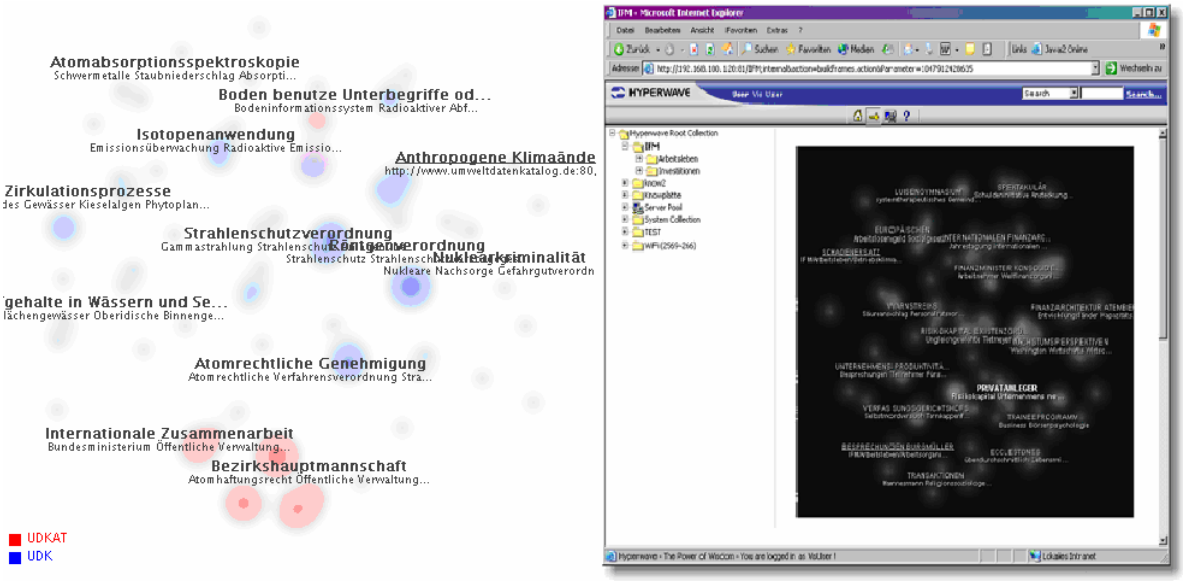


Abbildung 6-2: Unterschiedliche Anwendungsmöglichkeiten der WebRat. Links zum Vergleich von verschiedenen Suchergebnismengen und rechts zur Visualisierung der Zusammenhänge eines Verzeichnisses im Wissensmanagementsystem Hyperwave

Die Analyse von Suchergebnissen ist nur eine Anwendungsmöglichkeit der WebRat. Weitere Anwendungen sind die Analyse von Verzeichnissen in einem Wissensmanagementsystem (siehe Abbildung 6-2 rechts) und der Vergleich von verschiedenen Suchmaschinen (siehe Abbildung 6-2 rechts).

WebRat basiert auf einem Vector-Space-Model, welches unter Verwendung einer Charakter n-gram Zerlegung erstellt wird. Als Gewichtungsschema kommt hierbei das wohlbekannte Term Frequency/Inverse Document Frequency Termschema zum Einsatz (siehe Abschnitt 3.3.4). Die textuellen Daten werden dabei mittels HTTP Request von den unterschiedlichen Webseiten geladen. Das Laden und Vektorisieren der Dokumente wird dabei von dem in Abbildung 6-3 oben dargestellten *Grabber* übernommen, der die Vektoren dabei im geteilten Datenpool ablegt (Shared-Komponente in Abbildung 6-3).

Die Mapping-Komponente (Mitte Abbildung 6-3) transformiert den so erstellten hochdimensionalen Vektorraum mittels eines Force Directed Placement Algorithmus (siehe Abschnitt 4.1.2.1) in einen 1D-, 2D- oder 3D-Raum, wobei die hochdimensionalen Distanzen so gut wie möglich erhalten bleiben. Dieser 2D-Raum stellt auch die Grundlage für die Visualisierungskomponente dar, welche den 2D Raum entsprechend am Bildschirm darstellt.

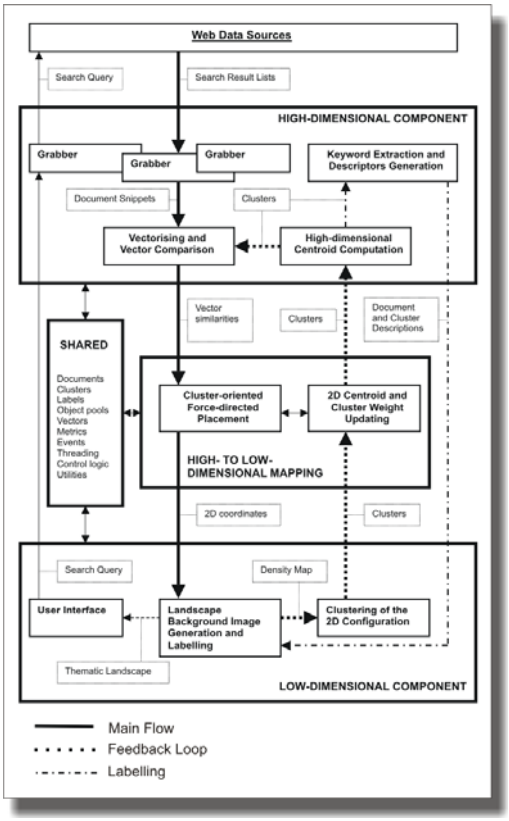


Abbildung 6-3: Überblick über die einzelnen WebRat Systemkomponenten

Das Laden von Dokumenten und die Transformation des Vektorraums erfolgen dabei dynamisch, d.h. während der Darstellung bzw. Transformation werden weitere Dokumente nachgeladen, vorverarbeitet und in die Transformation integriert, wodurch sich der Effekt einer sich entwickelnden Wissenslandkarte ergibt. Gleichzeitig werden auch die Ergebnisse der Visualisierung dazu verwendet, ein automatisiertes visuelles Clustering durchzuführen, dessen Ergebnisse in den Transformationsprozess rückgeführt werden. Dadurch kann der Transformationsprozess beschleunigt und zusätzlich an die visuelle Repräsentation angepasst werden. Für weitere Details sei der Leser auf die oben angeführten Publikationen verwiesen.

6.1.2 InfoSky

Die Beschreibung von InfoSky ist [Kappe et. al. 2002] entnommen, da dieser Artikel einen ausgezeichneten Überblick über die InfoSky Funktionalität bietet. Der Autor dieser Arbeit war an der Publikation ebenfalls beteiligt.

Während die WebRat auf die Analyse vollkommen unstrukturierter Dokumente aufbaut, fokussiert InfoSky auf die Analyse von schon vorstrukturierten Dokument-/Informationsbeständen und versucht über Ausnutzung der Strukturierung die Verfahren hinsichtlich Geschwindigkeit und Genauigkeit zu verbessern. InfoSky ist ein System, das Anwendern die effiziente Erforschung sehr großer, hierarchisch strukturierter Dokumentablagesysteme ermöglicht. Hierzu steht eine interaktive Visualisierung zur Verfügung, die - ähnlich einem echten Teleskop - eine planare grafische Repräsentation zeigt, welche geschwenkt und in beliebiger Vergrößerung betrachtet werden kann. Dokumente werden dabei als Sterne dargestellt; ähnliche Dokumente werden in geometrischer Nähe zueinander positioniert, wodurch thematisch verbundene „Sternhaufen“

entstehen. Elemente der Hierarchie werden in Form von Linienzügen, welche Sternhaufen umschreiben, angezeigt, wobei ähnliche Elemente wiederum in geometrischer Nähe zueinander platziert werden. Beschriftungen werden je nach Bedarf eingeblendet. Das Resultat ist eine Visualisierung, die erstaunlich an die bekannten astronomischen Aufnahmen von Galaxien erinnert und eine große Menge Wissen auf den ersten Blick expliziert. So ist etwa das häufige Problem, einem Themenkreis verwandte Dokumente aufzufinden, in InfoSky dadurch gelöst, dass alle ähnlichen Dokumente an einer Stelle zu finden sind. Auch kann aufgrund von Position und Dichte der Sternballungen eine intuitive Abschätzung der Strukturierung des gesamten Dokumentbestandes erfolgen. Für spezifische Suchen stehen mehrere Varianten der Resultatvisualisierung zur Verfügung, die unter anderem mit der Animation einzelner Elemente und mit Farbcodierung von Relevanzwerten arbeiten. In seiner letzten Ausbaustufe ist das System in der Lage, mehrere Millionen Dokumente in der beschriebenen Art interaktiv darzustellen und dabei wichtige Metainformationen wie Benutzerrechte zu berücksichtigen (siehe auch [Kappe et. al. 2002]).

InfoSky geht davon aus, dass die Dokumente einen signifikanten Text-Inhalt besitzen, der extrahiert und zur Bestimmung einer Ähnlichkeit zwischen den Dokumenten herangezogen werden kann, also z.B. HTML, PDF, oder Office-Dokumente.

Der Zugriff auf die Objekte (Dokumente und Ordner) kann durch Zugriffsrechte eingeschränkt sein. Abhängig von ihren Rechten, können Benutzer dadurch bestimmte Teile des Informationsraumes nicht sehen (der entsprechende Teil bleibt schwarz). Meta-Information, die mit den Objekten gespeichert wird (z.B. Autor, Erzeugungsdatum, Größe), kann von InfoSky verarbeitet und dargestellt werden, aber die räumliche Anordnung der Objekte beruht auf dem Inhalt der Dokumente.

Das Teleskop ist die Metapher für die Interaktion mit der Visualisierung. Benutzer können das Teleskop auf bestimmte Objekte richten, und über den Nachthimmel hin- und herfahren. Die Vergrößerung des Teleskops ist variabel, sodass man sowohl einen Überblick über den gesamten Raum haben kann, als auch durch eine „Zoom“ Operation auf kleine Details fokussieren kann.

Um die Navigation zu vereinfachen, stehen den Benutzern verschiedene Hilfsmittel zur Seite. So kann man sehr einfach den Fokus auf ein bestimmtes Objekt legen, welches dann in optimaler Größe dargestellt wird. Um den Schirm nicht durch eine Vielzahl von Informationen zu überladen, werden diese automatisch je nach Situation dargestellt bzw. weggelassen. Eine History-Funktion erlaubt das einfache Zurückgehen in der Navigation.

Benutzer können eine Suchanfrage eingeben, wobei als Resultat einzelne Sterne oder ganze Galaxien zurückgeliefert werden können. Die Objekte, auf die die Such-Anfrage zutrifft, werden hier (bis zu einer einstellbaren Relevanz) in der Darstellung durch Farben hervorgehoben. Dadurch kann man sehr leicht einen Überblick erlangen, „wo“ sich Suchresultate häufen, und diese Gebiete daraufhin genauer erforschen. Abbildung 6-4 zeigt die InfoSky Darstellung der DMOZ Computers Subhierarchie mit ca. 120.000 Dokumenten.

Aus dem Blickpunkt der KD-Techniken verwendet InfoSky ähnliche Algorithmen und Techniken wie die WebRat. Textdokumente werden in Worte zerlegt und als Wortvektoren repräsentiert. Die Gewichtung erfolgt bezogen auf den Ordner, in dem ein Dokument vorkommt und bezogen auf die Gesamtverteilung der Worte im gesamten Datenbestand, wobei wiederum ein TFIDF-Gewichtungsschema angewandt wird. Dokumente werden mittels Force Directed Placement Algorithmus platziert, wobei die Platzierung innerhalb eines Rechteckes erfolgt. Ein gewichteter Voronoi-Algorithmus dient der optimalen Platzierung der Ordner,

um möglichst wenig unnötige Freiräume zur erlauben, in welche die FDP-Rechteck Platzierungen eingeschrieben werden.

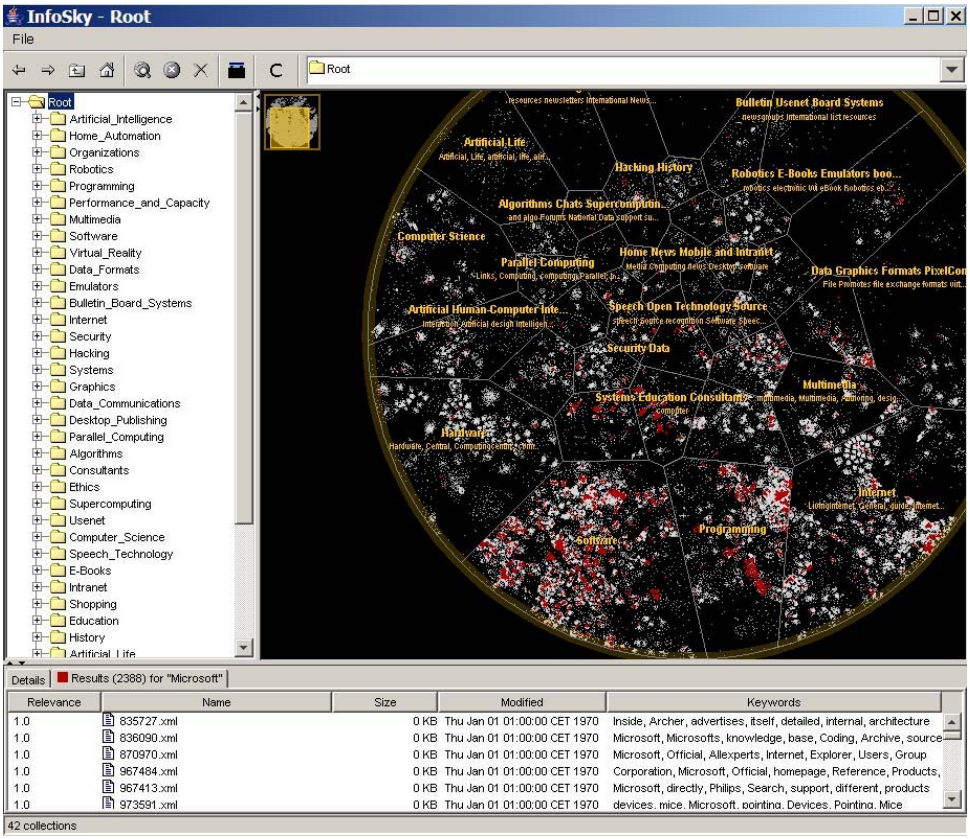


Abbildung 6-4: InfoSky Darstellung der DMOZ[35] Computers Subhierarchie mit ca. 120.000 Dokumenten. Rot farbkodiert ist die Suche nach dem Begriff „Microsoft“

Für die einzelnen Dokumente und Ordner werden automatisch Beschriftungen und textuelle Zusammenfassungen berechnet. Zusätzlich erfolgt die Indizierung der Dokumente in einem invertierten Index, um ein schnelles und effizientes Retrieval zu ermöglichen. InfoSky implementiert auch unterschiedliche Möglichkeiten zur Datenhaltung. So sind einerseits ein Dateisystem-basiertes Ablegen der von InfoSky benötigten Daten, andererseits aber auch eine Datenbankintegration inkl. Anbindung an das Wissensmanagementsystem Hyperwave realisiert. Die unterschiedlichen Datenhaltungssysteme sind vor allem für einen flexiblen Einsatz von InfoSky nötig.

6.1.3 DocClass: Hierarchische Dokumentklassifikation

Ähnlich wie InfoSky ging es beim KnowMiner Vorgängerprojekt „DocClass: Hierarchische Dokumentklassifikation“ um die Ausnutzung hierarchischer Strukturen zur Verbesserung von Genauigkeit und Geschwindigkeit von Textklassifikationsalgorithmen. Wie in Abschnitt 4.3 beschrieben, benötigen solche

[35] http://www.dmoz,.org

überwachten Lernverfahren zur Berechnung ihrer induktiven Klassifikationshypothese schon vorkategorisierte Trainingsbeispiele, welche im Trainingsschritt analysiert werden, um Regeln zur Zuordnung neuer Beispiele zu extrahieren.

Ähnlich wie in InfoSky, konstituiert die Ablagestruktur im Projekt DocClass ebenfalls eine IS-A Taxonomie, d.h. in den oberen Ebenen der Taxonomie befinden sich die thematisch generelleren Klassen, welche nach unten hin immer speziellere Themen adressieren. Den einzelnen Klassen ist eine Menge an Textdokumenten zugeordnet, welche als Ausprägung oder Instanzen dieser Klasse angesehen werden können. Im Allgemeinen kann ein Dokument mehreren Klassen zugeordnet werden, da Textdokumente meist mehrere Themenkomplexe abdecken. Die Struktur der Taxonomie ist zusätzlich als direkter azyklischer Graph ausgelegt, d.h. eine Klasse kann mehrere Elternklassen besitzen, wobei jedoch von einem eindeutigen Startknoten ausgegangen wird.

Im Projekt DocClass wurden Verfahren entwickelt, welche sowohl Klassifikationsgenauigkeit als auch Klassifikationszeit und Lernzeit reduzieren, indem Training und Klassifikation einem Divide & Conquer-Ansatz verfolgten. Zum Trainieren der Klassifikatoren wurden jeweils nur die den Teilbäumen der entsprechenden Klasse zugeordneten Dokumente betrachtet, anstatt alle Dokumente der gesamten Hierarchie in den Trainingsprozess einzubinden. Bei der Klassifikation erfolgte das Vorgehen ähnlich einem Pachinko-Modell, d.h. ein Dokument wird am Wurzelknoten der Taxonomie hinzugefügt und durchläuft die Taxonomie von oben nach unten. In jeder Klasse wird entschieden, ob das Dokument in der Klasse bleibt, weiter nach unten propagiert wird oder keine der beiden Möglichkeiten realisiert werden. Der Klassifikationsprozess ist beendet, wenn das Dokument in den verschiedenen Klassen hängen geblieben ist bzw. in keinen weiteren Teilbaum mehr propagiert wurde.

Die Entscheidung in den verschiedenen Klassen wurde durch verschiedene Standardklassifikationsmethoden wie z.B. Support Vektor Maschinen, Rocchio Klassifikatoren und Boosting (siehe Abschnitt 4.3) durchgeführt. Innerhalb des Projektes wurden die verschiedenen Ansätze sowie die Auswirkung von verschiedenen Ablagestrukturen (hierarchisch tief, hierarchisch breit, flach etc.) näher untersucht und miteinander verglichen. Um dies zu ermöglichen, war unter anderem die Entwicklung einer Benchmarking-Umgebung nötig, welche verschiedene Evaluierungskriterien anbot und verschiedene statistische Vergleiche zwischen den einzelnen Testläufen ermöglichte. Für Details im Hinblick auf das Klassifikationsmodell, die verwendeten Algorithmen und die Umsetzung sei auf [Granitzer & Auer 2005] und [Granitzer 2004a] verwiesen.

Neben der Ausnutzung hierarchischer Ablagestrukturen erwies sich die Anbindung unterschiedlicher Klassifikatoren sowie die Entwicklung einer entsprechenden Testumgebung als zentral, um Aussagen über die Qualität und Leistungsfähigkeit der einzelnen Algorithmen und Modelle tätigen zu können. Ähnlich wie in InfoSky mussten auch unterschiedliche Datenhaltungssysteme zur Anbindung vorgesehen werden, um die verschiedenen Datensätze einbinden zu können (e.g. Dateisystem, Oracle Datenbank, Hyperwave etc.).

6.1.4 Lessons Learned

Betrachtet man die drei Projekte InfoSky, WebRat und DocClass einzeln als KD-Anwendung, so können die folgenden gemeinsamen Eigenschaften und Charakteristika im Hinblick auf deren Umsetzung abgeleitet werden.

Anbindung verschiedener Datenquellen: Um flexibel in unterschiedlichen Szenarien einsetzbar zu sein, ist allen drei Anwendungen die Anbindung unterschiedlicher textueller Datenquellen eigen. Während sich InfoSky und DocClass mit der Anbindung einer Datenquellen pro Durchlauf begnügen, erhebt die WebRat zusätzlich die Ansprüche, verschiedene Datenquellen gleichzeitig abrufen und deren Ergebnisse in Echtzeit miteinander verbinden zu können. Die Anbindung unterschiedlicher Datenquellen kann auch als eine zentrale Eigenschaft der Datenselektion im ersten Schritt des KD-Prozesses (siehe Abschnitt 2 oben) angesehen werden und stellt für die effiziente Umsetzung und rasche Entwicklung von KD-Anwendungen einen zentralen Punkt dar.

Gemeinsame Verfahren zur Vorverarbeitung textueller Daten: In allen drei Projekten wurden ähnliche Verfahren zum Parsen und Vorverarbeiten von Dokumenten implementiert. Während die WebRat vornehmlich HTML-Seiten verarbeitet, parsen InfoSky und DocClass auch PDF-Dokumente. Nach Extraktion der Textdaten aus den verschiedenen Dokumenten erfolgt in den drei Anwendungen jeweils eine ähnliche Art der Vorverarbeitung, die Lemmatisierung und Merkmalsanalyse von Texten (siehe Abschnitt 3.1), sowie die Extraktion eines Vektormodells. Zwischen den Projekten wurde versucht, verschiedene Code-Teile auszutauschen, was jedoch aufgrund der fehlenden gemeinsamen Codebasis nur mit mäßigem Erfolg behaftet war. Die Entwicklung gemeinsamer Methoden zum Parsing von verschiedenen Dokumentformaten sowie zur Vorverarbeitung von Textanalyse-Methoden lässt sich aus allen drei Applikationen im Hinblick auf die Reduzierung der Entwicklungszeit dabei als Kriterium ableiten.

Einheitliche Repräsentation und Zugriff von Strukturen und Dokumenten: Während InfoSky auf hierarchischen Strukturen operiert, berücksichtigt die WebRat nur unstrukturierte Dokumentbestände; im Projekt DocClass werden beide Arten unterstützt. Bei der Berücksichtigung von strukturierten Zusammenhängen zwischen Dokumenten zeigte sich sowohl in InfoSky als auch im Projekt DocClass, dass man im Allgemeinen auf Basis-Algorithmen für unstrukturierte Datenbestände aufsetzt und diese für strukturierte Modelle erweitert bzw. entsprechend kombiniert. So teilen sich InfoSky und die WebRat ähnliche MDS-Algorithmen, wobei in InfoSky über spezielle Selektion und Vorverarbeitung der Dokumente die hierarchische Struktur mit berücksichtigt wird. Aus den Analysen der drei Projekte hinsichtlich Algorithmen und Strukturierung lässt sich ableiten, dass über eine einheitliche Repräsentation von strukturierten/unstrukturierten Daten bzw. einheitliche, wohl definierte Zugriffsmöglichkeiten sich die Entwicklungszeit und Stabilität von KD-Anwendungen reduzieren lassen. Zusätzlich kann über die Vereinheitlichung und Abstraktion von Operationen zwischen Dokumenten eine Trennung zwischen Daten und Algorithmen erreicht werden, wodurch eine rasche Transformation von Algorithmen eines Anwendungsfalls auf einen anderen Anwendungsfall bzw. anders gearteten Datenbestand möglich ist.

Parallelisierbarkeit: Vor allem bei der Entwicklung der WebRat zeigte sich die Notwendigkeit der Parallelisierung verschiedener Algorithmen und der Anwendung von KD-Techniken auf einem dynamischen Datenbestand. Aus mangelnder Konzeption bzw. durch Bereitstellen verschiedener abstrahierter Mechanismen zur parallelen Bearbeitung der gleichen Daten enthielt der Entwicklungsprozess der WebRat hohe Anteile an Debugging-Zeit für konkurrente Modifikationsfehler. Vor allem die Weiterentwicklung bzw. Integration neuer Techniken erwies sich dabei als äußerst problematisch, wodurch das Projekt im Prototypstadium stecken blieb. Zur Entwicklung von stabilen KD-Anwendungen, vor allem für den Multiuser-Betrieb, sind daher Möglichkeiten zur Parallelisierung von KD-Techniken essentiell und müssen, zur Reduktion der Zeit für die Fehlersuche, als Anforderung an ein allgemeines KD-Framework mit aufgenommen werden.

Client/Server Fähigkeiten: Für den Echteinsatz von KD-Anwendungen ist die Unterstützung von Client/Server-Architekturen unerlässlich. Dies zeigt auch das Beispiel InfoSky, bei dem die anfallenden Berechnungen sinnvoll nur auf einem leistungsstarken Serverrechner (bei entsprechendem Datenvolumen) ausgeführt werden können. Während die Client/Server-Fähigkeit kein essentieller Fokus bei der Umsetzung des ersten KnowMiner-Prototyps war, so wurde die Umsetzungstechnologie doch unter Berücksichtigung dieses Kriteriums gewählt.

Benchmarking: Benchmarking und der Vergleich von verschiedenen Algorithmen sind essentieller Bestandteil bei der Entwicklung von KD-Algorithmen, wie dies auch im Projekt DocClass gezeigt wurde. Benchmarking-Mechanismen sind dabei nicht nur im Bereich der Textklassifikation, sondern auch im Information Retrieval, Clustering etc. von essentieller Bedeutung für eine Selektion von „guten" Algorithmen in Abhängigkeit von einem gegebenen Datenbestand. Um die Anwendung und Integration von KD-Techniken im WM-Kreislauf im Hinblick auf Integrationszeit, Stabilität und Genauigkeit zu verbessern, sind somit die Berechnung von Performance-Maßzahlen und der Vergleich von Algorithmen essentiell.

Auf Basis der hier zusammengefassten Analyse, die im Entwicklungsteam rund um die erwähnten Projekte stattfand und an der der Autor in hohem Maß beteiligt war, wurde das erste Modell für das KnowMiner-Framework abgeleitet. Das Framework diente als Entwicklungsbasis für verschiedene Projekte und die daraus abgeleiteten Erkenntnisse flossen in den letzen Entwicklungsschritt des KnowMiner-Frameworks ein. Sowohl die abgeleiteten konzeptionellen Grundlagen, als auch die umgesetzten Projekte und abgeleiteten Erkenntnisse werden im Abschnitt 6.2 unten im Detail ausgeführt.

6.2 KnowMiner V 1.0: Entwicklung eines allgemeinen KD und IR Frameworks

Wie zuvor erwähnt, wurden die Erfahrungen in der Durchführung der Umsetzungsprojekte DocClass, WebRat und InfoSky als Anforderungen in die Entwicklung des KnowMiner-Frameworks integriert. Das KnowMiner-Framework verfolgt dabei das Ziel, eine gemeinsame Entwicklungsbasis für die Umsetzung verschiedener KD-Anwendungen in unterschiedlichen Anwendungskontexten zu schaffen und eine einfache Integration neuer Algorithmen sowie deren Evaluierung zu ermöglichen. Im Folgenden werden die Grundkonzepte, welche in der ersten Version des KnowMiner-Frameworks umgesetzt wurden, kurz illustriert.

Ziel diese Abschnittes ist es nicht, die konzeptionellen Grundlagen oder aber die Architektur des ersten KnowMiner-Frameworks im Detail auszuführen, dies passiert in den Kapiteln 7 und 8, sondern die grundlegenden Konzepte der ersten KnowMiner-Konzeption und -Umsetzung zu veranschaulichen und die aus den einzelnen Anwendungen abgeleiteten Erkenntnisse zu beleuchten.

Bei der Entwicklung des ersten KnowMiner-Frameworks standen vor allem die **Verallgemeinerung der Repräsentation** von Dokumenten mit Text- und Bildinhalten sowie die Vereinheitlichung der mit den Dokumenten assoziierten Metadaten im Zentrum. Es wurden Mechanismen geschaffen, um auf Basis eines simplen Typsystems einzelne Dokumente auf allgemeinen Merkmalsmengen abzubilden und diese Merkmalsmengen in flexibler Art und Weise miteinander zu kombinieren. Die Merkmale selbst waren nicht auf aus Texten extrahierte Merkmale beschränkt, sondern umfassten Merkmale beliebiger Modalitäten, wobei effektiv nur Bild-, Text- und Metadateninhalte angewendet wurden. Die Abstraktion der Merkmale war so ausgelegt, dass mathematische Operationen unabhängig von der dahinter liegenden Modalität und nur

abhängig von den jeweiligen Eigenschaften der Merkmale definiert werden konnten. Zusätzlich konnten verschiedene Merkmalstypen miteinander kombiniert und als Berechnungsgrundlage heran gezogen werden, wie dies z.B. in MAGICK (siehe Abschnitt 6.2.2) der Fall war.

Ein weiterer Punkt bei der Repräsentation war die Vereinheitlichung von Strukturen in die Form eines typisierten Graphen mit eindeutigem Startpunkt, dessen über typisierte Kanten miteinander verbundene Knoten je nach Typ Repräsentanten für Dokumente, Klassen, Personen etc. sein konnten. Damit konnten beliebige Strukturen realisiert und abgebildet werden. Über standardisierte Zugriffspunkte konnte der Graph in Abhängigkeit von Knoten- und Kantentypen breath-first oder depth-first traversiert werden, wobei Lockingmechanismen einen konkurrenten Zugriff sowohl auf die Graphstruktur als auch auf die Merkmalsstruktur ermöglichten.

Zur **Vereinheitlichung und Parallelisierung** der einzelnen KD-Algorithmen und -Techniken wurde das so genannte Worker-Konzept geschaffen, welches konzeptuell als State Maschine funktioniert und die thread-sichere Abarbeitung von Algorithmen ermöglicht. Worker erlauben es, Algorithmen sauber zu suspenden und bis zu einem beliebigen Zeitpunkt weiter laufen zu lassen sowie unterschiedliche Statusinformationen abzufragen oder gar einen Worker zu terminieren. Damit lässt sich unter anderem ein Load Balancing der einzelnen Worker in einer von der Implementierung der Algorithmen unabhängigen Art und Weise realisieren. Unter der Annahme, dass Worker lediglich auf der verallgemeinerten Repräsentation der Dokumente operieren, sind über deren Locking-Mechanismen auch konkurrente Zugriffe berücksichtigt.

Ein wichtiger Punkt im ersten KnowMiner-Konzept war auch die Vereinheitlichung der Anbindung von unterschiedlichen Datenquellen über ein spezielles **Import-Konzept**. Konzeptionell wurde beim Import von Daten zwischen dem Loader, dem Format-Normalisier, dem Content-Normalisier, dem Structure-Normalisier und dem Adder unterschieden, wobei diese Komponenten auch in der genannten Reihenfolge zum Einsatz kamen. Die Implementierung war, unter Anwendung des Worker Konzeptes, in einer Pipelinearchitektur hoch parallel ausgelegt. Der Loader war für das Laden der Daten verantwortlich, während der Format-Normaliser die Formate der geladenen Daten in ein internes Format überführte, welches vom Content-Normalisier in die oben erwähnte Merkmalsrepräsentation übertragen wurde. Parallel dazu überführte der Structure-Normalisier die unterschiedlichen Strukturdaten der Datenquelle. Den Abschluss bildete das Hinzufügen der inhaltlich und strukturell normalisierten Dokumente in die oben erwähnte Graph-Struktur über den GIR Adder.

Innerhalb der oben genannten Konzepte wurden **verschiedene Algorithmen** aus dem Bereich Information Retrieval, Clustering, Ordination und Textklassifikation umgesetzt. Im Speziellen waren dies Rocchio Klassifikatoren, Support Vector Maschinen, HAC und K-Means Clustering, Force Directed Placement Algorithmen sowie unterschiedliche Vorverarbeitungsmethoden wie Named Entity Extraktion und verschiedene Gewichtungsschemata wie TFIDF und Entropie Gewichtung (siehe Abschnitt 3 und 3.1 für Details). Neben den verschiedenen Algorithmen wurden auch Möglichkeiten zum **Benchmarken** von Algorithmen über Kontingenz-Tabellen sowie Precision/Recall und F_1-Werte entwickelt, welche in den Projekten entsprechend verfeinert wurden.

In den nächsten Abschnitten erfolgt die Darstellung und Analyse von auf Basis dieses KnowMiner-Frameworks umgesetzten Anwendungen, die als Ausgangspunkt für die zweite Iteration der KnowMiner-Entwicklung darstellten. Auf Basis dieses KnowMiner-Frameworks wurde das Projekt OnAIR – On APA Intelligent Retrieval mit der Austria Press Agency umgesetzt, welches thematisch auf das Retrieval und die

Analyse von Suchergebnissen ausgerichtet war (siehe auch [Kienreich et. al. 2005]). Zusätzlich wurden in der Diplomarbeit MAGICK ([Lux 2004]) die cross-modalen Fähigkeiten des KnowMiner-Frameworks und die Abstraktion zwischen Daten und Algorithmen näher untersucht, die Diplomarbeit KFetch WebMiner ([Juffinger 2005]) befasste sich näher mit den Klassifikationsfähigkeiten im Umfeld des focused Crawling.

6.2.1 OnAIR – On APA Intelligent Retrieval

Ziel im Projekt onAIR war die Verbesserung der Retrievalfunktionalität für den APA Online Manager[36]. Neben der Konzeption der Verbesserungen, wurde auch eine prototypische Implementierung unter Verwendung des KnowMiner-Frameworks durchgeführt. Im Projekt wurden folgende Retrievalfunktionalitäten umgesetzt:

1. *Suche nach ähnlichen Dokumenten:*
   Aus einem geöffneten Dokument heraus werden inhaltlich ähnliche und weiterführende Dokumente gesucht.
2. *Analyse der Trefferliste einer Suche:*
   Die von der Suche gelieferte Trefferliste wird hinsichtlich enthaltener Themen analysiert. Die gefundenen Themen werden entsprechend aufbereitet und dem Benutzer präsentiert.
3. *Automatisches Vorschlagen von relevanten Suchbegriffen:*
   Bei der Eingabe von Suchbegriffen werden dem Benutzer automatisch weiterführende Suchbegriffe vorgeschlagen. Zweck der Vorschläge ist es, große Treffermengen zu vermeiden.

Der Ablauf bei onAIR lässt sich in zwei Phasen einteilen. In der ersten Phase werden die APA-Dokumente vorverarbeitet und einer detaillierten Analyse unterzogen. Dabei erfolgt auf Basis von regulären Ausdrücken unter anderem die Extraktion von Orts- und Personenbezügen im KnowMiner - Content-Normaliser. Nach Lemmatisierung und Merkmalsanalyse erfolgt die Ablage der Ergebnisse in speziell indizierten Oracle-Tabellen, um ein effizientes Retrieval zu ermöglichen. Neben der Lemmatisierung der Ergebnisse erfolgt auch die Berechnung eines assoziativen Index, welcher zu einem beliebigen Suchbegriff weitere Suchbegriffe auf Basis einer Co-Occurence Analyse vorschlagen kann. Tabelle 6-1 zeigt die Vorschläge für das Wort „Virus“, für das Wort „Virus“ im Kontext von Computern und das Wort „Virus“ im Kontext von Menschen. Die Assoziationsindices wurden auf dem Reuters Corpus Volume 1 [Rose et. al. 2002] mit ca. 800.000 Dokumenten trainiert.

Die zweite Phase behandelt das Suchen und Darstellen der Ergebnisse über eine Client/Server-Architektur. Dazu wurde als Zwischenschicht ein Web-Service zur Kommunikation mit dem KnowMiner-Framework entwickelt, welches die Anbindung des .NET-Clients ermöglicht und die darunter liegende KnowMiner-Funktionalität kapselt. Im ersten Schritt erfolgt die Spezifikation des Informationsbedürfnisses durch Eingrenzung des Suchzeitraumes, durch Definition der zu durchsuchenden Quellen und durch Eingabe eines Suchbegriffes, wobei hier der assoziative Index zum Vorschlagen alternativer Suchbegriffe angewendet wird. Auf Basis des spezifizierten Informationsbedürfnisses wird die Suche innerhalb der Oracle-Datenbank durchgeführt und die retournierten Treffer werden einer detaillierten Analyse unterzogen. Dabei erfolgt die Vektorisierung der gefundenen Dokumente mit anschließendem Clustering, um thematische Gruppen zu

[36] http://www.aom.apa.at/ zum Stand 2003

bestimmen. Neben der Analyse der Suchergebnisse hinsichtlich thematischer Gruppen werden auch die verschiedenen, teils automatisch im Vorfeld extrahierten Metadaten analysiert und dargestellt. Abbildung 6-5 zeigt die Analyse des Suchergebnisses dargestellt im onAIR Client, Abbildung 6-6 zeigt die Darstellung der Metadaten im Zoom (Tab. Übersicht in Abbildung 6-5).

Top 7 Terms					
Virus		**Virus and Computer**		**Virus and Human**	
0,21	immune	0,25	MacAfee	0,51	immunodeficiency
0,19	infection	0,02	computers	0,44	hiv
0,16	infected	0,01	internet	0,31	aids
0,12	hiv	0,01	associates	0,17	infection
0,12	infects	0,01	infected	0,13	infected
0,12	infect	0,01	computer	0,06	human
0,09	vaccine	0,01	software	0,05	infections

Tabelle 6-1: Ergebnisse des auf dem Reuters Corpus Volume 1 trainierten Assoziationsindex (Trainingszeit 5h)

Die mittels Themenclustering automatisch extrahierten Themengruppen werden über automatische Verfahren verschlagwortet, um einen Eindruck über die Inhalte des Clusters geben zu können. Die thematischen Zusammenhänge inklusive der zeitlichen Komponenten und der entsprechenden Metadaten werden in verschiedenen Darstellungsformen (e.g. Themennetzwerk, Theme River) dem Benutzer näher gebracht, wobei alle Darstellungsformen miteinander synchronisiert sind. Selektiert der Benutzer einen Themencluster, so wird die Suchergebnisliste automatisch auf die in dem Cluster enthaltenen Dokumente eingeschränkt. Dies gilt auch für die entsprechenden Metadaten bzw. den Metadatenbaum, wie er in Abbildung 6-6 dargestellt ist.

Für die entsprechenden Analysen wird, wie bereits erwähnt, über ein anwendungsspezifisches Web-Service auf die einzelnen KnowMiner-Funktionalitäten, welche als Worker realisiert sind, zugegriffen. Dokumente sind als eine Menge von Merkmalsvektoren repräsentiert, welche sowohl die inhaltliche Komponente als auch die geographische und personenbezogene Komponente beinhalten. Durch die Trennung in mehrere Merkmalsvektoren können diese beliebig miteinander kombiniert werden.

Neben der Prototyp-Entwicklung umfasste das Projekt OnAIR auch die Evaluierung von verschiedenen Kriterien wie

1. *Qualität der Ähnlichkeitssuche und Trefferanalyse*
2. *Geschwindigkeit der Ähnlichkeitssuche und Trefferanalyse*
3. *Speicherbedarf Ähnlichkeitssuche und Trefferanalyse*

Zur Untersuchung dieser Kriterien wurden in Zusammenarbeit mit Mitarbeitern der APA DeFacto, einer Tochter der APA Gruppe, Testszenarien entwickelt und Dokumente entsprechend ihrer Relevanz bewertet. Diese Testszenarien dienten als Grundlage für die Analyse der Qualität der Ähnlichkeitssuche sowie von Geschwindigkeit und Speicherbedarf. Während Geschwindigkeit und Speicherbedarf nicht unbedingt standardisierbar sind, wurden zur Evaluierung der Genauigkeit die Evaluierungsmöglichkeiten des KnowMiner-Frameworks heran gezogen, über die unterschiedliche Metriken in Form von Precision/Recall-Kurven miteinander verglichen wurden. Für Details zum User Interface sei auf [Kienreich et. al. 2005] und für Details zu den Auswertungen auf [Granitzer 2004b] verwiesen.

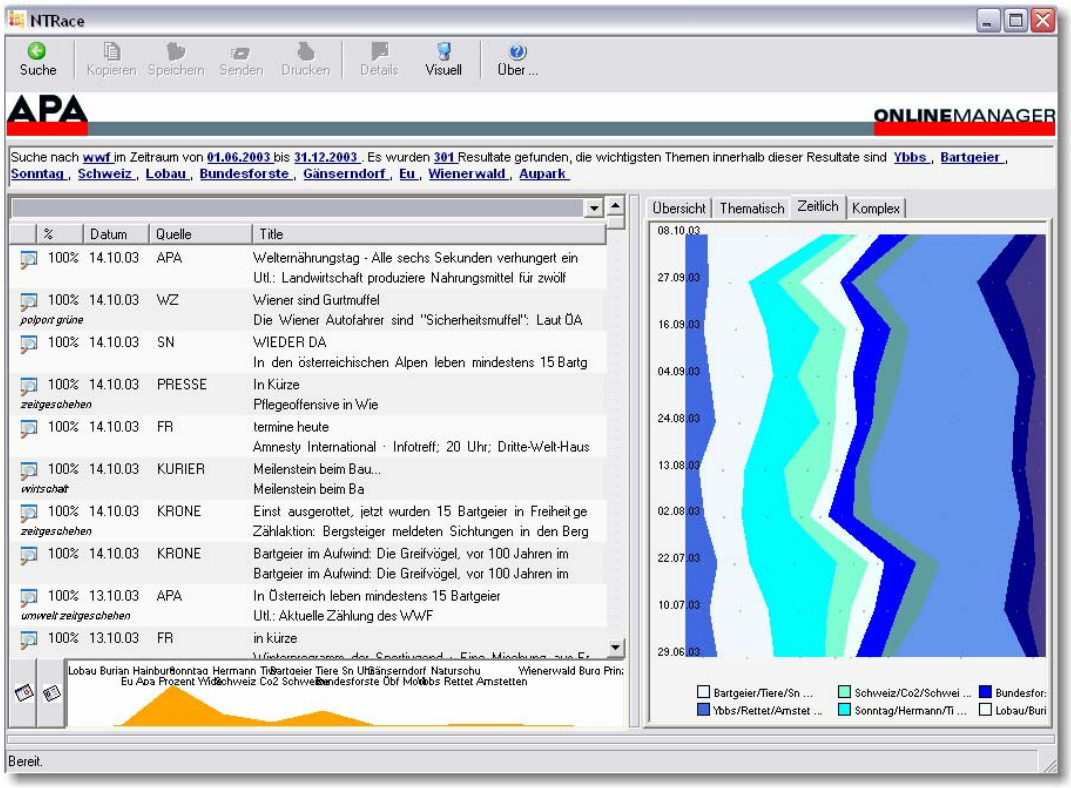


Abbildung 6-5: OnAIR Beispiel zum Suchbegriff „WWF“

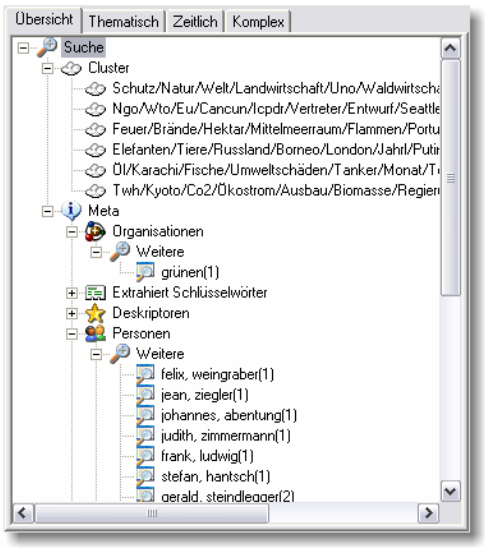


Abbildung 6-6: OnAIR: Analyse der Metadaten und Darstellung als Baum. Durch Navigation im Baum wird auch die Ergebnissmenge entsprechend eingeschränkt.

6.2.2 MAGICK- Ein Werkzeug für Cross-Media-Clustering und Visualisierung

MAGICK wurde im Rahmen einer Diplomarbeit auf Basis des KnowMiner-Frameworks entwickelt und stellt die Erweiterung der WebRat auf multimediale Datenbestände dar. Für das Clustern und Visualisieren multimedialer Dokumente wurden dabei cross-mediale Beziehungen zwischen den Dateien ausgenutzt und Ähnlichkeitsmaße über verschiedene Modalitäten hinweg definiert.

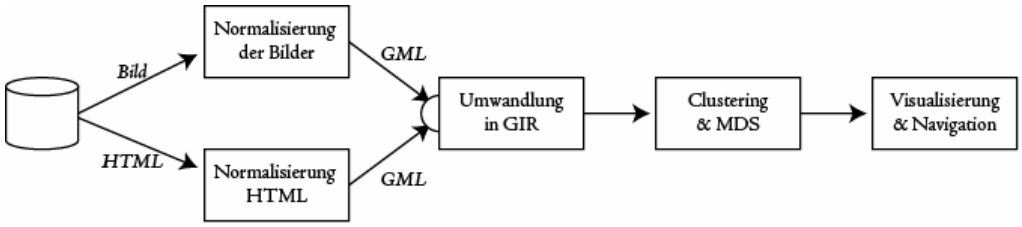


Abbildung 6-7: Überblick über den Ablauf in MAGICK (aus [Lux 2004])

MAGICK lädt vom Filesystem Bilder, HTML-Dokumente und Audiodateien und normalisiert einerseits die Bild-/Textinhalte und vereinheitlicht andererseits unterschiedliche Metadatenstandards. Der so entstandene normalisierte Inhalt wird in der so genannten GML (Generic Markup Language) abgebildet, welche in die KnowMiner-interne Repräsentation überführt wird. Dabei wird Dublin Core als Standard, auf den die verschiedenen Formate transformiert werden, verwendet. Tabelle 6-2 (entnommen aus [Lux 2004]) zeigt die Umwandlung der Metadaten der verschiedenen Formate auf Dublin Core.

Originalelement	**DC Element**
Title (HTML, PDF, ID3), Object Name (IPTC), Dateiname	Title
Author (HTML, PDF), Interpret (ID3), Creator (IPTC)	Creator
Keywords (HTML, PDF, IPTC), Album (ID3)	Subject
Description (HTML, PDF), Comment (ID3), Caption (IPTC), Comment (EXIF)	Description
-	Publisher
-	Contributor
Date Created (PDF, IPTC), Jahr (ID3), Creationdate der Datei, DateTime (EXIF)	Date
Mime Type der Datei	Type
Je nach Dateiendung entweder „Image“ oder „Text“	Format
Original Transmission Reference (IPTC), Die URI des Dokuments	Identifier
Source (IPTC)	Source
Language (HTML)	Language
Hyperlinks im Dokument (HTML)	Relation

Tabelle 6-2: Konvertierung von Inputformaten in MAGICK auf Dublin Core (aus [Lux 2004])

Auf Basis dieser Repräsentation erfolgen das Clustering und die Multi-Dimensionale-Skalierung, wobei unterschiedliche Ähnlichkeitsmetriken verwendet werden. Durch die im KnowMiner-Framework vorhandene Trennung zwischen Daten und Algorithmen/Operationen lässt sich die Ähnlichkeitsmetrik einfach austauschen, was eine der Grundlagen für MAGICK darstellt.

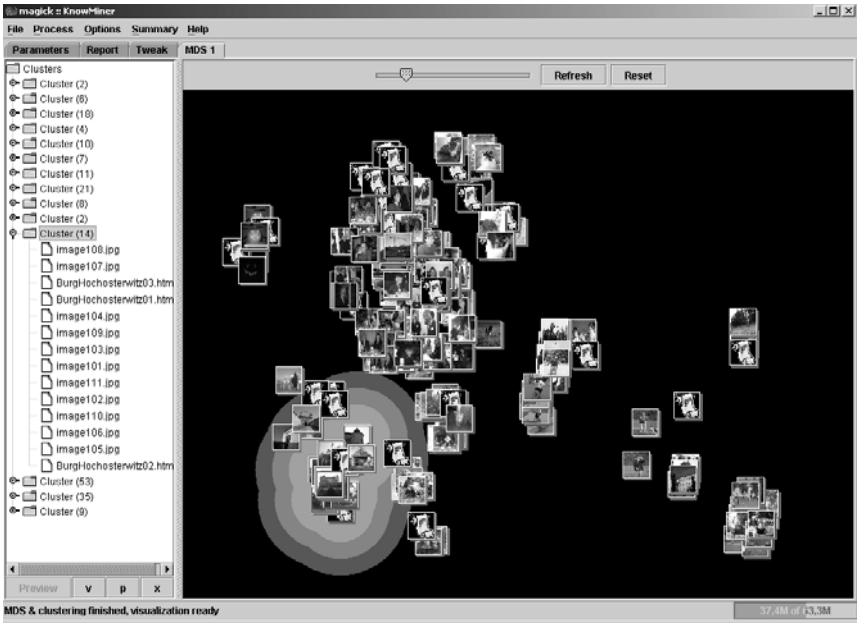


Abbildung 6-8: Screenshot aus MAGICK. Gemeinsames Clustering und Visualisierung von Bild-, Text- und Audiodaten

Der letzte Schritt besteht in der Visualisierung und Darstellung der gefundenen Zusammenhänge. Abbildung 6-7 zeigt einen Überblick über den Ablauf, Abbildung 6-8 zeigt die Darstellung der Ergebnisse in der MAGICK Applikation.

In der rechten Hälfte der Applikation werden die verschiedenen Zusammenhänge in Form einer Landscape-Visualisierung dargestellt. Die automatisch identifizierten Cluster und deren Sub-Cluster zeigen sich in der Baumdarstellung links im Screenshot.

Im Hinblick auf das Knowminer-Framework wurden vor allem die Möglichkeiten zur Auswertung, Entwicklung und Erweiterung von cross-modalen Metriken und deren Anwendung auf schon existierende Clustering- und Visualisierungsalgorithmen sowie die Möglichkeiten zur einfachen Integration verschiedener Formate vom MAGICK eingesetzt, wodurch sich die Umsetzungszeit entscheidend verkürzte.

6.2.3 KnowMiner Hierarchische Dokumentklassifikation

Neben den Visualisierungs- und Clusteringalgorithmen wurden auch Algorithmen zur Textklassifikation aus dem Projekt DocClass bzw. aus [Granitzer 2004a] ins KnowMiner-Framework integriert. Dabei wurde zwischen

- hierarchischen Klassifikationsmodellen
- flachen Klassifikationsmodellen
- dokumentbasierten Klassifikationsmodellen

unterschieden, wobei jedes dieser Klassifikationsmodelle verschiedene Klassifikationsmethoden wie Boosting, SVMs, Rocchio und k-Nearest Neighbour - Methoden anwenden kann. Im Zuge der Entwicklung der verschiedenen Modelle und Algorithmen, welche in der Beschreibung der Implementierung (siehe Abschnitt 8.4.5) näher erörtert werden, wurden auch die Benchmarking-Möglichkeiten aus dem Projekt DocClass übernommen bzw. in das KnowMiner-Framework integriert und evaluiert.

In der Diplomarbeit 'Focused crawling in the context of digital libraries' (siehe [Juffinger 2005]) erfolgte, unter Anwendung der Klassifikationstechniken des KnowMiner-Frameworks, die Integration von fokussierten Crawling-Techniken in das KnowMiner-Framework. Beim Crawling verfolgt der Crawler die Hyperlinks welche in WWW-Ressourcen enthalten sind, um so den Datenbestand des Webs zu traversieren und diesen zum Beispiel durch Suchmaschinen indizierbar zu machen. Dabei fallen im Allgemeinen hohe Datenvolumina an, welche im Netzwerk zu übertragen sind. Damit verbunden ist eine hohe Auslastung der Bandbreite. Focused Crawling versucht nun, das Crawling nur auf bestimmte vom Benutzer vorgegebene Themen zu fokussieren und nicht themenbezogene Seiten vom Crawl auszuschließen. Dazu werden die Links vor deren Verfolgung analysiert und es erfolgt die Bewertung, ob die Seite, auf welche der Link zeigt, das gewünschte Thema behandelt oder nicht.

Um diese Entscheidung treffen zu können, werden im Allgemeinen die textuelle Umgebung um den Hyperlink und etwaige vorhandene Metadaten mit Hilfe von Textklassifikation bewertet und entsprechenden Themen zugeordnet. Liegt das Thema im vom Benutzer definierten Themenfokus, so wird der Hyperlink verfolgt, ansonsten nicht. Dadurch kann bei Vorgabe eines Themas das zu übertragende Datenvolumen reduziert und der Crawl somit beschleunigt werden (für Details siehe [Juffinger 2005]).

Der in [Juffinger 2005] umgesetzte Focused Crawling - Mechanismus verwendet dabei die KnowMiner-Dokumentklassifikation, um den Themenbezug eines Hyperlinks zu ermitteln und damit den Crawl zu steuern.

6.2.4 Lessons Learned

Alle drei Projekte nutzen unterschiedliche Funktionen des KnowMiner-Frameworks und decken von der Anwendungsseite her ein breites Spektrum an KD-Anwendungen ab. Die Umsetzung der Projekte erfolgte in einem relativ großen Entwicklerteam, wodurch die verschiedenen Stärken und Schwächen des KnowMiner-Frameworks besser erkannt werden konnten. Während der Umsetzung der einzelnen Projekte erfolgte teilweise die Verfeinerung einzelner Komponenten im KnowMiner-Framework, nach Abschluss der einzelnen Projekte wurden Architekturschwächen identifiziert und Verbesserungsvorschläge für den nächsten Zyklus erstellt. Im Rest dieses Abschnittes werden die abgeleiteten Erkenntnisse einerseits vor dem Hintergrund einer effizienteren Algorithmenentwicklung und andererseits vor dem Hintergrund einer beschleunigten und robusteren Anwendungsentwicklung betrachtet.

Klare Spezifikation der internen Repräsentation: Während sich die interne Repräsentation als guter Abstraktionsmechanismus für die Darstellung von Dokumenten und ihrer Zusammenhänge eignete, zeigte sich bei der Anwendung jedoch, dass eine klare Spezifikation der Transformationsmöglichkeiten und der Eigenschaften der Dokumente fehlte. Die Repräsentation von Metadaten war nicht immer klar spezifiziert wodurch es zu Fehlern in der Datentransformation und, damit verbunden, zu langen Fehlersuchzeiten, vor allem bei Client/Server-Anwendungen, kam. Daraus abgeleitet wurden die Anforderungen einer klareren semantischen Spezifikation der Eigenschaften von Dokumenten und möglicher Transformationen von Eigenschaften.

Schnittstellen und Interface Design: Bei der Umsetzung der einzelnen Projekte konnten meistens Startprobleme erkannt werden, welche dadurch bedingt waren, dass die Schnittstellen zu den einzelnen Algorithmen und auch zur internen Datenrepräsentation meist nicht eindeutig genug spezifiziert worden waren und Mehrdeutigkeiten in der Anwendung zuließen. Die Anwender des KnowMiner-Frameworks benötigten viel Hintergrundwissen über die im KnowMiner-Framework verwendeten Repräsentationsformen

und Techniken, was in einer hohen Einarbeitungszeit und einer gewissen Hemmschwelle der Anwendung des KnowMiner-Frameworks resultierte. Aus diesem Punkt leitete sich für den nächsten Iterationszyklus eine klarere Spezifikation der Schnittstellen und der KnowMiner-internen Datenflüsse ab.

Datenquellenanbindung und Import: Während sich die flexible Anbindung von heterogenen Datenquellen als zufrieden stellend erwies, benötigten einige Prozesse keine inhaltliche Vorverarbeitung (Content-Normalising) und keine Strukturnormalisierung. Auch erwies sich die Platzierung dieser Komponenten direkt nach der Informationsakquise als suboptimal, da in verschiedenen Szenarien die Strukturierung z.B. nach der Akquise des gesamten Datenbestands erfolgen sollte und nicht schon während dem Laden der Einzeldokumente. Aus technischer Sicht erwies sich die Integration der Vorverarbeitung und Strukturnormalisierung in den Datenimport aufgrund dessen paralleler Natur als komplex, was die Zeit zur Fehlersuche innerhalb des Datenimports drastisch erhöhte. Aus diesem Grund leitete sich für die nächste Iteration die Anforderung einer Trennung von Datenimport und Vorverarbeitung sowie Strukturnormalisierung ab und die damit verbundene Simplifizierung des Datenimports an sich.

Zugriffsmechanismen auf die interne Repräsentation: Aufgrund der Graphennatur der internen Repräsentation erwies sich der Zugriff meist als schwierig, da er rein strukturbasiert erfolgte. Auch die verwendeten Locking-Mechanismen, welche jeweils eine Schreib-/Lesesperre auf Dokumentebene implementierten, erwiesen sich als zu starr und unhandlich für die effiziente Anwendung. So kam es des Öfteren dazu, dass gesperrte Objekte nicht wieder frei gegeben wurden und daher die Anwendung fehlerhaft terminierte. Ein Umstand, der ebenfalls die Entwicklungszeit von Algorithmen und KD-Anwendungen wegen erhöhtem Aufwand zur Fehlersuche ansteigen ließ. Aus diesem Punkt konnten die Anforderungen nach standardisierten Graphenzugriffen und verbesserten Lockingmechanismen auf Ebene der Eigenschaften (z.B. Merkmalsmengen) von Dokumenten sowie verbesserten Möglichkeiten zur Fehlersuche abgeleitet werden.

Persistierung und Caching der internen Repräsentation: In der ersten Version des KnowMiner-Frameworks existierten nur unzureichende Möglichkeiten, die interne Repräsentation persistent auf ein Sekundärspeichermedium auszulagern. Auch dem hohen Hauptspeicherverbrauch von KD-Anwendungen konnte nicht standardisiert begegnet werden, da Caching-Mechanismen, welche das effiziente Auslagern von Dokumenten aus dem Hauptspeicher ermöglichen, nicht vorhanden waren.

Erweiterung der Algorithmenbasis: Die Anwendung des KnowMiner-Frameworks zeigte auch die Defizite in der Breite der Algorithmenbasis auf. Im Speziellen fehlten Methoden aus dem Bereich der Informationsextraktion und zusätzliche Algorithmen in den Bereichen Clustering, Klassifikation und Merkmalstransformation (Ordination). Da die Entwicklung von Algorithmen einerseits meist zeitaufwändig und fehlerbehaftet ist und andererseits über verschiedenen Open Source Projekte Algorithmen aus unterschiedlichen Bereichen wie z.B. dem maschinellen Lernen (WEKA) zu Verfügung stehen, leitete sich die Anforderung der Integration von Drittprodukten in das KnowMiner-Framework ab.

Zusammenfassend konnten folgende Anforderungen für den nächsten Entwicklungszyklus abgeleitet werden, welche in die Definition des KnowMiner-Frameworks im nächsten Abschnitt einflossen und entsprechend berücksichtigt wurden.

1. Klare Spezifikation der Datenflüsse und der möglichen Datentransformationen im KnowMiner-Framework
2. Klare Spezifikation der verschiedenen Eigenschaften der Abstraktion der Einzeldokumente

3. Semantische Repräsentation und erweiterte Abfragemöglichkeiten zur Spezifikation der Beziehungen zwischen Dokumenten
4. Schaffung einer modularen, anwendungsorientierten Architektur zur Bereitstellung von KD-Funktionalitäten inklusive einer verbesserten Kapselung von Algorithmen zu KD-Funktionen
5. Integration von Techniken zur Informationsextraktion
6. Erweiterung der Algorithmenbasis durch Integration von verschiedenen Softwarebibliotheken wie z.B. GATE und WEKA
7. Implementierung von Persistierungs- und Cachingmechanismen

Der nächste Abschnitt führt die aus diesen Anforderungen und unter Berücksichtigung des Knowledge Discovery Processes entstandenen konzeptionellen Grundlagen für die finale Umsetzung des KnowMiner-Frameworks aus.

7 Konzeptionelle Grundlagen des KnowMiner-Frameworks

Dieser Abschnitt betrachtet die konzeptionellen Grundlagen des KnowMiner-Frameworks, führt die damit verbundenen Konzepte ein und zeigt die Verbindungen zwischen diesen auf. Ziel dabei ist, basierend auf den in den vorigen Kapiteln abgeleiteten Anforderungen die konzeptionellen Grundlagen für die in Kapitel 8 dargestellte Implementierung des KnowMiner-Frameworks zu schaffen.

Da Wissenserschließung ein sehr datenintensives Gebiet ist, fokussiert dieser Abschnitt auf die Analyse der zu verarbeitenden Daten einerseits vor dem Hintergrund der Art der zu verabeitenden Daten/Informationen und andererseits bezogen auf die Fragestellung, in welchen Schritten welche Transformationen Daten/Informationen verändern. Im Speziellen betrachtet dieses Kapitel folgende Aspekte in der dargestellten Reihenfolge:

1. *Detaillierte Analyse des Knowledge Discovery Prozesses (KDP, siehe Kapitel 2)*
   Auf Basis der in den Kapiteln 3 bis inkl. 5 erhobenen Anforderungen erfolgt die Detailanalyse des Knowledge Discovery Process um daraus relevante Daten- und Informationsströme sowie wichtige Transformationen ableiten zu können. Die detaillierte Beschreibung dieser Analyse erfolgt in Abschnitt 7.1.

2. *Ableitung des Daten- und Informationsflusses*
   Zentral bei der Entwicklung eines KD-Frameworks ist die Berücksichtigung der verschiedenen Informations- und Datenströme, sowie die Beschreibung dieser. Dadurch lassen sich einerseits modulare Komponenten und andererseits mögliche Engpässe definieren. Abschnitt 7.2 betrachtet diese Aspekte im Detail.

3. *Strukturierung der identifizierten Daten- und Informationsflüsse*
   Neben den Datenströmen sind auch die Strukturierung der Informationen und Daten und deren Zusammenhänge essentiell, vor allem um einen Trennung zwischen Algorithmik und Repräsentation von Information zu ermöglichen. Die detaillierte, auf den vorherigen Analysen basierende Strukturierung des Informationsraums stellt Abschnitt 7.3 dar.

4. *Definition der Konzepte und verwendeten Begrifflichkeiten im KnowMiner-Framework*
   Zur Vermeidung von Unklarheiten und zur Spezifikation der verwendeten Termini ist es wichtig, die verwendeten Konzepte explizit zu definieren und deren Zusammenhänge klar zu machen. Abschnitt 7.4 definiert die verschiedenen Konzepte sowie deren Zusammenhänge im Kontext des KnowMiner-Frameworks. Die Konzepte dienen als Grundlage für die weiteren Betrachtungen und der Beschreibung des Frameworks selbst in Kapitel 8.

5. *Identifikation der verschiedenen Transformationen im Informationsraum*
   Während die Untersuchung der Daten- und Informationsströme die benötigten Daten in den einzelnen Schritten untersucht, ist ein weiterer wichtiger Punkt in der Schaffung der konzeptionellen Grundlagen die Betrachtungen der möglichen Transformationen auf den Strukturen des Informationsraumes. Zentral für die Modularisierung von Komponenten sind die im Fokus von Abschnitt 7.5 liegenden Fragestellungen, welche Transformationen möglich bzw. relevant sind und welche Berechnungen zur Ausführung verschiedener Algorithmen benötigt werden.

6. *Identifikation der verschiedenen Klassen von Algorithmen und Komponenten*
   Aus den vorangegangenen Überlegungen lassen sich nun Algorithmen und Komponenten in Klassen gruppieren, welche damit eine erste Form der Modularisierung schaffen. Abschnitt 7.6 beschreibt diese Komponentenklassen und gibt Beispiele von Algorithmen. Diese Modularisierung dient in weiterer Folge der Definition der verschiedenen Dienste im KnowMiner-Framework, die detailliert in Abschnitt 8.5 beschrieben sind.

Die Analyse und Beschreibung dieser Punkte schafft die konzeptionelle Grundlage für die Entwicklung der dem KnowMiner-Framework zugrunde liegenden Software-Architektur. Die Berücksichtigung der unterschiedlichen Eigenschaften der Wissenserschließung im Kontext der erhobenen Anforderungen erlaubt eine konkrete Modularisierung der Komponenten bis hin zur Bereitstellung von KD-Diensten und ermöglicht die Spezifikation der verarbeitbaren Information. Die beschriebenen konzeptionellen Grundlagen sind Voraussetzung für die in Kapitel 8 dargestellte Implementierung des KnowMiner-Frameworks.

7.1 Der Knowledge Discovery Process im Detail

Der in Abschnitt 2 illustrierte KD-Prozess stellt die Grundlage für fast alle Anwendungen im Bereich der Wissenserschließung dar. Der folgende Abschnitt hebt die vor dem Hintergrund der bisher erhobenen Anforderungen zentralen Punkte des KDP hervor und liefert somit die Grundlagen für die weiteren Betrachtungen in diesem Kapitel.

7.1.1 Akquise und Homogenisierung

Der erste Schritt im KDP besteht in der Selektion von Datenquellen und dem Laden bzw. Akquirieren von Information. Zentral für KD-Anwendungen ist hierbei die kleinste zu betrachtende Informationseinheit, welche mittels verschiedener Eigenschaften, ermittelt aus den zugehörigen Daten, beschrieben wird. Datenselektion kann dabei über die direkte Anbindung von Datenquellen oder über verschiedenste Suchmöglichkeiten erfolgen.

Um heterogene Datenbestände mit unterschiedlichen Datenformaten und Datenstrukturen vergleichen bzw. im KD-Prozess verarbeiten zu können, muss in einem ersten Schritt eine verlustfreie Überführung in ein einheitliches Format erreicht werden. Dabei stellt sich auch die Frage, in wie weit datenquellenspezifische Metadaten, wie z.B. die Position innerhalb der Ablagestruktur eines WM-Systems, berücksichtigt und unterschiedliche Metadatenformate homogenisiert werden können. Informationsentitäten bezeichnen dabei im Folgenden die von den Datenquellen gelesenen Elemente, wie z.B. Dokumente, und formen die Einheiten für die nachfolgenden KD-Schritte.

Neben den einzelnen Informationsentitäten ist zusätzlich die Struktur der selektierten Datenquelle von Interesse. Zum Beispiel verfügen die meisten Wissensmanagementsysteme über eine unternehmensweite Ablagesystematik, in welcher Dokumente organisiert sind. In den nachfolgenden Schritten ist diese Struktur z.B. zur Verbesserung der Vorverarbeitung oder aber zum Lernen eines hierarchischen Klassifikationsmodells (siehe Abschnitt 4.3.2) verwendbar.

Zusammenfassend besteht der Akquisitionsschritt aus dem Sammeln der Daten, dem Normalisieren der Formate der einzelnen Informationsentitäten sowie der Normalisierung und Vereinheitlichung der für die nachfolgenden Schritte wichtigen Struktureigenschaften und Metadaten der Datenquellen. Im Allgemeinen werden diese normalisierten Daten in einem entsprechenden Backend abgelegt, um für die weiteren Schritte

verfügbar zu sein. Datenbanken in Kombination mit Tripelstores wie Jena [Carroll et. al. 2004] und Sesame [Broekstra et. al. 2002] eigenen sich gut als entsprechendes Backend, einerseits zur Verwaltung der meist großen Datenbestände und andererseits zur Verwaltung der unterschiedlichen Strukturen und Metadateninformationen.

7.1.2 Vorverarbeitung und Merkmalsgenerierung

Nach der Datenakquisition und entsprechenden Formatnormalisierungen müssen die Inhalte der einzelnen Informationsentitäten entsprechend ihrer Modalität vorverarbeitet und für die nachfolgenden Algorithmen aufbereitet werden. Die einzelnen Vorverarbeitungsmöglichkeiten hängen von der entsprechenden Modalität ab. Für Bilder sind z.B. im Umfeld von MPEG-7 [Kosch 2003] verschiedene Deskriptoren auf Basis der Farb- und Strukturverteilung vorgesehen. Diese Deskriptoren abstrahieren Bildinhalte in Form von Vektoren und ermöglichen somit Vektoroperationen und Ähnlichkeitsberechnungen. Ähnliches gilt für Text, wobei hier eine Zweiteilung der Vorverarbeitung ableitbar ist (siehe Abschnitt 3.1). Der erste Schritt besteht im Allgemeinen aus der Annotation des Originalinhaltes, d.h. es wird unter Anwendung von linguistischen Methoden zusätzliche Information zu Texten und Wörtern wie z.B. deren grammatikalische Funktion (Part-of-Speech), ermittelt. Auf Basis der Annotationen und des Originaltextes erfolgt die Extraktion von Merkmalen und die Überführung des Textdokumentes in einen so genannten logical view (siehe [Baeza-Yates et. al 1999]). Standardmäßig entspricht der logical view eines Dokumentes einer Vektorrepräsentation (siehe Abschnitt 3.3) der wichtigsten Terme/Worte. Die Vektorrepräsentation ermöglicht es in weiterer Folge, mathematische Operationen auf den verschiedenen Informationsentitäten durchzuführen.

7.1.3 Merkmalstransformation

Nach der Generierung von Merkmalen ist der nächste Schritt im KD-Prozess die Transformation der Merkmale, meist auf Basis von statistischen oder mathematischen Modellen. Im Unterschied zur Generierung von Merkmalen bei der Vorverarbeitung erfolgt im Transformationsschritt die Optimierung der Merkmalsmenge. Im Textbereich bzw. bei der Vektorrepräsentation von Texten kann dazu z.B. die Transformation des Vektorraummodells in einen niedrig dimensionalen Raum (siehe Abschnitt 4.1) dienen. Die Art der Transformation hängt dabei stark von den nachfolgenden Algorithmen bzw. der geplanten Anwendung ab und hat im Allgemeinen einen hohen Einfluss auf die Qualität und Geschwindigkeit der Abarbeitung.

7.1.4 Mining

Im Anschluss an den Transformationsschritt erfolgt der eigentliche Mining Schritt, d.h. das Auswerten und Entdecken von Zusammenhängen. Je nach Art der zu verarbeitenden Daten bzw. der entsprechenden Domäne kann hier zwischen Data Mining, Text Mining und Link Mining unterschieden werden. Data Mining fokussiert dabei auf numerische und nominale Daten, repräsentiert als Vektoren; Text Mining fokussiert auf unstrukturierte textuelle Daten, repräsentiert durch Vektoren und zusätzlich linguistische Strukturen wie beispielsweise Suffix Trees; und Link Mining fokussiert auf Zusammenhänge von Entitäten, repräsentiert durch Graphen. Zentral für die meisten Mining-Algorithmen ist die Definition der Ähnlichkeit zwischen den einzelnen Entitäten. Die Qualität von Clusteringverfahren und der damit ableitbaren Zusammenhänge hängt im Allgemeinen stark vom verwendeten Ähnlichkeitsmaß ab (siehe Abschnitt 3.5). Ähnlich verhält es sich im Bereich der Klassifikation und auch des Information Retrievals. Im Bereich der Ähnlichkeitsmaße (siehe Abschnitt 3.4) spielen vor allem Kernel-Methoden eine wichtige Rolle. Diese bieten auf unterschiedlichsten

Datenbeständen wie z.B. Graphen, Vektorräumen etc. die Möglichkeit zur flexiblen Definition von verschiedenen Ähnlichkeitsmaßen. Die Abstraktion der Ähnlichkeitsmaße sowie der entsprechenden mathematischen Operationen auf den unterschiedlichen Merkmalen von Informationsentitäten stellt somit den zentralen Aspekt zur Trennung von Algorithmen und Repräsentationsformen und der damit verbundenen Wiederverwendbarkeit von Algorithmen auf unterschiedlichen Modalitäten dar.

7.1.5 Aufbereitung und Visualisierung

Der letzte Schritt besteht in der Aufbereitung der Ergebnisse und Interpretation der extrahierten Zusammenhänge. Die Ergebnisse können wiederum in Form von Metadaten zu einzelnen Informationsentitäten hinzugefügt und so für weitere Verarbeitungsschritte verfügbar gemacht werden. UIMA (siehe Abschnitt 5.1.2) indiziert beispielsweise im KDP extrahierte Metadaten und ermöglicht so das Finden von Informationsentitäten auf Basis dieser Metadaten.

Der KDP stellt grundlegend keinen linearen Ablauf dar, da meist aufgrund der Interpretation der Ergebnisse eine Anpassung und Optimierung der Einzelschritte in der Prozesskette erfolgt. In diesem iterativen Charakter spiegelt sich auch die Möglichkeit wider, die im Mining extrahierten Muster und Metadaten in der nächsten Iteration als zusätzliche Merkmale bzw. Strukturierung zu verwenden oder verschiedene KD-Schritte miteinander zu kombinieren. So könnte z.B. ein KDP darin bestehen, einen gegebenen Datenbestand zu clustern und in Gruppen von ähnlichen Entitäten einzuteilen, während der darauf folgende KDP sich der Extraktion von Mustern aus den einzelnen Gruppen widmet. Visualisierungstechniken unterstützen diesen iterativen Charakter und ermöglichen meist einen fließenden Übergang zwischen den Teilschritten.

7.2 Daten und Informationsflüsse im KnowMiner-Framework

Aus der Betrachtung des KDP können folgende für das KnowMiner-Framework relevante Eigenschaften abgeleitet werden:

1. Einfache Anbindung von verschiedenen Datenquellen und Überführung der Formate der verschiedenen Informationsentitäten in eine einheitliche Struktur
2. Bereitstellen eines zentralen Daten-Backends zum Ablegen und Zugreifen auf die formatnormalisierten Entitäten
3. Abstraktion von Merkmalen und Operationen auf Merkmalsmengen zur Trennung von Algorithmen und Repräsentationsformen
4. Flexible Kombination der einzelnen Schritte zur Unterstützung des iterativen Charakters

Aus den relevanten Eigenschaften, der Analyse des KDP-Prozesses und dem in den Abschnitt 3 bis inkl. 5 dargestellten Stand der Forschung lässt sich der in Abbildung 7-1 dargestellte Datenfluss ableiten.

Im ersten Schritt erfolgt die Akquisition von Information durch Anbindung unterschiedlicher Datenquellen und Zugriffsmethoden. Neben der Anbindung herkömmlicher Datenbank- und File-Systeme können auch intelligentere Mechanismen zur Datenakquisition wie z.B. das Crawling von Hyperlink-Datenbeständen oder das Erfassen von RSS (Rich Site Summary) - Feeds eingesetzt werden. Zur weiteren Verarbeitung kann die Speicherung der akquirierten Inhalte auch optional in einem Content Repository erfolgen, was vor allem bei begrenzten Netzwerkbandbreiten hilfreich ist.

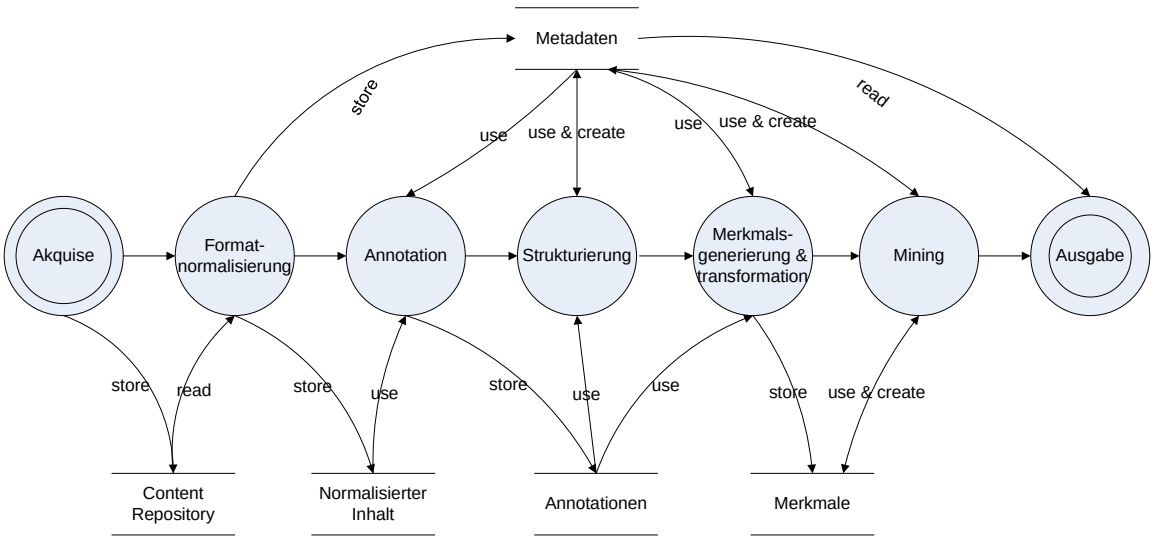


Abbildung 7-1: Abbildung des KnowMiner-Informationsflusses. In jedem Schritt werden andere Aspekte einer Informationsentität verarbeitet. KD-Algorithmen können auf alle Eigenschaften einer Informationsentität zugreifen

Wie auch im KD-Prozess erfolgt im Anschluss an die Akquisition die Formatnormalisierung. Dazu wandelt dieser Schritt die unterschiedlichen heterogenen Formate verlustfrei in ein einheitliches internes Format um und extrahiert zusätzlich im Format vorhandene Metadaten, wie z.B. den Autor, Titel, Schlüsselwörter etc. Die Vereinheitlichung der Metadaten in den verschiedenen Formaten ist in Abbildung 7-1 durch den „store"-Pfeil illustriert. Neben den inhaltlichen Metadaten werden in diesem Schritt auch die quellenspezifischen Metadaten, wie z.B. Erzeugungsdatum, strukturelle Einbettung in die Datenquelle etc., mit aufgenommen. Vor allem die Extraktion der datenquellenspezifischen Struktur, wie z.B. einer Ablagesystematik, und die Zuordnung eines Dokumentes zu dieser Ablagesystematik kann in vielen KD-Algorithmen zur Erhöhung der Genauigkeit bzw. zur Verringerung der Laufzeitkomplexität beitragen. Dies zeigt auch die Beispielanwendung InfoSky (siehe Abschnitt 6.1.2) bzw. die Anwendung von z.B. hierarchischer Textklassifikation (siehe Abschnitt 6.1.3). Die Zwischenspeicherung des formatnormalisierten Inhalts und dessen Bereitstellung für die nächsten Schritte ist ebenfalls möglich, wobei sich als Format zur Speicherung von Inhalten und Metadaten strukturierte Beschreibungssprachen wie XML anbieten. Aufgrund der umfassenden Repräsentationsmöglichkeiten dieser Beschreibungssprachen ist die Speicherung von unterschiedlichsten Inhalten und Metadatenformaten möglich. Bei der Formatnormalisierung ist vor allem die Heterogenität von Formaten in Form von Crosswalking zwischen Standards zu berücksichtigen [McCray et. al. 2001], wobei hier vor allem Methoden aus dem Umfeld des Semantic Web eine Hilfestellung bieten.

Auf Basis der formatnormalisierten Informationsentitäten erfolgt die Annotation der Inhalte, d.h. auf Basis verschiedener Algorithmen und Techniken werden zusätzliche Informationen zu Inhalten hinzugefügt. Für textuelle Daten ist dies mittels Part-Of-Speech Tagging und Informationsextraktion, wie in Abschnitt 3.1 und 3.2 beschrieben, erreichbar. Für Bilder oder Videos können z.B. Gesichtserkennungsalgorithmen spatio-temporale Annotationen des Videostreams erstellen. In Abhängigkeit von der Modalität bzw. von den angewandten Techniken sind unterschiedliche Datenstrukturen zur Erfassung der Annotation notwendig. Für die Definition der Annotation im Text reichen im Allgemeinen Start- und Endposition der jeweiligen Zeichen, wie dies z.B. in GATE (siehe Abschnitt 5.1.1) der Fall ist. Für Bild- bzw. Videoannotationen sind komplexere Beschreibungen nötig, da von rechteckigen Bildausschnitten bis hin zu komplexen Polygonen

unterschiedlichste Annotationsarten möglich sind (siehe [Grabner et. al 2006] für ein Beispiel zu Videoannotation). Das ATLAS-Framework (siehe Abschnitt 5.1.4) definiert Annotationen mittels Ontologien und bietet einen guten Ansatzpunkt hierfür. Auf jeden Fall dienen Annotationen als Grundlage für die weitere Verarbeitung bzw. für die Generierung der für KD-Algorithmen notwendigen Merkmale. Neben Annotationen von Inhalten besteht natürlich auch die Möglichkeit, Metadaten einer Informationsentität zu annotieren, um diese damit ebenfalls für die Merkmalsgenerierung vorzubereiten.

Neben der Detailanalyse von Einzelentitäten wie z.B. Dokumenten muss auch die Berücksichtigung von Zusammenhängen zwischen den Entitäten erfolgen. Solche Zusammenhänge sind einerseits auf Basis der den einzelnen Entitäten zugeordneten Metadaten wie z.B. Autoren, Kategorien, Hyperlinks etc. oder auf Basis extrahierter Annotationen wie z.B. Geo-Referenzen berücksichtigbar. Ziel dieser Strukturierung ist die Aufbereitung des Informationsraumes und die Vereinheitlichung der unterschiedlichen Datenquellen in einem gemeinsamen Informationsraum, wobei es möglich sein muss, gleichzeitig unterschiedliche Strukturierungsvarianten und Beziehungstypen zwischen einzelnen Entitäten zu unterstützen. Video bzw. Videosegmente können einerseits einer thematischen Organisation und andererseits einer zeitlichen Organisation folgen und müssen dementsprechend repräsentiert sein.

Im Anschluss an die Strukturierung erfolgen die Merkmalsgenerierung, -gewichtung und -transformation. Annotationen stellen neben den Metadaten dazu die Grundlage dar. Wie in Abschnitt 3.3 illustriert, werden Merkmale über statistische Analysen der Annotationen generiert, gewichtet und im Allgemeinen als Merkmalsvektor oder Merkmalsmenge repräsentiert. Neben der statistischen Analyse von Annotationen können, wie bereits erwähnt, auch Metadaten mit berücksichtigt werden. In Spezialfällen sind Metadaten auch direkt als Merkmale verwendbar, wie beispielsweise im Fall semantischer Metadaten (siehe [Lux et. al. 2006]). Die Strukturierung des Informationsraumes kann auch bei der Merkmalsgenerierung und Transformation eingesetzt werden, um einerseits die Genauigkeit zu verbessern bzw. andererseits die Laufzeitkomplexität zu reduzieren. Beispielsweise in [Koller et. al. 1997] reduzierte die Ausnutzung von hierarchischen Ablagestrukturen die Anzahl der notwendigen Merkmale zur Repräsentation von Dokumenten, was eine direkte Beschleunigung der Berechnungen zur Folge hatte.

Auf Basis der extrahierten Merkmale erfolgt die Anwendung der unterschiedlichen Mining-Algorithmen, welche im Detail in Abschnitt 3.1 beschrieben wurden. Generelles Ziel von Mining-Algorithmen ist die Extraktion von Metadaten über Einzelentitäten und die Identifikation von Zusammenhängen zwischen Entitäten, was wiederum als Transformation des Informationsraumes auffassbar ist.

Der hier dargestellte Datenfluss und der oben angeführte KD-Prozess stellen die Grundlage für die im nächsten Kapitel eingeführte Strukturierung des Informationsraumes dar.

7.3 Strukturierung des Informationsraumes

Bisher wurden das allgemeine KD-Prozessmodell sowie der technische Datenfluss untersucht und entsprechende Zusammenhänge abgeleitet. Wie aus Abbildung 7-1 ersichtlich, beschäftigen sich KD-Anwendungen vorrangig mit der Transformation von Information und dem Verwalten von extrahierten Daten. Daher kommt der Strukturierung der Einzelelemente und der Entwicklung einer möglichst flexiblen Datenstruktur eine zentrale Bedeutung für die Umsetzung von KD-Anwendungen zu. Zusätzlich sollen die entwickelten Datenstrukturen möglichst effizient mit großen Datenmengen umgehen können bzw. auf größeren Datenmengen skalierbar sein. Das WebLyzard System [Scharl 2004] führt z.B. wöchentliche

Analysen für viele verschiedene Medienberichterstattungen durch und überführt diese in eine ontologische Struktur. Solche Funktionen benötigten flexible und vor allem skalierbare Datenstrukturen. Auch in den verschiedenen Iterationszyklen des KnowMiner-Frameworks hat sich die Anforderung an eine flexible Datenstruktur mit möglichst einfachen Methoden zur Transformation von verschiedenen Formaten und Strukturierungen als kritisch heraus kristallisiert.

Vor diesem Hintergrund beschäftigt sich dieser Abschnitt mit den Grundlagen der Strukturierung des Informationsraumes, wobei verschiedene Anwendungsszenarien, vom einfachen Retrieval in Textdokumenten bis hin zur komplexen Analyse von Mediendaten, zu berücksichtigen sind. Während dieser Abschnitt die einzelnen Konzepte und deren Zusammenhänge nur umreißt, erfolgt in Abschnitt 7.4 eine exakte Definition der einzelnen Konzepte.

Abbildung 7-2 zeigt einen Überblick über die Zusammenhänge der einzelnen Konzepte in Form eines UML-Diagramms. Startpunkt bildet dabei der *Informationsraum*, welcher als Sammelbecken für alle anwendungsrelevanten Informationseinheiten und deren Beziehungen dient. Ein Informationsraum kann aus einer Datenquelle wie z.B. einem Wissensmanagementsystem oder aus mehreren heterogenen Datenquellen wie z.B. dem Intranet, dem E-Mail Server, verschiedenen Netzlaufwerken etc. einer Firma bestehen und muss daher eine flexible Abbildung sowohl von Strukturen als auch von Inhalten ermöglichen. Ein Wissensmanagementsystem verfügt meist über eine wohl definierte Ablagesystematik, während Netzlaufwerke im Allgemeinen eher unstrukturiert sind.

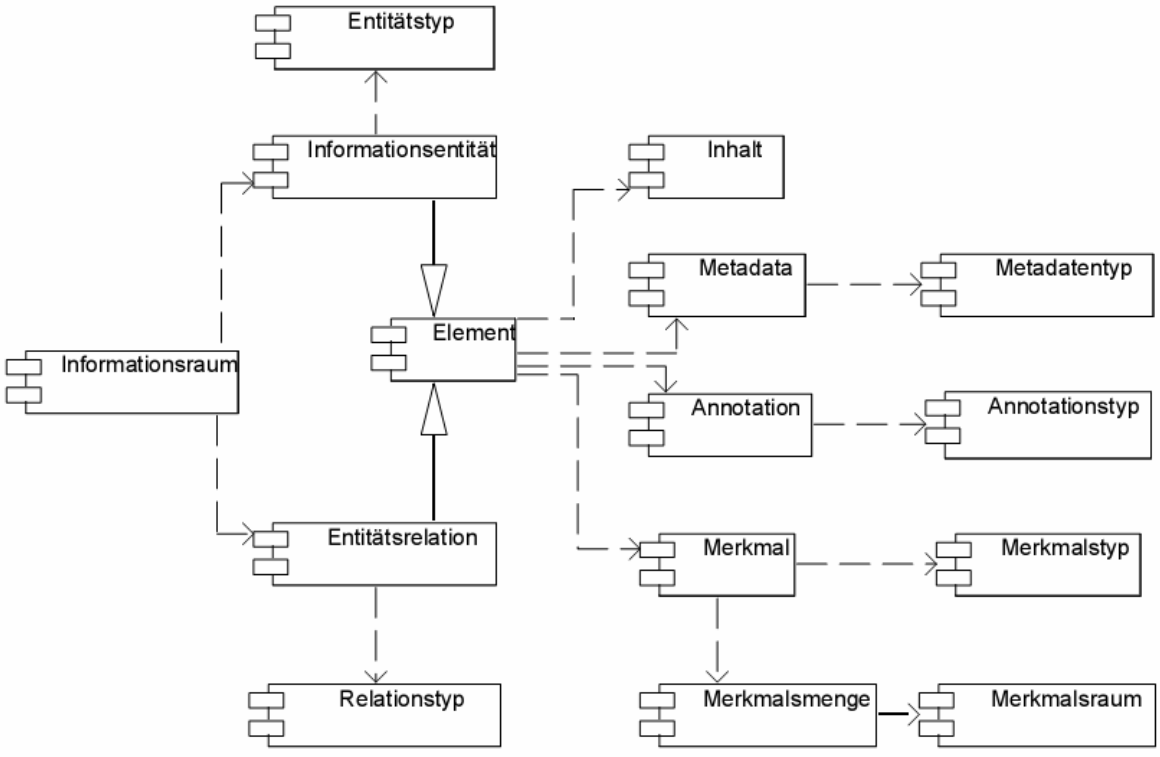


Abbildung 7-2: Überblick und Zusammenhang der KnowMiner Konzepte

Um einen solch allgemeinen Informationsraum zu ermöglichen, erfolgt die Teilung des Informationsraumes in (i) die zu betrachtenden Einzelelemente, die "atomaren" Einheiten, in weiterer Folge als *Informationsentität*

bezeichnet, und (ii) in die Beziehungen zwischen den einzelnen Informationsentitäten, den *Entitätsrelationen*. Die Granularität von Informationsentitäten und Informationsrelationen hängt dabei vom jeweiligen Anwendungsszenario ab. Um z.B. einfaches Retrieval auf bestimmten Dokument-/Informationsbeständen durch führen zu können, reicht es im Allgemeinen aus, die Dokumente als Informationsentitäten und Hyperlinks bzw. eine gegebene Ablagestruktur als Relationen zu betrachten. Beim Lernen von Ontologien aus Texten führt diese Granularität kaum zum Ziel, da hier Konzepte, Personen und Geo-Referenzen im Zentrum der Untersuchung stehen und damit die Informationsentitäten definieren. Entitätsrelationen zwischen den Entitäten können ebenfalls vielschichtiger Natur, wie z.B. IS-A - oder part-of - Beziehungen, sein, woraus sich die Notwendigkeit eines Typsystems für Relationen als auch für Entitäten ableitet.

Mit einem Typsystem können die möglichen Repräsentationsformen eines Informationsraumes klar spezifiziert und somit die vielschichtigen Ausprägungen semantisch eingeschränkt und definiert werden. Beispielweise sind Methoden zur hierarchischen Klassifikation von Textdokumenten, wie sie in [Granitzer & Auer 2005] vorgestellt wurden, lediglich auf einen (Teil-)Informationsraum, der in Form eines Baums vorliegt, anwendbar. Über Typisierung von Entitäten und Entitätsrelationen können semantisch vielschichtige Ausprägungen des Informationsraums erreicht werden. Zugelassen wären in einem Informationsraum in Form eines Baumes z.B. nur Informationsentitäten vom Typ Kategorie und Dokument sowie IS-A - Beziehungen zwischen Entitäten des Typs Kategorie und IS-PART-OF - Beziehungen zwischen Dokumenten und Kategorien. Die Typisierung ermöglicht eine frühzeitige Feststellung darüber, ob eine KD-Methode anwendbar ist oder nicht und erlaubt damit auch eine Prüfung der Anwendbarkeit vor der Ausführung der Algorithmik, ähnlich dem Syntax-Check in Compilern. Da KD-Anwendungen im Allgemeinen daten- und berechnungsintensiv sind, sparen solche Spezifikationen vor allem bei der Entwicklung von KD-Algorithmen Zeit, da schon vor der Anwendung von KD-Algorithmen deren Anwendbarkeit überprüft werden kann. Würde im Falle der hierarchischen Klassifikation z.B. ein zyklischer Graph als Ablagestruktur vorgegeben sein, so würden Algorithmen auf Basis des Pachinko-Modells (siehe [Granitzer & Auer 2005]) in der Trainingsphase nicht terminieren und auch die Fehlersuche bei einer Trainingszeit von mehreren Stunden erschweren.

Neben der Struktur des Informationsraumes sind auch die Eigenschaften der einzelnen Informationsentitäten und Entitätsrelationen von entscheidender Bedeutung für KD-Anwendungen. Ähnlichkeitsberechnungen, Inhaltsanalysen etc. erfolgen auf Basis der Eigenschaften der Informationsentitäten. Jedoch nicht nur Entitäten beinhalten auswertbare Eigenschaften, sondern auch Relationen an sich. Relationen können z.B. nur unter gewissen Voraussetzungen, wie z.B. in bestimmten Zeiträumen, korrekte Beziehungen zwischen Informationsentitäten darstellen oder aber auch definieren, warum die verbundenen Entitäten miteinander in Beziehung stehen. Dieses Warum könnte wiederum über z.B. Merkmale beschrieben werden und damit in Berechnungen einfließen.

Da Eigenschaften sowohl von Relationen als auch von Entitäten zu berücksichtigen sind, leiten sich in Abbildung 7-2 beide vom so genannten *Informationsraumelement* oder kurz *Element* ab, welches die Eigenschaften bzw. Eigenschaftsstruktur für Entitäten und Relationen kapselt. Die Strukturierung der Eigenschaften selbst orientiert sich an den schon im Datenfluss in Abschnitt 7.3 eingeführten Konzepten. Die Eigenschaften eines Elementes unterteilen sich in Metadaten, Annotationen und Merkmale, welche Informationsentitäten und Relationen näher beschreiben. Zusätzlich ist noch der Inhalt eine „natürliche“ Eigenschaft eines Elementes.

Ähnlich wie bei der Typisierung der Informationsentitäten und Entitätsrelationen ist auch eine Typisierung von Metadaten, Annotationen und Merkmalen wichtig. Die semantische Beschreibung über ein solches Typsystem erlaubt eine bessere Spezifikation der Algorithmen sowie die Prüfung, ob Algorithmen auf gewissen Eigenschaften operieren können oder nicht. Die Flexibilität innerhalb des Typsystems ist dabei ein wichtiges Kriterium. Ein zu starres Typsystem kann mögliche zukünftige Erweiterungen des Modells stark einschränken und eine schnelle Entwicklung neuer Algorithmen behindern, während ein zu lose gehaltenes Typsystem nur unvollständige Beschreibungen ermöglicht. Als Typisierungsschema sind unterschiedliche Möglichkeiten gegeben, welche von den unterschiedlichen Modalitäten und Standards abhängen. Eine Detaillierung dazu gibt Abschnitt 8.3.2.1.

Während Annotationen und Metadaten aufgrund ihrer Diversität keine weiteren Eigenschaften mehr teilen bzw. eine weitere Untergliederung zu feingranular und damit zu unflexibel für das KnowMiner-Framework wäre, ist die detaillierte Untersuchung der Merkmale wichtig für die weitere Anwendung in Mining-Algorithmen. Merkmale stellen die Grundlage für die weitere Verarbeitung in verschiedenen Mining-Algorithmen dar und abhängig von den Algorithmen müssen unterschiedliche Eigenschaften wie z.B. die Addierbarkeit der Merkmale gewährleistet sein. Zur Kapselung dieser Eigenschaften erfolgt die Gruppierung von Merkmalen zu Merkmalsmengen, die wiederum in Merkmalsräumen zusammengefasst sind. Diese Merkmalsräume kapseln algebraische und mathematische Eigenschaften der Merkmalsmengen und gestatten somit die Abstraktion von Berechnungsmechanismen für Mining-Algorithmen von den darunter liegenden Daten. Unter Berücksichtigung der Transformationsmechanismen zwischen Inhalt, Metadaten, Annotationen und Merkmalen (siehe Abschnitt 7.5) und der auf den abstrahierten Merkmalsoperationen anwendbaren Mining-Algorithmen ist somit eine Trennung zwischen Repräsentationsform und Algorithmus und, damit verbunden, die unabhängige Implementierung und Anwendung von Algorithmen möglich. Wie im Kapitel 8 zur Implementierung des KnowMiner-Frameworks beschrieben, müssen natürlich unterschiedliche Repräsentationsformen von Merkmalsmengen wie z.B. Merkmalsvektoren, -listen oder -graphen berücksichtigt werden, um eine laufzeit- und speichereffiziente Anwendung zu ermöglichen.

Neben der Typisierung und Repräsentation der Eigenschaften einer Entität ist noch der Kontext, in welchem die Eigenschaften betrachtet werden, von zentraler Bedeutung für die Durchführung und das Ergebnis des Mining-Prozesses. Betrachtet man den Kontext einer Entität näher, so kann zwischen der Sichtweise auf das Element und der Sichtweise auf die Beziehungen des Elements zu anderen Elementen unterschieden werden. Für unterschiedliche Algorithmen macht es Sinn, unterschiedliche Mengen von Eigenschaften einer Entität zu betrachten, was in weiterer Folge als *Entitätskontext* bezeichnet wird. Zum Beispiel benötigt ein Suffix Tree Clustering-Algorithmus [Zamir et. al. 1998] als Merkmalseigenschaft die Sätze eines Dokumentes, während der K-Means Algorithmus eine Vektorrepräsentation benötigt. Beide Darstellungsformen sind Merkmale und in weiterer Folge Eigenschaften von Informationsentitäten, die in unterschiedlichen Kontexten anzuwenden sind. Der Entitätskontext kann also als Sichtweise auf die Eigenschaften einer Entität in einer bestimmten Situation aufgefasst werden.

Neben diesem Entitätskontext ist zusätzlich der Umgebungskontext, d.h. die Beziehungen von Entitäten zu anderen Entitäten zu berücksichtigen. Wie schon zuvor illustriert, können z.B. durch Berücksichtigung von hierarchischen IS-A - Beziehungen zwischen Entitäten die Anzahl der notwendigen Merkmale einer Entität reduziert oder aber auch wie im Projekt InfoSky (siehe Abschnitt 6.1.2) die Laufzeitkomplexität deutlich verringert werden. Daraus lässt sich ableiten, dass die Berücksichtigung des Umgebungskontextes ebenfalls

von Bedeutung bei der Abarbeitung der KD-Algorithmen ist und vor allem Einfluss auf Genauigkeit, Geschwindigkeit und Speicherbedarf nimmt.

Zusammenfassend betrachtet, gliedert sich die Strukturierung des Informationsraums in Aspekte bezogen auf die Struktur des Informationsraumes und in Aspekte bezogen auf die Eigenschaften der Einzelelemente des Informationsraumes. Ziel dabei ist die Abstraktion der Operationen auf beiden Aspekten, um eine möglichst galvanische Trennung zwischen Algorithmen und Datenrepräsentation zu erreichen. Zusätzlich zu dieser Trennung sind unterschiedliche Sichtweisen auf Einzelentitäten und auf den Informationsraum notwendig, um verschiedene Kontexte und damit situationsbedingte Einsatzmöglichkeiten zu unterstützten.

Der nächste Abschnitt definiert in weiterer Folge die verwendeten KnowMiner-Konzepte im Detail, um Unklarheiten in der Namensgebung und Begriffsdefinition zu vermeiden und das Verständnis der einzelnen Konzepte durch Beispiele zu fördern. Anschließend erfolgt die Detaillierung der möglichen Transformationen auf Informationsraum und Entitätsebene sowie die Spezifikation der Voraussetzung für die Ausführung der einzelnen Algorithmen.

7.4 Konzepte im KnowMiner-Framework

Um Unklarheiten zu vermeiden, wird im Folgenden eine Definition der in der weiteren Beschreibung verwendeten KnowMiner-Konzepte/Begriffe gegeben. Für eine klare Spezifikation wird dabei auf die Definition von Konzepten im so genannten semiotischen Dreieck [Ogden & Richards 1923] zurückgegriffen (siehe auch Abschnitt 4.6).

Die Beschreibung der Konzepte erfolgt anhand ihrer Intension, also Definition, ihrer Extension, also Beispielen für die Konzepte, und ihrer lexikalischen Begriffe. Zusätzlich werden bei der Beschreibung der Konzepte die Zusammenhänge zu anderen Konzepten und deren Anwendung angeführt.

Grundlegend ist dabei zu beachten, dass je nach Anwendung die Ausprägung der KnowMiner-Konzepte unterschiedlich sein kann. In einer Anwendung als Metadaten gehandelte Eigenschaften können in einer anderen Anwendung als Merkmale dienen. Beispielsweise ist für ein inhaltliches Gruppieren von Dokumenten der Autor eines Dokumentes - als Metadatum gegeben - von zweitrangiger Bedeutung, während bei der Zitationsanalyse das Metadatum Autor ein zentrales Merkmal zur Ermittlung von Ähnlichkeiten zwischen Dokumenten darstellt. Daher sind die aufgeführten KnowMiner-Konzepte stark an ihrer Funktionsweise im Framework angelehnt und nicht an den verarbeiteten Inhalten an sich.

7.4.1 Informationsentität

Intension: Informationsentitäten sind die kleinste zu betrachtende Informationseinheit im KnowMiner-Framework, auf die KD-Algorithmen angewendet werden. Informationsentitäten besitzen einen Entitätstyp, einen Status, einen Inhalt, eine Beschreibung durch Merkmale sowie der Informationsentität zugeordnete Metadaten. Über typisierte Relationen (siehe Entitätsrelation unten) werden Informationsentitäten miteinander in Beziehung gesetzt.

Über die Angabe eines Typs können Entitäten näher spezifiziert werden, was vor allem für das Stellen von Abfragen im Informationsraum essentiell ist. Ähnlich verhält es sich mit der Angabe eines Status, welcher den aktuellen Zustand der Entität widerspiegelt.

Die Basistypen einer Entität sind über folgende, beliebig erweiterbare Typhierarchie definiert:

- DataEntity
  - Document
  - Search Result
  - Concept
- Container
  - Search
  - Cluster
  - Collection
- Concept
  - Person
  - Location
  - ..

Abhängig von der Art der Anwendung ist die Konvertierung von Entitäten eines Typs in Entitäten eines anderen Typs möglich. Zum Beispiel basiert im Anwendungsfall Ontology Learning die Analyse eines Dokumentkorpus und die Bildung von Konzeptnetzwerken auf den in den Sätzen gemeinsam vorkommenden Worten. Jedes Konzept des Netzwerkes kann aber wiederum als Entität mit dem Typ Concept dargestellt werden.

Extension:

- Dokumente: Importierte Dokumente bzw. Dokumentteile
- Collections/Ordner: Allgemeiner Container, der als Gruppierung von Informationsentitäten aufgefasst werden kann
- Suche: Container, der eine Suche bzw. Suchanfrage kapselt
- Cluster: automatisch erzeugte Gruppen von Entitäten
- Konzepte, wie oben definiert

Anwendung: Entitäten können als Objekte verstanden werden, welche in Beziehung zu anderen Objekten stehen und Eigenschaften wie Merkmale, Annotationen und Metadaten kapseln. Sie ermöglichen das Abfragen und Traversieren des Informationsraums und dienen als Basiseinheit für die Verarbeitung von Algorithmen und Vorverarbeitungskomponenten.

7.4.2 Entitätsrelation

Intension: Eine Entitätsrelation beschreibt eine gerichtete und typisierte Beziehung zwischen zwei Informationsentitäten. Eine solche Relation ist immer Element eines Informationsraumes und kann ähnlich einer Informationsentität mittels Metadaten, Merkmalen und Inhalt beschrieben werden.

Extension:

- IS-A Beziehung zur Erstellung von Taxonomien
- Hyperlinks zwischen Informationsentitäten (i.e. Dokumenten)
- Container von Beziehungen, welche die Zuordnung einer Informationsentität zu einer Menge von Informationsentitäten beschreiben

- Part-of Beziehung

Anwendung: Entitätsrelationen dienen, gemeinsam mit Informationsentitäten, zur Strukturierung des Informationsraumes und zur Navigation innerhalb eines solchen. In Abhängigkeit vom zugrunde liegenden Datenmaterial ist die Abbildung semantisch reichhaltiger Relationen oder aber auch einfacher Relationen wie Hyperlinks möglich. Neben der Vorgabe von Relationen aus Datenquellen ist unter Anwendung von KD-Algorithmen auch die automatische Erzeugung von Relationen zwischen Entitäten möglich (z.B. Zitationen von Dokumenten).

7.4.3 Informationsraum

Intension: Ein Informationsraum definiert sich über die Menge der enthaltenen Informationsentitäten und deren Relationen zueinander. Bedingt durch die der Relation und Entität zugeordneten Merkmale, Metadaten und Inhalte kapselt der Informationsraum ebenfalls die Menge der in ihm enthaltenen Merkmale und Metadaten. Im KnowMiner-Framework sind Informationsräume als gerichteter und typisierter Graph ausgelegt, wodurch beliebige Strukturen realisierbar sind.

Extension:

- Dokumentrepository
- Konzeptnetzwerk
- WWW

Anwendung: Der Informationsraum kapselt den Zugriff auf Entitäten und deren Relationen unter Anwendung von verschiedenen Anfragemethoden.

7.4.4 Element

Intension: Das Element kapselt die gemeinsamen Eigenschaften von Entitäten und Relationen sowie deren Bezug zum Informationsraum. Gemeinsame Eigenschaft von Entitäten und Relationen ist die Verwaltung von Merkmalen und Metadaten. Das Element kann als künstliches Konzept angesehen werden und dient als Überbegriff von Entität und Relation.

Extension:

- Informationsentität
- Entitätsrelation

Anwendung: Elemente im KnowMiner-Framework kapseln die gemeinsamen Eigenschaften von Relationen und Entitäten zur Vereinfachung von Notation, Implementierung und Zugriff auf den Informationsraum.

7.4.5 Eigenschaften von Elementen

Nach der Spezifikation der strukturrelevanten Einheiten sollen im Folgenden die Eigenschaften der Entitäten spezifiziert werden.

7.4.5.1 Inhalt

Intension: Der Inhalt einer Entität bzw. einer Relation ist der binäre Datenstrom, der eine Entität oder eine Relation beschreibt, wobei dieser auf ein einheitliches Format reduziert ist.

Extension:

- Formatnormalisierter Text
- Bilddaten, Base 64 kodiert
- Audiostream

Anwendung: Der Inhalt ermöglicht den Zugriff auf die formatnormalisierten Originaldaten einer Informationsentität inkl. der damit bereitgestellten Metadaten (deduktive Metadaten). Die Haltung der Daten kann z.B. in einem XML Format erfolgen und zusätzlich als Serialisierungsformat dienen.

7.4.5.2 Annotationen

Intension: Annotationen sind Beschreibungen von Teilinhalten. Im Gegensatz zu Merkmalen dienen Annotationen nicht als Grundlage für Berechnungen (außer, sie werden in Merkmale transformiert) und werden auch nicht zum Vergleich von Entitäten und Relationen verwendet. Annotationen beschreiben Teile des Inhaltes näher und müssen somit auf den Ursprungsinhalt referenzieren, z.B. über die Position eines Wortes im Text, im Gegensatz zu Merkmalen, welche losgelöst vom Inhalt existieren können.

Extension:

- Beschreibung von Worten (z.B. Wortstamm, Part-of-Speech , Start-/Endposition)
- Beschreibung von Sätzen (z.B. Beginn/Ende eines Satzes)
- Beschreibung von Bildsegmenten

Anwendung: Die Vorverarbeitung extrahiert Annotationen und beschreibt dadurch die Inhalte von Entitäten im Detail. Annotationen ermöglichen jedoch keinen Vergleich bzw. keine standardisierten Berechnungen über eine Menge von Entitäten. Daraus folgt, dass sich Annotationen nur aus der Prozessierung einer Entität ergeben, wie dies z.B. bei der Informationsextraktion der Fall ist. Zu beachten ist, dass Annotationen eine Referenz zum Inhalt darstellen und im Gegensatz zu Merkmalen nicht als Reduktion, sondern als Erweiterung der Information anzusehen sind.

7.4.5.3 Metadaten

Intension: Metadaten sind als Daten über Daten definiert und beschreiben auf die entsprechende Anwendung bezogene wichtige Informationen über eine Entität. Metadaten können dabei deduktiv vorgegeben sein (z.B. Ursprungsformat, Erstellungsdatum) oder induktiv aus Annotationen und Merkmalen ermittelt werden (z.B. Personen, Keywords etc.). Metadaten können unterschiedliche Komplexitätsgrade aufweisen. So werden z.B. semantische Metadaten als Graph abgebildet, während EXIF Metadaten sich mittels Key-Value Paaren darstellen lassen.

Extension:

- Erstellungsdatum

- Dateigröße
- Format der Datei
- Hintergrund bzw. transparente Farbe eines Bildes
- Zugeordnete Klasse

Anwendung: Metadaten sind ausgezeichnete Daten, die bezogen auf die jeweilige Anwendung ein Element des Informationsraumes näher beschreiben. Deduktive Metadaten werden während der Formatnormalisierung extrahiert und abgelegt. Induktive Metadaten sind, in Abhängigkeit von der Anwendung, aus Annotationen und Merkmalen extrahierbar. Beispielsweise können Metadaten, wie z.B. das Metadatum Autor, aus den in Annotationen erkannten Personen abgeleitet und spezifiziert werden.

Dabei ist zu beachten, das Metadaten anwendungsspezifisch wichtige Eigenschaften von Entitäten und Relationen definieren, während Annotationen und Merkmale zur Verarbeitung von Entitäten dienen. Annotationen und Merkmale sind dabei in Metadaten konvertierbar und umgekehrt.

7.4.5.4 Merkmale

Intension: Ein Merkmal ist eine ein Element beschreibende Eigenschaft, die für Berechnungen zwischen Elementen bzw. für die Abarbeitung von Elementen bereitgestellt wird. Ein Merkmal ist die kleinste beschreibende Einheit zu einem Element und es wird im Allgemeinen aus Annotationen bzw. aus den Metadaten erzeugt. Ein Merkmal kann unterschiedliche Repräsentationsformen haben, wobei am häufigsten Merkmale als Key/Value Paare realisiert sind. Der Key enthält dabei einen eindeutigen Schlüssel zur Identifikation eines Merkmales und der Value die Ausprägung des Merkmales. Zu beachten dabei ist, dass bei gleichem Schlüssel Merkmale als gleich angesehen werden und dadurch miteinander vergleichbar sind.

Ausprägungen (Values) können Zahlen, Zeichenketten oder komplexeren Objekten entsprechen. Auf diesen müssen jedoch verschiedene Operationen wie Addition, Subtraktion etc. definiert sein.

Essentiell für Merkmale ist die Eigenschaft, dass sie einen Vergleich zwischen Elementen eines Informationsraumes ermöglichen, d.h. gleiche Merkmale teilen sich den gleichen Schlüssel über verschiedene Entitäten hinweg und können über ihre Ausprägung miteinander verglichen werden.

Extension:

- Term-Gewicht-Paare für die Vektor Space Repräsentation von Objekten
- Metadatum und deren nominale Ausprägungen (z.B. Temperatur={Warm, Kalt, Mittel})
- Binäres Term Gewicht Pärchen für die Darstellung von binären Merkmalsvektoren

Anwendung: Merkmale und deren Strukturierung kommen vor allem bei der Berechnung von Ähnlichkeiten, sowie bei der Ausführung von maschinellen Lerntechniken zum Einsatz.

7.4.5.5 Merkmalstyp

Intension: Ein Merkmalstyp bezeichnet die Verwendung bzw. Semantik eines Merkmals. Durch die Typisierung von Merkmalen können diese in anwendungsspezifisch sinnvolle Gruppen unterteilt und miteinander verglichen werden. Während die technische Vergleichsmöglichkeit über die Merkmalsklasse

definiert ist, adressiert der Typ zusätzlich eine semantische Definition von Merkmalen, d.h. wie Merkmale Entitäten beschreiben.

Extension:

- Wort-Gramme: Der Merkmalsschlüssel repräsentiert Wortgruppen der Länge n und der Wert repräsentiert die Anzahl der Vorkommnisse
- Charakter n-Gramme: Der Schlüssel entspricht Zeichenketten der Länge n und Werte wiederum deren Vorkommnis.
- Named Entities: Der Merkmalsschlüssel beinhaltet die Zeichnkette der benannten Entität und der Wert wiederum das Vorkommnis. Alternativ können Named Entities auch weiter in Personen, Orte etc. unterteilt werden.
- Color Layout Descriptor
- Scalable Color Descriptor

Anwendungen: Merkmalstypen dienen der Typisierung der möglichen Merkmale und der Kapselung von Merkmalen gleicher Art. Zusätzlich können aufgrund von Merkmalstypen auch die Anforderungen eines Algorithmus spezifiziert werden, wie z.B. dass die Extraktion von Schlüsselwörtern nur auf Wortbasis erfolgen kann und das Charakter n-Gramme dabei keinen Sinn machen.

7.4.5.6 Merkmalsklasse

Intension: Merkmalsklassen definieren die technische Realisierung von Merkmalen, d.h. in welcher Form z.B. Schlüssel und Werte abgelegt sind. Eine semantische Unterscheidung bezüglich der Anwendung der Merkmale wird in der Merkmalsklasse nicht getroffen, sondern in den auf die Merkmalsklassen aufbauenden Merkmalstypen.

Die Merkmalsklasse kann im Fall von Key/Value Paaren auch als Tupel aus Schlüsselklasse und Wertklasse aufgefasst werden, wobei die Schlüsselklasse die Klasse der Repräsentation von Schlüsseln (e.g. Zeichenketten, Ganzzahlen) und die Wertklasse die Klasse der Repräsentation von Werten (e.g. Ganzzahl, reellwertige Zahlen etc.) darstellen.

Im Zusammenhang mit Merkmalstypen kann eine Merkmalsklasse in mehreren Merkmalstypen verwendet werden, aber ein Merkmalstyp hat immer nur eine Merkmalsklasse

Extension:

- <Zeichenketten, reellwertige Zahlen>: Zur Abbildung von Wordgrams, Charakter n-grams etc.
- <Ganzzahl, reellwertige Zahlen>: Abbildung von z.B. Histogrammen, 2D- und 3D-Räumen (wenn der Schlüssel auf 1-3 beschränkt ist)

Anwendung: Die Merkmalsklasse kapselt Merkmale gleicher Repräsentationsform, ohne diese semantisch zu unterscheiden. Sowohl Wordgrams als auch Named Entities oder Charakter n-grams können als String/Zahl-Paar aufgefasst werden und haben eine gemeinsame Repräsentation. Durch das gemeinsame Verwalten von Schlüsselwerten kann das aufkommende Datenvolumen reduziert werden, da jeder Schlüssel nur einmal im Speicher gehalten werden muss.

7.4.5.7 Merkmalsmenge/Feature Set

Intension: Eine Merkmalsmenge bezeichnet eine Menge von Merkmalen eines bestimmten Merkmalstyps. Merkmalsmengen können als Sammelbehälter für Merkmale gemeinsamen Typs aufgefasst werden, die mathematische Operationen wie Addition, Subtraktion etc. (abhängig natürlich von der Arte der Merkmalsmenge) zwischen Merkmalsmengen des gleichen Typs ermöglichen.

Extension:

- Merkmalsserie: Eine Merkmalsserie entspricht einer reihenfolgeabhängigen Merkmalsmenge mit mehrdeutigen Schlüsseln, wie sie z.B. eine Abbildung von diskreten Zeitserien darstellt. Ein Merkmal mit gleichem Schlüssel kann mehrmals an verschiedenen Positionen vorkommen. Merkmalsmengen sind zueinander unterschiedlich, wenn die Reihenfolge unterschiedlich ist, auch wenn die darin enthaltenen Merkmale gleich sind.
- Merkmalsvektor: Ein Merkmalsvektor entspricht einer reihenfolgeunabhängigen Merkmalsmenge mit eindeutigen Merkmalsschlüsseln. Jeder Merkmalsschlüssel bildet eine Dimension des Vektors, der Wert bestimmt die Ausprägung entlang der Dimension. Ein binärer Merkmalsvektor dient zur Darstellung einer reihenfolgeunabhängigen Merkmalsmenge, d.h. ein Merkmal ist in der Menge enthalten oder nicht, und zwei Mengen sind gleich, wenn sie die gleichen Merkmale, egal in welcher Reihenfolge diese aufscheinen, enthalten.
- Merkmalsgraph: Ein Merkmalsgraph stellt eine Merkmalsmenge mit eindeutigen Schlüsseln und Verbindungen zwischen den Merkmalen dar.

Anwendung: Merkmalsmengen dienen dem Vergleich von Informationsentitäten und der Durchführung von mathematischen Operationen auf bzw. zwischen Informationsentitäten. Die Merkmalsmengen abstrahieren und komprimieren den Inhalt und die auf dem komprimierten Inhalt ausführbaren mathematischen Operationen einer Informationsentität.

7.4.5.8 Merkmalsraum/Feature Space

Intension: Ein Merkmalsraum ist die Sammlung von Merkmalsmengen eines bestimmten Merkmalstyps von verschiedenen Informationsentitäten. Er abstrahiert die gleiche Ausprägung von verschiedenen Informationsentitäten und definiert mathematische Eigenschaften wie das Kommutativgesetz oder die Dreiecksungleichung etc.

Der Merkmalsraum ermöglicht das Durchführen von mathematischen Operationen zwischen Merkmalsmengen.

Extension:

- Vektorraum (laut mathematischer Definition)
- Hilbertraum (entspricht einen Vektorraum plus Definition des inneren Produktes zwischen Vektoren)
- Abelsche Gruppe: Menge von Merkmalen, für die das Kommutativgesetz gilt
- Körper: eine ausgezeichnete algebraische Struktur, in der die Addition, Subtraktion, Multiplikation und Division durchgeführt werden kann

7.4.5.9 Entitätskontext

Intension: Der Entitätskontext beschreibt die Sichtweise auf Eigenschaften, im Speziellen auf Metadaten, Merkmale und Annotationen eines Elementes. Mit dem Kontext können für unterschiedliche Abarbeitungsschritte unterschiedliche Eigenschaften sozusagen aktiviert werden. Zum Beispiel benötigen Ordinationstechniken eine hochdimensionale Repräsentation von Entitäten zur Berechnung der Ähnlichkeiten, und sie erzeugen eine niederdimensionale Repräsentation der Entität. Je nach Sichtweise oder Kontext betrachtet man entweder die niedrigdimensionale oder die hochdimensionale Repräsentation oder beide.

Der Kontext kann auch als Gruppierung und Filter für die Eigenschaften von Entitäten verstanden werden.

Extension:

- Hochdimensionale Merkmalsdaten
- 2D-Koordinaten

Anwendung: In Anwendungen erfüllt der Kontext zwei Aufgaben. Einerseits können über den Kontext die Eigenschaften einer Entität für Algorithmen bereitgestellt werden, andererseits können Algorithmen definieren, in welchem Kontext sie sich bestimmte Eigenschaften von Entitäten erwarten. Beispielsweise braucht ein K-Means Clusterer eine auf einem Vektorraum basierende Merkmalsmenge, die die Berechnung von Ähnlichkeiten ermöglicht, während ein Suffix Tree Clusterer Annotationen vom Typ „Satz" benötigt. Der Kontext kapselt somit die Merkmals- und Annotationstypen, den Zugriff auf Merkmale/Annotationen und die den Merkmalsräumen zugrunde liegenden Berechnungseigenschaften.

7.4.5.10 Umgebungskontext

Intension: Der Umgebungskontext beschreibt den Kontext einer Informationsentität in Bezug auf andere Informationsentitäten. Dabei kann von einem induktiven oder einem deduktiven Umgebungskontext ausgegangen werden. Im induktiven Fall definiert sich der Umgebungskontext durch eine Menge von Informationsentitäten und Relationen im gleichen Informationsraum. Der deduktive Kontext definiert sich über eine Menge von Entitäten und Relationen in anderen Informationsräumen bzw. durch extern vorgegebene Ressourcen, wie z.B. Taxonomien oder Ontologien, welche die Informationsentität mit anderen Entitäten innerhalb und außerhalb des eigenen Informationsraumes in Beziehung setzen.

Extension:

- Mengen von Entitäten zur Ermittlung von Merkmalsstatistiken
- Beziehungen zwischen Entitäten
- Externe Ressourcen wie Taxonomien, Ontologien etc.

7.5 Transformationen im Informationsraum

KD besteht vornehmlich aus der Transformation von Daten. Neben den im KnowMiner-Framework verwendeten Konzepten und den in den einzelnen Schritten zu verarbeitenden Daten müssen daher auch die möglichen Transformationen des Informationsraumes und die Möglichkeit zur Durchführung

unterschiedlicher Berechnungen im KnowMiner-Framework berücksichtigt werden, um eine konzeptuelle Abstraktion der einzelnen Komponenten zu erreichen.

Grundlegend kann zwischen der Transformation einer Entität und der Transformation des Informationsraumes unterschieden werden. Diese Transformationen stellen auch eine Dimension der Kategorisierung von KD-Algorithmen dar und dienen als konzeptionelle Grundlage zur Strukturierung der Abläufe und Abfolgen von KD-Aufgaben. So benötigt z.B. ein Clusteringalgorithmus zur erfolgreichen Abarbeitung Entitäten mit zugeordneten Merkmalen eines bestimmten Typs, woraus sich als Vorgängeraufgabe die Extraktion von Merkmalen aus Annotationen ergibt. Im Folgenden bezeichnen *Eigenschaftstransformationen* auf eine Entität begrenzte Transformationen und *Strukturtransformationen* Transformationen des Informationsraumes, wie sie z.B. im Hinzufügen und Löschen von Entitäten und Relationen bestehen.

7.5.1 Eigenschaftstransformation

Eigenschaftstransformationen definieren sich dadurch, dass sie die Eigenschaften einer Informationsentität ändern, jedoch nicht die strukturelle Einbettung der Entität im Informationsraum selbst. Für die Durchführung der Transformation wird im Allgemeinen aber die strukturelle Einbettung als induktiver Umgebungskontext der Transformation berücksichtigt. Dies geschieht beispielsweise bei der Errechnung von Merkmalsstatistiken, indem nur Informationsentitäten der gleichen Klasse in der Statistik Berücksichtigung finden. Neben diesem induktiven Umgebungskontext ist es auch möglich, externe Ressourcen und Wissensbasen als deduktiven Umgebungskontext für die Transformation zu verwenden, was in der Regel speziell bei linguistischen Verfahren Anwendung findet.

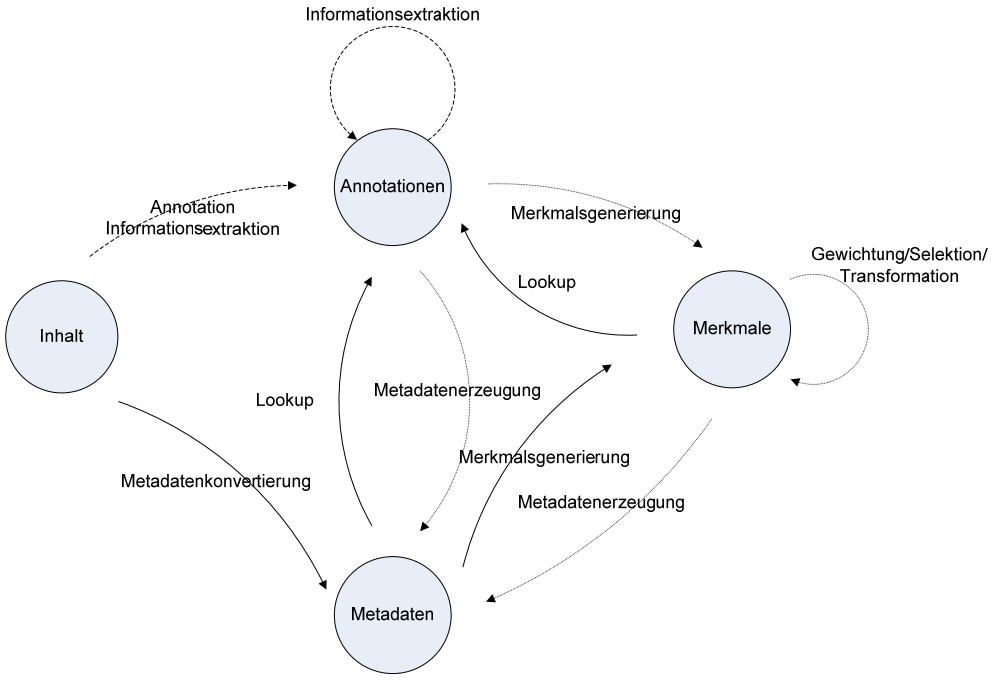


Abbildung 7-3: Transformationen zwischen Eigenschaften von Elementen eines Informationsraumes. Punktierte Linien beschreiben vorwiegend informationsreduzierende Transformationen, durchgezogene Linien informationserhaltende Transformationen und strichlierte Linien informationsanreichernde Transformationen

Der Einfluss auf den Informationsgehalt einer Entität definiert dabei ein wichtiges Kriterium für die Unterscheidung von Transformationen. Im Allgemeinen ist das Ziel der Wissenserschließung die Reduktion bzw. Kompression der Information auf die im aktuellen Kontext sinnvollste Größe. Um dies zu erreichen, müssen Transformationen die Anforderung der sinnvollen, d.h. im Anwendungskontext möglichst verlustfreien Reduktion der Eigenschaften einer Informationsentität berücksichtigen. Reduzierende Transformationen können auch als deskriptive Transformationen verstanden werden, die Informationsentitäten in einer komprimierten Form beschreiben. Invers dazu existieren Transformationen, die Informationsentitäten anreichern, d.h. auf Basis gewisser prozeduraler Abläufe wie z.B. der Klassifikation oder unter Einbringung externer Ressourcen den Informationsgehalt von Entitäten erweitern. Solche Transformationen können auch als prädiktive oder anreichernde Transformationen verstanden werden. In die letzte Transformationskategorie fallen umwandelnde Transformationen, welche Eigenschaften lediglich umwandeln ohne dabei den Informationsgehalt zu verändern.

Die im KnowMiner-Framework abgebildeten Eigenschaftstransformationen inklusive deren Kategorisierung bezüglich Informationsgehalt sind in Abbildung 7-3 zusammengefasst.

Inhalt-Annotationen: Beginnend beim auf ein einheitliches Format normalisierten Inhalt einer Entität erfolgt unter Anwendung von z.B. Informationsextraktion die Erzeugung von Annotationen des Inhaltes. Dieser Schritt stellt im Allgemeinen eine informationsanreichernde Transformation dar, da Teile des Inhaltes detaillierter beschrieben werden. Im Textbereich bedeutet dies z.B. die Annotation der einzelnen Wörter mit ihrem sprachlichem Gebrauch (Part-of-Speech Tagging) oder ihrer reduzierten Wortform (Stemming). Im Bildbereich kann dies z.B. die Auszeichnung spezieller Regionen von Bildern, z.B. von Personen in einem Videobild, sein. Annotationen stellen die Basis für die weiteren Transformationen im KnowMiner-Framework dar und ermöglichen somit das Extrahieren von Merkmalen und Metadaten auf Basis eines angereicherten, erweiterten Inhalts. Annotationen können auch zur Erzeugung weiterer Annotationen führen, wie dies z.B. im GATE-Framework (siehe Abschnitt 5.1.1) der Fall ist. Unter Einbindung externer Ressourcen erfolgt die Annotation von Worten bzw. Wortgruppen, d.h. ob es sich bei diesen z.B. um den Vornamen einer Person handelt. Unter Anwendung von Grammatiken und Regeln wird bestimmt, ob es sich bei mit Vornamen annotierten Wortgruppen um einen Namen handelt (z.B. nächstes Wort nach dem Vornamen ist ein Substantiv) oder nicht.

Inhalt-Metadaten: Neben der Annotation des Inhaltes erfolgt auch die Extraktion/Konvertierung der von außen vorgegebenen deduktiven Metadaten. In Abhängigkeit vom jeweiligen Anwendungsfall können dies z.B. die Dateigröße, der externe Ablageort, das Erzeugungsdatum etc. sein. Zu berücksichtigen hierbei ist, dass die Definition des formatnormalisierten Inhalts auch die damit verbundenen externen Metadaten beinhaltet und dass es in diesem Schritt lediglich um die Transformation dieser in relevante, für die Entität ausgezeichnete Metadaten geht.

Annotationen-Merkmale: Aus den Annotationen erfolgt die Generierung der für die weiterführenden Berechnungen nötigen Merkmale. Diese Transformation kann meist als Zählen von Annotationen (z.B. Häufigkeit von Worten) und als Filtern von Annotationen (z.B. das Entfernen von Stoppwörtern) aufgefasst werden. Neben der puren statistischen Auswertung erfolgt auf Basis der generierten Merkmale meist auch eine Merkmalstransformation, d.h. Merkmale werden aufgrund von statistischen Eigenschaften wie deren Vorkommnis in verschiedenen Entitäten entsprechend gewichtet (siehe Abschnitt 3.3) oder aber entfernt.

In die umgekehrte Richtung existiert ebenfalls die Möglichkeit, Merkmale in Annotationen zu überführen, wobei hier lediglich ein Lookup von Merkmalen in Annotationen möglich ist. Da Merkmale laut Definition keine Beziehung mehr zum Inhalt aufweisen, kann es zu keiner Erzeugung von Annotationen kommen, da diese sich ja über die Referenz auf den Inhalt definieren. Eine Rückführung von Merkmalen auf Annnotationen ist vor dem Hintergrund eines Nachschlagens von zu einem Merkmal passenden Annotationen möglich und kann z.B. dazu genutzt werden, um die statistisch wichtigsten Annotationen einer Entität zu ermitteln.

Ein kleines Beispiel zur Illustration der Annotationen-Merkmale Transformationen: Aus dem Text eines Dokumentes erfolgt die Annotation des Wortstamms der einzelnen Worte. Die Wortstämme bilden die Basis für die Extraktion von Merkmalen und mittels TFIDF-Gewichtung (siehe Abschnitt 3.3.4) lassen sich die wichtigsten Wortstämme extrahieren. Über den entsprechenden Wortstamm ist nun eine Identifikation der wichtigsten Annotationen und somit die Überführung des Wortstamms in für den Benutzer aussagekräftige Beschreibungen möglich. Dieses Verfahren wurde zur Ermittlung von Beschreibungen für Gruppen von Dokumenten in den Projekten WebRat und InfoSky angewendet, welche im Abschnitt 6.1 näher beschrieben sind.

Merkmale-Metadaten: Eine zentrale Transformation bildet die Überführung von Merkmalen in Metadaten, welche auf mehrere Arten erfolgen kann. Eine direkte Überführung transformiert im Anwendungskontext wichtige Merkmale direkt in Metadaten, wie dies z.B. bei der Generierung von Schlüsselwörtern bzw. wichtigen Begriffen für eine Entität gemacht wird. Hierbei handelt es sich um einen deskriptiven Prozess, welcher Merkmale weiter reduziert und in Form von Metadaten zur Verfügung stellt. Zusätzlich zu dieser direkten Konvertierung setzen prädiktive Algorithmen die Merkmale einer Entität zur Voraussage von gewissen Eigenschaften ein, wie z.B. im Anwendungsfall der Textklassifikation (siehe Abschnitt 4.3), wo auf Basis der Merkmalsmengen von Entitäten diese automatisch einer vordefinierten Klassenstruktur zugeordnet werden. Die Klasse ist dabei als Metadatum aufzufassen.

Eine ebenfalls zentrale Transformation zwischen Metadaten und Merkmalen ist die Merkmalsgenerierung aus Metadaten. In den meisten Anwendungsfällen ist es wichtig, Metadaten für die Verarbeitung von Entitäten und den Vergleich von Entitäten als Merkmale zu verwenden. Als Beispiel seien hier Spam-Filter angeführt, welche E-Mails nicht nur auf Basis des Inhaltes einer E-Mail, sondern auch auf ausgezeichneten Metadaten wie Absender und Zeit in Spam oder Nicht-Spam klassifizieren.

Metadaten-Annotationen: Neben der Transformation von Merkmalen in Metadaten spielt auch die Transformation von Annotationen in Metadaten eine wichtige Rolle. Während Merkmale stärker aufgrund statistischer Auswertungen überführt werden, erfolgt die Transformation von Annotationen in Metadaten meist auf Basis von Regeln, Grammatiken und, im Textbereich, anhand linguistischer Analysen. Zum Beispiel ermöglichen über Annotationen identifizierte Personen inklusive deren Rolle (z.B. Autor, Koautor) die Überführung dieser in entsprechende Metadaten z.B. das Feld Autor im Dublin Core Standard. Ähnlich ist es auch bei der Summarisation, dem automatischen Zusammenfassen von Entitäten. Da es im Allgemeinen sinnfrei ist, alle Annotationen in Metadaten zu überführen, kann diese Transformation als informationsreduzierend aufgefasst werden.

Umgekehrt ist die Transformation von Metadaten in Annotationen, ähnlich wie bei den Merkmalen, ein Nachschlagen von Annotationen auf Basis der gegebenen Metadaten. Hintergrund ist, dass Metadaten nicht notwendigerweise einen Bezug zum Inhalt einer Entität haben müssen. Wie oben angeführt, kann z.B. mittels

Textklassifikation die Klasse einer Entität in einer Ablagestruktur ermittelt werden, was nicht notwendigerweise auf Inhaltsebene annotierbar ist. Dadurch entspricht die Überführung von Metadaten in Annotationen einer Konvertierung.

7.5.2 Strukturtransformation

Neben Transformationen einzelner Informationsentitäten sind noch Transformationen des Informationsraumes selbst zu betrachten. Während strukturelle Transformationen beliebige Komplexitätsgrade aufweisen können – ausgehend vom einfachen Gruppieren von Dokumenten bis hin zum automatischen Erstellen von typisierten Relationen zwischen einzelnen Informationsentitäten – sollen in diesem Abschnitt Transformationen auf einer abstrakteren Ebene betrachtet werden. Dies trägt dem Umstand Rechnung, dass der Informationsraum in Abhängigkeit von der jeweiligen Anwendung viele verschiedene Repräsentations- und Transformationsmöglichkeiten hat und sich damit eine Konkretisierung der Transformation als schwierig erweist.

Wie in Abschnitt 7.3 beschrieben stellt der Informationsraum einen Graphen mit Informationsentitäten als Knoten und Entitätsrelationen als Kanten dar. Zusätzlich können sowohl Entitäten als auch Entitätsrelationen unterschiedlichen Typen (z.B. Dokument, Verzeichnis, Konzept, Is-A, is-container-of) aus einem gegebenen Typsystem angehören. Aus der gewählten Repräsentationsform ergeben sich die grundlegenden Graphenoperationen wie das Einfügen, Aktualisieren und Löschen von Knoten und Kanten als Transformationsmöglichkeiten. Diese weisen vor allem in der Umsetzung und unter Berücksichtigung von konkurrenten Zugriffen einen hohen Komplexitätsgrad auf.

Konzeptuell ist lediglich das flexible Typsystem für Relationen und Entitäten von Interesse, das die anwendungsspezifische Strukturierung des Informationsraumes ermöglicht. Grundlegend wird dabei zwischen Strukturierungsentitäten, so genannten Containern, und Datenentitäten, so genannten Data Entities unterschieden. Bezogen auf diese grobe Typisierung lassen sich neben den oben erwähnten Graphenoperationen Strukturtransformationen unterscheiden. Zu diesen gehören z.B. die Zuordnung von Datenentitäten zu Strukturierungsentitäten (z.B. mittels Klassifikation), die Erzeugung neuer Strukturierungsentitäten über eine Menge von Datenentitäten (z.B. Clustering), die Zerlegung von Datenentitäten (z.B. Text Segmentation) oder die Zusammenführung von Datenentitäten (z.B. Summarisation). Die detaillierte Spezifikation der Transformationen ist, wie bereits erwähnt, von der Anwendung abhängig, wobei Möglichkeiten zur Spezifikation der Transformationen im nächsten Abschnitt erläutert werden.

7.6 KD Komponenten und Algorithmen

Nach der Beschreibung der KnowMiner-Konzepte und der möglichen Transformationen, sollen im Folgenden die unterschiedlichen Komponenten und Algorithmen von KD-Anwendungen näher behandelt werden. Als Grundlage für die Definition der einzelnen Komponentenklassen dienen dabei der Stand der Wissenschaft, wie er in den Kapiteln 2-5 dargestellt ist, die in Abschnitt 5.2 analysierten KD-Standards sowie die bisher in diesem Abschnitt erarbeiteten Grundlagen.

Im Groben unterscheiden sich KD-Komponenten hinsichtlich ihrer Transformationseigenschaften, also ob sie Entitäten oder die Struktur des Informationsraumes modifizieren. Erstere ändern die Eigenschaften von Entitäten und zweitere die Beziehungen zwischen Entitäten. Als dritte Gruppe sind noch KD-Komponenten

ohne Transformationseigenschaften zu unterscheiden, im Folgenden als nicht-modifizierende Komponentenklassen bezeichnet.

Komponentenklasse	**Ziel**	**Beispiele für Algorithmen & Techniken**[37]
Annotation	Annotation von Entitäten	Named Entity Extraction, Part-of-Speech Tagging
Merkmalsgenerierung	Generieren von Merkmalen einer Entität	Vectorizing, Co-Occurrence Analyse*
Merkmalsgewichtung und -selektion	Selektieren/Gewichten von Merkmalen einer Entität ohne Änderung des Merkmalraums	Entropy Weighting, TFIDF Weighting, Frequenz basierte Termselektion
Merkmalstransformation	Transformation von Merkmalen eines Raumes in einen anderen Raum	Latent Semantic Indexing, Force Directed Placement, FastMap
Metadatenextraktion	Erzeugen von Metadaten aus Annotationen und/oder Merkmalen von Entitäten	Schlüsselwort Extraktion, Template Extraction*, Summarisation*
Metadatenklassifikation	Lernen von Regeln zur Zuordnung von Metadaten zu Entitäten und nachfolgende Zuordnung von Metadaten zu neuen Entitäten	Support Vektor Maschinen, Rocchio, Boosting*, Naive Bayes Klassifikatoren*, Decision Trees*

Tabelle 7-1: Überblick über die verschiedenen Entitäten-modifizierenden KD-Komponentenklassen

Komponentenklasse	**Ziel**	**Beispiele für Algorithmen & Techniken**[38]
Import	Import von Datenquellen und Einbettung in den Informationsraum	Web-Crawling, Datenbankimport, Parsing, Metadata Crosswalking*
Strukturerzeugung	Gruppieren von Entitäten	Clustering (K-Means, HAC, WARD, Suffix Tree Clustering, Density Based Clustering*), Document Subsumption*, Hearst Patterns*
Strukturklassifikation	Lernen der Regeln zum Einfügen von Beziehungen zwischen Entitäten und erzeugen von Relationen zwischen neuen Entitäten auf Basis der Regeln	Supervised Learning (wie unter Metadatenklassifikation beschrieben) mit Anwendung auf Strukturen[39]
Strukturerweiterung	Erweiterung der Struktur auf Basis einer	Ontology Alignment*, Ontology Extension*, Ontology Refinement*, Graph Mining

Tabelle 7-2: Überblick über die verschiedenen Struktur-modifizierenden KD-Komponentenklassen

Komponentenklasse	**Ziel**	**Beispiele für Algorithmen & Techniken**[40]
Inidizierung und Retrieval	Erstellung einer Datenstruktur zum Retrieval von Entitäten und dementsprechende Möglichkeiten zur Suche	Vector-Space-Model, SPARQL/Triple Stores, probabilistisches Modell*, Boolesches Retrieval

Tabelle 7-3: Überblick über die verschiedenen nicht-modifizierenden Komponentenklassen

Auf dieser Basis und unter Berücksichtigung der in den Kapiteln 2-5 betrachteten Techniken erfolgt in diesem Abschnitt die Definition unterschiedlicher Komponentenklassen, welche als Grundlage der Strukturierung der

[37] Mit * gekennzeichneten Algorithmen sind nicht im KnowMiner-Framework umgesetzt

[38] Mit * gekennzeichneten Algorithmen sind nicht im KnowMiner-Framework umgesetzt

[39] Im KnowMiner-Framework erfolgte bis dato keine Anwendung dieser Technicken zur Strukturklassifikation

[40] Mit * gekennzeichneten Algorithmen sind nicht im KnowMiner-Framework umgesetzt

Algorithmen und Dienste des KnowMiner-Frameworks, dienen. Die im KnowMiner-Framework implementierten Algorithmen zu den einzelnen Komponentenklassen sind im Detail in Kapitel 8 dargestellt. Ziel der Komponentenklasse ist eine möglichst modulare Organisation der unterschiedlichen KD-Techniken, welche in weiterer Folge die Grundlage für die Definition der softwaretechnischen Schnittstellen bilden.

Tabelle 7-1 gibt einen Überblick über die verschiedene Entitäten modifizierenden Komponentenklassen, Tabelle 7-2 über die Struktur modifizierenden Komponentenklassen und Tabelle 7-3 über die nicht-modifizierenden Komponentenklassen. Zu den einzelnen Komponentenklassen sind dazu auch Beispiele für Algorithmen und Techniken angegeben. Nicht im KnowMiner-Framework enthaltene Algorithmen bzw. Komponenten sind entsprechend gekennzeichnet. Die hier vorgestellten Komponentenklassen liefern dabei einen guten Ansatzpunkt zur Klassifikation von KD-Komponenten, sowie zur Erarbeitung von Workflows für verschiedene Aufgaben wie z.B. Retrieval & Clustering oder Ontology Learning durch Verschaltung der verschiedenen Komponentenklassen.

7.6.1 Entitäten modifizierende Komponentenklassen

KD-Komponenten dieses Typs modifizieren zwar Entitäten bzw. deren Eigenschaften, nehmen jedoch auf die Struktur des Informationsraumes, also auf die Beziehungen zwischen Entitäten, keinen Einfluss.

Die erste Klasse stellen *Annotationskomponenten* dar, welche Inhalte von Entitäten in Annotationen überführen, wie beispielsweise die Extraktion benannter Entitäten (siehe Abschnitt 3.2). Die Komponente kann dabei auch auf schon vorgegebenen Annotationen weitere Annotationen durchführen und diese beispielsweise verfeinern.

Die Komponentenklasse der *Merkmalsgenerierung* benötigt Entitäten mit Annotationen und erzeugt aus diesen neue Merkmale für die jeweilige Entität. Die Vektorisierung ist ein Beispiel für eine solche Komponente, wobei meist nur gewisse Annotationen in den Merkmalsvektor übernommen werden. Diese Filterung reduziert die Größe der entstehenden Merkmalsvektoren.

Die *Merkmalsgewichtung und -selektion* formt eine etwas interessantere Komponentenklasse. Die Merkmale einer Entität, welche über den Entitätskontext definiert sind, werden gewichtet, modifiziert und gegebenenfalls gelöscht. Wichtig dabei ist, dass diese Komponente keine neuen Merkmale erzeugt. Dies ist ausschließlich der *Merkmalstransformation* vorbehalten, die Merkmale aus einem Merkmalsraum in einen anderen transformiert. Dies reicht vom einfachen Kopieren bis hin zur Anwendungen komplexer Algorithmen wie z.B. des LSI (siehe Abschnitt 4.1.1). Eine TFIDF-Gewichtung erzeugt man beispielsweise durch Merkmalsgenerierung aus den Wort-Annotationen, mit anschließender Merkmalstransformation in einen eigenen Merkmalsraum und nachfolgender Merkmalsgewichtung. Die Merkmalstransformation dient in diesem Beispiel dazu, die Originalmerkmale für weitere Operationen bei zu behalten. Der TFIDF gewichtete Vektorraum kann nun entweder mittels Merkmalsselektion beschnitten oder mittels Merkmalstransformation in einen niedrig dimensionaleren Raum transformiert werden.

Nach Transformationen in Richtung Merkmale stellt die nächste Gruppe die Extraktion von Metadaten in den Vordergrund. Die *Metadatenextraktion* definiert eine Komponentenklasse, die auf Basis von Annotationen und Merkmalen zusammenfasst und als Metadaten der Entität anhängt. Summarisation ist eine Ausprägung einer *Metadatenextraktion.* Sie fasst textuelle Annotationen zusammen und stellt diese in Form eines Metadatums als Schlüsselwort bereit. Die Definition der Metadaten einer Entität ist im Allgemeinen stark von

der Applikationsdomäne abhängig, wodurch die Komponentenklasse der Metadatenextraktion ein breites Spektrum abdecken muss.

Eine ähnliche Funktionsweise bietet auch die *Metadatenklassifikation*, die die Funktionalität überwachter Lernmethoden zur Extraktion von Metadaten kapselt. Wie bei Lernverfahren üblich teilt sich das Verfahren in eine Lern- und Klassifikationsphase. Die Lernphase analysiert die gegebenen Entitäten mit den zugeordneten Metadaten und leitet eine Klassifikationshypothese ab. Die Klassifikationshypothese dient in der Klassifikationsphase dem Hinzufügen von Metadaten zu noch nicht gesehenen Entitäten. Wichtig dabei ist, dass sich diese Art der Klassifikation von der Strukturklassifikation unterscheidet. Bei der Strukturklassifikation (siehe unten) erfolgt die Erzeugung von Relationen zwischen Entitäten, während die *Metadatenklassifikation* lediglich Eigenschaften hinzufügt.

7.6.2 Struktur modifizierende Komponentenklassen

Die Struktur modifizierenden Komponenten beginnen beim *Import*, der externe Datenquellen in interne Strukturen überführt. Des Weiteren hängt der Import, was ihn zusätzlich zu einer Entitäten modifizierenden Komponente macht, Metadaten und Inhalte and Entitäten an. Beispiel für Techniken im Bereich des Imports reichen von komplexen Crawling Techniken bis hin zu einfachen Datenbankanbindungen.

Bei Struktur modifizierenden Komponentenklassen kann von den verschiedenen möglichen Operationen auf Strukturen bzw. zwischen Strukturen ausgegangen werden. Zu Beginn erfolgt im Allgemeinen die Erzeugung von Strukturen, einerseits über den oben genannten Import, andererseits aber auch über z.B. unüberwachte Lernverfahren wie Clustering. Auf Basis einer Menge von Entitäten mit entsprechenden Merkmalen erfolgt die Erzeugung von Relationen. Hierarchische Clusteringverfahren können z.B. IS-A Beziehungen abbilden, während linguistische Ontology Learning Verfahren auch komplexere Relationen wie part-of erzeugen können.

Nach der Erzeugung kann die Struktur modifiziert oder erweitert werden. Tabelle 7-2 fasst dies unter den Komponentenklassen *Strukturklassifikation* und *Strukturerweiterung* zusammen. Bei der Klassifikation erfolgt die Erstellung von Relationen auf Basis zuvor induktiv erstellter Hypothesen (Lernphase), im Gegensatz zur Strukturerweiterung, die neue Entitäten über andere Kriterien wie z.B. die oben genannten linguistischen Regeln zuordnet und keine zuvor erlernten Hypothesen benötigt. Grundlage für beide Komponenten ist ein strukturierter Informationsraum und neu zuzuordnende Entitäten, die mittels Relationen in den Informationsraum eingehängt werden. Sowohl bei der Strukturklassifikation als auch bei der Strukturerweiterung können die zuzuordnenden Entitäten ebenfalls über Relationen verbunden sein. Sie formen damit einen Eingabegraph, der in den Informationsraum eingehängt wird. Somit ist die Problemstellung des Ontology Alignments über diese Komponentenklasse abbildbar.

Die Strukturmodifikation geht, im Gegensatz zur Klassifikation und Modifikation, lediglich von einer gegebenen Struktur aus, welche hinsichtlich verschiedener Geschichtspunkte modifiziert wird, ohne dabei neue Entitäten einzufügen. Beispiele hierfür sind Link Mining [Getoor & Diehl 2005] Techniken, in welchen Graphenstrukturen auf Grund gleicher Strukturen miteinander verschmolzen oder aufgrund sehr unterschiedlicher Strukturen aufgeteilt werden.

7.6.3 Nicht-modifizierende Komponentenklassen

Nicht-modifizierende Komponentenklassen führen im Gegensatz zu den oben diskutierten Komponentenklassen keine Transformationen auf Entitäten oder dem Informationsraum durch. Im Speziellen handelt es sich dabei um Techniken aus dem Bereich des Retrievals

Retrieval und Indizierung können als Zugriffsmethoden auf Entitäten verstanden werden. Die Indizierung erstellt spezielle Datenstrukturen (i.e. invertierten Indizes) zum schnellen Zugriff auf Teile des Informationsraums. Ausgehend von einer gegebenen Entität (der Suchanfrage oder eines Beispiels im Falle von Query-by-Example) oder eines gegebenen Teilgraphen erfolgt die Suche nach Entitäten. Suchen nach Teilgraphen können beispielsweise über einen Tripelstore wie Jena und der Anfragesprache SPARQL erfolgen. Traditionelles content based Retrieval kann dazu beispielsweise auf dem Vector-Space-Model beruhen.

7.7 Zusammenfassung

In diesem Abschnitt wurde die konzeptionelle Grundlage für die Entwicklung des KnowMiner-Frameworks vorgestellt und entwickelt. Ausgehend von den Erfahrungen der Iterationszyklen der KnowMiner-Entwicklung und der Analyse des KD-Prozesses erfolgte die Ableitung der Strukturierung des Informationsraumes, die Definition der verwendeten Konzepte sowie die Beschreibung der Transformationsschritte. Den Abschluss des Abschnitts bildete die Diskussion der verschiedenen Komponentenklassen zur Realisierung von KD-Aufgaben, die den Rahmen zur Ableitung entsprechender Softwareschnittstellen und zur Organisation der KnowMiner-Komponenten in Kapitel 8 bilden.

Auf Seite der Strukturierung des Informationsraums erfolgte eine Trennung zwischen Eigenschaften von Entitäten und der Beziehungen zwischen Entitäten. Diese Trennung ermöglicht eine Definition der verschiedenen Operationen und Transformationen, sowie eine Charakterisierung der Eigenschaften von KD-Komponenten.

Die entwickelten Eigenschaften dienen als eine Möglichkeit, KD-Komponenten semantisch zu beschreiben und somit für die Anwendung in verteilten, serviceorientierten Architekturen vorzubereiten. Zusätzlich ermöglicht die Einteilung der verschiedenen Komponentenklassen eine erweiterte Spezifikation der einzelnen KD-Komponenten im Hinblick auf deren Transformationsfähigkeiten. Neben dem Fernziel über die semantisch erweiterte und ontologisch formal definierte Form dieser Beschreibung die Orchestrierung von Services zu automatisieren, ermöglicht die informale, verbale Beschreibung dieser Eigenschaften bei der Entwicklung von KD-Komponenten ein besseres Verständnis zwischen Anwender und Entwickler. Dadurch wird der Einsatz erleichtert und die Wiederverwendbarkeit verbessert.

Die in diesem Abschnitt dargestellten Grundlagen dienen auch dem besseren Verständnis der im nachfolgenden Abschnitt beschriebenen Implementierung des KnowMiner-Frameworks.

8 Implementierung des KnowMiner-Frameworks

Vor dem Hintergrund der zuvor eingeführten konzeptionellen Grundlagen erfolgt in diesem Abschnitt die Beschreibung der Implementierung des KnowMiner-Frameworks. Die Umsetzung erfolgte in Java, wodurch sowohl Plattformunabhängigkeit als auch eine gute Leistungsfähigkeit erreichbar sein sollte. Das Framework selbst ist modular aufgebaut, um einen höchstmöglichen Grad an Flexibilität und Erweiterbarkeit zu sichern. Diese Modularität ermöglicht auch die einfache Einbindung von verschiedenen Open Source Bibliotheken und Frameworks, wie z.B. der Java basierten Suchmaschine Lucene, dem Textengineering Framework GATE und diversen Machine Learning und Lineare Algebra Bibliotheken wie z.B. dem Matrix Java Toolkit.

Die Einbindung von solchen hoch spezialisierten Bibliotheken erweitert das Anwendungsspektrum und bietet die Möglichkeit der Einbettung von leistungsfähigen Algorithmen aus den verschiedensten Bereichen. Die Schaffung eines modularen Aufbaus und die damit verbundene einfache Integration solcher Komponenten ist ein wichtiges Kriterium bei der Implementierung und Integration des Frameworks.

Des Weiteren wartet das KnowMiner-Framework zur Erleichterung der Implementierung von Algorithmen und Anwendungen folgende Eigenschaften auf:

- Abstraktion von Merkmalstypen und Kapselung der Funktionalität zur Verwaltung von Merkmalen
- Abstraktion von Berechnungseigenschaften von Merkmalsräumen sowie Kombinationsmöglichkeiten von verschiedenen Merkmalsräumen
- Abstraktion von Informationsräumen auf Basis semantischer Beschreibungssprachen
- Locking-Mechanismen zur Verwaltung von konkurrenten Zugriffen auf Entitätseigenschaften und Informationsräume
- Komplexes Pipeline Konzept zur einfachen Realisierung verschiedener Annotationsmethoden unter Berücksichtigung wieder verwertbarer Komponenten
- Mechanismen zur parallelen Abarbeitung von Algorithmen
- Transparente Anbindung von Persistierungsmechanismen

Neben den Eigenschaften der KnowMiner-Kernkomponenten werden in diesem Abschnitt auch die Basisdienste, welche als Schnittstelle zum KnowMiner-Kern entwickelt wurden, beschrieben. Diese Basisdienste wurden in den in Kapitel 9 beschriebenen Applikationen angewendet und programmatisch orchestriert, um KD-Anwendung en und deren Einbettung in WMS zu realisieren.

Das vorliegende Kapitel beginnt mit der Diskussion der Basisarchitektur und beschreibt anschließend die verschiedenen Komponenten im Detail. Den Abschluss bildet die Beschreibung der auf den Kernkomponenten aufbauenden Service-orientierten Architektur.

8.1 Basisarchitektur

Vor der Detailbeschreibung der einzelnen Komponenten beschreibt dieser Abschnitt kurz die Grundarchitektur mit ihren Kernkonzepten. Angelehnt an Fayyad's Prozessmodell (siehe Abschnitt 2), kann

in der KnowMiner-Architektur zwischen einem Knowledge Acquisition Teil, einen Prozessierungsteil, einem Persistierungsteil und einem Repräsentationsteil unterschieden werden.

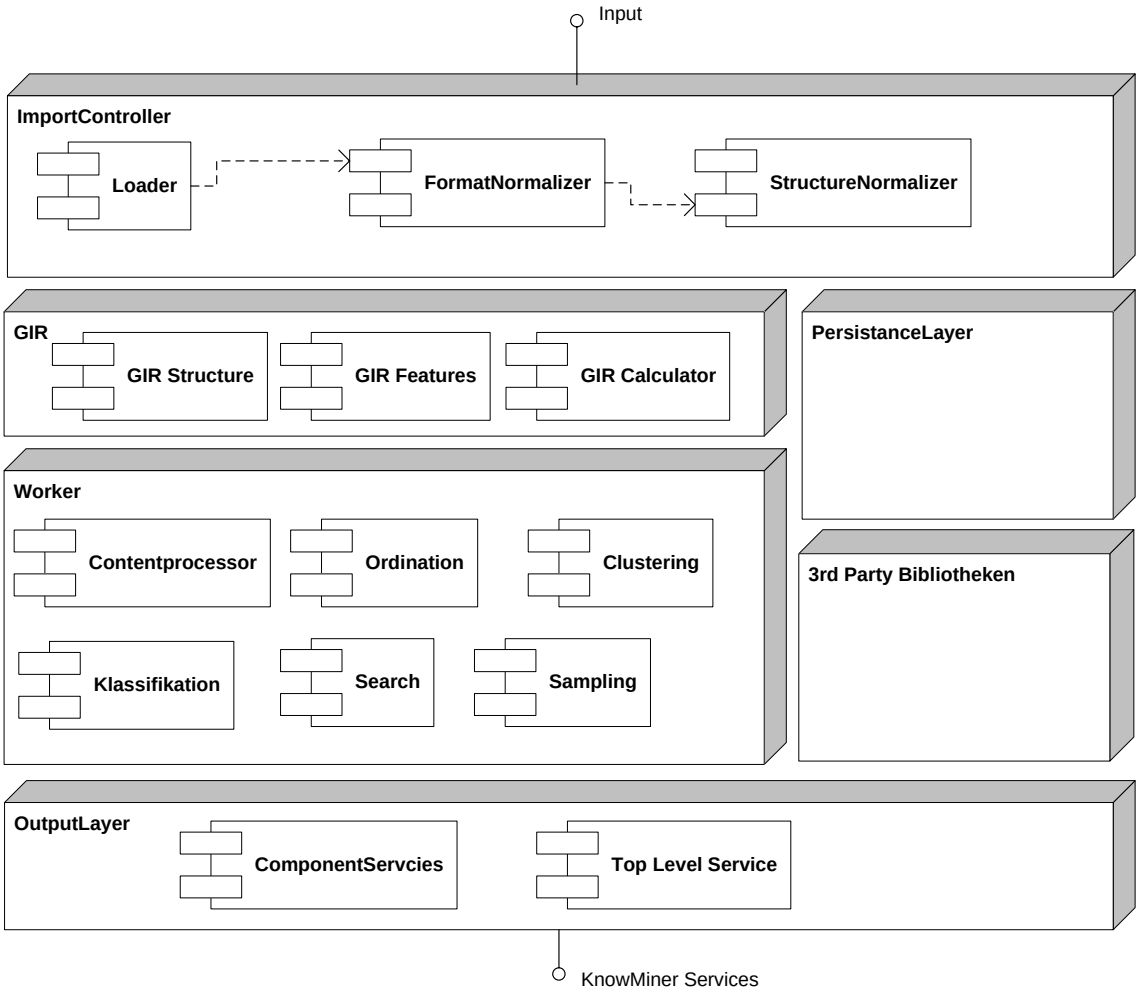


Abbildung 8-1: KnowMiner-Komponentenarchitektur mit ImportController, Worker, Generische Interne Repräsentation, Persistence Layer, Output Layer.

Beginnend bei der Anbindung von Informationsquellen und der Akquisition von Information, kann das KnowMiner-Framework als eine auf einer gemeinsamen Datenrepräsentation arbeitende Pipeline-Architektur verstanden werden. Die Datenrepräsentation, bezeichnet als Generische Interne Repräsentation, ist daher neben der Abstraktion der einzelnen Verarbeitungsschritte und Algorithmen - den Workern - eine zentrale Komponente der gesamten Architektur. Abbildung 8-1 zeigt einen groben Überblick über die im Folgenden beschriebene KnowMiner-Architektur, wobei hinsichtlich der Worker nicht alle Algorithmen angeführt sind. Ein Überblick über die verfügbaren Algorithmen gibt Abschnitt 8.4.

8.1.1 Import Controller

Die Verarbeitung von Information beginnt im *ImportController* (Abbildung 8-1 oben), welcher die Aufgaben der Anbindung verschiedener heterogener Datenquellen und deren Vorverarbeitung über hat. Die *Loader*

Komponenten des *ImportController's* binden dabei die unterschiedlichen Datenquellen wie z.B. Datenbanken, Suchmaschinen oder Dateisysteme an und ermöglichen ein systematisches Abrufen der Informationsentitäten dieser. Konnektoren sind dabei so ausgelegt, dass sie einerseits lokal, andererseits aber auch remote lauffähig sind und den KnowMiner-Framework mit Information über standardisierte Schnittstellen versorgen.

Nach der Akquise von Information erfolgt die Normalisierung der unterschiedlichen Formate der einzelnen Informationsentitäten im so genannten *FormatNormalizer*, welcher die unterschiedlichen properitären Formate in ein generisches internes XML Format, genannt *GML* (Generic Markup Language), überführt. *GML* dient einerseits der Überführung heterogener Informationen in ein gemeinsames Format, stellt aber auch eine standardisierte Form der Persistierung der Inhalte dar (siehe Abschnitt 8.2).

Die Vereinheitlichung der Struktur von verschiedenen Informationsentitäten bzw. Informationsräumen erfolgt im *StructureNormalizer*, der entsprechend den vorigen Schritten die Informationsentitäten in die Generische Interne Repräsentation, anhand verschiedener applikationsabhängiger Ordnungskriterien, einhängt. So ist z.B. einerseits die Erzeugung zeitlich orientierter Informationsstrukturen, andererseits inhaltsorientierter Organisationsstrukturen wie z.B. Ablagestrukturen je nach Applikationsbedarf möglich. Im Speziellen können auch Hyperlinks und andere Relationen zwischen Informationsentitäten für die weitere Verarbeitung gesetzt werden. Durch die flexible Kombination der Einzelkomponenten sind unterschiedliche Szenarien und Anwendungen schnell und einfach realisierbar. Die Strukturnormalisierung ist ein optionaler Verarbeitungsschritt, dessen Durchführung auch im Nachhinein ohne Einschränkungen möglich ist.

In früheren Versionen des KnowMiner-Frameworks war im *ImportController* auch die Inhaltsnormalisierung angesiedelt. Des Weiteren ermöglichte der *ImportController* auch die parallele Verarbeitung und Analyse von Informationsentitäten. Es hat sich jedoch in den verschiedenen Anwendungsszenarien (siehe Abschnitt 6) gezeigt, dass einerseits keine Notwendigkeit für den Einsatz eines parallelen *ImportController's* bestand und andererseits dieser in der Anwendung und Fehlersuche viel Zeit in Anspruch nahm. Aus Gründen der Kompatibilität existiert zwar noch die Möglichkeit zum parallelen Import im KnowMiner, er wird aber nicht mehr zur Applikationsentwicklung angewendet.

8.1.2 Generische Interne Repräsentation (GIR)

Die Generische Interne Repräsentation verwaltet alle im KnowMiner-Framework zu bearbeitenden Objekte und wird über den ImportController befüllt. Grundsätzlich teilt sich die GIR in zwei unterschiedliche Teilbereiche, den Strukturbereich und den Eigenschaftsbereich, auf.

Dem *Strukturbereich* obliegt dabei die Verwaltung des Informationsraums, im Speziellen der einzelnen Informationsentitäten und deren Beziehungen zueinander. Um eine möglichst flexible Abbildung von „echten" Informationsräumen in dieser Datenstruktur zu erreichen, erfolgt die Datenhaltung im Strukturbereich in Form eines typisierten Graphen. Dabei können sowohl einzelnen Informationsentitäten als auch Relationen zwischen Entitäten unterschiedliche Typen zugeordnet werden.

Funktionell unterstützt der Strukturbereich den konkurrenten Zugriff auf Teilgraphen sowie die flexible Selektion von verschiedenen Subgraphen unter Verwendung einer entsprechenden Anfragesprache.

Im *Eigenschaftsbereich* erfolgt die Verwaltung der Eigenschaften, im Speziellen von Merkmale, Metadaten und Annotationen einzelner Informationsentitäten und Relationen. Im Gegensatz zu anderen Frameworks ermöglicht das KnowMiner-Framework auch das Verwalten von Eigenschaften für Relationen, wodurch einer Informationsentität in Abhängigkeit ihrer Beziehung zu anderen Informationsentitäten unterschiedliche

Eigenschaften zugewiesen werden können. Beispielsweise kann hierüber einem Objekt vor dem Hintergrund einer Suche eine Relevanz von 1.0, vor dem Hintergrund einer anderen Suche eine Relevanz von 0.5 zugeordnet werden. Der Eigenschaftsbereich verwaltet auch die unterschiedlichen Werte- und Schlüsselbereiche von Merkmalen, sowie die Typisierung von Merkmalen, Annotationen und Metadaten.

Kalkulatoren kapseln Funktionalitäten rund um Berechnungsmöglichkeiten zwischen Informationsentitäten und stellen die funktionelle Kapselung der GIR dar, wobei von den Eigenschaften einer Entität Merkmale die Berechnungsgrundlage darstellen. In Abhängigkeit vom verwendeten Kalkulator können so z.B. einfache Ähnlichkeitsberechnungen zwischen Objekten auf Basis des inneren Produkts der korrespondierenden Termvektoren oder aber auch komplexe Ähnlichkeitsberechnungen unter Berücksichtigung von zusätzlichen Metadaten oder zusätzlichen Modalitäten (Bild, Text und Audio) durchgeführt werden.

Diese Dreiteilung von Informationsräumen eignet sich hervorragend zur Kapselung unterschiedlichen Operationen, wie z.B. dem Manipulieren der Struktur der Informationsentitäten oder aber der Verarbeitung der Merkmale. Jeder dieser Teilbereiche stellt auch unterschiedliche Anforderungen an die jeweilige Implementierung. Der Strukturbereich zielt auf eine semantisch möglichst eindeutige Strukturierung zur Aufbereitung der Informationsentitäten ab. Hierfür wird im KnowMiner-Framework auf Techniken der Wissensrepräsentation wie RDF bzw. RDFS zurückgegriffen, wodurch die semantische Qualität des Informationsraumes auf Basis eines wohl etablierten Standards erhöht wird. Der Merkmalsbereich wiederum benötigt möglichst speicher- und laufzeiteffiziente Datenstrukturen und Implementierungen, um die Verarbeitung großer Datenmengen zu ermöglichen. Zum Beispiel wäre die Repräsentation eines hochdimensionalen Vector-Space-Models, wie es für viele Retrievalanwendungen benötigt wird, in Form eines Graphen undenkbar. Durch Abstraktion der unterschiedlichen Repräsentationsformen von Merkmalen und Merkmalsräumen kann hierzu im KnowMiner-Framework auf effiziente, HashMap basierte Vektorrepräsentationen zurückgegriffen werden, wodurch Operationen im Vektorraum schnell berechenbar sind. Detailbeschreibungen der GIR sind in Abschnitt 8.3 zu finden.

8.1.3 Worker

Neben der Datenhaltung und -repräsentation in Form der GIR definiert das Worker Konzept eine zentrale Komponente zur Implementierung von unterschiedlichen Algorithmen. Worker definieren hier ein Grundgerüst und liefern abstrakte Basisimplementierungen zur einfachen Umsetzung von KD-Algorithmen wie z.B. Clustering oder Klassifikation. Hauptaugenmerk dabei ist die Berücksichtigung einer möglichst effizienten, parallelen Abarbeitung von Algorithmen unter Ausschluss von Dead Locks und unter Berücksichtigung von konkurrenten Zugriffen. Dazu baut die Worker-Architektur auf einem Finit-State Maschinen-Konzept auf und berücksichtigt auch Möglichkeiten zur Lastverteilung sowie zum Stoppen und Neustarten von Algorithmen.

Zusätzlich zur Abstraktion der Verarbeitungsart werden auch unterschiedliche KD und Retrieval Funktionalitäten innerhalb der Worker Architektur definiert und abstrahiert. Aktuell existieren Schnittstellen für *Information Retrieval, Clustering, Klassifikation* und die *Ordination* von Informationsentitäten sowie unterschiedliche Implementierungen verschiedener Algorithmen wie K-Means Clustering, K-NN Klassifikation, Force Directed Placement etc.

Neben der Abstraktion und Implementierung unterschiedlicher KD-Funktionalitäten sind auch Worker zur Transformation der Merkmale einer Informationsentität vorhanden, welche z.B. Gewichtungen von Vektoren oder die Selektion von Merkmalen auf Basis statistischer Maße durchführen. Zentral hierbei ist auch der

ContentProcessor, der auf Basis einer Pipeline Architektur die Annotation von Dokumenten durchführt und somit die Vorbereitung für die Merkmalsgenerierung, -gewichtung und –transformation legt.

Um den Implementierungsaufwand für verschiedene Algorithmen gering zu halten, greift das KnowMiner-Framework auch auf als Open Source verfügbare Komponenten wie z.B. GATE oder das lineare Algebra Packet MTJ zurück. Durch die Offenheit des Frameworks kann eine Integration meist einfach durchgeführt und auf Spezialfunktionalität von verschiedenen Techniken zurückgegriffen werden. Auch die oben genannte Abstrahierung der Schnittstellen ermöglicht ein einfaches Austauschen der dahinter liegenden Funktionalität, wodurch z.B. die dahinter liegende Suchmaschine ohne Änderung der eigentliche Applikationslogik ersetzbar ist. Die dadurch erreichte Flexibilität ist essentiell für die rasche Adaption in unterschiedlichen Anwendungsszenarien. So kann z.B. die KnowMiner eigene Suchmaschine verwendet oder aber auch auf die schon im WMS integrierte Suchmaschine aufgesetzt werden, ohne dabei die restliche Applikation großartigen Änderungen unterwerfen zu müssen.

Im Allgemeinen können Worker als die kleinsten Bausteine für die Erstellung von Applikationen bzw. die Implementierungen der KnowMiner Dienste (siehe Abschnitt 8.5) verstanden werden und sind zentral für die Erstellung KnowMiner basierter Applikationen.

Die einzelnen Worker sowie die dahinter liegenden Implementierungsdetails stellt Abschnitt 8.4 dar.

8.1.4 Persistence Layer

Der Persistence Layer ist eng an die GIR gekoppelt und sorgt für das Ablegen der GIR in ein externes Datenhaltungssystem, wobei zwischen der Persistierung und dem Caching unterschieden werden muss. Beim Caching werden GIR Teile, welche für die momentane Verarbeitung nicht benötigt werden, auf ein Sekundärspeichermedium ausgelagert, wodurch die Verarbeitung großer Datenmengen ermöglicht wird, ohne dabei Speichermanagementfunktionalität explizit in den Algorithmen/Workern zu implementieren. Die Persistierung wiederum sorgt für das von Workern explizit veranlasste, konsistente Ablegen der GIR Struktur auf ein Sekundärspeichermedium, sodass bei einem Neustart der Applikation auf schon durchgeführte Berechnungen zurückgegriffen werden kann.

Persistierung und Caching betrachten die einzelnen Bereiche der GIR getrennt. Für die Persistierung des Strukturbereiches wird aufgrund dessen Graphenstruktur auf RDF-Tripelstores wie Jena [Carroll et. al. 2004] und Sesame [Broekstra et. al. 2002] zurückgegriffen. Die Verwaltung des Merkmalsbereichs erfolgt über Datenbankanbindungen bzw. die Java internen Möglichkeiten zur Serialisierung von Java Objekten.

8.1.5 Output Layer

Dieser Teil des KnowMiner-Frameworks stellt Schnittstellen und Dienste für die Nutzung der Einzelkomponenten zur Verfügung. Der Output Layer definiert dabei verschiedene Services, welche über RMI bzw. als Web-Service ansprechbar sind. Die Services teilen sich dabei in Basis-Dienste und Top-Level Dienste auf. Basis-Dienste kapseln Knowledge Discovery Tasks und kombinieren dabei die Funktionalität mehrerer Worker mit dem Ziel, eine einfachere Ansteuerung der einzelnen Techniken zu ermöglichen, und können als Bausteine für die Erstellung eines Top-Level Dienstes angesehen werden. Für einen Überblick hierzu sei auch auf [Klieber et. al. 2006] verwiesen, eine Publikation aus der ein Teil der vorliegende nBeschreibung übernommen wurde.

Die Umsetzung im KnowMiner-Framework erfolgte in der Spezifikation der einzelnen KD-Basis-Dienste (Basis-Services) sowie eines Knowledge Discovery Service (KDService, Applikations-Service), dass die Kernfunktionalität aller Basis-Services zusammenfasst. Für die Konfiguration der Services und Algorithmen wird das Spring-Framework[41] verwendet, das eine einfache Konfiguration von Parametern und Instanzen sowie die Inspektion des Zustands eines laufenden Systems erlaubt. Neben dem KDService sind bereits weitere Applikations-Services in Arbeit, die unterschiedliche Anforderungen anderer Anwendungsgebiete berücksichtigen, wie zum Beispiel Ontology Learning.

Für die Umsetzung von SOA in KDD Prozessen können die Basis-Services in Zugriffs-, Verwaltungs-, und Bearbeitungs-Services für den Informationsraum und die Informationsentitäten unterteilt werden. Die Bearbeitungs-Services kapseln dabei die unterschiedlichen Funktionalitäten und Manipulationen des Informationsraumes, wie in Abschnitt 5.3 und 7.6 dargestellt, und bieten unter anderem die Möglichkeit den aktuellen Prozess abzubrechen, anzuhalten oder fortzusetzen. Des Weiteren können über Bearbeitungs-Services Statusinformationen wie verbleibende Verarbeitungszeit, Verarbeitungsgeschwindigkeit etc. abgefragt werden.

Die Schnittstellen der Bearbeitungs-Services (e.g. Clustering Service) sind so gestaltet, dass eine beliebige Kombination möglich ist. Die Input und Outputkonfiguration erfolgt über so genannte *Kontexte* und *ElementContainer*. *Kontexte* legen den Informationsraum, Merkmalsraum sowie die zu verwendenden Metriken fest und definieren somit die Sichtweise auf die einzelnen Informationsentitäten. Die Spezifikation lehnt sich dabei an die Eigenschaften der in Abschnitt 7.6 beschriebenen KD-Komponentenklassen an. *ElementContainer* beinhalten die Informationsentitäten, mit denen gearbeitet wird. In vielen Anwendungsfällen arbeiten die Bearbeitungs-Services mit wenigen Kontexten und übernehmen direkt die verarbeiteten Entitäten der vorhergehenden Services, wodurch sich die Orchestrierung in solchen Fällen sehr einfach und intuitiv gestaltet. Für die Zukunft ist der Einsatz einer Pipeline-Architektur auf Service Ebene geplant, welche die Orchestrierung in komplexeren Anwendungsfällen regelt und Konflikte über die definierten Eigenschaften zum Zeitpunkt der Konfiguration feststellt (e.g. die Ausgabe einer Komponente kann nicht als Eingabe in eine andere Komponente verwendet werden).

Je nach Orchestrierung der Bearbeitungs-Services können schnell unterschiedliche KD-Aufgabenstellungen realisiert und abgearbeitet werden. Die Orchestrierung wird am Beispiel eines erweiterten Information Retrieval Anwendungsfalls gezeigt: Die Indizierung erfolgt durch das Importieren der Daten mit dem Import-Service, gefolgt von optionaler Extraktion weiterer Metadaten und Annotationen und der abschließenden Indizierung durch das Index Service. Die Suche und Ergebnisaufbereitung erfolgt durch das Ausführen einer Suche mit dem Such-Service, dem anschließenden Finden von thematischen Ähnlichkeiten mit dem Cluster-Service, der Generierung einer 2D Abbildung durch das Ordinalitätsservice und der finalen Darstellung der Abbildung in einer Wolkenvisualisierung durch das Visualisierungs-Service.

Assembly Services werden verwendet, um verschiedene Basis-Services zu kombinieren, damit die für einen bestimmten Applikations-Kontext benötigte Funktionalität einfach bereitgestellt werden kann. Ein solches Assembly Service ist zum Beispiel ein Metadaten Service, welches auf Benutzerebene eine einfache Handhabung der Annotationen, Merkmale und Metadaten erlaubt. Der Benutzer gibt als Input die gewünschten Arten von Metadaten wie Personen, Key-Wörter oder Dublin Core an, und das Metadaten

[41] http://www.springframework.org/

Service kümmert sich um die richtige Ansteuerung der entsprechenden Basis-Services, um diese Metadaten zu erzeugen und auszulesen.

8.2 GML Generic Markup Language

Nach dem Überblick über die Basisarchitektur behandelt dieser Abschnitt eine detaillierte Beschreibung von GML, der Generic Markup Language, welche im *ImportController* erzeugt wird und in weiterer Folge Informationsentitäten in einer formatneutralen Art und Weise beschreibt.

Der ImportController beinhaltet, wie zuvor beschrieben, den *Loader*, den *Format-Normaliser* und den *StructureNormalizer*. *Loader* und *FormatNormalizer* bilden den so genannten Connector, dessen Aufgabe es ist aus den angebundenen Datenquellen Informationsentitäten sowie datenquellenspezifische Eigenschaften auszulesen, zu normalisieren und in die Generic Markup Language (GML) zu überführen. GML ist ein XML Schema zur allgemeinen Beschreibung von Informationsentitäten und ist das zentrale Austauschformat für Informationsentitäten im KnowMiner-Framework.

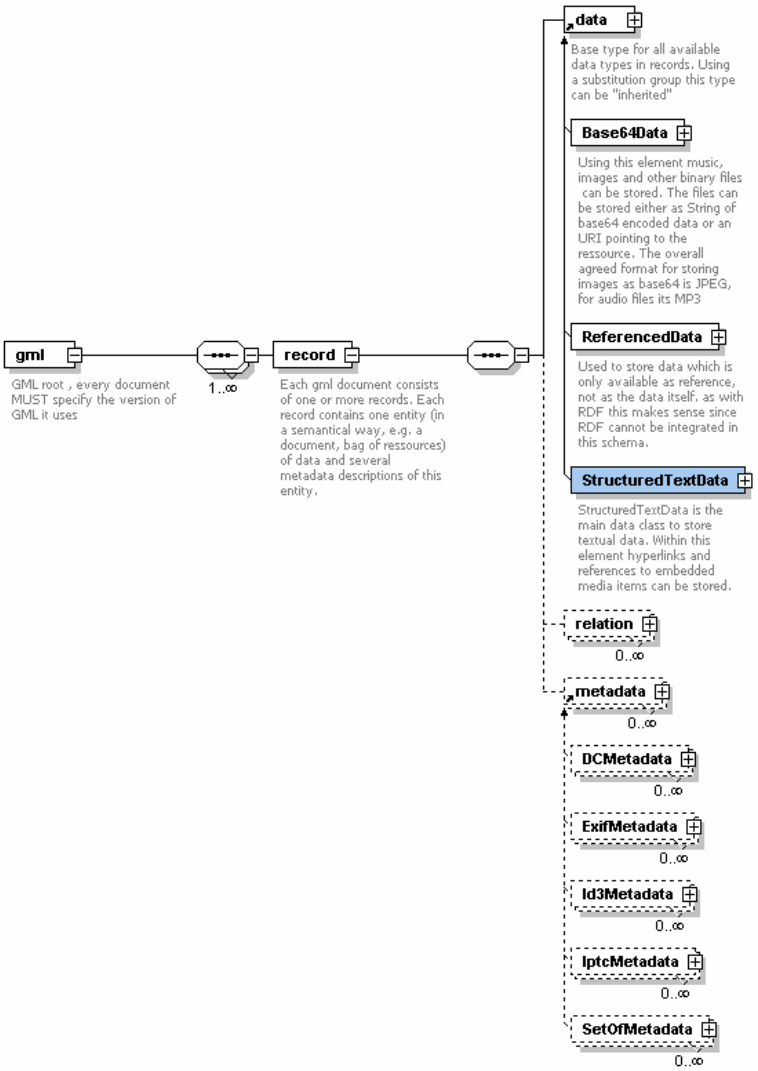


Abbildung 8-2: XML Schema Struktur von GML

Abbildung 8-2 zeigt die zentralen Elemente des GML XML Schemas, wobei ein GML Dokument im Allgemeinen eine Informationsentität beschreibt. Jedes GML Dokument besteht aus einer Menge an Records, welche die unterschiedlichen Daten einer Informationsentität und deren Metadaten kapseln. Im XML Schema definieren *data* und *metadata* einen abstrakten Datentyp der unterschiedliche Implementierungen für die einzelnen Modalitäten eines Dokumentes hat. Neben Daten und Metadaten beinhaltet ein GML Record noch ein *relation* Element, welches eine beliebige Anzahl von Verweisen mit Quelle, Ziel und Typ beinhalten kann und somit Objekte bzw. Records miteinander verbindet.

Die abstrakten Datentypen *data* und *metadata* definieren die Basiselemente eines Records von denen sich die speziellen Datentypen ableiten. Beide Datentypen sind überblicksmäßig in Abbildung 8-3 dargestellt.

Der Datentyp *data* kapselt dabei alle Daten eines Records und beinhaltet dabei mindestens eine URI zur eindeutigen Identifikation der Daten eines Records, einen Mime Type zur Spezifikation des Typs der Daten. Optional können noch Metadaten für die Daten des Records und ein Hash Code zur Überprüfung der Korrektheit der Daten angehängt werden. Vom abstrakten Datentyp leiten sich der Base64Data Type, der ReferencedData Type und der StructuredTextData Type ab:

- Base64Data: Beinhaltet binär kodierte Daten wie z.B. Bilder, Musik oder Videostreams, kodiert als Base 64 String.
- ReferencedData: Beinhaltet Daten welche nur als Referenz auf andere Ressourcen vorliegen. RDF wäre ein solches Beispiel, da sich RDF Daten nicht in dieses Schema integrieren lassen und daher per Referenz übergeben werden müssen.
- StructuredTextData: Beinhaltet Textdaten, welche hierarchisch aufschlüsselbar sind. Dadurch ist die Berücksichtigung der Struktur von Dokumenten im GML möglich. Neben rein textuellen Daten können auch eingebettete Medien wie z.B. Bilder oder Musik durch Referenzen auf anderen GML Records mit berücksichtigt werden, wodurch multimodale Dokumentstrukturen abbildbar sind.

Der Datentyp *metadata* kapselt unterschiedliche Metadatenformate wie Dublin Core, EXIF und IPTC. Für unspezifizierte Metadaten ermöglicht der *SetOfMetadata* Typ die Berücksichtigung einer Menge von Key/Value Paaren, über die verschiedene unstrukturierte Metadaten berücksichtigbar sind. Im abstrakten Datentyp können auf alle Fälle die Version, das Erzeugungsdatum, das Datum der letzten Änderung und das Tool mit dem das Metadatum erzeugt wurde, für jeden Typ erfasst und abgelegt werden.

GML hat sich vor allem als formatneutrales Austauschformat im KnowMiner-Framework bewährt. Zum Beispiel wird GML zur Persistierung verwendet, da aus GML wiederum alle Merkmale und Annotationen erzeugbar sind und der Ballast der Formatnormalisierung weg fällt. Dadurch reduziert sich auch die Vorverarbeitungszeit, vor allem bei komplexen Formaten wie PDF. Während im Moment keine einfache Möglichkeit besteht, Annotationen und Merkmale über GML mit abzulegen, ist die Erweiterung von GML in diese Richtung geplant.

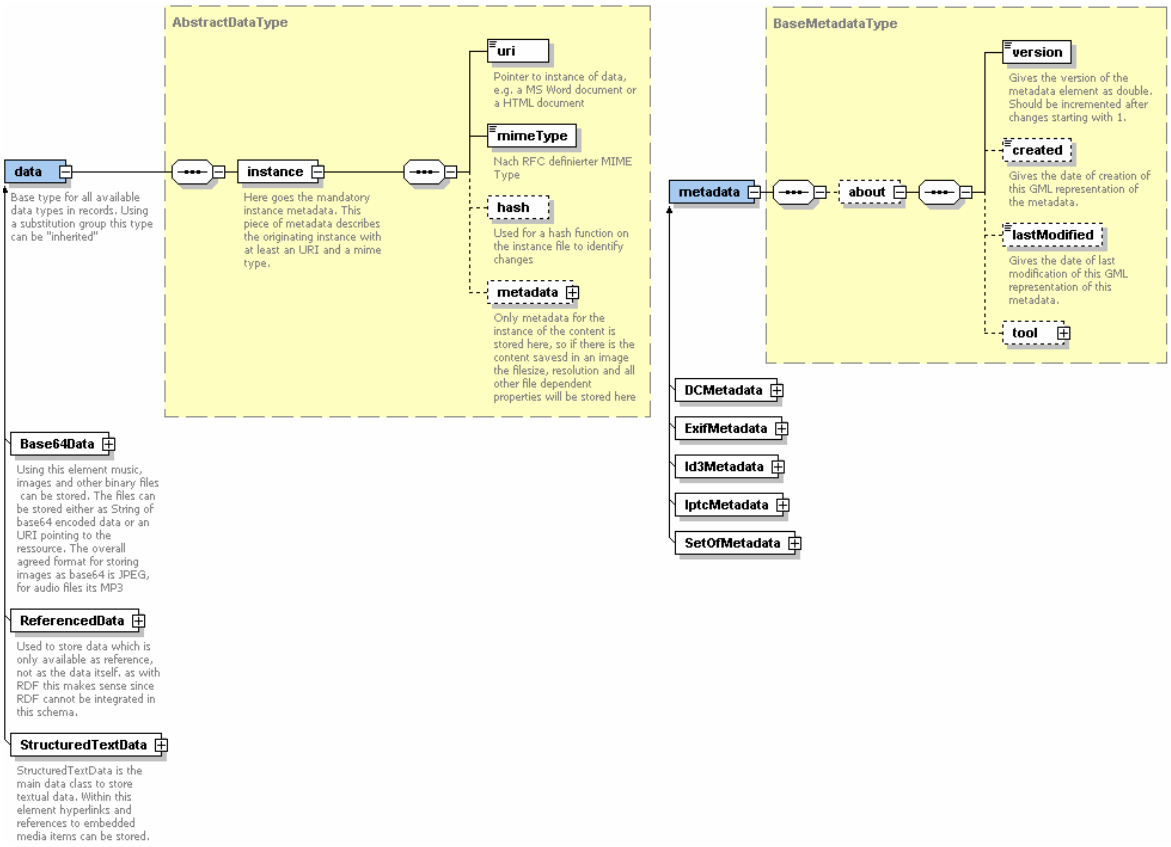


Abbildung 8-3: Die abstrakten Datentypen data und metadata im Detail

8.3 Generische Interne Repräsentation

Wie bereits oben erwähnt, stellt die Generische Interne Repräsentation, kurz GIR, die zentrale Datenhaltungskomponente des KnowMiner-Frameworks dar. In dieser Komponente werden sowohl der zu bearbeitende Informationsraum, dessen Merkmale und Topologie, als auch die Eigenschaften der einzelnen Informationsentitäten abstrahiert und für die weitere Verarbeitung zu Verfügung gestellt.

Zur Verwaltung von Informationsräumen wird dabei auf RDF [Lassila & Swick 1998] als strukturierte Beschreibungssprache zurück gegriffen, welche die Abbildung beliebiger Topologien ermöglicht. Die Abbildung von Informationsentitäten eines Informationsraumes und deren Relationen untereinander erfolgt mittels typisierter und gerichteter Graphen, d.h. sowohl Informationsentitäten als auch deren Relationen zueinander sind typisiert. Dies ermöglicht es, unterschiedliche Arten von Entitäten wie z.B. Dokumente, Collections, Suchergebnisse etc. sowie deren Beziehungen zueinander wie z.B. is-Part-of, is-A, is-related-to etc. zu verwalten.

Wie in Kapitel 7 beschrieben, gruppieren sich die Eigenschaften von Entitäten in Annotationen, Merkmale und Metadaten, welche Informationsentität oder Entitätsrelation zugewiesen sind. Neben dem Strukturaspekt beschreibt der vorliegende Abschnitt die Integration dieser zweiten Kerneigenschaft der GIR. Aufgrund der Anforderungen hinsichtlich. Speicher- und Laufzeitkomplexität erfolgt mit Ausnahme der Metadaten keine Abbildung der Eigenschaften von Entitäten und Relationen in RDF. Während die Abstraktion von Annotationen und Metadaten einfach gehalten ist, kapselt die Implementierung der Merkmale unterschiedlichen Repräsentationsformen (wie z.B. Vektoren und Graphen) und abstrahiert Operationen auf Merkmalsmengen wie Ähnlichkeitsberechnung, Addition, Division.

8.3.1 Repräsentation von Strukturen

Um die oben genannten Strukturen flexibel verwalten zu können, greift das KnowMiner-Framework auf RDF zurück, welches neben der effizienten Verwaltung von Graphen auch Möglichkeiten zur Typisierung und Inferenz bietet.

Derzeit existieren verschiedene Open Source Backends wie Jena [Carroll et. al. 2004] und Sesame [Broekstra et. al. 2002], die den Umgang mit großen RDF Graphen ermöglichen. Im KnowMiner-Framework wird mit Jena als RDF Backend gearbeitet, wobei die Verwaltungsschicht für RDF abstrakt gehalten wurde, um einen einfachen Tausch des RDF Backends zu ermöglichen und die Anwendung im Kontext KnowMiner zu erleichtern.

Wie in Abschnitt 8.1 eingeführt, setzt das KnowMiner-Framework auf die Verwaltung von Informationsräumen auf, in denen Informationsentitäten und deren Relationen zueinander verwaltete werden. Ähnlich stellt sich auch die Klassenstruktur zur Verwaltung der Informationsräume dar, welche in Abbildung 8-4 dargestellt ist. Vor Betrachtung der Einzelkomponenten sei darauf hingewiesen, das alle RDF Operationen hinsichtlich Manipulation der RDF Graphstruktur durch eine abstrakte API gekapselt sind (repräsentiert durch *Model, QueryResult, ModelFactory* in Abbildung 8-4), wodurch ein einfacher Austausch des RDF Backends möglich wird

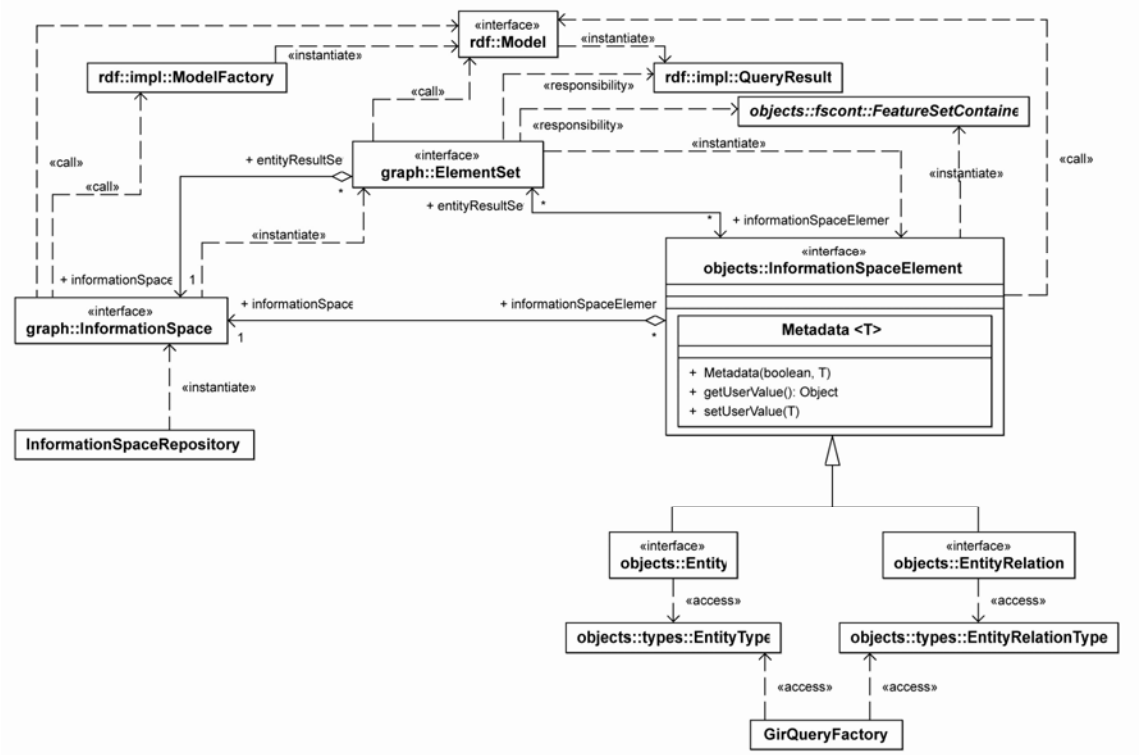


Abbildung 8-4: Hauptklassen für die Umsetzung der Strukturverwaltung auf Basis einer abstrahierten RDF Schnittstelle

Die zentralen Komponenten sind der Informationsraum, dargestellt durch die Klasse *InformationSpace* (verwaltet über das Factory Pattern *InformationSpaceRepository*) und die Elemente des Informationsraumes, dargestellt durch die Klasse *InformationSpaceElement*, welche immer einem Informationsraum zugeordnet sind. Wichtig dabei ist, dass ein Element des Informationsraumes entweder eine Informationsentität (Klasse *Entity*) oder eine Relation darstellt (Klasse *EntityRelation*). Grund dafür ist, dass neben der Informationsentität auch der Relation zwischen zwei Entitäten Metadaten und Merkmale zugewiesen werden können. Aus Anwendungssicht ermöglicht dies, Attribute zu Beziehungen zwischen Entitäten zuzuordnen und somit Entitäten im Kontext anderer Entitäten zu betrachten. Beispielsweise kann ein Dokument, dargestellt als Informationsentität, sowohl eine Zugehörigkeit zu einem automatisch errechneten Cluster, als auch zu einer Suche haben. Sowohl Cluster und Suche, beide abgebildet durch eine Informationsentität, stellen unterschiedliche Kontexte für ein Dokument dar und sind über eine Relation mit dem Dokument verbunden, wobei der Relation jeweils Attribute über den Grad der Zugehörigkeit zur Suche bzw. zum Cluster angehängt sind.

Aus dem Gesagten lassen sich somit auch die Eigenschaften für die Super-Klasse von *Entity* und *EntityRelation*, das *InformationSpaceElement*, ableiten. Zentrale Aufgabe für das *InformationSpaceElement* ist das Anbieten von Zugriffsmöglichkeiten auf Merkmale und Metadaten (siehe die innere Klasse *Metadata<T>*) sowie die Vergabe von eindeutigen Schlüsseln für ein Element. Die Klasse bildet die Grundlage für alle Elemente, die im Informationsraum verwaltet werden können. Darauf aufbauend abstrahiert die Klasse *Entity* nur die Knoten des Graphs und fügt lediglich die Verwaltung des Typs eines Knotens zur Funktionalität eines *InformationSpaceElement* hinzu. Ähnliches gilt auch für die *EntityRelation*, welche den Relationstyp, die Quellentität und die Zielentität der Relation zusätzlich verwaltet. Es sei angemerkt, dass es sich hierbei um typisierte gerichtete Kanten handelt. Falls die Erstellung von ungerichteten Kanten für eine Applikation

notwendig ist, kann dies flexibel über eine entsprechende Typisierung der Kante geregelt werden. Die Typisierung des Informationsraums wird flexibel über die Klassen *EntityType* und *EntityRelationType* abgehandelt.

Die Klasse *InformationSpace* bietet den zentralen Zugriffspunkt auf den Informationsraum und die Möglichkeit zur Abfrage von Entitäten und Relationen im Informationsraum. Über sie können strukturierte Suchen, in Form von SPARQL Queries, abgesetzt und in Form der *ElementSet* Klasse retourniert werden. Zusätzlich sind die Konfiguration des Informationsraumes inkl. der Merkmalsbereiche und die Möglichkeit der Abfrage von Statistiken wie Anzahl der Objekte, Anzahl der Relationen etc. über den *InformationSpace* gegeben. Fremdklassen können sich als Listener beim Informationsraum registrieren und über Änderungen im Informationsraum wie das Hinzufügen, Löschen und Ändern von Elementen informiert bzw. notifiziert werden, wodurch eine flexible Reaktion auf Aktionen im Zusammenhang mit den Informationsraum möglich ist.

8.3.1.1 Abfragen des Informationsraums

Die Erstellung von neuen Objekten über die Klasse *InformationSpace* ist aus Gründen der Unterstützung von konkurrierenden Zugriffen nicht möglich und wird daher über die Klasse *ElementSet* abgebildet. Grund ist, dass die *ElementSet* Klasse Locking und Concurrency Mechanismen bereitstellt und somit ein sicheres dezentrales Erstellen von Elementen ermöglicht, welches im Folgenden beschrieben wird.

Die *ElementSet* Klasse wird über das jeweilige *InformationSpace* Objekt unter Angabe einer entsprechenden SPARQL Query erzeugt. Die Erzeugung der *ElementSet* Klasse erfolgt dabei über den *InformationSpace*, wobei neben SPARQL Queries auch programmatische Konstrukte zur Erzeugung von nicht über SPARQL abbildbaren Abfragen möglich sind. Rekursive Abfragen stellen beispielsweise eine solche Klasse dar.

Die Abfragen liefern dabei als Ergebnis immer eine Liste von Informationsentitäten zurück, die der *ElementSet* verwaltet. Diesem obliegt auch das Sperren oder Freigeben von Entitätseigenschaften, damit Algorithmen gezielt auf einzelne Eigenschaften konkurrierend zugreifen können. Den einzelnen Algorithmen können somit genau die benötigten Eigenschaften bereitgestellt werden. Da die Bereitstellung auf der feingranularen Ebene der Eigenschaften eines Elementes des Informationsraumes beruht, ist somit die gleichzeitige Bearbeitung unterschiedlicher Sichtweisen bzw. Entitätskontexte möglich.

Abbildung 8-5 zeigt die Ausführung einer Abfrage und die Bereitstellung von Elementen über die *ElementSet* Klasse im Detail: Über die Anwendung erfolgt die Spezifikation eine Suchanfrage, welche entweder in SPARQL oder aber programmatisch beschrieben ist. Durch die Übergabe dieser Suchanfrage bzw. der Parameter bei programmatischen Anfragen an das entsprechende *InformationSpace* Objekt (Methode *select()*), erfolgt die Erzeugung des *ElementSet* Objektes welches über das dahinter liegende RDF Modell (Klasse *Model)* die Ausführung der Suche übernimmt. Für jedes im RDF Graphen gefundene Element des Informationsraumes (*InformationSpaceElement*) wird eine Instanz der Klasse *InformationSpaceElement* resp. des Typs *Entity* oder *EntityRelation* erzeugt. Wichtig hierbei ist, dass gleiche Elemente in unterschiedlichen *ElementSet* Instanzen auch unterschiedliche Java Objekte sind, die auf die gleiche Referenz im Modell verweisen. Die Synchronisation kann daher im Modell erfolgen, und alle unterschiedlichen *InformationSpaceElement* Instanzen können ohne Wissen von einander existieren. Erfolgt ein Zugriff auf Eigenschaften eines Elements, so erfolgt die Prüfung der Existenz des Elements sowie des Locking Status bei diesem Zugriff und nicht schon im Vorfeld.

Um auf dieser Basis konkurrierende Zugriffe zu ermöglichen, muss die Anwendung im Vorfeld Eigenschaften sperren, dann bearbeiten und zum Abschluss wieder frei geben. Das dahinter liegende Modell trägt dafür Sorge, dass jeweils nur eine Ergebnismenge schreibend auf die Eigenschaften zugreift, wobei natürlich die Anforderungen paralleler Lesezugriffe erhalten bleiben.

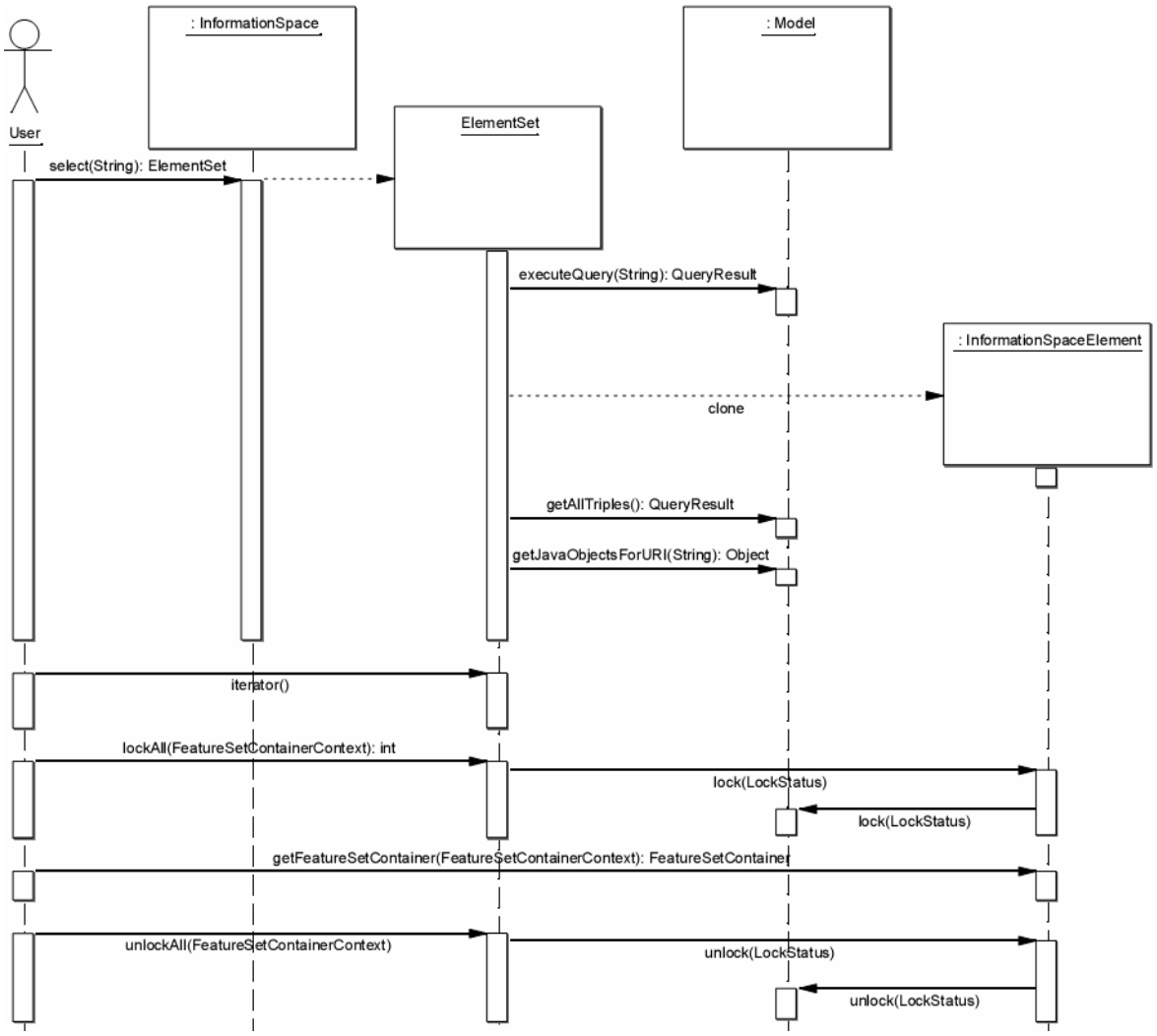


Abbildung 8-5: Sequenzdiagramm zur Erzeugung einer GIR Ergebnismenge und Iteration über die enthaltenen Objekte nach erzeugen eines Readlocks

Das Modell verfolgt also eine multiple read/single write Lock-Strategie, d.h. das Sperren von Schreibzugriffen ist auf Basis exklusiver Write Locks auf Originaldaten gelöst. Ein Ergebnis im *ElementSet* kann nur modifiziert werden, wenn keine andere Ergebnismenge auf die gleichen Eigenschaften lesend oder schreibend zugreift. Sind wiederum eine Entität bzw. deren Eigenschaften schreibgesperrt, kann keine andere Ergebnismenge lesend oder schreibend darauf zugreifen. Die Sperre ist also exklusiv. Dadurch besteht mitunter die Gefahr, dass auf einen Schreibzugriff lange gewartet werden muss und der schreibende Prozess verhungert, ermöglicht aber andererseits speichereffiziente lesende und schreibende Zugriffe. Die Möglichkeit, Sperren auf Eigenschaftsebenen durch zu führen, reduziert dabei die Wahrscheinlichkeit, dass gleichzeitig auf die gleichen Eigenschaften zugegriffen wird und verbessert somit das konkurrente Verhalten.

Zur Illustration ein paar Beispiele:

- Ordination: Auf Basis der hochdimensionalen Merkmale von Entitäten werden niedrigdimensionale Merkmale erzeugt. Die Sperre der Eigenschaften ist nun so ausgelegt, dass auf die hochdimensionalen Merkmale lesend und auf die niedrigdimensionalen Merkmale schreibend zugegriffen wird und damit die niedrigdimensionalen Merkmale exklusiv nur der Ordination zu Verfügung stehen. Die Benutzung der hochdimensionalen Merkmale in z.B. Clustering od. Klassifikationsalgorithmen ist nach wie vor möglich, zumindest für den Fall, dass diese nicht verändert werden.

- Gewichtung eines Vektorraums: Über Merkmale von Entitäten werden Statistiken erstellt, welche zur Gewichtung der Merkmale dienen. Die zu verändernden Merkmale werden für den Schreibzugriff gesperrt, wodurch kein anderer Algorithmus mehr damit arbeiten kann. Ein paralleler Clustering-Algorithmus müsste warten, bis die Gewichtung für die von ihm benötigten Entitäten abgeschlossen ist und somit die richtigen Daten bereit stehen. Aus diesem Beispiel ist auch ersichtlich, das Locking Mechanismen nicht nur zur gleichzeitigen Verarbeitung von Entitäten, sondern auch zur Wahrung der Konsistenz der Entitäten notwendig sind. Würde der Clustering-Algorithmus nicht warten, so würde die Abarbeitung auf einer inkonsistenten Repräsentation der Entitäten erfolgen, wodurch ein falsches Ergebnis erzeugt werden würde. Während programmatische Fehler in KD-Anwendung „einfach“ zu finden sind, ist das Auffinden solcher Inkonsistenzen meist ein relativ aufwändiges Unterfangen. Lockingmechanismen unterstützen hierbei bzw. helfen beim Verhindern von Inkonsistenzen.

Um zusätzlich ein schnelles Freigeben der beschriebenen Eigenschaften zwecks Weiterverarbeitung oder Ausgabe (z.B. dynamische Visualisierung der niedrigdimensionalen Repräsentation wie in der WebRat Anwendung) zu ermöglichen, wurde auf eine effiziente Implementierung der Locking-Mechanismen geachtet. Im Zusammenhang mit dem Worker Konzept (siehe Abschnitt 8.4) können hier dynamische Algorithmen realisiert werden, welche nach Abarbeitung einer Iteration die Schreibsperre aufheben und somit die kurzfristige Ausgabe der Zwischenergebnisse ermöglichen. In der nächsten Iteration erfolgt wiederum das Sperren der Entitäten und die weitere Verarbeitung.

Realisiert wird der Lockingmechanismus mittels dem *ElementSet*. Über keine andere Klasse bzw. Schnittstelle können Elemente eines Informationsraumes bearbeitet, erstellt oder gelöscht werden, womit der *ElementSet* den idealen Synchronisationspunkt darstellt. Abbildung 8-5 zeigt den Ablauf des Sperrens für lesenden Zugriff. Über eine Anfrage liefert das RDF Model (*Model)* eine Ergebnismenge zurück, welche im *ElementSet* gekapselt wird. Um nun über diese Ergebnismenge iterieren zu können, werden die Elemente der Ergebnismenge für lesende Zugriffe gesperrt. Die Anfrage der Sperre wird zum RDF Modell weitergeleitet das die Verwaltung der Sperren der einzelnen Elemente über hat. Gleichzeitig erfolgt auch das Aktualisieren des Sperrstatus bei jedem *InformationSpaceElement*, damit dieses den Zugriff entsprechend kontrollieren kann. Hierbei sind folgende implementierungstechnische Punkte zu berücksichtigen:

1. Alle *InformationSpaceElement*'s sind Klone der Originalelemente und halten nur die Verweise (in Form von URIs des RDF Modells) auf die abgefragten Eigenschaften der Entität.

2. Der Zugriff auf Merkmale (i.e. *FeatureSetContainer*), Metadaten und Annotationen erfolgt über das *InformationSpaceElement*, wobei der Zugriff wiederum zum Modell delegiert wird.

3. Die benötigten Schreib/Lesezugriffe können eigenschaftsselektiv durchgeführt werden, d.h. Teile der Ergebnismenge können für lesende Zugriffe gesperrt werden, während andere Teile für schreibende Zugriffe gesperrt sind.
4. Bereits lesegesperrte Ergebnisse können mittels „lock upgrade“ in schreibende Zugriffe umgewandelt werden.

Neben den Zugriff auf Elemente des Informationsraumes ermöglicht die *ElementSet* Klasse auch das Erstellen bzw. das Löschen von Elementen, wobei sowohl Erstellung als auch Löschen eine atomare Operation ist, wodurch es zu keinen Konflikten beim Multithreading kommen kann.

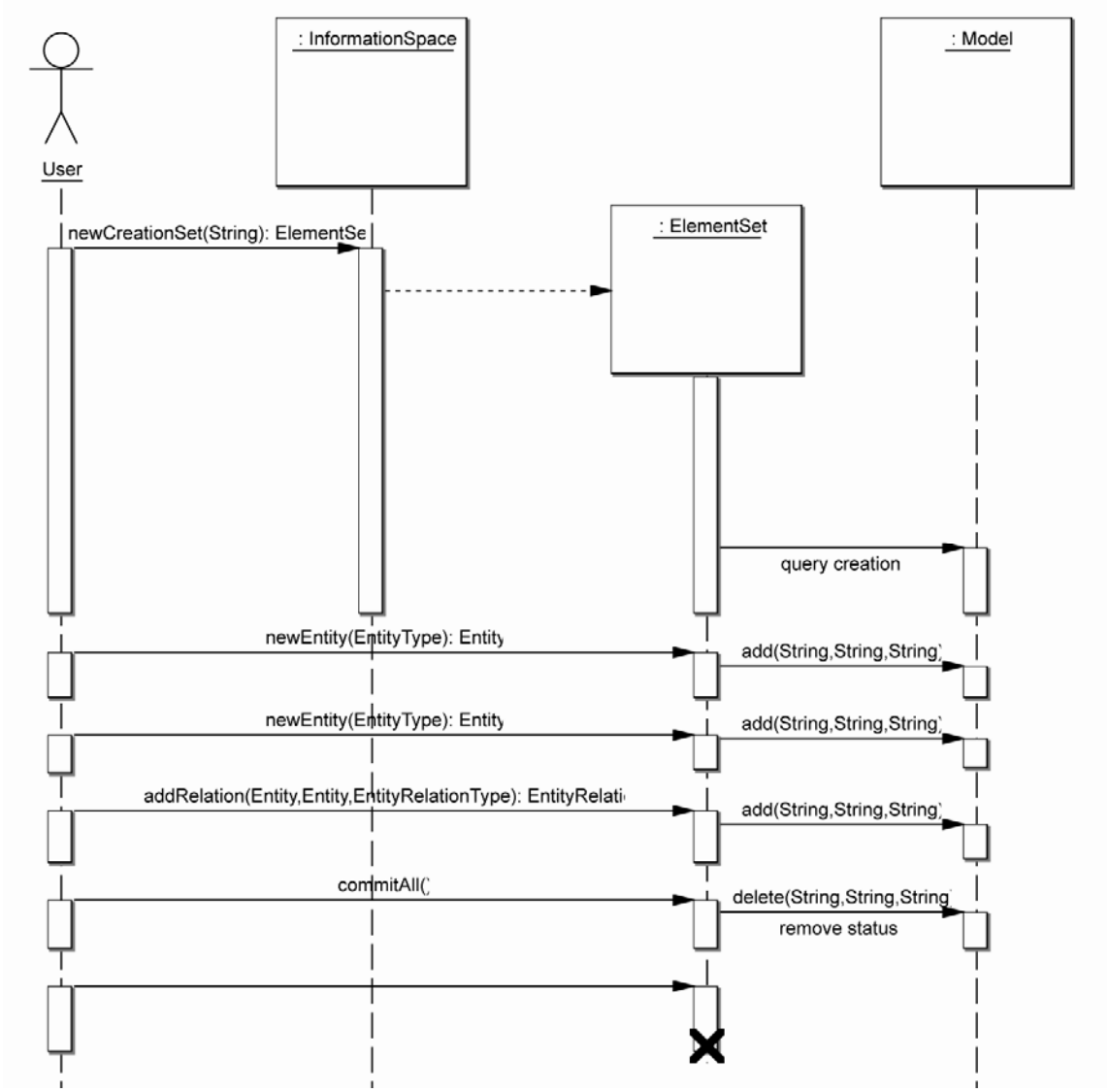


Abbildung 8-6: Beispielhafte Erzeugung von Entitäten und Relationen mittels ElementSet.

Für das Löschen eines Elementes ist zusätzlich eine Schreibsperre auf alle Eigenschaften des Elements nötig. Das Erzeugen von Entitäten hingegen braucht keine parallelen Zugriffe zu berücksichtigen. Nachdem eine Entität neu erzeugt wurde, erhält diese den Status *uncommited*, wodurch sie nicht über andere Anfragen zugreifbar gemacht werden kann. Die Funktion *commit* im ElementSet erlaubt das Entfernen des Status, um

das Element ebenfalls für andere verfügbar zu machen. Das Hinzufügen von Relationen zwischen Entitäten hat dabei den Status einer Entität zu berücksichtigen. Hintergrund ist, das nicht bestätigte Entitäten (also mit Status *uncommited*) nicht zum bestätigten Graphen hinzugefügt werden sollten, da mitunter ein Traversieren der Kanten einen Zugriff auf diese ermöglichen würde. Aus diesem Grund können Relationen nur zwischen bereits bestätigten Entitäten oder, für den Aufbau eines komplett neuen Teilgraphen, zwischen zwei unbestätigten Elementen erfolgen. Letzteres ermöglicht den Aufbau eines kompletten Teilgraphen und die Integration von diesem in den Informationsraum mittels einfachen Hinzufügens einer Relation inkl. Bestätigung aller unbestätigten Elemente. Abbildung 8-6 zeigt beispielhaft die Erzeugung von Entitäten und Relationen als UML Sequenz Diagramm. Das Löschen von Elementen beinhaltet eine ähnliche Sequenz wie das Iterieren über die Ergebnismenge (siehe Abbildung 8-5), lediglich dass die Elemente zum Schreiben gesperrt sein müssen und beim *ElementSet* die Funktion *remove* bzw. *removeAll* aufgerufen wird.

Als letzte Eigenschaft vom *ElementSet* sei noch die Möglichkeit zum Aktualisieren einer Abfrage erwähnt. Dabei werden jene Elemente der Ergebnismenge hinzugefügt, welche der Abfrage entsprechen und beim letzten Aufruf noch nicht im Informationsraum enthalten waren. Dadurch ist die Realisierung dynamischer, sich über die Zeit ändernder Ergebnismengen möglich. Zum Beispiel können während der Berechnung einer 2D Visualisierung neue Entitäten über den ImportController in den Informationsraum eingefügt werden, welche in der nächsten Iteration ebenfalls zu visualisieren sind (z.B. für iterative Visualisierungen von Suchergebnissen in der WebRat, siehe Abschnitt 6.1.1). Dazu stellt der *ElementSet* eine *Refresh* Methode bereit, welche effizient die neu hinzugekommenen Elemente der aktuelle Ergebnismenge dem *ElementSet* hinzufügt.

8.3.1.2 RDF Backend

Alle hinsichtlich der Verwaltung der Struktur eines Informationsraums vorgestellten Funktionalitäten sind über ein RDF Backend realisiert. Um möglichst generisch zu bleiben existiert eine abstrakte Schicht welche das darunter liegende Backend kapselt. Im Moment existieren Implementierungen für Jena und Sesame, wobei die Anbindung weiterer Frameworks relativ einfach von statten gehen kann. Dadurch ist das KnowMiner-Framework flexibel in der dahinter liegenden Technologie und kann schnell auf neue Backends zur Verwaltung von Graphen überführt werden. Im Folgenden wird lediglich die abstrakte Sichtweise auf das RDF Modell beschrieben. Für eine detaillierte implementierungstechnische Beschreibung sei auf [Zechner 2006] und auf die KnowMiner JavaDoc verwiesen.

Neben der Verwaltung der strukturellen Daten obliegt dem Modell ebenfalls die Verwaltung der Ressourcen, auf welche in RDF verwiesen wird. Solche Ressourcen sind z.B. die einzelnen Merkmalsmengen, Metadaten, Annotationen (alle als Java Objekte) sowie der formatnormalisierte Inhalt in Form von GML.

Zur Verwaltung der Elemente eines Informationsraumes müssen diese in RDF bzw. in Subjekt-Prädikat-Objekt Tripel überführt werden. Zur Beschreibung der RDF Struktur erfolgt im Folgenden die Betrachtung der Einzelelemente, im Speziellen des *InformationSpaceElement*, der *Entität* und der *Entitätsrelation.*

RDF Repräsentation eines InformationSpaceElement

Wie in Abschnitt 8.3 dargestellt, verwaltet die Klasse *InformationSpaceElement* Merkmale, Metadaten und Annotationen und ermöglicht die eindeutige Identifikation des Elementes im Informationsraum. Ein *InformationSpaceElement* besteht dabei aus folgenden Attributen:

- **E**: Ressource mit eindeutiger URI für eine Entität
  Eindeutig über alle *InformationSpaceElement*
  Kardinalität: 1 pro Entität

- **F**: Ressource für einen FeatureSetContainer
  Eindeutig über alle *InformationSpaceElement*
  Kardinalität: 1-n pro Entität, wobei n die maximale Anzahl der unterschiedlichen Merkmalskontexte darstellt

- **G**: Ressource für GML
  Eindeutig über alle *InformationSpaceElement*
  Kardinalität: 1 pro Entität

- **M**: Ressource für ein Metadatum
  Eindeutig über alle *InformationSpaceElement*
  Kardinalität: 1-n pro Entität, wobei n die maximale Anzahl unterschiedlicher Metadatenkontexte darstellt

- **A**: Ressource für eine Annotation
  Eindeutig über alle *InformationSpaceElement*
  Kardinalität: 1-n pro Entität, wobei n die maximale Anzahl unterschiedlicher Annotationskontexte darstellt

- **C**: Ressource für den Kontext von Merkmalen (*FeatureSetContainer*), Metadaten (*Metadata*) und Annotationen (*Annotation*)
  Eindeutig über alle Kontexte
  Kardinalität: 1 Kontext pro Metadatum, Annotation und Merkmalscontainer

RDF Repräsentation einer Entität

Entitäten bauen auf dem *InformationSpaceElement* auf und fügen jeweils noch den Typ und den Status einer Entität ein.

- E_t: Ressource mit dem Typ der Entität
  Eindeutig über alle Typen
  Kardinalität: 1 pro Entität

- S: Ressource für den Status der Entität
  Eindeutig pro Status, *Entitäten* referenzieren immer zur gleichen Ressource
  Kardinalität: 1-n, wobei n die maximale Anzahl unterschiedlicher Stati darstellt

RDF Repräsentation einer Relation

Relationen bauen ebenfalls auf den *InformationSpaceElement* auf und verbinden zwei Entitäten miteinander. Eine Relation definiert sich immer aus ihrer Quelle und ihrer Senke, d.h. es kann nur im Kontext der beiden Entitäten existieren. Zusätzlich zu Quelle und Senke ist auch der Typ der Relation definiert.

- R_t: Ressource mit dem Typ der Entität
  Eindeutig über alle Typen
  Kardinalität: 1 pro Entität

- E_{source}: Tripel mit Verweis auf das Quellelement der Relation
  Kardinalität: 1 pro Entität

- $E_{destination}$: Tripel mit Verweis auf das Zielelement der Relation
  Kardinalität: 1 pro Entität

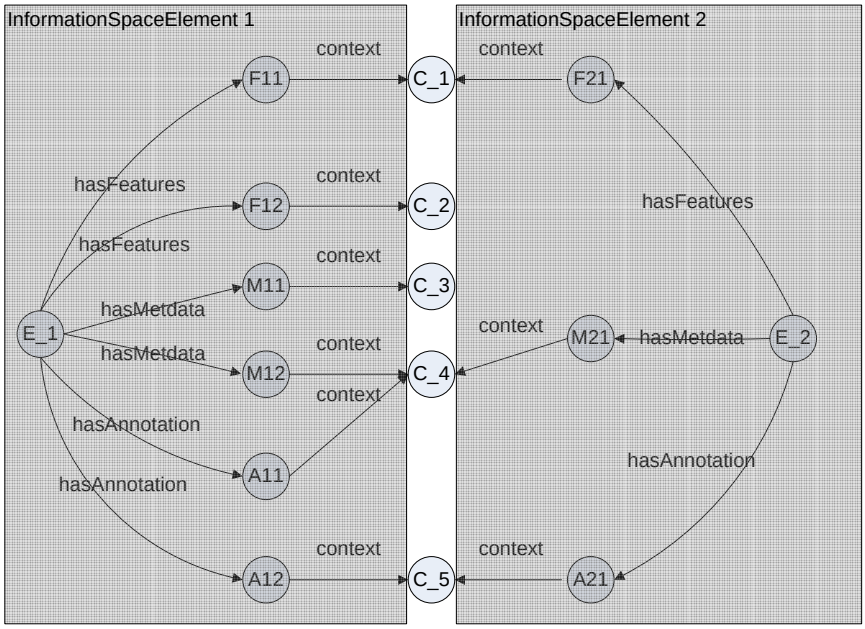


Abbildung 8-7: Darstellung von zwei InformationSpaceElement's als Graph. Merkmale, Annotationen und Metadaten unterschiedlicher Elemente verweisen dabei jeweils auf den gleichen Kontextknoten.

Auf Basis der eingeführten Abkürzungen zeigt Abbildung 8-7 zwei *InformationSpaceElement's*. Eine Entität verweist dabei auf eine Menge von Merkmalscontainern (*F_1..F_n*), Annotationen (*A_1..A_n*) und Metadaten (*M_1..M_n*). Die Kontextknoten werden von den Annotationen, Metadaten und Merkmalen unterschiedlicher Entitäten referenziert. Eine Abbildung des Kontextes als Literal wäre ebenfalls denkbar, jedoch würde dies die Größe des Graphs und auch die Anfragegeschwindigkeit erhöhen, da für jedes Element ein eigener Kontextknoten existieren würde. Die Elementknoten selbst (*E_1* und *E_2*), sowie alle Merkmalscontainer-, Metadaten- und Annotationsknoten werden durch eine eindeutige URI, welche in der Abbildung abgekürzt dargestellt sind, repräsentiert. Die Klassen bzw. Java Objekte, welche die eigentlichen Merkmale, Eigenschaften und Annotationen enthalten werden, wie oben erwähnt, in einem separaten Repository gehalten. Hintergrund dafür ist, dass die RDF Backends auf Zeichenketten und URIs restriktiert sind und keine Abbildung beliebiger binärer Objekte erlauben. Auch die Serialisierung und Deserialisierung von Java Objekten in und von Zeichenketten würde eine extreme Erhöhung der Laufzeiteffizienz bedeuten.

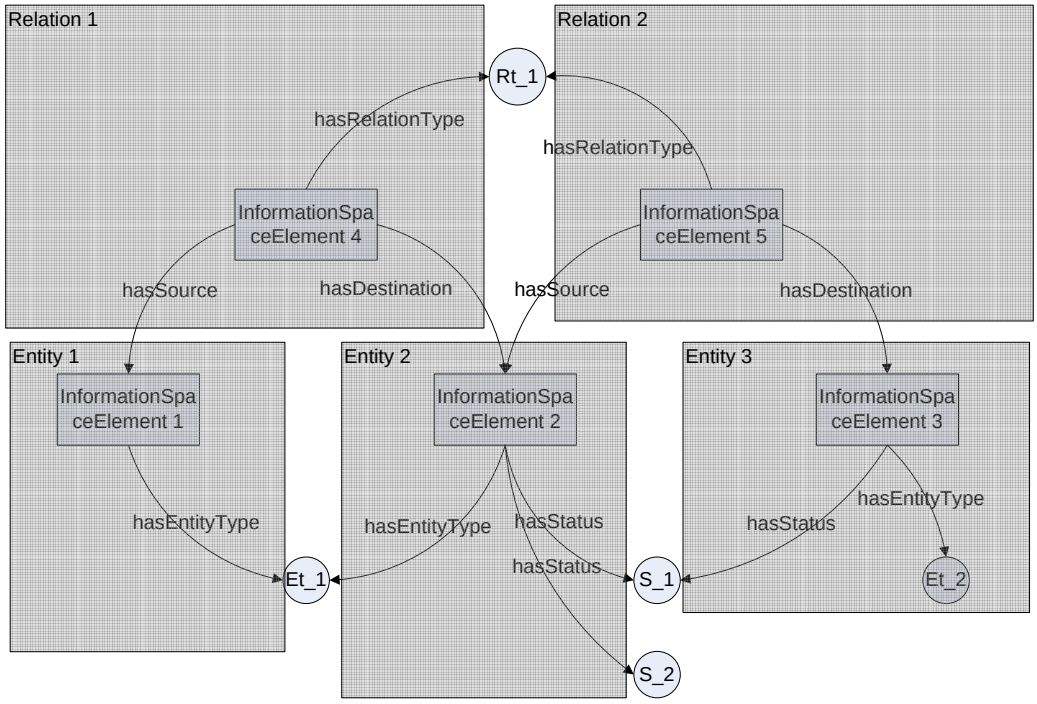


Abbildung 8-8: Graph-Abbildung von Entitäten und Relationen auf Basis von InformationSpaceElement's, welche wiederum als Graph abgebildet sind (siehe Abbildung oben)

Darauf aufbauend zeigt Abbildung 8-8 die Darstellung von Entitäten und Relationen, wobei die Subgraphen der Elemente durch ein Rechteck repräsentiert sind, um die Darstellung zu vereinfachen. Ähnlich wie beim Kontext referenzieren auch Entitätstyp, Status und Relationstyp bei gleicher Ausprägung auf den gleichen Knoten im Graphen, wiederum vor dem Hintergrund der Speicher- und Abfrageoptimierung.

8.3.2 Repräsentation von Eigenschaften von Elementen eines Informationsraumes

Nach der Behandlung der Strukturrepräsentation eines Informationsraumes im KnowMiner-Framework behandelt dieser Abschnitt die Repräsentation und Implementierung der Eigenschaften von Elementen eines Informationsraumes, also der Merkmale, Annotationen und Metadaten.

Wie in Abschnitt 7.4.5 eingeführt, behandeln Eigenschaften von Elementen unterschiedliche Aspekte und beinhalten auch den Großteil der Daten eines Elements. Zusätzlich ist die Verwaltung und Benutzung von Eigenschaften im Allgemeinen zeitkritisch, und es werden je nach Anwendung unterschiedliche Datenstrukturen für die Verwaltung von Merkmalen, Annotationen und Metadaten benötigt. Aus diesen Gründen wurde bei der Umsetzung des KnowMiner-Frameworks auf eine Abbildung von Merkmalen und Annotationen in RDF verzichtet. Bei Metadaten stehen beide Möglichkeiten zur Auswahl, also einerseits die Abbildung als proprietäres Java Objekte und andererseits mittels RDF.

Der weitere Abschnitt behandelt zuerst Implementierungsdetails zu den Merkmalen und stellt dabei das Merkmals-Typsystem sowie die Verwaltung von Merkmalsmengen und Merkmalsräumen (siehe Abschnitt 7.4.5) dar. Der Abschnitt schließt mit der Illustration der Implementierung von Metadaten und Annotationen,

womit auch die Repräsentation und Implementierung der Eigenschaften von Elementen hinreichend erläutert sein sollte.

8.3.2.1 Merkmal-Typsystem

Der folgende Abschnitt stellt das flexible KnowMiner Typsystem vor, mit welchem Möglichkeiten zur Abbildung verschiedener Merkmalstypen und Wertebereiche innerhalb des KnowMiner-Frameworks ermöglicht werden. Der Grad der Formalisierung ist dabei ein wichtiges Unterscheidungskriterium. Während Frameworks wie z.B. GATE (siehe Abschnitt 5.1.1) beliebige Annotierungstypen und Schemata über eine freie Namensvergabe erlauben, ist das im KnowMiner abgebildete Typsystem strikter in Form von Key-Value Paaren aufgebaut. Sowohl der Schlüssel (Key) als auch der Wert (Value) eines Merkmals sind über Java Klassen gekapselt, welche mittels Factory-Pattern instanziert werden. Idee hinter dem Typsystem ist die semantische Beschreibung von Merkmalen bzw. deren programmatische Umsetzung. Mögliche Typen sind zum Beispiel 2D oder 3D Koordinaten oder auch TFIDF Vektoren.

Durch diese striktere Typisierung sinkt auch die Fehlerwahrscheinlichkeit hinsichtlich der Verarbeitung und dem Vergleich von falschen Merkmalen miteinander. So wäre beispielsweise eine Ähnlichkeitsberechnung zwischen der 2D Repräsentation einer Entität und des TFIDF Vektors einer anderen Entität zwar möglich, jedoch semantisch bzw. für den Algorithmus sinnfrei. Das Typsystem ist sowohl zur Laufzeit als auch zur Compilezeit erweiterbar, wobei die Laufzeiterweiterung unterschiedlichen Restriktionen unterliegt.

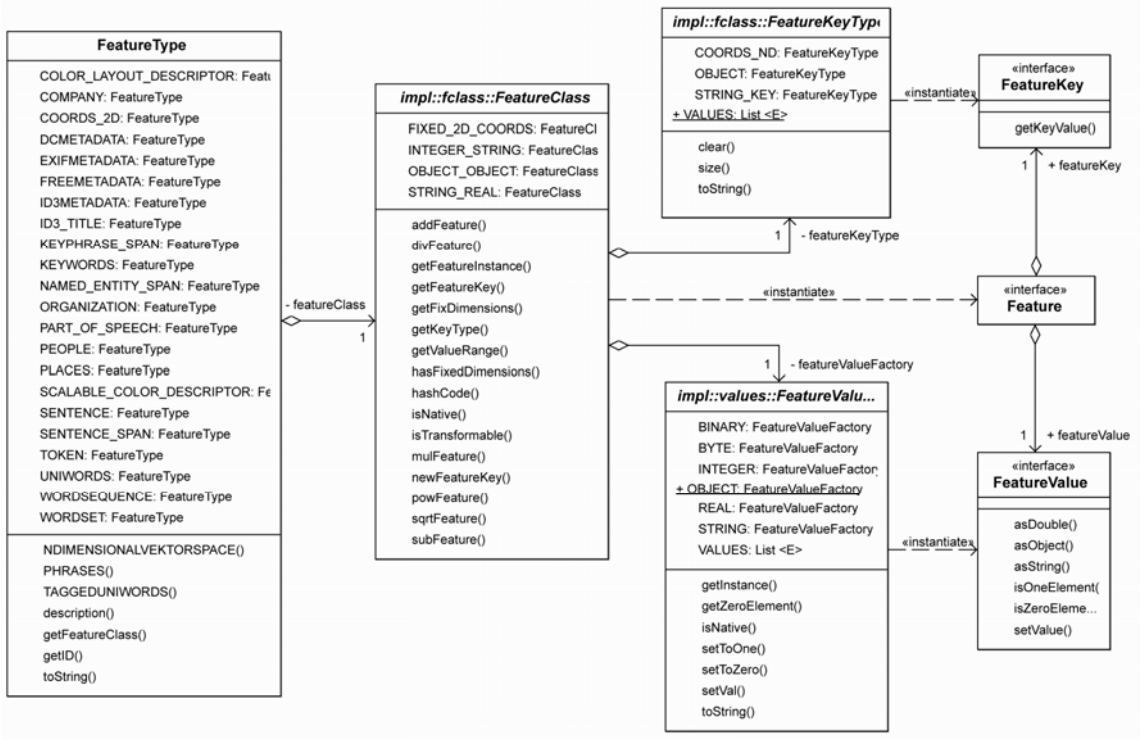


Abbildung 8-9: Klassendiagramm zum KnowMiner Merkmal-Typsystem. Rechts sind die Schnittstellen abgebildet, links die Merkmalstypen. Die Instanzierung von Merkmalen, deren Schlüsseln und deren Merkmalswerten erfolgt über Factory Patterns.

Abbildung 8-9 zeigt den Aufbau des KnowMiner Merkmal-Typsystems als UML Klassendiagramm. Das Interface *Feature* kapselt den Zugriff auf den Schlüssel (über die Schnittstelle *FeatureKey*) und den Wert eines

Merkmals (über die Schnittstelle *FeatureValue*). Sowohl *FeatureKey* als auch *FeatureValue* werden über die Factory Pattern Klassen *FeatureKeyType* resp. *FeatureValueRange* instanziert. Während im Falle der Merkmalswerte dieser Ansatz lediglich eine Kapselung der Instanzierung und dynamischen Lademöglichkeit darstellt, obliegt der *FeatureKeyType* Klasse ebenfalls die Aufgabe, die unterschiedlichen *FeatureKey* Instanzen in Form von Pools zu verwalten. Das Pooling von Schlüsselwerten dient der speichereffizienten Verwaltung von Merkmalen. In den meisten KD-Anwendungen bzw. KD-Modellen, wie z.B. dem Vector Space Model, ist die Anzahl der unterschiedlichen Schlüsselwerte begrenzt (z.B. Anzahl der unterschiedlichen Wörter im Vector Space Model), wodurch die Wiederverwendung von Schlüsselwerten für verschiedene Informationsobjekte sinnvoll ist. Da Java keine Speicherzeiger unterstützt, wird diese Implementierung mittels Factory Pattern gekapselt.

Aktuell werden reellwertige, boolesche, nominale und auf Zeichenketten basierende Merkmalswerte unterstützt, wobei nominale Merkmalswerte über Ganzzahlkonstrukte realisiert werden. Zusätzlich zu diesen einfachen Merkmalswerten besteht noch die Möglichkeit, komplexere Merkmalswerte über die Implementierung der Schnittstelle *CustomFeatureValue* in das Merkmalssystem einzubringen. Diese Schnittstelle kapselt neben der Verwaltung der Merkmalswerte auch Operationen wie Addition, Subtraktion etc. für ein komplexes Merkmalsobjekt. Solche Merkmalsobjekte können z.B. Metadaten wie Dublin Core oder EXIF kapseln und auf diesen Metadaten benutzerdefinierte Operationen erlauben, womit genügend Freiheitsgrade für die Implementierung spezieller Merkmalswerte gegeben sind. Abbildung 8-10 zeigt die Zusammenhänge und Klassenstruktur der Merkmalswerte mittels UML Klassendiagramm.

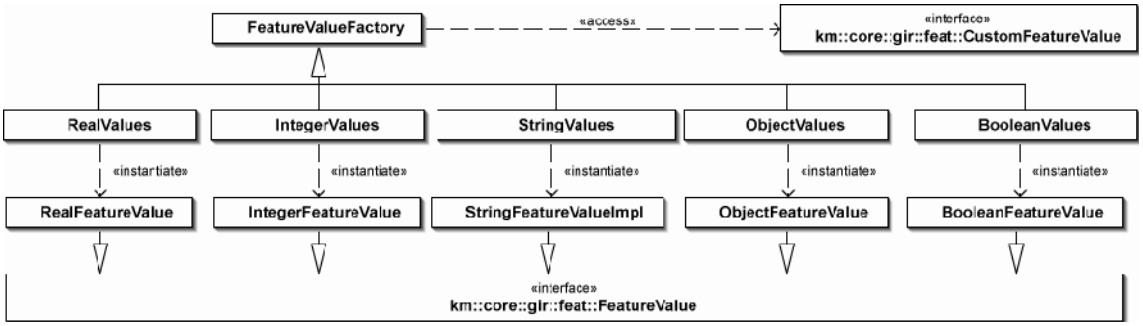


Abbildung 8-10: UML Klassendiagramm für die Verwaltung von Merkmalswerten im KnowMiner-Framework. Die FeatureValue-Factory sorgt für Instanzierung der unterschiedlichen Wertebereiche.

Das Implementierungsmuster für die Erstellung von Merkmalsschlüssel (Schnittstelle *FeatureKey*) ist äquivalent zu dem der Merkmalswerte (siehe Abbildung 8-11). Hauptgrund für die Verwaltung unterschiedlicher Schlüsselklassen sind Speicher- und Laufzeitoptimierungen sowie die Möglichkeit unterschiedlicher Implementierungsvarianten für dichte bzw. karg verteilte Merkmalsräume. So liegt beispielsweise dem IR Vektorraummodell ein karger Merkmalsraum zugrunde, welcher unter Verwendung von auf Hashing basierenden Datenstrukturen effizient verwaltbar ist. Dies bringt wiederum die Anforderung von guten Hash basierten Merkmalsschlüsseln mit sich. Unter Anwendung der dargestellten Architektur für die Verwaltung von Merkmalsschlüsseln sind solche Adaptionen einfach durch zu führen.

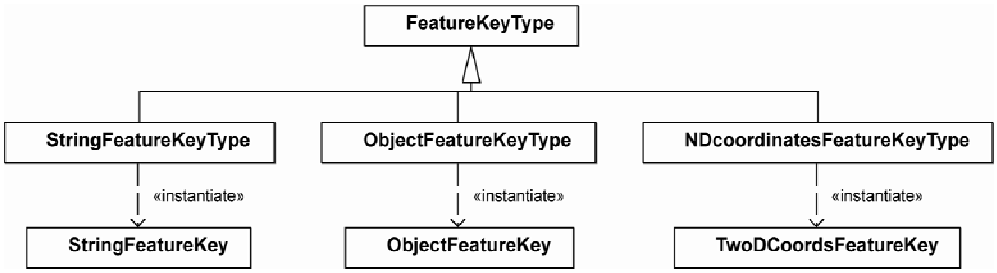


Abbildung 8-11: UML Klassendiagramm für die Verwaltung von Merkmalsschlüssel. Wiederum erfolgt die Instanzierung der Schlüssel über ein Factory-Pattern (FeatureKeyType).

Da das KnowMiner-Framework sowohl einen flexiblen, als auch einfachen Umgang mit Merkmalen von Informationsentitäten ermöglichen soll, erfolgt der Zugriff auf die Merkmalsschlüssel und Merkmalswerte nicht direkt, sondern über eine abstrahierte Typstruktur. Ein solcher Merkmalstyp, implementierungstechnisch realisiert durch die Klasse *FeatureType*, kann als Abbildung zwischen semantischem Merkmalstyp und technischer Realisierung verstanden werden. Ein Merkmalstyp wäre z.B. eine Termrepräsentation in einem Vektorraum oder aber Named Entities welche mittels Techniken der Informationsextraktion aus Texten extrahiert wurden. Beide Typen können durch die gleichen Merkmalsschlüssel und Merkmalswerte realisiert werden und sind auch in topologisch gleichen Merkmalsräumen abbildbar, weshalb sie sich vernünftigerweise die gleichen Implementierungen für Merkmalsschlüssel und Merkmalswerte teilen sollten. Obwohl die dahinter liegende Implementierung gleich ist, muss die semantische Bedeutung der Merkmale unterschieden werden. Diese Unterscheidung treffen die Klassen *FeatureType* und *FeatureClass*, in denen *FeatureClass* die Implementierung von semantischen Typen und *FeatureType* die semantischen Typen selbst kapselt.

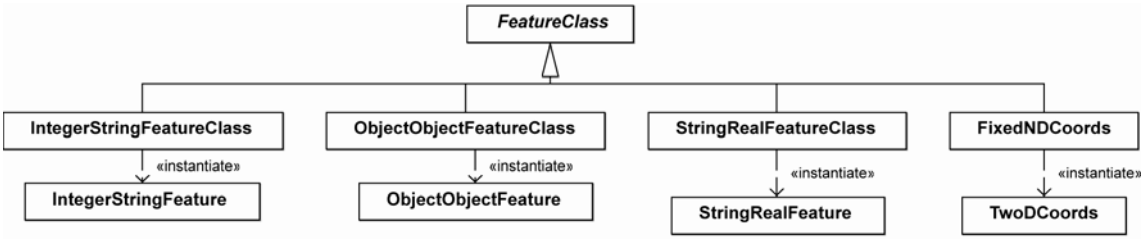


Abbildung 8-12: UML Klassendiagramm für die Verwaltung von Merkmalen. Grundlegend greift die Klasse FeatureClass auf die Klassen zur Erzeugung von Merkmalsschlüsseln und Merkmalswerten zurück, um ein entsprechendes Merkmal zu instanzieren.

Beide Klassen sind dabei nach dem Factory Design Pattern implementiert, wodurch die Verwaltung der Typen abstrahiert wird. *FeatureClass* besteht (siehe auch Abbildung 8-9) aus den beiden Klassen zur Erzeugung von Merkmalsschlüsseln (*FeatureKeyType*) und Merkmalswerten (*FeatureValueRange*); jede Klasse von Merkmalen kann also als Tupel aus Schlüsseltyp und Wertetyp angesehen werden. Abbildung 8-12 zeigt die Implementierung der Merkmalsklassen und Abbildung 8-11 resp. Abbildung 8-10 der Merkmalstypen resp. Merkmalsmengen. Für die Darstellung der konzeptuellen Zusammenhänge sei auf Abbildung 8-9 verwiesen.

Um die Funktionsweise zu illustrieren, ein kleines Beispiel: Eine TFIDF Vektorrepräsentation eines Objektes spiegelt sich im Merkmalstyp UNIWORDS (*FeatureType.UNIWORDS*) wider. Die Implementierung dieses Typs setzt die Merkmalsklasse (*FeatureClass*) *StringRealFeatureClass* um, welche aus Zeichenketten als Schlüssel

(*StringFeatureKeyType*) und reellwertigen Zahlen als Merkmalswert (*RealValues*) besteht. Eine ähnliche Repräsentationsform, wenngleich die semantische Bedeutung unterschiedlich ist, können extrahierte benannte Entitäten ebenfalls haben.

Idealerweise sollte der Zusammenhang zwischen semantischem Typ (*FeatureType)* und technischer Realisierung (*FeatureClass*) frei konfigurierbar sein, was im Framework nicht umgesetzt wurde. Die derzeitige feste Verdrahtung von Typ und Klasse ist aktuell ausreichend, kann aber bei Bedarf durch Modifikation der Klasse *FeatureType* sehr einfach auf eine konfigurierbare Verknüpfung geändert werden, ohne dabei weitere KnowMiner Teile zu adaptieren.

Die eingeführte Typstruktur dient als zentraler Anknüpfungspunkt für die anderen KnowMiner Module und durch die Kapselung der dahinter liegenden Funktionalität ist die Umsetzung von Algorithmen und Anwendungen losgelöst von der Verwaltung von Merkmalen, außer im Falle einer notwendigen Erweiterung des Typsystems.

8.3.2.2 Merkmalsräume und Merkmalsmengen

Nachdem die Implementierung der einzelnen Merkmale beschrieben wurde, soll dieser Abschnitt Aufschluss über die Implementierung und Anwendung von Merkmalsmengen und Merkmalsräumen, also wie Einzelmerkmale miteinander kombiniert werden können, geben. Im Allgemeinen behandelt die Umsetzung die folgenden 4 Hauptpunkte:

1. Merkmalsmengen desselben Merkmalstyps werden in einem Merkmalsraum zusammengefasst, womit die Vergleichbarkeit dieser Merkmalsmengen gewährleistet ist. Es macht beispielsweise wenig Sinn, TFIDF Merkmalsvektoren mit ungewichteten Merkmalsvektoren zu vergleichen. Die Vermeidung solcher Fehler kann Zeit in der Entwicklung sparen.

2. Ein Merkmalsraum definiert die verschiedenen mathematischen Operationen über Merkmalsmengen und definiert zwischen den Merkmalsmengen Metrik bzw. Ähnlichkeit. Je nach Art des Raumes können Operationen auf Arten der Repräsentation von Merkmalen Rücksicht nehmen.

3. Unterschiedliche Merkmalsmengen können mittels so genannter Merkmalsmengencontainer in einem Entitätskontext zusammengefasst werden. Dadurch können Algorithmen auf verschiedene Aspekte einer Entität in einem Kontext zugreifen. Beispielsweise sollen sowohl inhaltliche Ähnlichkeiten in Form von TFIDF Vektoren als auch die Ähnlichkeiten bezogen auf die extrahierten Named Entities in die Berechnung einfließen. Da beide Merkmalsmengen unterschiedliche Ähnlichkeitsmaße (z.B. Kosinus und Hamming Metrik) verwenden können und damit in unterschiedlichen Merkmalsräumen existieren, ist die Kombination über einen Merkmalscontainer trotzdem einfach möglich.

4. Kalkulatoren definieren Operationen, wie die Berechnungen auf Merkmalsmengen in einem Merkmalsmengencontainer gehandhabt werden. Zum Beispiel kann die maximale oder durchschnittliche Ähnlichkeit aller Ähnlichkeiten der Merkmalsmengen in einem Container als Ähnlichkeit für den Container herangezogen werden. Kalkulatoren abstrahieren dies und bauen dabei auf den abstrahierten Operationen der Merkmalsräume auf.

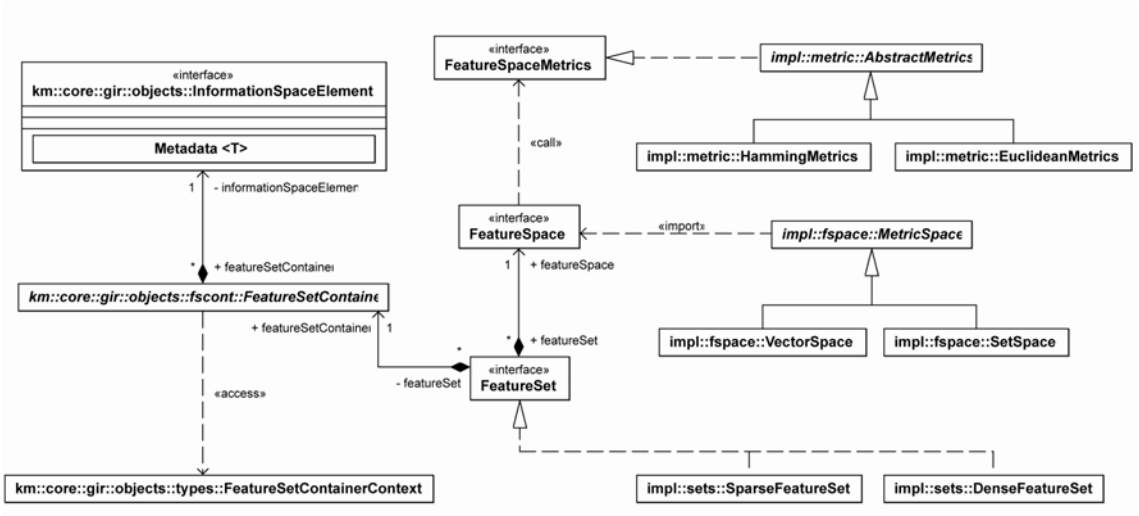


Abbildung 8-13: UML Klassendiagramm für Merkmalsräume und Merkmalsmengen

Abbildung 8-13 zeigt die Zusammenhänge zwischen Merkmalsmengencontainer (*FeatureSetContainer*), Merkmalsmengen (*FeatureSet*), Merkmalsräumen (*FeatureSpace*) und den zu Räumen zugehörigen Metriken (*FeatureSpaceMetrics*). In der Klasse *FeatureSet* werden dabei die Merkmale gleichen Typs (*FeatureType*) zusammengefasst, wobei der *FeatureSet* jeweils dem Merkmalsraum (*FeatureSpace*) für den entsprechenden Merkmalstyp zugeordnet ist. Der Merkmalsraum beinhaltet also alle Merkmalsmengen eines bestimmten Merkmalstyps und definiert die zulässigen Operationen auf und zwischen den Merkmalsmengen. Zum Beispiel werden Merkmalsvektoren in der *FeatureSet* Klasse abgebildet, welche einem Vektorraum (*VectorSpace*) als Merkmalsraum zugeordnet sind. Einen wichtigen Aspekt des Merkmalraums, nämlich die Metrik für Ähnlichkeits- und Distanzberechnungen, kapselt die Klasse *FetaureSpaceMetrcis*. Hintergrund ist, dass unterschiedliche Metriken für einen bestimmten Typ von Räumen verwendet werden können. Die Normierung eines Vektorraums kann z.B. über die euklidische oder die L_1 Norm erfolgen, eine Eigenschaft die für alle Merkmalsmengen in dem Merkmalsraum gleich sein muss, damit korrekte Ergebnisse erzielt werden können. Die Austauschbarkeit der Metrik reduziert auch die Komplexität der Implementierung, da Basisoperationen im Merkmalsraum die Zusammenhänge zwischen Mengen aber in der entsprechenden Metrik abgebildet sind. Der Aufruf der Operationen erfolgt über die *FeatureSpace* Klasse, welche auf die assoziierte Metrik zurückgreift. Die Zusammenhänge zwischen Raum und Metrik werden dabei mittels Konfiguration festgelegt und sind in weiterer Folge bei der Benutzung transparent.

Im Unterschied zu anderen Machine Learning und Data Mining Systemen besteht die Möglichkeit unterschiedliche Merkmalsmengen zu gruppieren. Hintergrund hierfür ist, dass Merkmalsmengen in unterschiedlichen Räumen mit unterschiedlichen Metriken/Operationen existieren (vergleiche Abschnitt 3.5), aber ein Element mehrere solcher Merkmalsmengen zu seiner Beschreibung enthalten kann. Um nun Operationen über eine Menge von Merkmalsmengen abstrahieren zu können und somit eine einfache Benutzbarkeit zu erreichen, wurde die Klasse *FeatureSetContainer* eingeführt. Diese kapselt die Kombination von unterschiedlichen Merkmalsmengen und ist über die *InformationSpaceElement* Klasse, unter Angabe eines Kontextes, zugänglich. Zum Beispiel können über dieses Konzept TFIDF Vektoren und Mengen von extrahierten Personennamen in einem *FeatureSetContainer* über den Kontext *HDFeatureSpace* gekapselt werden. Der Merkmalsmengencontainer kapselt nun einerseits die Merkmale eines Elements hinsichtlich extrahierter

Personen und andererseits hinsichtlich Inhalt über den Termvektor. Dies ermöglicht die Kombination unterschiedlicher Merkmalsmengen innerhalb eines Kontextes.

Die Kombination der Operationen über einen Merkmalsmengencontainer (*FeatureSetContainer*) erfolgt dabei durch die so genannten Kalkulatoren, wobei zwischen auf Entitäten operierenden Kalkulatoren und auf Relationen operierenden Kalkulatoren unterschieden wird. Die Zusammenhänge und Implementierung der Kalkulatoren illustriert Abbildung 8-14. Hintergrund hierfür ist, das mitunter Operationen auf Relationen anders durchgeführt werden (um dem Aspekt der Relation gerecht zu werden) als auf Entitäten. Die Vereinheitlichung der Operationen über Merkmalsmengencontainer ist notwendig, um eine einheitliche Benutzung des Containers zu ermöglichen. Beispielsweise werden Ähnlichkeitsberechnungen zwischen Containern auf die Ähnlichkeitsberechnung der in ihnen enthaltenen Merkmalsmengen zurückgeführt, und dem Container bzw. dem assoziierten Kalkulator obliegt die Kombination der Ähnlichkeitswerte über z.B. Mittelwert- oder Medianbildung, abhängig von der entsprechenden Konfiguration. Um eine flexible Erweiterung der Kalkulatoren zu ermöglichen, wurde eine abstrakte Basisklasse mit den grundlegenden Verwaltungsfunktionen implementiert (*AbstractEntityCalculator*), von welcher spezielle Kalkulatoren (e.g. *AveragingEntityCalculator*, *MaximumEntityCalculator*) abgeleitet werden.

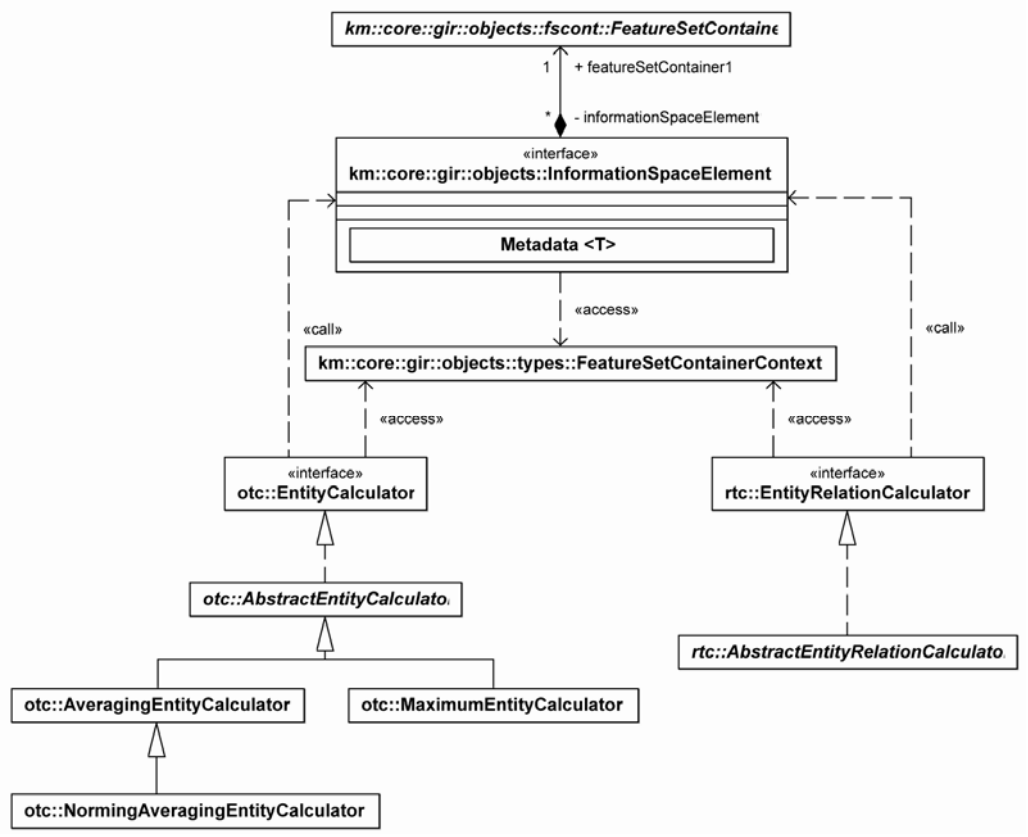


Abbildung 8-14: UML Klassendiagramm bzgl. den verwendeten Kalkulatoren.

Anwendungen bzw. Algorithmen operieren lediglich mit diesen Kalkulatoren, wobei ein Kalkulator immer nur auf einem gegebenen Kontext (*FeatureSetContainerContext*) operiert. Kontextübergreifende Operationen sind daher nicht möglich, ein Umstand der auf die Definition des Kontexts als Sichtweise auf ein Objekt

zurück zu führen ist. Über Merkmalsmengencontainer besteht aber nach wie vor die Möglichkeit beliebige Sichtweisen zu definieren.

Folgende Operationen werden dabei von Containern abstrahiert, wobei nicht alle Operationen für jeden Container implementiert sein müssen.

- Vektoroperationen
- Mengenoperationen
- Graphoperationen
- Ähnlichkeits- und Distanzberechnungen

Zusammenfassend können folgende Eigenschaften dem KnowMiner Merkmalssystems zugeordnet werden:

- Flexibles Typsystem, sowohl auf semantischer als auch auf funktionaler Ebene
- Abstraktion von Merkmalsmengen und Merkmalsräumen
- Abstraktion von Operationen über Merkmalsmengen und Merkmalsräumen
- Kombination von Merkmalsmengen zu einem Kontext inkl. Operationen auf der Kombination

Wie anhand der mit KnowMiner-Framework umgesetzten Applikationen (siehe Kapitel 9) gezeigt werden konnte, bietet das Merkmalssystem genügend Flexibilität auf der einen, aber auch eine starke Strukturierung auf der anderen Seite. Beides sind zentrale Elemente für die rasche Entwicklung und Umsetzung von KD-Anwendungen.

8.3.2.3 Metadaten

Aufgrund der Reichhaltigkeit der Formate von Metadaten ist die Repräsentation unterschiedlicher Metadaten im KnowMiner-Framework einfach gehalten. Zu jeder Entität kann unter einem bestimmten Kontext ein Metadatum abgelegt werden, wobei die Struktur der Metadaten nicht festgelegt ist. Einzige Unterscheidung bilden im RDF Format vorliegende Metadaten oder nicht im RDF Format vorliegende Metadaten. Für erstere erfolgt die direkte Integration in das RDF Modell des KnowMiner Backends, während zweitere als Java Objekte (POJO, Plain Old Java Objects) der Entität angehängt werden.

RDF Metadaten bieten dadurch den Vorteil mittels Queries auf das RDF Backend suchbar zu sein.

8.3.2.4 Annotationen

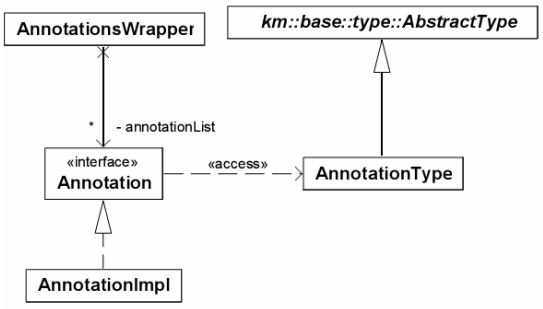


Abbildung 8-15: UML Klassendiagramm zum Annotationssystem

Im Gegensatz zu Metadaten sind Annotationen komplexer aufgebaut, wenngleich der Komplexitätsgrad weit unter dem des Merkmalssystems liegt. Abbildung 8-15 zeigt das UML Klassendiagram dazu.

Alle Annotationen leiten sich vom *Annotation* Interface ab und beschreiben den Typ der Annotation über die *AnnotationType* Klasse, die sich wiederum vom *AbstractType* Interface ableitet. Das *AbstractType* Interface wird von jeder im KnowMiner verwendeten Typ Beschreibung implementiert. Da Annotationen im Allgemeinen einem Ordnungskriterium unterliegen und meist in Form von Listen abgearbeitet werden, kapselt die *AnnotationWrapper* Klasse grundlegende Listenoperationen auf Annotationen und bietet die Basisfunktionalität für den Umgang mit Annotationen.

Zentral bei Annotationen ist die Verwaltung eines Typs für jede Annotation und wenn nötig die Kapselung von Annotationen gleichen Typs in einer Liste. Damit sind eine effiziente Abarbeitung und ein effizientes Verteilen von Annotationen in der Content Processor Pipeline (siehe nächster Abschnitt) möglich. Da Annotationen hauptsächlich in dieser Pipeline verwendet werden, wurde die Datenstruktur darauf hin optimiert.

8.4 Worker Konzept und Algorithmen

Im folgenden Abschnitt werden implementierungstechnische Details zum Worker-Konzept und zu den verschiedenen umgesetzten Algorithmen dargestellt.

8.4.1 Implementierungsdetails zum Worker Konzept

Wie oben erwähnt, dienen Worker zur Implementierung von datenbezogenen Aufgaben, welche in kleine Schritte unterteilt sind und bieten eine Thread-sichere Möglichkeit der parallelen Abarbeitung dieser innerhalb einer Java Virtual Machine an. Dazu zerlegen Worker die Gesamtaufgabe in so genannte *Work Units* und iterieren über diese, wobei eine Work Unit in jeder Iteration vom Worker abgearbeitet wird. Worker sind so aufgebaut, dass nach jeder Iteration die Möglichkeit besteht, den Worker von außen sauber abzubrechen oder zu suspenden und damit den Worker in einem konsistenten Zustand zu lassen.

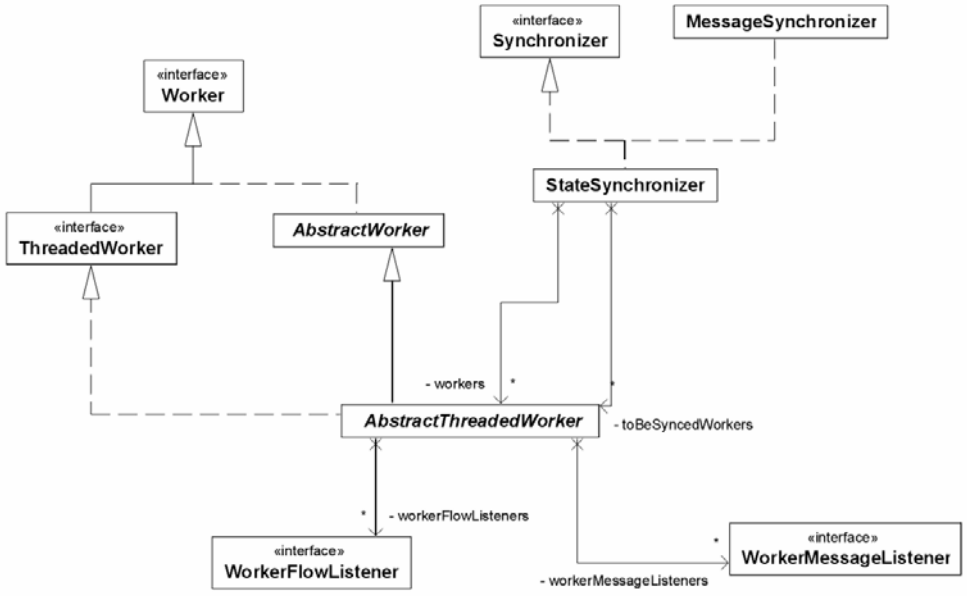


Abbildung 8-16: UML Diagramm der Worker-Klassen.

Java bietet mit seinem Concurrency Package leider keine Möglichkeit dies zu tun, woraus sich die Notwendigkeit der Implementierung dieses Konzeptes ergeben hat. Neben der Eigenschaft der parallelen Abarbeitung können Worker über den so genannten Worker-Monitor verwaltet und überwacht werden (siehe dazu Abschnitt 8.6). Das Konzept unterstützt ebenfalls die Integration von Scheduling-Techniken für Worker und damit verbunden load balancing Möglichkeiten.

Vom Entwicklungsstandpunkt im Team betrachtet, bietet das Worker-Konzept eine Abstraktion der Implementierung von KD-Algorithmen und eine einheitliche Verwendung. Dadurch erhöht sich die Wiederverwendbarkeit im Team und die Fehleranfälligkeit bei der Implementierung sinkt, da auf ein gut getestetes, stabiles Framework aufgesetzt werden kann.

Abbildung 8-16 zeigt das UML Klassendiagramm für die Worker Infrastruktur. Das *Worker* Interface definiert die Basisfunktionalität, von der alle Worker Klassen und Implementierungen ableiten. Während das *Worker* Interface noch keine asynchrone Abarbeitung erlaubt, erfolgt die Definition der entsprechenden Methoden im *ThreadedWorker* und die Implementierung der Basisfunktionen in der abstrakten Klasse *AbstractThreadedWorker*. Die abstrakte Basisfunktionalität für Worker, welche nicht in einem eigenen Thread laufen, sich also direkt vom *Worker* Interface ableiten, ist in der abstrakten Klasse *AbstractWorker* gekapselt, von welcher sich auch der *AbstractThreadedWorker* ableitet. Der *AbstractThreadedWorker* verwendet standardmäßig einen eigenen Thread zur Ausführung, kann aber über die Wahl des entsprechenden Threading-Modells (siehe JavaDoc für Details) auf einen nicht Threading Worker (i.e. dem *AbstractWorker*) rückgeführt werden. Die *AbstractThreadedWorker* Klasse bietet auch die Möglichkeit, Listener über das Interface *WorkerMessageListener* zu registrieren, wodurch Callbacks im asynchronen Betrieb möglich sind. Die *StateSynchronizer* Klasse ermöglicht das Synchronisieren von verschiedenen *AbstractThreadedWorkern* anhand ihres aktuellen Status. So ist es z.B. möglich über den *StateSynchronizer* zu warten bis mehrere Worker gleichzeitig mit der Verarbeitung fertig werden. Für Details zur Implementierung und Anwendung sei der Leser auf die entsprechende Dokumentation verwiesen.

8.4.2 ContentProcessor

Ziel des *ContentProcessor* ist die Erzeugung bzw. Verfeinerung von Annotationen aus dem Inhalt von Entitäten. Aufgrund der notwendigen Flexibilität und Generik des KnowMiner-Frameworks ist die Art der Inhaltsnormalisierung stark applikationsgetrieben, wodurch flexible Verarbeitungsmechanismen zum Einsatz kommen können. Aus diesem Grunde ist der Aufbau der Inhaltsnormalisierung über den *ContentProcessor* als flexible Pipeline-Architektur umgesetzt. Zusätzlich können zusammengesetzte Pipelines wiederum als Einzelkomponenten einer weiteren Pipeline verwendet werden, wodurch hierarchische Pipe-Architekturen möglich sind. Die hier im Folgenden vorgestellte Pipe-Architektur ist in ihrer Anwendung nicht auf den *ContentProcessor* beschränkt, sondern eignet sich auch zum Verwalten von Pipelinestrukturen auf Serviceebene. Eine Entwicklung, die sich zum jetzigen Zeitpunkt bereits in Planung befindet.

Im Gegensatz zu den komplexen Aufgaben der anderen Komponenten betrachtet die ContentProcessor Pipeline immer nur einzelne Informationsentitäten und berücksichtigt keine Beziehungen zwischen unterschiedlichen Informationsentitäten. Diese Einschränkung dient vor allem zur Komplexitätsreduktion der Pipeline bzw. der einzelnen Komponenten und der damit verbundenen leichten Entwicklung von neuen Pipe-Komponenten. Zusätzlich ist eine effizientere Abarbeitung z.B. im Parallelbetrieb möglich, da Einzelkomponenten per Definition immer nur ihre aktuelle Informationsentität betrachten und somit keine konkurrierenden Zugriffe entstehen.

8.4.2.1 Aufbau der Pipeline

Die KnowMiner Pipeline, im Detail beschrieben in [Klieber 2006b], zielt auf einen möglichst flexiblen Einsatz ab, der nicht nur auf den *ContentProcessor* beschränkt ist. wobei sie jedoch momentan nur in diesem genutzt wird. Dieser Abschnitt gibt einen kurzen Einblick in den Aufbau der Pipeline (für Details siehe [Klieber 2006b], die entsprechende Dokumentation in [Sabol, Klieber, Granitzer 2006]) und die grundlegende Funktionsweise. Anschließend erfolgt die Beschreibung der von diesen Komponenten abgeleiteten Annotations-Pipeline, welche im *ContentProcessor* eingesetzt wird.

Abbildung 8-17 zeigt die wichtigsten Komponenten der Pipeline Architektur in Form eines UML Diagramms.

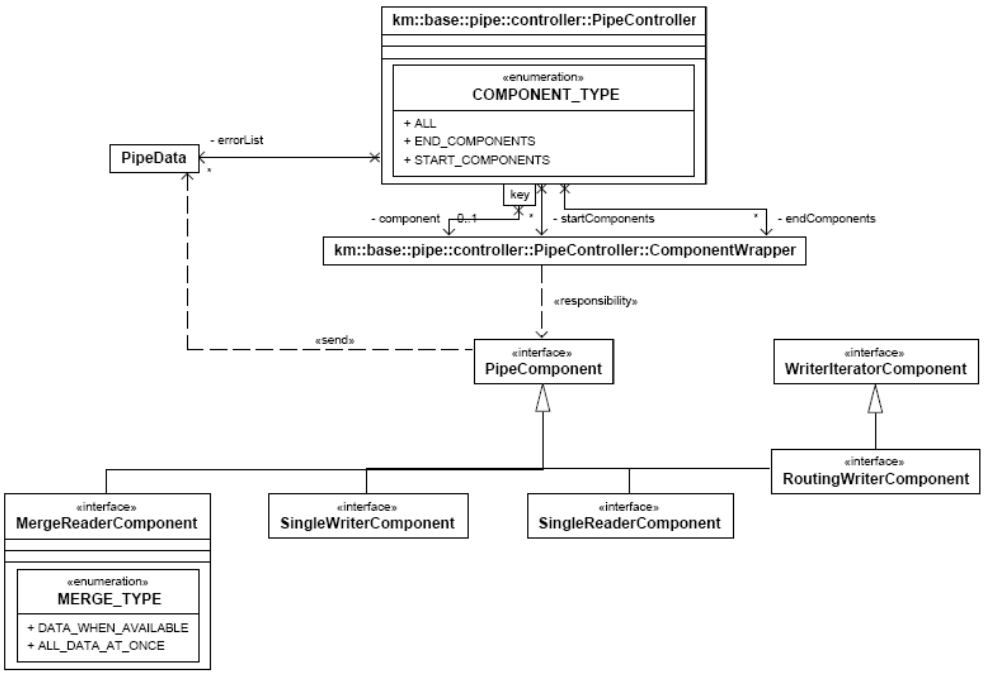


Abbildung 8-17: UML Klassendiagramm der Pipeline zur Inhaltsnormalisierung. PipeData kapselt die Daten der Pipeline und PipeComponent die einzelnen Komponenten. Die Pipeline ist hierarchisch organisiert sodass eine PipeComponent wiederum als Komponenten einer anderen PipeComponent verwendet werden kann.

Zentrale Komponente ist das Interface *PipeComponent*, welches von der Klasse *PipeController* verwendet wird, um die Pipeline aufzubauen. Der *PipeController* verwendet intern einen Wrapper, um den einzelnen Pipe-Komponenten, im Speziellen den Klassen, die das *PipeComponent* Interface implementieren, zusätzlich einen Typ zu geben und die verschiedenen Verbindungen von der Pipe-Komponente weg und zur Pipe-Komponente hin zu verwalten.

Damit die Pipe-Komponente Eingabedaten lesen bzw. schreiben kann, muss diese die folgenden vom *PipeComponent* Interface abgeleiteten Interfaces implementieren:

- *SingleWriterComponent*: Komponente, welche alle Ausgangsdaten an *n* Komponenten weiter geben kann.

- *SingleReaderComponent*: Komponente, welche Eingangsdaten von einer Vorgängerkomponente lesen kann
- *MergeReaderComponent:* Komponente, welche Eingangsdaten von *n* Vorgängerkomponenten lesen kann
- *RoutingWriterComponent:* Komponente, welche selektiv Ausgangsdaten an *n* Pipe-Komponenten weiter geben kann. Im Unterschied zur *SingleWriterComponente* werden nicht alle Ausgangsdaten an alle *n* Pipes weiter gegeben.

Die Klasse *PipeData* kapselt die Daten, die von den einzelnen Pipe-Komponenten gelesen oder geschrieben werden. Aus Übersichtsgründen zeigt Abbildung 8-17 nur eine Abhängigkeit von *PipeData* zu *PipeComponent*, wobei grundsätzlich jedes der abgeleiteten Interfaces eine Abhängigkeit zu *PipeData* hat.

Der *PipeController* kapselt die Erstellung und Abarbeitung der Pipeline sowie die richtige Verschaltung. Durch Übergabe einer Menge an *PipeComponenten* und durch Verdrahtung dieser kann der *PipeController* die Pipeline aufbauen und abarbeiten.

8.4.2.2 Pipeline des ContentProcessor

Auf Basis der oben eingeführten Pipe-Architektur wurde die Annotations-Pipeline für den *ContentProcessor* entwickelt. Zusätzlich zur einfachen Weiterleitung von Daten definiert diese Pipeline speziell auf die Annotation von Entitäten zugeschnittene Eigenschaften, die im Folgenden kurz illustriert werden.

Ein wichtiger Punkt in der Erstellung der Annotations-Pipeline sind die Eigenschaften der zu verarbeitenden Daten und deren Wechselwirkung mit den Eigenschaften der verarbeitbaren Daten der Pipelemente. Die verarbeitbaren Daten der Einzelkomponenten der Annotations-Pipeline definieren sich durch die Art der Attribute, die verarbeitet werden können, unter welchem Kontext bzw. Schlüssel die Attribute der Komponente bereit gestellt werden, welche Attribute eine Komponente liefert und unter welchem Kontext bzw. Schlüssel die Attribute bereitgestellt werden.

Zentral für die Einzelkomponenten ist die Definition der verarbeitbaren Attribute und welche Attribute von einer Komponente bereitgestellt werden. Die Eigenschaften der verarbeitbaren Attribute definieren sich über das Tripel <*Kontext, Attributstyp, Datenstrukturen*>, welches im Folgenden näher beschrieben wird:

- **Kontext**: Der Kontext kann als Schlüssel angesehen werden, unter welchem auf die zu verarbeitbaren Attribute zugegriffen wird, ähnlich dem Merkmalskontext in der GIR (siehe Abschnitt 8.3). Die Übergabe des Kontexts einer Einzelkomponente erfolgt dabei von außen, womit verschiedene Datenkanäle in einer Pipeline verwaltet und konfiguriert werden können. Unter Zuhilfenahme des Kontextes können also verschiedene Ausprägungen von Attributen für unterschiedliche Komponenten bereitgestellt werden, eine Eigenschaft, die implementierungstechnisch über die unten beschriebenen Kanalkomponenten realisiert wird.
- **Attributstyp**: Zur Spezifikation der verarbeitbaren Attribute einer Komponente dient der Attributstyp. Attributstypen hängen von der jeweiligen Modalität (also Text, Bild und Audio) ab und sind über ein flexibles Typsystem definiert. Im Grunde kann als Typ ein beliebiger String verwendet werden, wobei zusätzlich ein validiertes Typsystem unterstützt wird. Dieses Typsystem ist

hierarchisch aufgebaut, wodurch eine Schachtelung von Typen möglich ist. Attributstypen werden immer auf ihrer generellsten Ebene in den Verarbeitungskomponenten definiert, d.h. alle Untertypen eines angegebenen Typs können ebenfalls verarbeitet werden. Diese Strukturierung dient zur Definition der Eigenschaften der Attribute in Form einer IS-A Beziehung, d.h. Untertypen erben die Eigenschaften ihrer Übertypen. Beispiel für einen Typ wären *Text.Sentence* oder *Text.Token*, wobei Texttokens weiter unterteilt werden können in *Text.Token.Primitives*, *Text.Token.OffsetToken* etc. Kann nun eine Komponente mit dem Typ *Text.Token* umgehen, so ist auch eine Abarbeitung von *Text.Token.OffsetToken* möglich. Umgekehrt ist dies nicht der Fall, da *Text.Token.OffsetToken* spezielle Eigenschaften wie die Start- und Endposition eines Tokens enthält, die von der Komponente benötigt werden könnte. Neben modalitätsspezifischen Attributstypen sind hier auch Metadaten spezifizierbar. Benötigt z.B. eine Komponente die Sprache einer Entität als Metadatum, so kann dies entsprechend definiert werden.

- **Datenstrukturen**: Attribute von einem Typ können in unterschiedlichen Datenstrukturen verwaltet werden, vor allem für den Fall der Einbindung anderer Frameworks wie z.B. GATE. Da nicht jede Komponente mit allen Datentypen umgehen können muss (wodurch der Implementierungsaufwand exorbitant steigen würde), können Einzelkomponenten spezifizieren mit welchen Datenstrukturen zu einem Attributstyp sie umgehen können.

Äquivalent zur Definition der verarbeitbaren Attribute einer Komponente werden auch die von einer Komponente bereitgestellten Daten definiert. Wiederum kommt das Tripel <*Kontext, Attributstyp, Datenstrukturen*> zur Beschreibung, in welchem Kontext, welche Attribute in Form welcher Datenstruktur von der Komponente bereitstellt werden, zum Einsatz. Im Gegensatz zur Definition der verarbeitbaren Daten, wird bei der Bereitstellung der Daten nur eine mögliche Datenstruktur, die von der Komponente erzeugt wird, angegeben, wodurch die Komplexität der Benutzung der gesamten Pipeline reduziert wird.

Grundlegend definiert die Implementierung einer Komponente die Menge der Attributstypen und Java Datenstrukturen, die sie verarbeiten kann, und die von ihr bereitgestellte Datenstruktur und Attributstypen. Im Gegensatz dazu erfolgt das Setzen des Kontexts von außen und ist damit ein topologisches Merkmal für die Architektur der Pipeline. Diese Typisierung von Eingangs-/Ausgangsdaten definiert damit, welche Komponenten mit welchen Komponenten sowohl auf semantischer als auch auf Ebene der Datenstrukturen verschachtelt werden können, und erlaubt somit eine Vorab-Prüfung der Richtigkeit der Pipeline.

Komponente	Input/ Output	Kontext	Attributtype	Datenstrukturen
Tokenizer	Input	„Text“	Zeichenkette	String StringBuffer
	Output	„Text“	Text.Token	List<String>
POS Tagger	Input	„Text“	Text.Token	List<String>
		„Text“	Metadata.lang	String
	Output	„Text“	Text.Token.POSToken	List<Annotations>
Named Entity Detector	Input	„Text“	Text.Token	List<Annotations> List<String>
	Output	„Text“	Text.NamedEntities	List<Annotations>
Language Detektor	Input	„Text“	Text.Token	List<String>
	Output	„Text“	Metadata.lang	String

Tabelle 8-1: Eigenschaften von Beispielkomponenten aus dem Bereich Textvorverarbeitung

Beispielsweise definiert ein Part-Of-Speech Tagger, dass er auf einer Menge von Tokens (=Attributstyp) arbeitet, welche in Form einer Liste von Zeichenketten (i.e. *List<String>* in Java) gehalten werden. Die Unterscheidung multipler Datenstrukturen pro Typ ermöglicht dabei, wie oben erwähnt, eine einfache Integration von Komponenten anderer Frameworks. So könnte z.B. die Part-Of-Speech Tagging Komponente auch die Fähigkeit besitzen, mit Datenstrukturen aus dem GATE-Framework umzugehen und somit mit GATE Komponenten verschaltbar zu sein. Eine Aufteilung zwischen Typ und Implementierung macht solche Verarbeitungen möglich.

Tabelle 8-1 zeigt beispielhaft mögliche Konfigurationen für Textkomponenten. Der Tokenizer benötigt Inputdaten in Form einer Zeichenkette, welche als Java *String* oder *StringBuffer* Klasse angeliefert wird und erzeugt aus dieser eine Liste von Strings vom Typ *Text.Token*, welche wiederum von der POS Tagger Komponente weiter verarbeitet werden können.

Abbildung 8-18 zeigt das UML Diagramm der Implementierung dieser Annotations-Pipeline des ContentProcessor, welche auch als Kontext-Pipe bezeichnet wird, auf Basis der im vorigen Abschnitt beschriebenen Pipeline-Elemente.

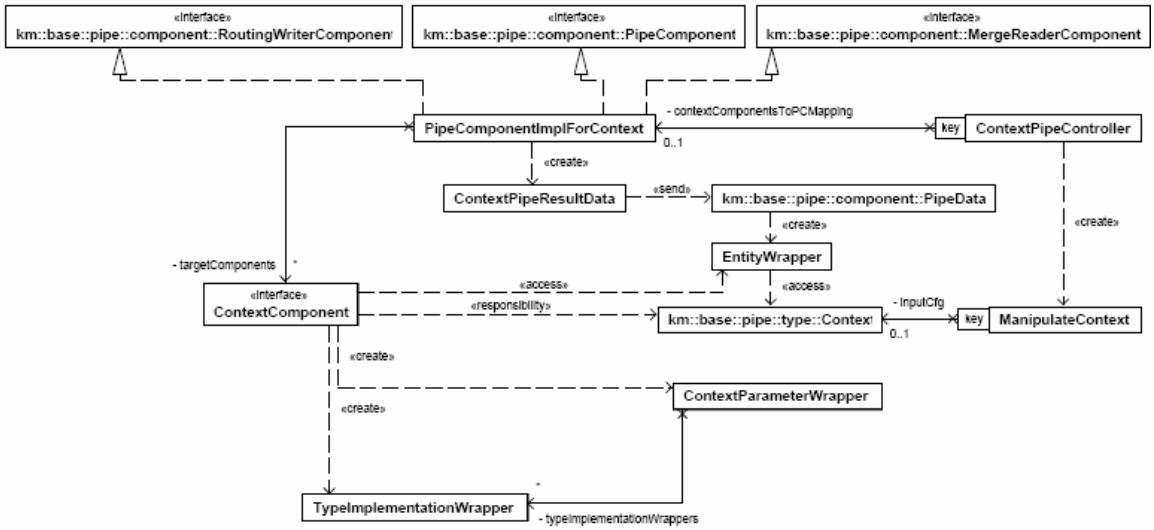


Abbildung 8-18: UML Diagramm der Context Pipeline zur Annotierung und Vorverarbeitung von Inhalten.

Aus Sicht des Entwicklers von Pipe-Komponenten für die Kontext-Pipe gestaltet sich die Implementierung einfach. Das Interface *ContextComponent* muss für die umzusetzenden Komponenten implementiert werden, wobei der Entwickler sich lediglich um die Spezifikation der Attribute und Datenstrukturen und natürlich die Implementierung der eigentlichen Funktionalität selbst kümmern muss. Den Aufbau der Pipe übernimmt dabei der *ContextPipeController*, welcher sich vom *PipeController* der Basis Pipe ableitet. Der *ContextPipeController* benutzt zur Abarbeitung der einzelnen Pipe-Elemente die *PipeComponentImplForContext* Klasse, welche sich aus den Pipe-Interface-Komponenten (*RoutingWriterComponte, PipeComponent und MergeReaderComponent*) der Basis-Pipeline ableitet und über ein Proxy-Pattern die jeweilige *ContextComponente* bedient. Anders gesagt, erfolgt also die Ansteuerung der jeweiligen Implementierung (i.e. Implementierung der *ContextComponente*) über die Klasse *PipeComponentImplForContext,* welche über den *ContextPipeController* verwaltet und mit anderen *PipeComponentImplForContext* verschachtelt wird. Über den *ContextPipeController* können auch Attribute und Kontexte der Pipe-Komponenten konfiguriert und verwaltet werden. Der Kontext und die Attribute selbst

werden über die Klassen *EntityWrapper, Context, TypeImplementationWrapper, ContextParameterWrapper und MainpulateContext* verwaltet, wobei für eine detaillierte Beschreibung auf die entsprechende JavaDoc verwiesen sei.

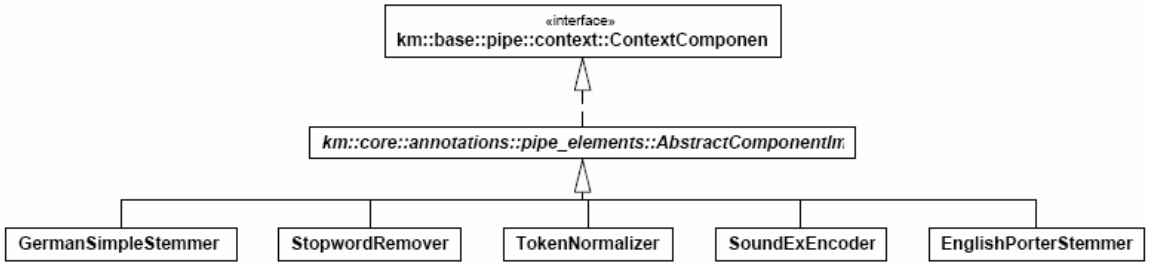


Abbildung 8-19: Implementierungen des ContextComponent Interface für verschiedene im KnowMiner verwendete Stemmer

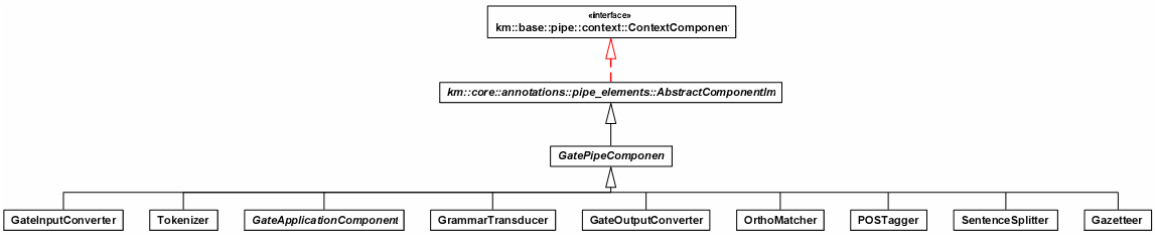


Abbildung 8-20: Implementierungen des ContextComponent Interface für die Implementierung von GATE Komponenten im KnowMiner

Über diese flexible Pipe-Architektur und über das verwendete Typen- bzw. Kontextsystem können relativ einfach verschiedene Vorverarbeitungsverfahren je nach Anwendungsfall integriert werden. Des Weiteren ist die Implementierung neuer Komponenten einfach zu realisieren, da nur ein relativ einfaches Interface zu implementieren ist. Das Attribut- und Kontextsystem sorgt wiederum dafür, dass Erzeuger von Pipe-Komponenten exakt spezifizieren, welche Daten verarbeitet werden können und somit das semantisch korrekte Zusammenstellen einer Pipeline für Anwender der Pipe-Architektur erleichtert wird. Durch dieses System erfolgt also einerseits die Unterstützung der Komponentenentwickler und andererseits die der Pipeanwender. Abbildung 8-19 und Abbildung 8-20 zeigen das Vererbungsdiagramm für die Implementierung von verschiedenen Pipe-Komponenten als Ableitung vom *ContextComponent* Interface und sollen zum Abschluss dieses Abschnitts einen kleinen Ausschnitt aus den im KnowMiner-Framework enthaltenen Pipe-Funktionen darstellen.

8.4.3 Clustering

Das KnowMiner-Framework bietet eigene Worker-Klassenstrukturen für die Umsetzung von Clustering-Algorithmen an, wie dies in Abbildung 8-21 dargestellt ist. Der Implementierung des Clustering-Algorithmus selbst erfolgt über das Interface *Clusterer*, welches vom *ThreadedWorker* Interface ableitet und damit die Worker-Funktionalität inkludiert. Verschiedene Clustering-Algorithmen benötigen auch so genannte Seed-Finder, welche die Startpunkte für das Clustering finden. Die Benutzung dieser erfolgt über das Interface *SeedFinder*, das in den unterschiedlichen Algorithmen angewendet wird. Derzeit existiert ein auf Zufallszahlen

basierender Seed-Finder (Klasse *RandomSeedFinder*) und der auf Entitäten basierende *GIRObjectSeedFinder*, welcher zufällig ausgewählte Entitäten zu einem Start-Cluster zusammenfasst.

Die erzeugten Cluster stellt der *Clusterer* in Form von Klassen die das Interface *Cluster* implementieren zu Verfügung. *Cluster* können über den *ClusterWriterWorker* zurück in die GIR-Struktur geschrieben werden, wobei der *ClusterWriterWorker* wiederum einen Threaded-Worker implementiert.

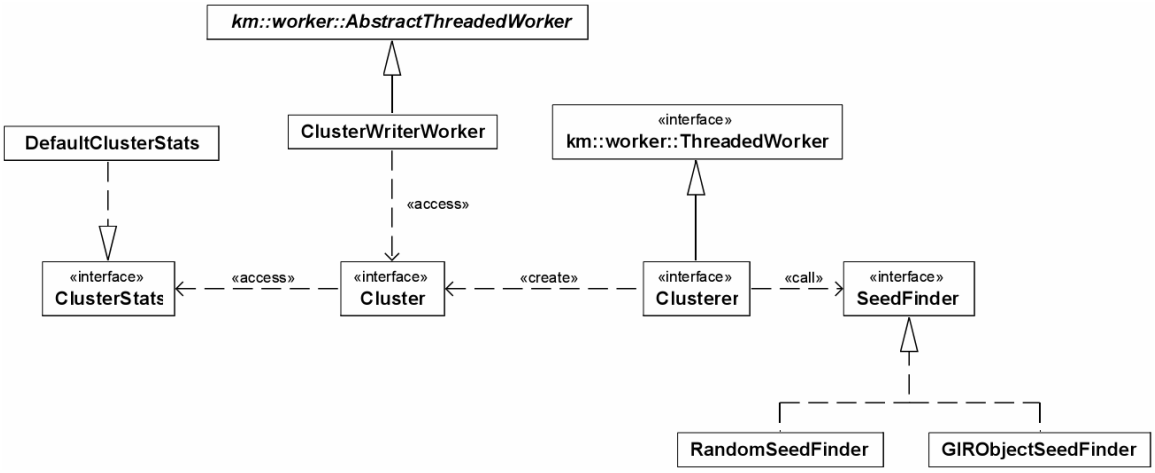


Abbildung 8-21: Beschreibung der Cluster Interfaces und deren Einbettung in die Worker Infrastruktur

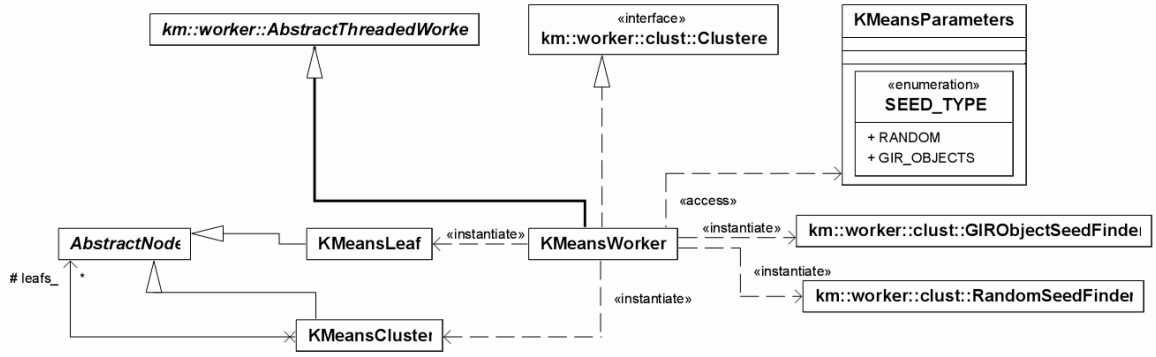


Abbildung 8-22: UML Diagramm zur Implementierung des K-Means Clustering-Algorithmus auf Basis der Interfaces und Worker Struktur

Direkt im KnowMiner implementiert sind derzeit K-Means, HAC mit verschiedenen Linkage Varianten und einer laufzeiteffizienten WARD Implementierung, Fuzzy K-Means und Suffix Tree Clustering. Abbildung 8-22 zeigt beispielhaft die Umsetzung des K-Means Clusterer anhand eines UML-Diagramms. Für Details dazu sei auf die umfangreiche Java Dokumentation verwiesen.

8.4.4 Ordination und Merkmalstransformation

Die Ordination und Merkmalstransformation wird, wie in Abschnitt 7.6 spezifiziert, auf eine Menge von Entitäten angewendet, deren Merkmale von einem Merkmalsraum in einen anderen transformiert werden sollen. Im KnowMiner-Framework sind aktuell Merkmalsgewichtung, Multi Dimensionale Skalierung (als Force Directed Placement und FastMap Algorithmus) und LSI implementiert. Dieser Abschnitt gibt einen Überblick über die Implementierung der Techniken, wobei für Beschreibungen der einzelnen Algorithmen

auf Abschnitt 4.1 und für weitere Details zur Implementierung auf die JavaDoc des KnowMiner-Frameworks verwiesen sei.

Merkmalsgewichtung: Das KnowMiner-Framework bietet verschiedene Methoden zur Merkmalsgewichtung an, wobei zwischen frequenzbasierten und informationstheoretischen Gewichtungsverfahren unterschieden wird.

Der erste Schritt bei der Merkmalsgewichtung besteht im Aufbauen der Statistik durch Iteration über die vorgegebene Elementmenge. Für frequenzbasierte Verfahren obliegt dies dem *FrequencyStatisticWorker*; für informationstheoretische Verfahren dem *EntropyStatisticWorker*. Dabei wird die Häufigkeit von Merkmalen in einem Element bzw. über die verschiedenen Elemente einer Selektion berechnet. Auf Basis der berechneten Statistik kann unter anderem eine TFIDF (Term Frequency Inverse Document Frequency, siehe Abschnitt 3.3.4) und eine Entropie-basierte Gewichtung durchgeführt werden. Gleichzeitig erfolgt die Ermittlung verschiedener statistischer Momente (i.e. Median und Varianz), welche zum nachfolgenden Beschneiden der Merkmalsvektoren dienen.

Multi Dimensionale Skalierung (MDS): Wie in Abschnitt 4.1.2 beschrieben, zielt MDS darauf ab, einen höherdimensionalen Merkmalsraum unter Beibehaltung der Distanzen der einzelnen Objekte in einen niedrigdimensionalen Merkmalsraum zu transformieren, wobei die Zieldimensionalität frei wählbar ist. Im Allgemeinen ist das Einsatzgebiet von MDS Algorithmen die visuellen Aufbereitung hochdimensionaler Daten (i.e. Transformation in 1D, 2D oder 3D). Die Techniken können aber auch, wie in [Platt 2005], zur Dimensionsreduktion für Ähnlichkeitssuchen heran gezogen werden.

Im KnowMiner-Framework sind zwei Algorithmen zur Multi Dimensionalen Skalierung implementiert, nämlich der FastMap und der Force Directed Placement (FDP) Algorithmus. Beide Algorithmen kamen in verschiedenen Projekten wie InfoSky, WebRat, MAGICK und onAIR zum Einsatz und haben sich hinsichtlich Anwendbarkeit bewährt.

Während FastMap die multidimensionale Skalierung in linearer Laufzeit der Objektanzahl berechnet, benötigt der implementierte FDP Algorithmus $O(n \log n)$ in seiner schnellsten Variante. Diese beiden Varianten zeichnen sich zwar durch eine effiziente Laufzeit aus, jedoch ist damit nicht immer eine optimale Erhaltung der hochdimensionalen Distanz verbunden. Um in unterschiedlichen Anwendungsszenarien flexibel auf die jeweiligen Anforderungen reagieren zu können, sind insgesamt folgende FDP Varianten implementiert:

- In der Original FDP Version interagiert jede Entität mit jeder anderen der Menge, woraus sich eine Zeitkomplexität von $O(n^3)$ und eine Speicherkomplexität von $O(n^2)$ ergibt.
- Zur Reduktion der Laufzeitkomplexität wird in der Sampling Variante des FDPs ein Ausschnitt aus der Entitätsmenge herangezogen, mit welchen die restlichen Entitäten interagieren resp. zwischen denen in jeder Iteration die Kräfte berechnet werden (siehe Abschnitt 4.1.2.1). Die Sampling Menge wird während der Ausführung modifiziert, um eine gute Qualität zu gewährleisten. Dadurch erreicht man eine Zeitkomplexität von $O(n^2)$ und eine Speicherkomplexität von ebenfalls $O(n^2)$
- Grundvoraussetzung beim hierarchischen FDP ist eine gegebene Taxonomie. Zur Beschleunigung des Verfahrens interagieren die Entitäten einer Klasse miteinander und mit den Klassenrepräsentanten ihrer direkten Nachbarn und Eltern in der Hierarchie. Dadurch lässt sich eine Zeitkomplexität von $O(n^2*\log(n))$ (unter Vorberechnung der Ähnlichkeiten $O(n^2)$) und eine Speicherkomplexität von $O(\log(n))$ erreichen.

Da beide Verfahren ähnlichkeits- bzw. distanzbasiert arbeiten, profitieren sie von der Abstrahierung der Ähnlichkeitsmetriken im KnowMiner-Framework und können durch Wechsel der Metrik schnell auf andere Anwendungsfälle adaptiert werden. Diese Adaptierbarkeit zeigt sich auch in den Anwendungsbeispielen MAGICK (siehe Abschnitt 6.2.2) und onAIR (siehe Abschnitt 6.2.1), in denen die MDS Algorithmen einerseits auf textuellen Daten und andererseits cross-modal, d.h. auf Bild, Audio, Text und Metadaten eingesetzt wurden.

Latent Semantic Indexing: Während es verschiedene Implementierungsarten für die Berechnung der LSI Transformation gibt, unter anderem eine probabilistische Version von Hoffmann [Hofmann 1999], ist im KnowMiner-Framework die einfachste Form auf Basis einer Singulärwertzerlegung implementiert. Da dies ein numerisch anspruchsvolles Verfahren aus dem Bereich der linearen Algebra darstellt und es frei erhältliche lineare Algebra Bibliotheken gibt, wurde die MTJ Bibliothek in den KnowMiner integriert. Unter Anwendung der Matrix Toolkit for Java (MTJ) Bibliothek[42] erfolgt auch die LSI Zerlegung, wobei die Merkmale in eine für MTJ verwertbare Repräsentation überführt werden und nach Berechnung der Singulärwertzerlegung eine Rücktransformation in die KnowMiner Repräsentation durchgeführt wird.

8.4.5 Klassifikation

Die KnowMiner Klassifikationskomponente setzt überwachte Lernverfahren zur Klassifikation von Entitäten und zur Zuordnung von Metadaten um. Abbildung 7-20 zeigt einen Überblick über die KnowMiner Komponenten zur Klassifikation, zu denen das Klassifikationsmodell, die jeweilige Klassifikationshypothese, die Selektion von Trainings- bzw. Testbeispielen, sowie die Evaluierungskomponenten zählen.

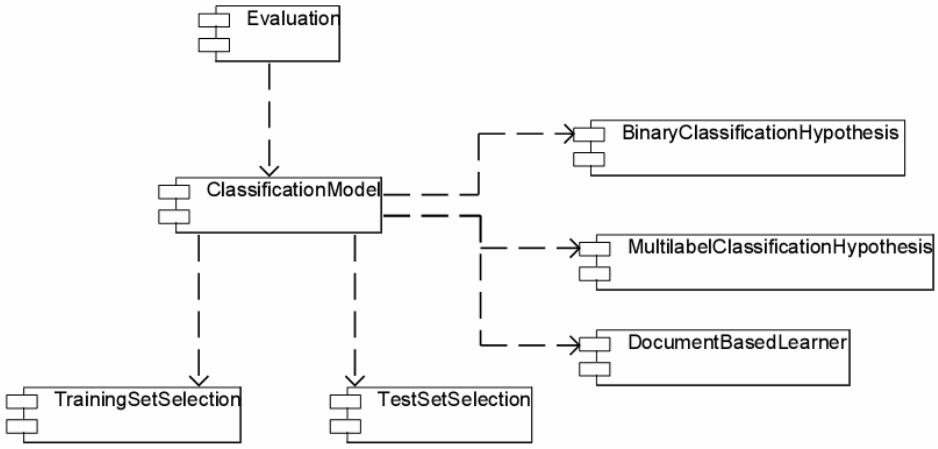


Abbildung 8-23: Überblick über die Komponenten der Klassifikation

Das Klassifikationsmodel entscheidet dabei, wie das Training bzw. die Evaluierung des Klassifikationsmodelles laufen und berücksichtigt spezielle Anforderungen an die jeweiligen Datenstrukturen. So werden zum Beispiel hierarchische und flache Modelle unterschieden. Bei der hierarchsichen Klassifikation erfolgt dabei die Berücksichtigung von Beziehungen zwischen Klassen in Form eines direkten azyklischen Graphen (DAG), während bei der flachen Klassifikation Relationen zwischen Entitäten unberücksichtigt

[42] http://rs.cipr.uib.no/mtj/

bleiben. Jeweils beim Training und bei der Zuordnung von Entitäten müssen dabei unterschiedliche Herangehensweisen beachtet werden, welche im entsprechenden Modell gekapselt sind. Als drittes Modell im KnowMiner ist ein Online Learning Modell integriert, welches die Klassifikationshypothesen auf Basis jedes präsentierten Beispiels adaptiert, und dem somit nicht alle Beispiele auf einmal präsentiert werden müssen.

Die Komponenten *TrainingSetSelection* und *TestSetSelection* stellen dabei die Trainings- bzw. Testdaten bereit.. Die Kapselung der Bereitstellung der Trainingsdaten zielt auf einfache Integration verschiedener Strategien ab, um z.B. wie in [Granitzer 2004a] die Robustheit der Klassifikationsentscheidungen über die geeignete Selektion von Trainingsbeispielen zu beeinflussen.

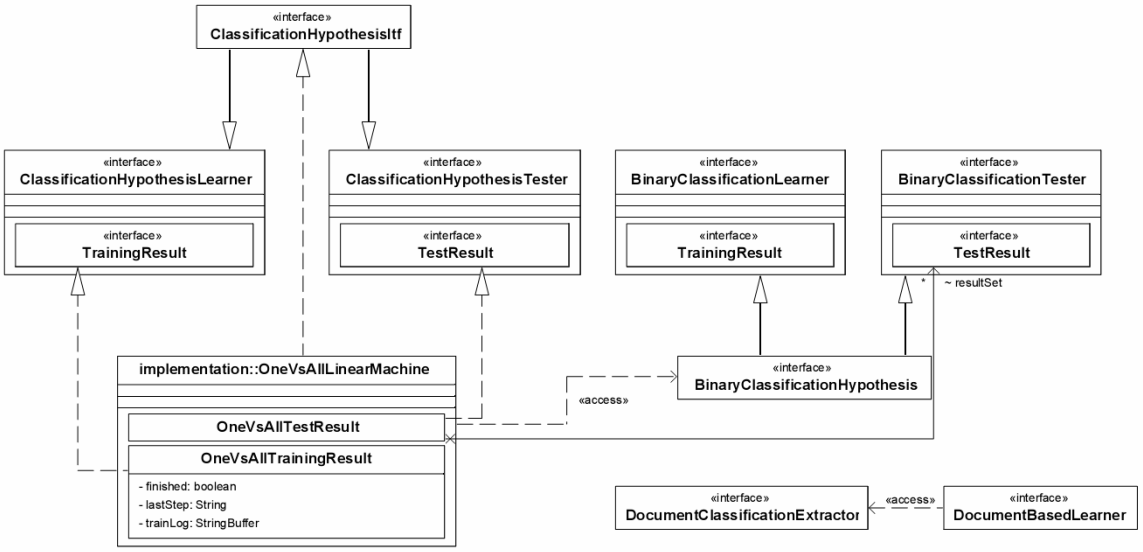


Abbildung 8-24: Schnittstellen zur Implementierung der Klassifikationshypothesen /-algorithmen

Die Interfaces *BinaryClassificationHypothesis, MultilabelClassificationHypothesis* und *DocumentBasedLearner* enthalten die eigentlichen Lernalgorithmen. Damit erreicht man eine unabhängige Implementierung der Klassifikationsalgorithmen vom Klassifikationsmodell. Abbildung 8-24 zeigt die Zusammenhänge zwischen den Klassifikationshypothesen im Detail. Hypothesen werden unterteilt in eine Lernkomponente (e.g. *ClassificationHypothesisLearner, BinaryClassificationLearner, DocumentBasedLearner*) und in eine Zuordnungskomponente (e.g. *ClassificationHypothesisTester, BinaryClassificationTester*), die jeweils ihre eigenen Datenstrukturen zur Rückgabe der Ergebnisse definieren (e.g. *TestResult* und *TrainingResult*). Durch die Trennung zwischen Lern- und Zuordnungskomponente ist eine getrennte Implementierung der beiden Aspekte möglich und somit können z.B. gelernte lineare Klassifikatoren in unterschiedlichen Testkomponenten unterschiedlich ausgewertet werden (z.B. Winkelähnlichkeit, inneres Prdukt etc.). Die Unterscheidung zwischen binären Klassifikatoren (*BinaryClassification*….) und Klassifikatoren, welche Mehrfachzuordnungen erlauben (*ClassificationHypothesis….),* ermöglicht eine effiziente Kombination der beiden. So können z.B. Mehrfachzuordnungen auf binäre Zuordnungen zurück geführt werden (siehe auch [Sebastiani 2002]), wie dies über die Klasse *OneVsAllLinearMachine* im KnowMiner-Klassifikationsframework erfolgt.

Die Evaluierung erfolgt über verschiedene Evaluierungskomponenten, welche Maßzahlen wie Precision und Recall ermitteln. Die Komponenten werden wiederum über ein Interface gekapselt, um verschiedene

Implementierungen zu ermöglichen. So ist es mitunter bei großen Datenmengen nötig, Klassifikationsergebnisse in einer Datenbank abzulegen, um diese auswertbar zu machen.

Mit dieser Struktur ist die Anwendung und Erweiterung der Klassifikation flexibel ausgelegt. Über das hierarchische Modell ist z.B. die Textklassifikation auf strukturierten Ablagesystematiken möglich, während die einzelnen Hypothesen natürlich auch direkt in anderen KnowMiner Komponenten anwendbar sind.

8.4.6 Information Retrieval

Information Retrieval stellt den letzten Aspekt des hier vorgestellten KnowMiner-Frameworks dar. Während IR nicht direkt zu den Methoden der Wissenserschließung zählt, so stellen Methoden aus diesem Gebiet doch Grundbausteine im Umgang mit großen Textdatenmengen dar. Da heutzutage im Unternehmenskontext schon meist Retrievallösungen im Einsatz sind, bietet das KnowMiner-Framework hier einen hohen Abstraktionsgrad zur Anbindung unterschiedlicher Retrievalsysteme an. Standardmäßig erfolgt im KnowMiner-Framework die Anwendung der Open Source Suchmaschine Lucene (siehe [Hatcher et. al. 2004]).

Für die Anbindung unterschiedlicher Suchmaschinen wurde die in Abbildung 8-25 dargestellte Interface-Struktur entwickelt. Grundlegend kann in Suchmaschinen zwischen dem Ablauf der Indizierung und dem Ablauf der Suche unterschieden werden.

Die Indizierung zielt auf die Analyse von Inhalten und die Überführung dieser in für das Retrieval effiziente Datenstrukturen wie z.B. einen invertierten Index ab (siehe Abschnitt 4.2). In der KnowMiner Interface-Struktur wird dies über das *SearchIndexFilter* Interface abgewickelt, welches Entitäten analysiert und entsprechend der jeweiligen Suchmaschine die Indizierung vornimmt. Die zu indizierenden Felder sind über das *SearchField* Interface definiert.

Die Kapselung der unterschiedlichen Suchmaschinen erfolgt über das *SearchEngineBackend* Interface, welches mittels eines Factory Design Patterns über die Klasse *SearchEngineFactory* instanzierbar ist. In Abbildung 8-25 ist die abstrakte Implementierung des *SearchEngineBackend* Interface über die Klasse *AbstractSearchEngineBackend* angedeutet. Für die restlichen abgebildeten Interfaces existieren ebenfalls abstrakte Implementierungen, welche gemeinsame Funktionalitäten kapseln, hier aber aus Gründen der Übersichtlichkeit nicht angeführt sind. Die Implementierungen des *SearchEngineBackend* Interface sind für die Instanzierung der Indizierungskomponenten (*SearchIndexFilter*) und der Instanzierung der Suchkomponenten zuständig.

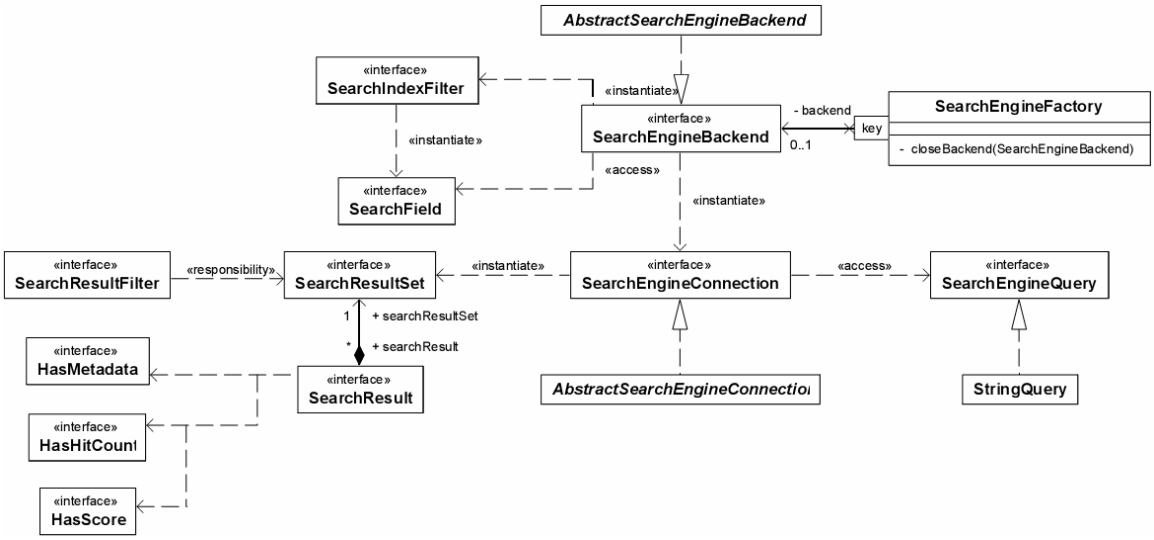


Abbildung 8-25: Interface Struktur der KnowMiner Retrieval Funktionalität

Die Kapselung der Suchkomponenten erfolgt über das *SearchEngineConnection* Interface, wobei jede Suchanfrage ihre eigene *SearchEngineConnnection* Klasse als Dreh- und Angelpunkt für die Manipulation der Suche instanziert. Die Instanzierung des *SearchEngineConnection* Interface erfolgt dabei über die von *SearchEngineBackend* abgeleiteten Klassen. Pro Verbindung kann immer nur eine Suche abgesetzt bzw. ein Suchergebnis verwaltet werden. Das Interface *SearchEngineQuery* kapselt dabei die Syntax der Suchanfrage. Neben der Berücksichtigung verschiedener Query-Sprachen (z.B. Boolesche Anfragen, Anfragen aus dem Vector-Space-Model) ist somit auch die Kapselung unterschiedlicher Anfragearten wie z.B. einer Query-by-Example, falls dies von der Suchmaschine unterstützt wird, möglich.

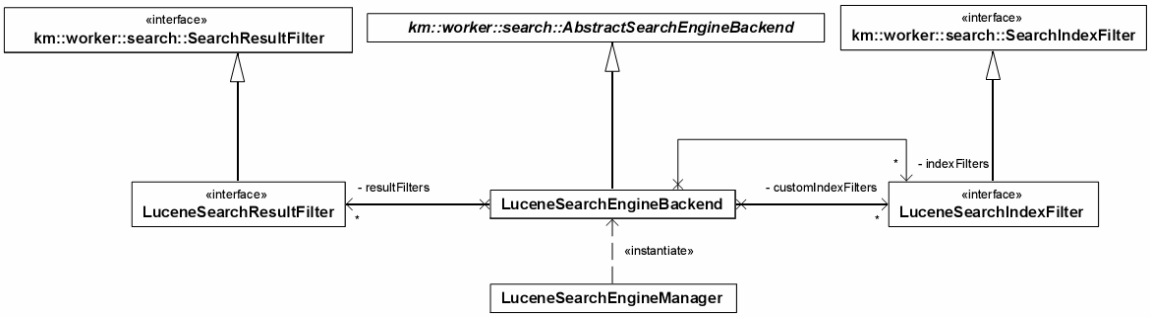


Abbildung 8-26: UML Diagramm zur Implementierung der Suchmaschine Lucene auf Basis der oben dargestellten Interface Struktur.

Die über eine *SearchEngineConnection* abgesetzte Suchanfrage liefert eine Menge an Treffern zurück. Die Verwaltung dieser Treffermenge obliegt den vom Interface *SearchResultSet* abgeleiteten Klassen. Die bereitgestellte Funktionalität hängt dabei natürlich von der darunter liegenden Suchmaschine ab. Charakteristisch für die Ergebnismenge ist, dass sie eine geordnete Menge an Suchergebnissen kapselt und die einzelnen Treffer einer Suche über das Interface *SearchResult* zu Verfügung stellt. SearchResult benutzt dabei Marker Interfaces, um die Eigenschaften eines Treffers zu verwalten. So gibt z.B. das Interface *hasMetadata* an, dass der Treffer über auslesbare Metadaten verfügt. Suchergebnisse können auch nach weiteren Kriterien, wie

z.B. nach vom Benutzer spezifizierten Metadaten, gefiltert und sortiert werden, Funktionalitäten welche über das Interface *SearchResultFilter* gekapselt werden.

Abbildung 8-26 zeigt die Implementierung der Interfaces *SearchEngineBackend*, *SearchIndexFilter* und *SearchResultFilter* für die Lucene Suchmaschine. Um das Diagramm übersichtlich zu halten werden die Implementierungen für *SearchResult* und *SearchQuery* sowie der dazu gehörigen Interfaces nicht angegeben.

Aktuell sind im KnowMiner-Framework die Suchmaschine Lucene ([Hatcher et. al. 2004]) und das TERRIER [43] Framework der Universität Glasgow implementiert. TERRIER arbeitet auf Basis probabilistischer Retrieval Methoden (siehe Abschnitt 4.2) und ist auf die Analyse und Untersuchung verschiedener Ranking Methoden auf großen Datenmengen ausgelegt. Lucene hingegen arbeitet auf Basis des Vektorraummodells und eignet sich hervorragend für den praktischen Einsatz. Lucene stellt eine stabile und leistungsfähige Retrievalplattform dar, welche auch auf die Suche in multimedialen Datenbeständen erweiterbar ist[44].

In der ersten Version des KnowMiner-Frameworks, welche im Projekt OnAIR (siehe Abschnitt 6.2.1) eingesetzt wurde, erfolgte zusätzlich die Anbindung von Retrievalmethoden auf Basis der ORACLE 8i Datenbank. Die dazu implementierten Klassen und Interfaces deckten jedoch nur einen Teil der Abstraktionsmechanismen der jetzigen Retrieval-Komponente ab und wurden aus Kompatibilitätsgründen nicht in die aktuelle KnowMiner-Version übernommen. Dennoch konnten Erkenntnisse für die aktuelle Architektur, im Speziellen im Bereich der Aufbereitung der Suchergebnisse und in der Verwaltung von Verbindungen zur Suchmaschine, abgeleitet werden.

Die hier vorgestellte Architektur zur Anbindung von Suchmaschinen erwies sich für die Anwendung als vorteilhaft und entwickelte sich, wie bereits erwähnt, aus den verschiedenen durchgeführten Projekten (siehe Abschnitt 6). Die Anwendbarkeit der Architektur zeigt sich auch in den in Kapitel 9 dargestellten Projekten, in denen die Architektur zum Einsatz kam.

Die vorgestellte Retrieval-Architektur diente auch zur Entwicklung des Assoziationsindex, der zu gegebenen Termen im Kontext weiterführende Terme findet. Der Assoziationsindex ermöglicht somit z.B. die Suche nach ähnlichen Wörtern zu einer Suchanfrage. Ausgehend von einer Suchanfrage können weiterführende Begriffe automatisch aus dem Datenbestand extrahiert und als zusätzliche Suchbegriffe oder aber zur Erweiterung von synonymen Begriffen in einem Thesaurus (siehe Projekt XTrend) verwendet werden.

Der Assoziationsindex ist auf einem invertierten Vektorraummodell aufgebaut, in dem Merkmale bzw. Terme durch die Objekte, in denen sie vorkommen, repräsentiert sind. Jedem Term wird also ein Objektvektor zugeordnet. Ausgehend von der Objekt-Merkmalsmatrix $\boldsymbol{O}$, in der jede Zeile den Merkmalsvektor zu einem Objekt beinhaltet, erhält man durch einfaches Transponieren die Merkmals-Objektmatrix $\boldsymbol{M}=\boldsymbol{O}^T$, in der sich nun in jeder Zeile der Objektvektor eines Merkmals befindet. Auch in der Merkmals-Objektmatrix sind die Anwendung von Gewichtungsschemen und Techniken zur Merkmalsselektion möglich, sogar erwünscht. Die Suche von ähnlichen Termen innerhalb der Merkmals-Objektmatrix erfolgt nun durch eine Ähnlichkeitssuche innerhalb der Objektvektoren der Merkmale, wobei bekannte Ähnlichkeitsmaße wie z.B. das Kosinusmaß Anwendung finden können.

---

[43] http://ir.dcs.gla.ac.uk/terrier/

[44] http://www.semanticmetadata.net/

Implementiert ist der Assoziationsindex über das KnowMiner Suchframework und macht sich dabei Funktionalitäten von Lucene zu Nutze. Die Erzeugung des Index erfolgt über den normalen Suchindex, welcher als Datenstruktur für das Finden von Entitäten dient. In diesem Index referenziert ja jedes Merkmal auf die Entitäten in denen er enthalten ist. Zur Erstellung des Assoziationsindex wird diese Merkmalsliste traversiert und jedes Merkmal als Entität aufgefasst, welche als Inhalt die URIs der Entitäten des Originalindex beinhaltet. Die so erzeugte Entität wird wiederum mittels des Suchframeworks indiziert, woraus ein invertierter Index entsteht, in welchem jede Entität auf eine Liste von Merkmalen verweist.

Die Suche erfolgt nun in zwei Schritten. Der erste Schritt startet mit einer normalen Suche im Originalindex, um die relevantesten Dokumente für diese Suche zu ermitteln. Die gefundenen Entitäten und deren URIs können als Entität des Assoziationsindex aufgefasst werden und als Suchanfrage für eine Query-by-Example im Assoziationsindex angewendet werden. Der Assoziationsindex liefert nun die zu der Originalsuche ähnlichen Merkmale und damit auch weiterführende Suchbegriffe zur besseren Formulierung des Informationsbedürfnisses.

Der Assoziationsindex wurde im Projekt XTrend und MISTRAL in dieser Form angewendet und zeigt die flexiblen Einsatzmöglichkeiten des KnowMiner Retrieval Frameworks.

8.5 Die KnowMiner Service-orientierte Architektur

Die bisher beschriebene Kernfunktionalität des KnowMiner-Frameworks kann direkt in Java Programmen verwendet werden. Um jedoch eine möglichst einfache Anwendung, auch in verteilten Systemen, zu gewährleisten, erfolgt die Kapselung der KnowMiner-Funktionalität in Form von Knowledge Discovery Diensten.

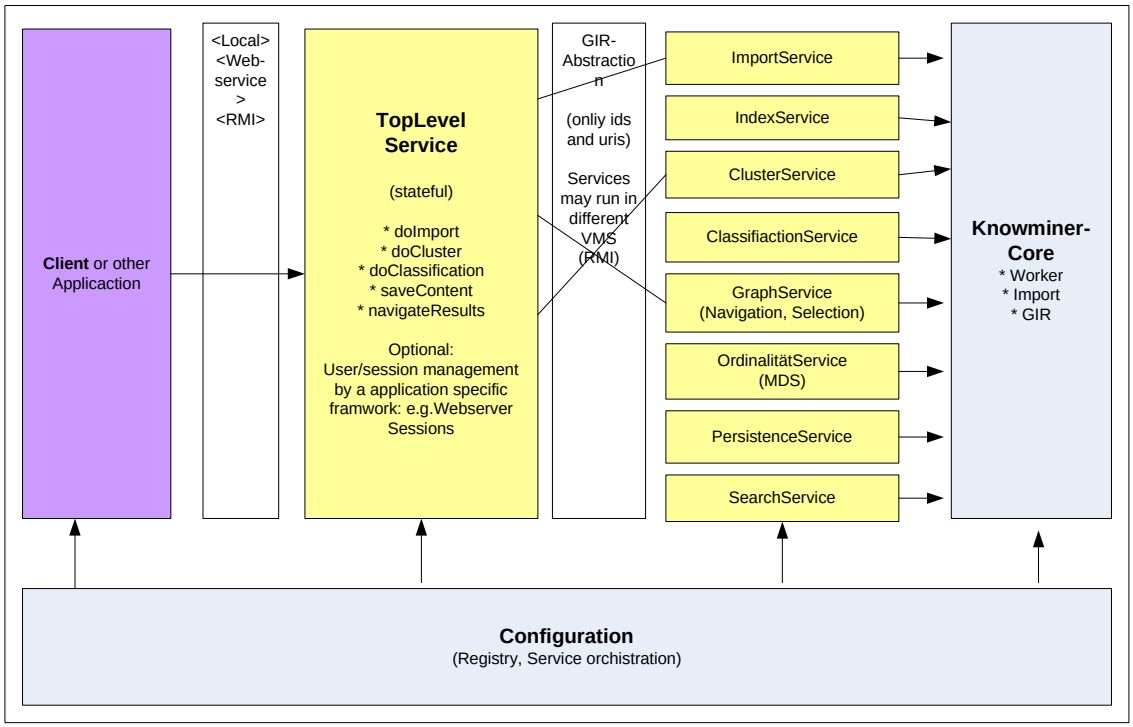


Abbildung 8-27: KnowMiner Service Architektur. Am KnowMiner Core setzen die Basis-Services für die einzelnen Funktionalitäten auf, welche im als Web-Service ansprechbaren TopLevelService kombiniert werden

Das KnowMiner-Framework stellt daher unter Anwendung der oben beschriebenen Kernfunktionalitäten Basis-Services bereit. Diese sind in sich gekapselte, funktional voneinander unabhängige Blöcke die unterschiedliche Funktionen bereitstellen. Das Funktionsspektrum ist dabei an die in Abschnitt 7.6 erarbeiteten Komponentenklassen angelehnt, wobei nicht alle Komponentenklassen implementiert sind.

Unter Verwendung der Basis-Services stellt das KnowMiner-Framework ein als Web-Service ansprechbares Top-Level-Service bereit, in welchem die einzelnen Basis-Services miteinander kombiniert sind. Das Top-Level-Service abstrahiert dabei die Funktionalität der Basis-Dienste und kombiniert verschiedene, häufig benötigte Operationen. Beispielsweise erfolgt mit dem Import von Dateien auch meist das Indizieren derselben; Funktionen, die in zwei unterschiedlichen Basis-Services realisiert sind. Im Top-Level-Service erfolgt die Kombination der beiden Services in einer Funktion. Abbildung 8-27 zeigt einen Überblick über die Service Architektur im KnowMiner-Framework.

Der Rest dieses Abschnittes gibt einen Überblick über die im KnowMiner abgebildeten Basis und Assembly Services, wobei auf die verschiedenen Komponentenklassen aus Abschnitt 7.6 und die den Services zugrunde liegenden Worker aus Abschnitt 8.4 referenziert wird. Der Abschnitt schließt mit der Beschreibung der Implementierungsdetails der Service Architektur und damit verbunden der Ansteuerung der Dienste sowie deren Orchestrierungsmöglichkeiten. Teile der Texte sind dabei der Publikation [Klieber et. al. 2006] entnommen, an der der Autor dieser Arbeit mitgewirkt hat.

8.5.1 Basis und Assembly Services

Funktional sind die Basis-Services des KnowMiner-Frameworks in Verwaltungs- und Bearbeitungs-Services untergliedert. Verwaltungs-Services ermöglichen dabei den Zugriff auf Entitäten und die Erzeugung von Informationsräumen. Dazu zählen im aktuellen KnowMiner-Framework das *DataAccessService* und das *RepositoryService*. Das *DataAccessService* bietet dabei Funktionalität zum Auslesen und Manipulieren des Informationsraumes an. So können z.B. der GIR Graph traversiert und Elemente ausgelesen werden. Verschiedene Repositories wie z.B. Datenbanken können im *RepositoryService* gekapselt werden. Damit ist es möglich, den Originalinhalt von Entitäten, wie z.B. HTML Seiten oder PDF Dokumente, abzulegen und für den Anwender verfügbar zu halten. Implementiert ist momentan ein Persistierungs-Service auf Basis des Open Source Tools EH Cache[45], welches in diesem Bereich rudimentäre Funktionalität anbietet.

Zentral für die Service-orientierte Architektur sind die Bearbeitungs-Services. Diese kapseln die unterschiedlichen Algorithmen und deren Manipulation im Informationsraum. Sie bieten unter anderem die Möglichkeit, die aktuelle Bearbeitung abzubrechen, anzuhalten oder fortzusetzen (siehe Abschnitt 8.5.2). Des Weiteren können über Bearbeitungs-Services Statusinformationen wie verbleibende Verarbeitungszeit, Verarbeitungsgeschwindigkeit etc. abgefragt werden. Um einen Eindruck über die unterschiedlichen Bearbeitungs-Services und die Funktionalität des KnowMiner-Frameworks zu geben, werden im Folgenden die wichtigsten Bearbeitungsdienste - inklusive deren Bezug zu den dargestellten Workern und Komponentenklassen - beschrieben.

- **Import Service:** Dieses Service importiert Daten von beliebigen Datenquellen in den Informationsraum unter Berücksichtigung unterschiedlicher Strukturierungsmerkmale.

[45] http://ehcache.sourceforge.net/

Komponentenklasse: Import, Struktur modifizierend
KnowMiner Core: ImportController

- **Information Extraktion Service:** Dieses Service extrahiert Annotationen aus einzelnen Informationsentitäten. Aus Textdokumenten werden Annotationen wie Personen, Orte und Organisationen extrahiert.
  Komponentenklasse: Annotation, Entitäten modifizierend
  KnowMiner Core: Content Processor

- **Vectorizing Service:** Das Vectorizing Service erzeugt die Merkmale einer Informationsentität aus dessen Annotationen.
  Komponentenklasse: Merkmalsgenerierung, Entitäten modifizierend
  KnowMiner Core: Content Processor

- **Feature Transform Service:** Das Feature Transform Service führt Merkmalsgewichtungen und -selektionen aus. Zum Feature Transform Service gehören unter anderem das Feature Normalisation, Feature Selection und Feature Weighting Service.
  K**omponentenklasse:** Merkmalsgewichtung und -selektion, Entitäten modifizierend
  KnowMiner Core: Merkmalsgewichtung

- **Feature Space Transform Service:** Das Feature Space Transform Service führt Merkmalstransformationen aus. Zum Feature Space Transform Service gehören unter anderem Funktionalitäten wie MDS und LSI, sowie einfache Kopierfunktionen die einen Merkmalsraum in einen anderen kopieren.
  K**omponentenklasse:** Merkmalstransformation, Entitäten modifizierend
  KnowMiner Core: LSI, MDS

- **Index Service:** Das Index Service erlaubt die Indizierung von selektierten Entitäten zur nachfolgenden Suche. Während dem Indizieren kann auch der zuvor beschriebene Assoziationsindex erstellt werden.
  Komponentenklasse: Retrieval (Indizierung) , nicht-modifizierend
  KnowMiner Core: Information Retrieval, *SearchEngineBackend*

- **Such-Service:** Das Such Service erlaubt das Absetzen von Anfragen und das Suchen in zuvor indizierten Entitätsbeständen.
  Komponentenklasse: Retrieval (Suche) , nicht-modifizierend
  KnowMiner Core: Information Retrieval, *SearchEngineBackend*

- **Clustering-Service:** Dieses Service identifiziert Gruppen von ähnlichen Objekten. Zurzeit erfolgt die Anwendung von Standard-Algorithmen wie K-means und Hierarchical Agglomerative Clustering. Zur Steigerung der Leistungsfähigkeit werden diese miteinander kombiniert.
  Komponentenklasse: Strukturerzeugung, Struktur modifizierend
  KnowMiner Core: Clustering

- **Klassifikations-Service**: Mittels dieses Services erfolgt die Zuordnung einzelner Informationsentitäten zu Klassen bzw. Konzepten. Aktuell wird auf Service Ebene nur die Metadatenklassifikation, d.h. die Zuordnung von einem Metadatum zu einer Entität, unterstützt. Strukturklassifikation wird zwar im KnowMiner Core unterstützt, aber noch nicht als Service

angeboten. .

Komponentenklasse: Metadatenklassifikation, Entitäten modifizierend

KnowMiner Core: Klassifikation, *DocumentBasedClassifier*

Assembly Services kommen zum Einsatz um verschiedene Basis-Services zu einem neuen Service zu kombinieren um damit häufig benötigte Funktionalitäten einfach bereitzustellen. Ein solches Assembly Service ist zum Beispiel ein Metadaten Bereitstellungs-Service, das auf Benutzerebene eine einfache und einheitliche Handhabung der Annotationen, Merkmale und Metadaten im Informationsraum erlaubt. Der Benutzer gibt als Input die gewünschten Arten von Metadaten wie Personen, Schlüsselwörter oder Dublin Core an, und das Metadaten Bereitstellungs-Service kümmert sich um die richtige Ansteuerung der entsprechenden Basis-Services, um diese Metadaten zu erzeugen und bereitzustellen. Assembly Services können für verschiedene Anwendungsfälle dezidiert aus den Basis-Services entworfen werden.

Aktuell bietet das KnowMiner-Framework zwei Assembly Services an:

- MISTRAL Top-Level-Service: Hier erfolgt die Verdrahtung aller Basis-Services im oben erwähnten Top-Level-Service, welches in MISTRAL (siehe Abschnitt 9.2) verwendet wird.
- APOSDLE Ontology Engineering Support Service: Dieses Service kombiniert lediglich einen Teil der Basis-Services und stellt Funktionalität zur Unterstützung eines Ontology Engineering Prozesses dar. Dazu gehören unter anderem die Extraktion von Schlüsselwörtern für Textdokumente und die Zuordnung von Dokumenten zu Konzepten einer Ontologie. Es setzt sich dabei aus dem Import, Vectorize, Feature Transform, Clustering und Classification Services zusammen.

Die vorgestellten Assembly Services stellen sehr einfache Kompositionen aus Basis-Services dar. Nichtsdestotrotz bieten sie anwendungsspezifische, einfach erstellbare Funktionalität an. Nach diesem Überblick über die im KnowMiner-Framework enthaltenen Services sollen nun im nächsten Abschnitt die Implementierungsdetails beleuchtet werden.

8.5.2 Implementierungsdetails

Die Implementierung von Services ergibt sich aus zwei verschiedenen Sichtweisen, der Datensicht und der Prozesssicht. Die Datensicht ergibt sich aus der Betrachtung, welche Entitäten ein Service bearbeitet und wie diese dem Service bereitgestellt werden. Die Prozesssicht behandelt die Fragestellung, wie die Ausführung der Services erfolgt. Speziell im Multi Threading Betrieb muss die Möglichkeit des Startens, Stoppens und Suspendens von Services sowie die Möglichkeit zum Abfragen des aktuellen Status gewährleistet sein.

Die in Abbildung 8-28 dargestellte Interface Struktur berücksichtigt sowohl den Processing- als auch den Datenaspekt und je nach Notwendigkeit können Services alle oder nur Teilaspekte implementieren. Jedes KnowMiner Service leitet sich vom Interface *KnowMinerService* ab, welches Methoden zum Initialisieren und zum Freigeben eines Service beinhaltet.

Die Datensicht kapselt das *GIREnabledService* und alle von ihm abgeleiteten Services. Das *GIREnabledService* stellt Funktionalität zum Setzen und Auslesen des *GIRWorkspace* Objektes bereit, welches Referenzen zum *InformationSpace* und damit zu allen Entitäten und Relationen enthält. Dieser Workspace definiert auch den Arbeitsraum eines Service. Sollen Services auf den gleichen Informationsentitäten arbeiten, so muss der Workspace von der Orchestrierung den Services bereitgestellt werden. Der Workspace kann also als

Ausschnitt aus dem Informationsraum verstanden werden, der von Service zu Service zur Bearbeitung weiter gereicht wird.

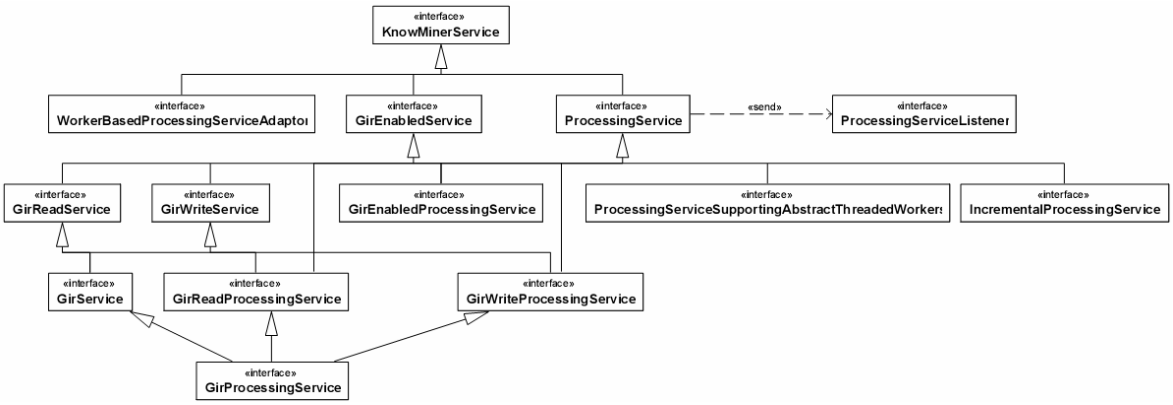


Abbildung 8-28: Daten und Processing Interfaces der Service Architektur

Bei der Datensicht wird auch zwischen den *GIRReadService* und den *GIRWriteService* Interfaces unterschieden, die sich vom *GIREnabledService* ableiten. Wie der Name schon sagt, lesen Klassen, die sich vom *GIRReadService* ableiten aus der GIR, schreiben aber nicht zurück; das Schreiben erfolgt über vom *GIRWriteService* abgeleitete Klassen, die jedoch nicht lesen. Das Import Service ist beispielsweise ein typisches *GIRWriteService*. Die Unterscheidung macht vor allem bei dynamischen Orchestrierungen Sinn, da z.B. ein *GIRWriteService* nicht auf Daten eines Vorgängerservice zugreifen kann. Im Allgemeinen implementieren Services jedoch sowohl den Write als auch den Read Aspekt, was sie zu einem vollständigen *GIRService* macht, welches lediglich als Marker Interface verstanden werden kann.

Services, deren Ablauf steuerbar und kontrollierbar ist, müssen das *ProcessingService* Interface implementieren. Die Ansteuerung der von diesem Interface abgeleiteten Services kann sowohl synchron als auch asynchron erfolgen. *ProcessingServices* implementieren ein Finit State Machine Konzept, welches im nachfolgende Abschnitt näher erläutert wird. Über verschiedene, vom *ProcessingService* bereitgestellte Methoden kann der Status des Service abgerufen sowie das Prozessieren gestartet, gestoppt und in Wartestellung gelegt werden. In Form eines *ElementSet* erfolgt die Übergabe der zu bearbeitenden Daten aus dem *GIRWorkspace* an das *ProcessingService*, welches damit seine Aufgabe (engl. Task laut Definition im Java Interface) kennt. Die Prozessierungsfunktionalität knüpft stark an das Worker Konzept, nur eben auf Serviceebene, an, unterscheidet sich jedoch durch die Möglichkeit Listener Klassen zu registrieren (*ProcessingServiceListener*) sowie durch ein restriktiveres State Konzept. Während Worker relativ frei in der Wahl der verschiedenen Zustände sein können, sind Processing Services auf 6 Zustände und definierten Übergängen zwischen den Zuständen begrenzt. Dadurch ist die Semantik für die Abarbeitung von KnowMiner-Services für den Klienten klar definiert. Zur einfachen Kombination von Workern und Processing Services bietet das *ProcessingServiceSupportingAbstractThreadedWorkers* Interface Methoden an. Klassen, die sich von der abstrakten Implementierung ableiten können einfach eine Menge von Workern an die entsprechenden Funktionen übergeben, welche nacheinander auf dem gleichen *ElementSet* (entspricht der abzuarbeitenden Datenmenge eines *ProcessingService*) abgearbeitet werden. Über diesen Mechanismus können bei gegebenen Worker einfach und schnell Services realisiert werden. Als letztes ist noch das *IncrementalProcessingService* anzuführen, welches

ein leicht modifiziertes Zustandsmodell zur Abarbeitung von inkrementellen Aufgaben besitzt (siehe auch nächster Abschnitt).

Um sowohl Daten- als auch Prozesssicht in einem Service abbilden zu können kombiniert das *GIRProcessingService* und deren Super Interfaces *GIRReadProcessingService* und *GIRWriteProcessingService* die definierten Prozessierungs- und Datenverwaltungsmethoden. Für jedes der in Abbildung 8-28 dargestellten Interfaces und Kombinationen existieren abstrakte Implementierungen, von denen für die Implementierung eines Service abgeleitet werden kann. Da Java jedoch auf Klassenebene keine multiple Vererbung ermöglicht, muss diese mittels Delegation aufgelöst werden, wodurch sich eine äußerst komplexe Klassenstruktur ergibt. Aus Gründen der Übersichtlichkeit und da diese Struktur für das Verständnis der Architektur nicht benötigt wird, wurde hier auf eine Darstellung verzichtet.

Nach dem Überblick über die Strukturierung der Service Architektur gehen die nächsten beiden Abschnitte auf Details der Processingservices und der Daten Services ein.

8.5.2.1 Abstraktion der Prozessaspekte

Processingservices dienen zur Bereitstellung von Kontrollmöglichkeiten über den Ablauf und aktuellen Status eines Service. Um dies in einer semantisch eindeutigen Art und Weise zu ermöglichen, funktionieren alle Processing Services auf einem klar spezifizierten Finite State Machine Konzept, welches Übergänge zwischen Zuständen des Services über Aufruf bestimmter Methoden ermöglicht.

Abbildung 8-29 zeigt das Finite State Modell eines Processing Services in Form eines UML-Aktivitätsdiagramms. Nach dem Erzeugen des Service nimmt die Finite State Machine (FSM) den Zustand *notInitalised* ein, welcher auch beim Setzen der Parameter und des Tasks als *ElementSet* nicht verlassen wird. Über das Initialisieren des Processing Service mittels der *initialise* Methode wechselt der Zustand der FSM in *readyToBegin,* und das Service ist bereit mit der Abarbeitung anzufangen. Über die Methode *process(amount)* kann das Service zur Abarbeitung einer bestimmten Anzahl von Schritten oder der gesamten Aufgabe veranlasst werden und wechselt dabei in den Zustand *processing.*

Im Zustand *processing* angelangt, arbeiten die Services die gegebene Aufgabe, meist die einzelnen Entitäten eines *ElementSet's*, ab. Nach jedem Einzelschritt kann ein Service durch den Aufruf der Methoden *suspend* schlafen gelegt werden und stellt dabei die Abarbeitung ein. Der Zustand des Services wechselt nach *suspend.* Ein erneuter Aufruf von process weckt das Service wieder auf und lässt die Verarbeitung mit dem Zustand *processing* fortfahren. Das Service markiert das Ende der Abarbeitung durch den Wechsel in den Zustand *taskCompleted.* Hier trennen sich die Zustandsübergänge in inkrementelle und nicht inkrementelle Processing Services (siehe Abbildung 8-28) auf. Über das Setzen einer neuen Aufgabe und den *process* Aufruf können inkrementelle Services weitere Daten abarbeiten (im Status *process*), während dies bei normalen Processing Services nicht möglich ist.

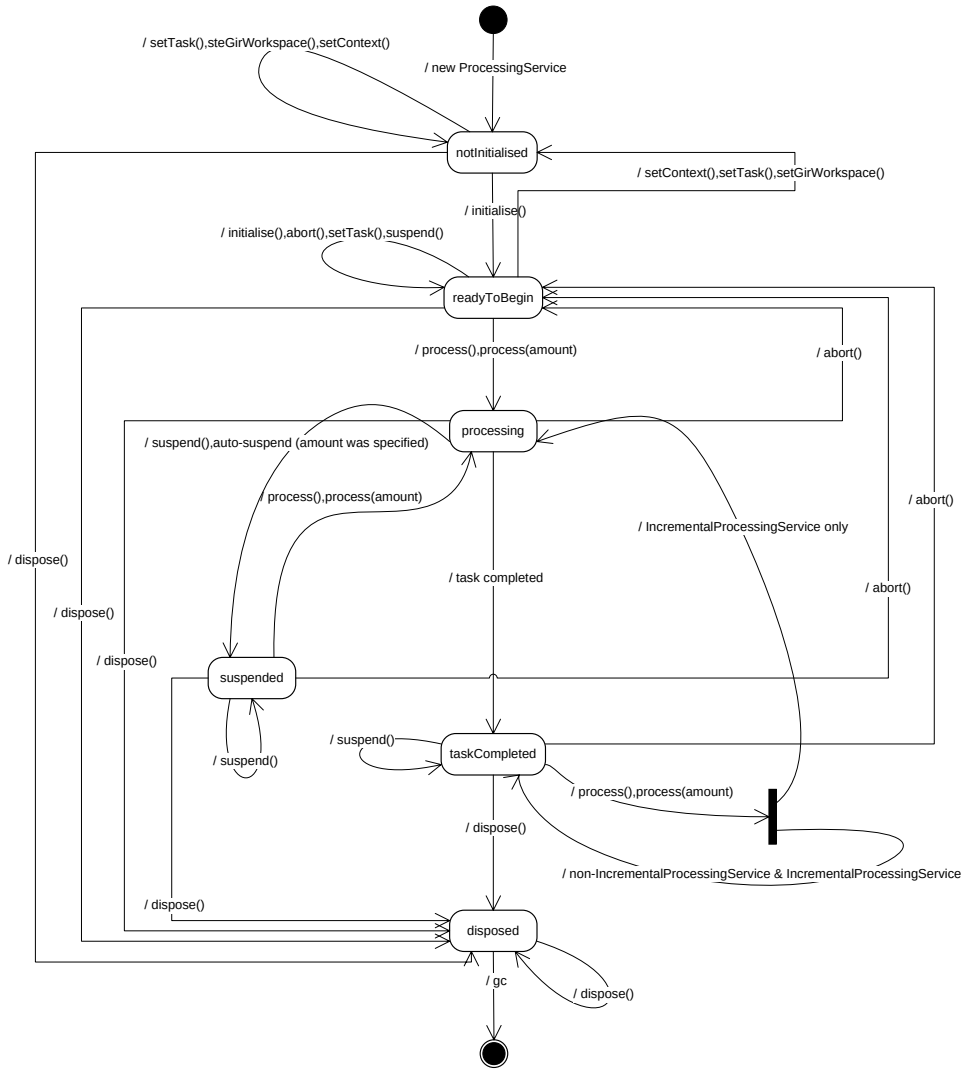


Abbildung 8-29: Finite State Modell von Processing und Incremental Processing Services (Quelle [Sabol, Klieber, Granitzer 2006])

Aus den Zuständen *processing*, *taskCompleted* und s*uspended* kann die aktuelle Abarbeitung abgebrochen und das Service in den Zustand *readyToBegin* zurückgesetzt werden. Im *readyToBegin* Zustand ist wiederum das Setzen von neuen Aufgaben möglich, wobei dabei das Service zurück in den Zustand *notInitalised* wechselt und wieder initialisiert werden muss.

Das Freigeben eines Service erfolgt durch Aufruf der Methoden *dispose,* wodurch das Service in den Zustand *dispose* wechselt. Die Methode *dispose* kann aus jedem beliebigen Zustand aus aufgerufen werden. Da Java kein einfaches Abbrechen von Threads ermöglicht und auch eine einfache Unterbrechung von Threads ungewollte Zustände des Informationsraumes hinterlassen würde, kann das Processing Service nur am Ende jedes Einzelschritts in den Zustand *dispose* überführt werden. Daraus leitet sich ab, dass die Granularität der Einzelschritte, welche in *process* abgearbeitet werden, die Zeit angibt, bis ein Service aus dem *process* Status in den *dispose* Status gelangt. Im Zustand *dispose* ist das Service, wie gesagt, beendet und wartet darauf, von der Java Virtual Machine aus dem Speicher entfernt zu werden (Grabage Collected, GC).

Die vorgestellte Finite State Machine legt zwar Restriktionen hinsichtlich der Implementierung von Services auf, indem der Ablauf von Algorithmen in Einzelschritten umgesetzt werden muss, birgt aber den Vorteil für den Anwender eines Dienstes, das Service immer in gleicher Art und Weise benutzen zu können, unabhängig von der Funktionalität. Zusätzlich bieten die Processing-Services auf eine einfache Art und Weise volle Kontrolle über den Ablauf eines Services.

8.5.2.2 Abstraktion der Datenaspekte

Neben den Steuerungsaspekten vereinheitlicht die KnowMiner Service Architektur auch die Datenaspekte von Services. Die Betrachtung der Dateneigenschaften von Services erfolgt dabei aus zwei Richtungen:

1. Die Bereitstellung der zu verarbeitenden Daten an ein Service oder der Transport von Daten zwischen Services

2. Die Spezifikation der vom Service benötigten und bereitgestellten Daten

Die Bereitstellung von Daten an ein Service erfolgt, wie bereits vorher erwähnt, über das *GIRWorkspace* und über ein *ElementSet* Objekt. Der *GIRWorkspace* definiert dabei den Informationsraum in dem ein Service arbeitet und liefert für die Aufgaben einen Startknoten im Informationsraum. An diesem Startknoten (auch Task-Knoten genannt) hängen Services neu erzeugte Entitäten und Strukturen an. Er kann als Startpunkt zur Traversierung der Daten eines Dienstes verstanden werden. Mit der Übergabe eines *GIRWorkspace* Objektes definiert sich also für ein Service sein Arbeitsbereich und durch Weitergabe des gleichen *GIRWorkspace* Objektes an verschiedene Services, egal ob parallel oder seriell, erfolgt die Weitergabe des Arbeitsbereichs innerhalb der Service-orientierten Architektur. So kann beispielsweise für einen Benutzer ein Arbeitsbereich erstellt und an verschiedene Services zur Bearbeitung weiter gereicht werden.

Neben dem Arbeitsbereich definiert das an das Service übergebene *ElementSet* (siehe Abschnitt 8.3) die eigentliche Aufgabe für ein Service im Arbeitsbereich. Hintergrund dafür ist, dass verschiedene Services zwar auf gleichen Arbeitsbereichen operieren können, jedoch in den Arbeitsbereichen nicht die gleichen Entitäten bearbeiten. Während beispielsweise ein Clustering der Dokument-Entitäten in einem Arbeitsbereich erfolgt, kann ein anderes Service an die im Arbeitsbereich enthaltenen Klassen-Entitäten neue Metadaten anhängen. Der Arbeitsbereich ist für beide Services der Gleiche, jedoch ändert sich die über den *ElementSet* definierte Aufgabe. Welche Entitäten ein Service abarbeiten soll, definiert also das *ElementSet* und kann dabei laut Architektur nur an *GIREnabled* Services übergeben werden. Die Art der benötigten Entitäten hängt natürlich von der Funktionalität des Service ab. Im Standardfall beinhaltet ein *ElementSet* dabei alle zu verarbeitenden Entitäten und Relationen. Da dies jedoch nicht immer praktikabel ist, wie z.B. beim Lernen einer Klassifikationshypothese, wozu manchmal hunderttausende von Entitäten benötigt werden, beinhaltet das *ElementSet* nur jene Entitäten, die vom Service zum weiteren Abruf der zusätzlich benötigten Entitäten notwendig sind. Im einfachsten Falle der hierarchischen Textklassifikation enthält das *ElementSet* lediglich die Klassenstruktur mit der Voraussetzung, dass an jeder Klasse die der Klasse zugeordneten Dokumente hängen. Das Service kann nun für jede Klasse die an der Klasse hängenden Dokumente zum Lernen der Klassifikationshypothese verwenden.

Welche Entitäten und welche Eigenschaften von Entitäten ein Service benötigt, spezifiziert das so genannte *GIRContext* Objekt. Die Spezifikation erfolgt dabei einerseits für die dem Service zur Verfügung zu stellenden

Entitäten (Precondition) und andererseits für die vom Service daraus errechneten Entitäten bzw. deren Eigenschaften. Beispielsweise benötigt das Vectorize Service annotierte Entitäten (Input *GIRContext*) und liefert Merkmalsvektoren zu Entitäten (Output *GIRContext*). Während in einfachen Fällen eine Spezifikation des Eingabe- und Ausgabekontexts leicht möglich ist, so erwies sich die Spezifikation in komplexen Fällen, beispielsweise bei der Definition der benötigten Struktur des Informationsraumes eines Services, als problematisch. Es erfolgte daher nur eine teilweise Umsetzung des *GIRContext* für Services (siehe dazu Kapitel 9). In der aktuellen KnowMiner Version unterstützt der *GIRContext* die Spezifikation der benötigten Merkmals- und Annotationstypen sowie die für das Locking von Entitäten wichtige Spezifikation, ob Lese- oder Schreiboperationen durchgeführt werden. Des Weiteren kapselt der GIRContext auch die Sichtweise auf Entitäten, unter denen die benötigten Typen verfügbar sind oder verfügbar gemacht werden. Ohne Berücksichtigung bleibt die benötigte Struktur des Informationsraumes. Nichtsdestotrotz stellt der *GIRContext* die Grundlage für die Verfeinerung der Pre- und Postconditions für Services dar und formt die Basis für weitere Arbeiten im Bereich der semantischen Beschreibung und Orchestrierung von KD-Diensten.

8.5.3 Grafisches Service Frontend

Bisher erfolgte die Betrachtung der verschiedenen bereitgestellten Dienste sowie deren Implementierung. Im KnowMiner-Framework enthalten ist auch ein grafisches Service Frontend, welches den Aufruf und das Abarbeiten von einzelnen Services ermöglicht. Das Frontend dient dabei zur Prüfung der Funktionalität durch den Entwickler, und nicht zur Benutzung durch den Endanwender. Trotzdem kann anhand dieses Frontends schön die Benutzung der einzelnen Dienste demonstriert werden.

Abbildung 8-30 zeigt das grafische Frontend. Die Liste links beschreibt dabei die zu Verfügung stehenden Services und der Klick auf ein Element zeigt den Dialog zum entsprechenden Service, von welchem es auch gestartet werden kann. Rechts unten ist die Statusanzeige zu den einzelnen Services und deren aktueller Zustand ersichtlich.

Bei den einzelnen Services nehmen das Selection und Visualization Service eine spezielle Rolle ein. Das Selection Service ermöglicht das Setzen der im nächsten Aufruf eines Processing Service zu verarbeitenden Entitäten und erlaubt damit eine benutzergesteuerte Selektion von Entitäten. Um den Selektionsaufwand zu reduzieren, erfolgt nach Abarbeitung jedes Services das Setzen der Selektion auf das Ergebnis der Verarbeitung. Dadurch stehen beispielsweise nach dem Import von Daten diese automatisch für den nächsten Verarbeitungsschritt wie z.B. die Indizierung zur Verfügung. Im Visualization Service können einzelne Entitäten, deren Struktur und Eigenschaften betrachtet werden.

Das Frontend ermöglicht zusätzlich die Orchestrierung der einzelnen Dienste über eine integrierte Java Script Engine, wodurch einfache Abläufe abbildbar sind. Ein solcher Ablauf ist z.B. die Suche nach Dokumenten unter Angabe eines Suchbegriffs. Das Search Service stellt die Treffer der Suche als Selektion bereit, worauf das Information Extraction Service benannte Entitäten aus den gefundenen Treffern extrahiert und das Vectorize Service Merkmalsvektoren generiert. Die Merkmalsvektoren können im nächsten Schritt zum Clustern der Ergebnismenge und im finalen Schritt zur Errechnung einer Landscape-Visualisierung verwendet werden.

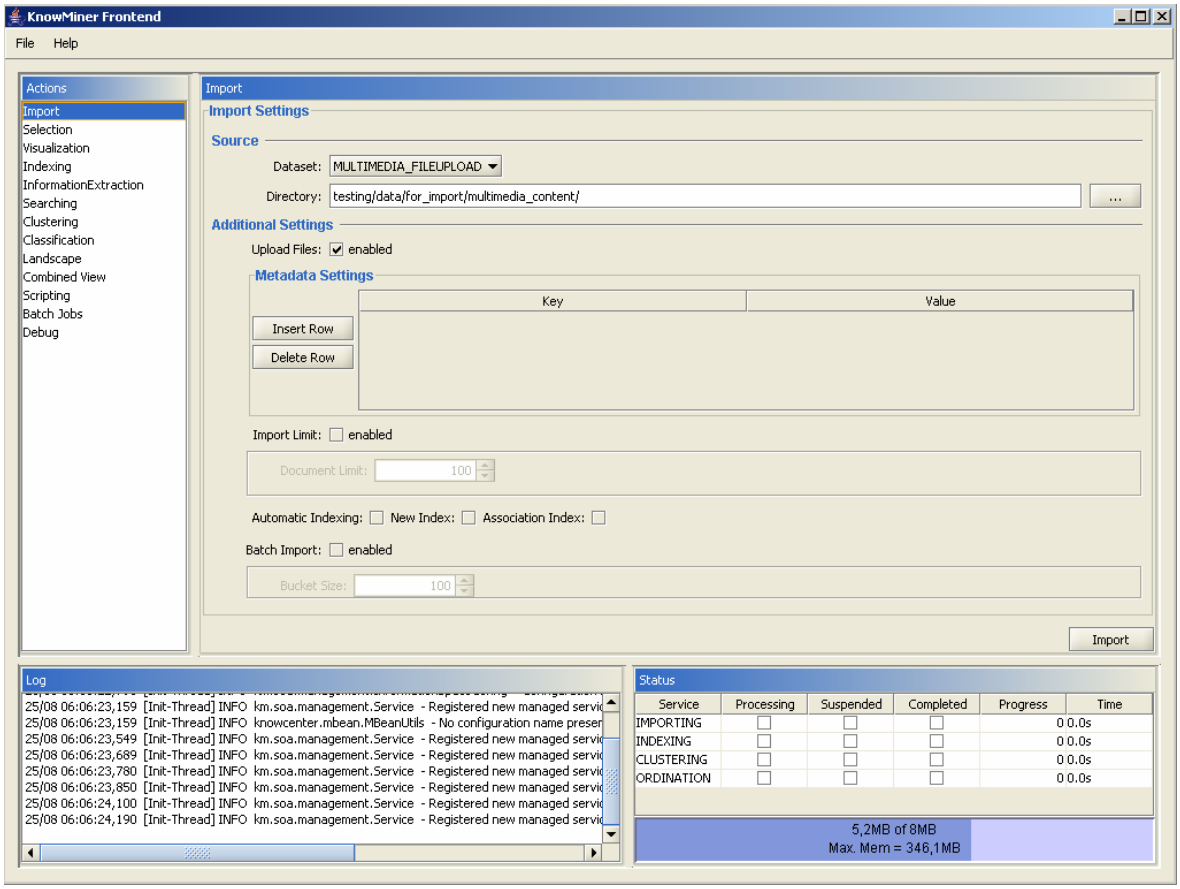


Abbildung 8-30: Grafisches Service Frontend des KnowMiner-Frameworks

Das Frontend integriert zusätzlich Konfigurations- und Debugging-Tools, deren Beschreibung im nächsten Abschnitt erfolgt.

8.6 Konfiguration, Monitoring und Debugging Tools

Zum Abschluss der Präsentation des KnowMiner-Frameworks soll noch kurz auf die unterstützenden Möglichkeiten zur Konfiguration, zum Monitoring der Worker und auf die im Zuge der Erstellung des Frameworks entwickelten Debugging Tools eingegangen werden. Diese Methoden stellen keine funktionalen Komponenten dar, verdeutlichen aber die Möglichkeiten zur Anwendungsentwicklung im Framework.

Möglichkeiten zur Konfiguration sind essentiell im Anbieten von KD-Diensten. Auf Serviceebene kann dabei zwischen den benötigten Konfigurationsparametern, wie z.B. Datenbankkonfiguration und Benutzerparameter, wie z.B. der Anzahl der gewünschten Cluster beim Clustering, unterschieden werden. Für beide Parameterarten kommt im KnowMiner-Framework das Spring-Framework[46] zum Einsatz. Das Spring-Framework ist ein Applikationsframework, welches neben Konfiguration auch Konzepte wie Dependency-Injection und Aspect-Oriented-Programming unterstützt. Im KnowMiner-Framework wird derzeit lediglich

[46] http://www.springframework.org/

die XML basierte Konfiguration verwendet, um Services und einzelne Algorithmen einfach konfigurierbar zu gestalten.

Neben der Konfiguration sind während der Entwicklung vor allem Tools zur Fehlersuche notwendig, um eine schnelle Entwicklung von Algorithmen zu ermöglichen. Neben dem oben erwähnten Service Frontend und einer breiten Testabdeckung in Form von JUnit Tests für die KnowMiner Funktionalität, beinhaltet das Framework zur Darstellung des aktuellen Prozessierungsstands von Workern den so genannten Worker Monitor (Abbildung 8-31). Der Worker Monitor stellt alle laufenden Worker dar und deren aktuellen Zustand sowie Statistiken über Anzahl der Aufrufe, Frequenz und Dauer eines Schrittes etc. Neben der reinen Darstellung verfügt der Worker Monitor auch über Möglichkeiten zum Starten, Stoppen und Suspenden von Workern, wodurch der Entwickler die Kontrolle über den Ablauf hat und somit unterschiedliche Szenarien testen kann.

Neben dem Worker Monitor als Tool zur Darstellung des Prozessierungsstatus des KnowMiner-Frameworks dient der GIR Debugger zur Darstellung der Daten im KnowMiner-Framework. Er bietet dabei die Möglichkeit an, die unterschiedlichen Startknoten im GIR Graph zu finden und einen davon als Startknoten zu setzen. Über eine einfache Graphenvisualisierung sind das Browsing und die Navigation im GIR Graph (siehe Abbildung 8-32) möglich und für jede Entität können die Eigenschaften dieser abgerufen und dargestellt werden. Zusätzlich besteht die Möglichkeit, direkt das RDF Backend über simple Tripelabfragen abzurufen.

Wie bereits erwähnt werden die vorgestellten Tools nicht zur Anwendung des KnowMiner-Frameworks benötigt. Sie erleichtern jedoch die Entwicklung mit dem Framework.

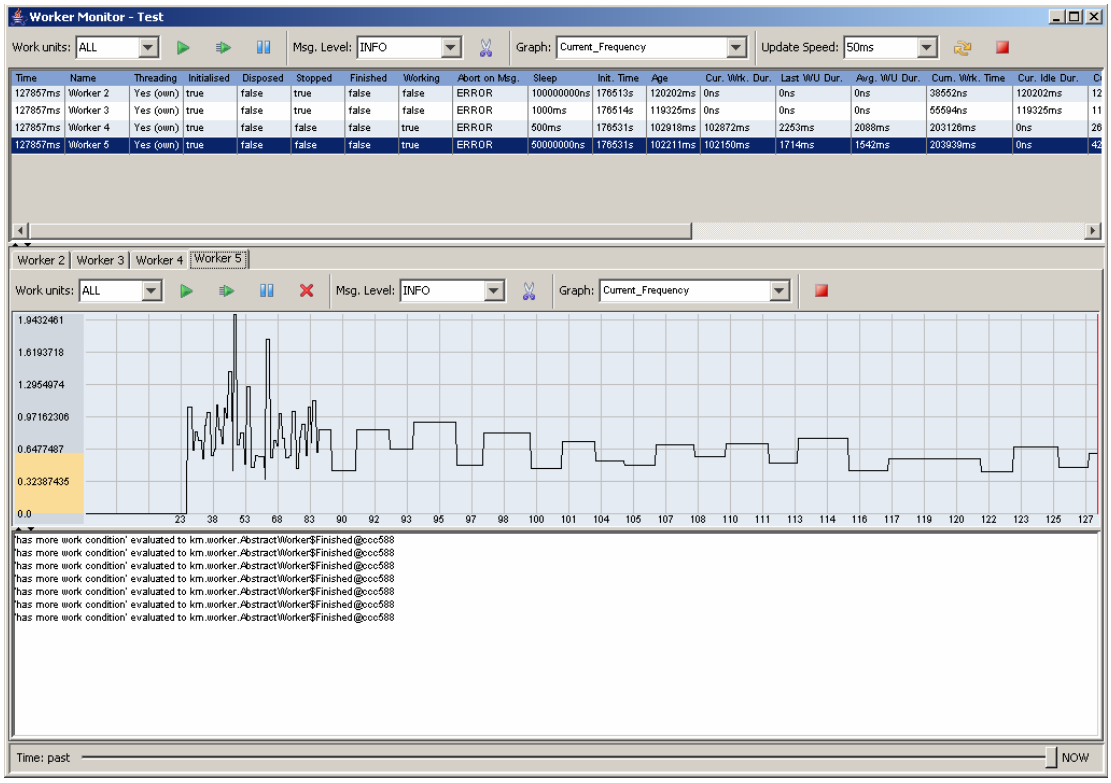


Abbildung 8-31: Screenshot des KnowMiner Worker Monitors

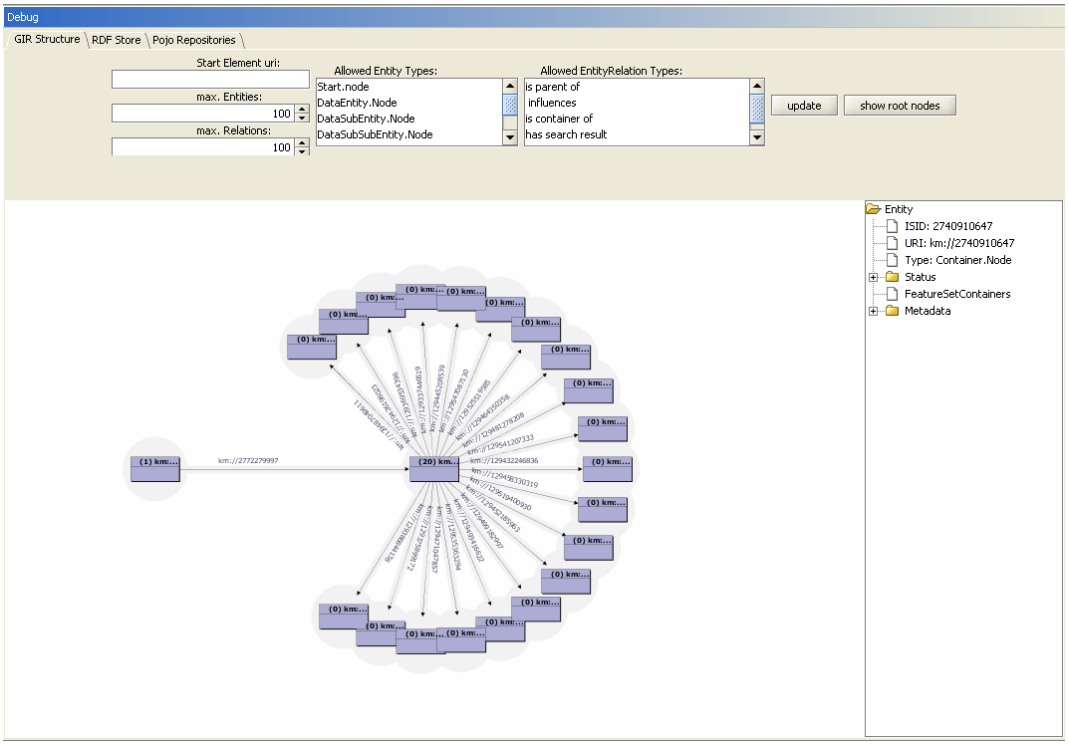


Abbildung 8-32: GIR Debugging Tool. Darstellung des GIR Graphen

8.7 Zusammenfassung

Im vorliegenden Kapitel wurde das KnowMiner-Framework vorgestellt, beginnend mit einem Systemüberblick mit nachfolgender Beschreibung der Kernkomponenten bis hin zur - auf den Kernkomponenten basierenden - Service-orientierten Architektur. Die Umsetzung des Frameworks behandelt dabei die in den vorigen Kapiteln beschriebenen Anforderungen und setzt auf dem in Kapitel 7 entwickelten Grundlagen auf.

Betrachtet man das vorgestellte Framework, so können die im Folgenden dargestellten Anforderungen (fett gekennzeichnet) aus den Kapiteln 2-5 wieder gefunden werden:

- Die Funktionsblöcke decken die Unterstützung zum

  a. **Erfassen und Speichern von Information**

  b. **Analysieren von Information und zur Extraktion von Zusammenhängen**

  c. **Suchen, Finden und Darstellen von Information**

  in Bezug auf Text und Text-Metadaten ab und erfüllen somit die Anforderungen in Kapitel 2.

- Die vorgestellte Repräsentationsform von Informationsräumen, Entitäten und Eigenschaften von Entitäten stellen **Abstraktionen zur Repräsentation verschiedener Zusammenhänge zwischen Objekten** und **flexible Datenstrukturen zur Abbildung von Merkmalen** bereit. Sie ermöglicht auch die **Abstraktion von Ähnlichkeiten und Operationen auf Merkmalen** und die **Trennung zwischen Repräsentation der Daten und Algorithmen**, wodurch wiederum die **Wiederverwertbarkeit von Algorithmen** erhöht wird.

- Durch Anbindung eines RDF Backends ist die Abbildung von heterogenen Informationsräumen mittels **semantischer Beschreibungssprachen berücksichtigt,** und das Typsystem für Eigenschaften von Entitäten ermöglicht die **Beschreibung der unterschiedlichen Merkmals- und Annotationstypen**

- Aus Sicht der Prozessierung bietet das Worker Konzept eine Umgebung zur **gleichzeitigen Abarbeitung unterschiedlicher Algorithmen,** und durch die oben beschriebenen Locking Mechanismen erlaubt das KnowMiner-Framework **konkurrente Zugriffe auf den Informationsraum.**

- Die vorgestellten Funktionalitäten **decken einen Großteil der in Kapitel 4 und 5 vorgestellten Algorithmen** ab und ermöglichen damit die in **Kapitel 2 erarbeiteten Anwendungsszenarien.** Durch die Integration der **Open Source Software MTJ und GATE** deckt das KnowMiner-Framework ein breites Funktionsspektrum ab.

- Der Content Processor und die vorgestellte Importarchitektur ermöglichen die **Anbindung unterschiedlicher Informationsarten** und die **Überführung unterschiedlicher Datenquellen in**

Standards der Wissensrepräsentation, im Speziellen nach RDF**.** Die im Content Processor enthaltene Pipe-Struktur erlaubt die Erzeugung **flexibler und austauschbarer Vorverarbeitungsmodule** zur **Annotation von Objekten inkl. Extraktion von Merkmalen aus Annotationen.**

- Die vorgestellte Service-orientierte Architektur stellt **Schnittstellen und Modelle der einzelnen Methoden** sowie **semantisch eindeutige Möglichkeiten zur Prozessierung der Services** bereit. Dies und die Beschreibung der vom Service benötigten resp. gelieferten Daten ermöglicht einen einfache **Komposition der einzelnen Methoden** sowie eine klare Definition der **Schnittstellen und Modelle der einzelnen Methoden**

Der Vergleich des umgesetzten KnowMiner-Frameworks zu den Anforderungen zeigt, dass die erhobenen Anforderungen im Framework umgesetzt wurden. Offen bleibt noch die Frage der Anwendbarkeit des Frameworks in unterschiedlichen Szenarien, welche das nächste Kapitel beantwortet.

9 Anwendungen und abgeleitete Ergebnisse

Um zu zeigen, dass das entwickelte Framework nicht nur die erhobenen Anforderungen erfüllt, sondern ein

flexibles Software Framework zur Umsetzung von Knowledge Discovery und Information Retrieval Diensten und die Möglichkeit der Einsetzbarkeit dieses Frameworks in unterschiedlichen Anwendungsgebieten

- wie es die Zielsetzung der Arbeit formuliert - darstellt, beschreibt dieser Abschnitt die auf Basis des KnowMiner-Frameworks im letzten Iterationszyklus umgesetzten Anwendungen. Neben der Demonstration der Anwendbarkeit des Frameworks erfolgt zusätzlich eine Analyse der Vor- bzw. Nachteile verschiedener Konzepte, im Speziellen auf Ebene der unterschiedlichen Services.

Im letzten Iterationszyklus erfolgte die Umsetzung von zwei Anwendungen im Rahmen zweier Großprojekte am Know-Center. Neben den beiden Projekten wurden noch mehrere kleinere Anwendungen im Rahmen von Diplomarbeiten umgesetzt, welche ebenfalls zur Bewertung des KnowMiner-Frameworks heran gezogen werden.

Im ersten Projekt mit Namen Xtrend war das Ziel die Entwicklung von Anwendungen, welche die Analyse von Zeitungsdaten durch die Anwendung von KD-Methoden unterstützt. In Zusammenarbeit mit den Firmen APA DeFacto[47] und MediaWatch[48] konnte ein System geschaffen werden, welches im Umfeld von Medienresonanzanalysen und Nachrichtendiffusionsanalysen den Umgang mit mehreren Millionen Dokumenten erleichtert.

MISTRAL - Measurable Intelligent and Reliable Semantic Extraction and Retrieval of Multimedia Data – stellt die zweite große Anwendung des KnowMiner-Frameworks dar. MISTRAL[49] ist ein im Rahmen des FIT-IT Programms gefördertes Projekt mit dem Ziel, Retrieval in multimedialen Datenbeständen unter Ausnutzung von cross-modalen Zusammenhängen zwischen Informationen zu verbessern. Während MISTRAL ein breites Spektrum an Anforderungen definiert, welche nicht im vorliegenden KnowMiner-Framework berücksichtigt sind, fokussierte die Anwendung des KnowMiner-Frameworks auf die im Rahmen von MISTRAL zu verarbeitenden textuellen Daten und stellte dazu Dienste in Form von Web-Services bereit.

Dieser Abschnitt beschreibt im Folgenden diese zwei Projekte. Zusätzlich gibt er einen Überblick über Projekte, die gerade unter Anwendung des KnowMiner-Frameworks umgesetzt werden bzw. kleinere Anwendungen des KnowMiner-Frameworks im Rahmen von Diplomarbeiten. Der Abschnitt schließt mit der Darstellung der aus der Anwendung des KnowMiner-Frameworks abgeleiteten Erkenntnisse.

9.1 Xtrend

Die APA MediaWatch bietet Kunden als Dienstleistung die so genannten Medienresonanzanalysen an. Nachrichtenmeldungen werden hinsichtlich Nennungen von Personen, Firmen und Produkten untersucht, um Aussagen über die Nennungen und die Nennungsqualität (positiv, negativ oder ambivalent) bezogen auf ein bestimmtes Thema (z.B. Umwelt, Wirtschaft) treffen zu können. Diese Analysen basieren dabei auf

[47] http://www.apa-defacto.at/

[48] http://www.apa-mediawatch.at/

[49] http://www.mistral-project.at/

statistischen Kenngrößen wie Anzahl der Nennungen einer Person oder eines Produktes sowie auf intellektuell evaluierten Meldungsinhalten. Resultat solcher Analysen sind Aussagen über Nennungshäufigkeit, positiven oder negativen Nennungskontext und andere auf das Kundeninteresse abgestimmte Skalen.

Im Projekt XTrend wurde eine weitgehend automatisierte Umgebung zur Planung und Durchführung solcher Analysen geschaffen und auf die Bedürfnisse der Mitarbeiter der APA MediaWatch abgestimmt. Die Umsetzung der algorithmischen Wissenserschließungstechniken erfolgte unter Einbeziehung des KnowMiner-Frameworks, wobei die KnowMiner Servicearchitektur (siehe Abschnitt 8.5) zum Einsatz kam. Bestandteile der Umgebung waren:

- Verwaltung von Thesauri mit Möglichkeiten zum automatischen Vorschlagen von zusätzlichen weiterführenden Suchbegriffen
- Verwaltung von unterschiedlichen Analyseprojekten
- Anbindung an externe Suchmaschinen und Optimierung der Selektion von Nachrichtenmeldungen aus großen Dokumentbeständen
- Informationsextraktion zur automatischen Analyse von Nachrichtenmeldungen hinsichtlich Nennung von Personen
- Automatische Zuordnung von Nennungen zu Themen und deren Bewertung
- Optimierte Benutzerschnittstelle zur intellektuellen Kontrolle der Zuordnungen und Nennungen
- Statistische Auswertung von Nennungen und deren Qualität

Weitere Rahmenbedingung im Projekt war die Integration der genannten Funktionalität in eine bereits bestehende IT Infrastruktur. Da die APA mit der APA Powersearch [Mitterholzer & Horvath 2005] über eine leistungsfähige Such- und Clusteringmaschine zum Durchsuchen des APA Datenbestands mit über 30 Millionen Dokumenten verfügt, wurde die Anbindung dieser Suchmaschine in das KnowMiner-Framework vorgenommen.

Im Folgenden wird kurz ein Überblick über den Ablauf einer Medienresonanzanalyse gegeben und auf die Systemarchitektur eingegangen. Um den Rahmen nicht zu sprengen, werden nur die KnowMiner relevanten Teile sowie die implementierten Algorithmen im Detail beschrieben. Den Abschluss der XTrend Beschreibung bildet ein Überblick über die Evaluierung und deren Ergebnisse.

9.1.1.1 Medienresonanz-Analyse

Eine Medienresonanz-Analyse in der von APA MediaWatch durchgeführten Form analysiert die Präsenz (Nennungshäufigkeit) von Akteuren (Personen, Produkten, Unternehmen) in Meldungen und leitet daraus statistische Informationen und individuell gewonnene Einschätzungen der Qualität von Nennungen (z.B. positiv-negativ, aktiv-passiv) ab.

Grundlegend wird im Produktionsprozess zwischen zwei extremen Ausprägungen der Analyse unterschieden, die sich nach Detailgrad der Fragestellung und nach nötiger Genauigkeit unterscheiden:

- Individualanalysen werden im Auftrag von Kunden durchgeführt und beantworten spezifische Fragen der Auftraggeber. Zwar werden intern vereinheitlichte „Themenkörbe“ verwendet, doch die Definition von Merkmalen und die Bestimmung von Merkmalsausprägungen erfolgt weitgehend

intellektuell da eine hundertprozentige Genauigkeit der Analyse erwünscht ist und auch kontrolliert wird.

- Standard-Analysen werden nach Feststellung von allgemeinem Bedarf definiert und stehen der Allgemeinheit zur Verfügung. Kunden „kaufen" Standard-Analysen und eventuell Anpassungen in geringem Ausmaß. In den Standard-Analysen ist keine hundertprozentige Genauigkeit in der Bestimmung von Merkmalsausprägungen erforderlich.

Diese beiden Ausprägungen der Analyse bilden im Wesentlichen Extremformen innerhalb einer Skala die vom Detailgrad der Fragestellungen sowie von der gewünschten Genauigkeit geprägt wird. Innerhalb dieser Skala sind – insbesondere in Hinblick auf eine Vermarktung via Web – auch andere Positionierungen und Angebote denkbar.

Suche relevanter Dokumente: Am Beginn einer Medienresonanz-Analyse steht die Suche nach relevanten Dokumenten. Solche Dokumente werden über eine Reihe von vom Auftraggeber bzw. von APA MediaWatch definierten Schlüsselwörtern und entsprechenden Synonymen mittels APA PowerSearch ermittelt. Auf dem gefundenen Dokumentset basiert die weitere Analyse.

Präsenzbestimmung: Hier wird innerhalb der Menge gefundener Dokumente nach Vorkommnissen der Akteure gesucht. Gespeichert werden sowohl statistische Angaben zur Vorkommenshäufigkeit als auch die konkreten Stellen, an denen Akteure vorkommen, um eine spätere intellektuelle Analyse des Nennungskontextes zu ermöglichen.

Kontrolle ermittelter Nennungen: An die automatisierte Präsenzbestimmung schließt sich eine intellektuelle Kontrolle ermittelter Nennungen an. Zweck ist die Sicherstellung, nur korrekte Nennungen erfasst zu haben. Es werden alle Nennungen intellektuell nachkontrolliert, um eine hohe Qualität zu erreichen.

Merkmalsbestimmung: Sind die Nennungen erst bekannt, so kann die intellektuelle Bestimmung des Kontextes von Nennungen und damit die Bestimmung von Merkmalen wie „positiv erwähnt" erfolgen. Dieser Abschnitt der Medienresonanzanalyse erfolgt – zumindest in Individualuntersuchungen – vollständig intellektuell und muss zu einhundert Prozent korrekt sein.

Verfassen der Analyse: Sind alle benötigten Daten bestimmt, so wird innerhalb der Microsoft Office Umgebung das Analysedokument erstellt. Dabei werden statistische Daten in Microsoft Excel aufbereitet und als Diagramme dargestellt, während die textuelle Beschreibung der Resultate und der Gesamtsatz in Microsoft Word erfolgen. Schließlich wird die Statistik-Software SPSS für verschiedene statistische Berechnung eneingesetzt.

9.1.1.2 Systemarchitektur

Zur Realisierung der Medienresonanzanalyse wurde eine Client/Server Architektur entwickelt, welche zu einem großen Teil auf das KnowMiner-Framework aufsetzt, aber auch anwendungsspezifische Komponenten wie z.B. Projekt- und Benutzerverwaltung beinhaltet. Die Systemarchitektur berücksichtigt ebenfalls unterschiedliche Datenquellen wie z.B. die APA PowerSearch oder aber RDBMS, welche Projekt- und Nutzdaten beinhalten.

Um ein möglichst dezentrales Arbeiten zu ermöglichen wurde als Kommunikationsschnittstelle zwischen Client/Server eine Web-Service Architektur eingesetzt. Abbildung 9-1 zeigt einen Überblick über die Gesamtarchitektur.

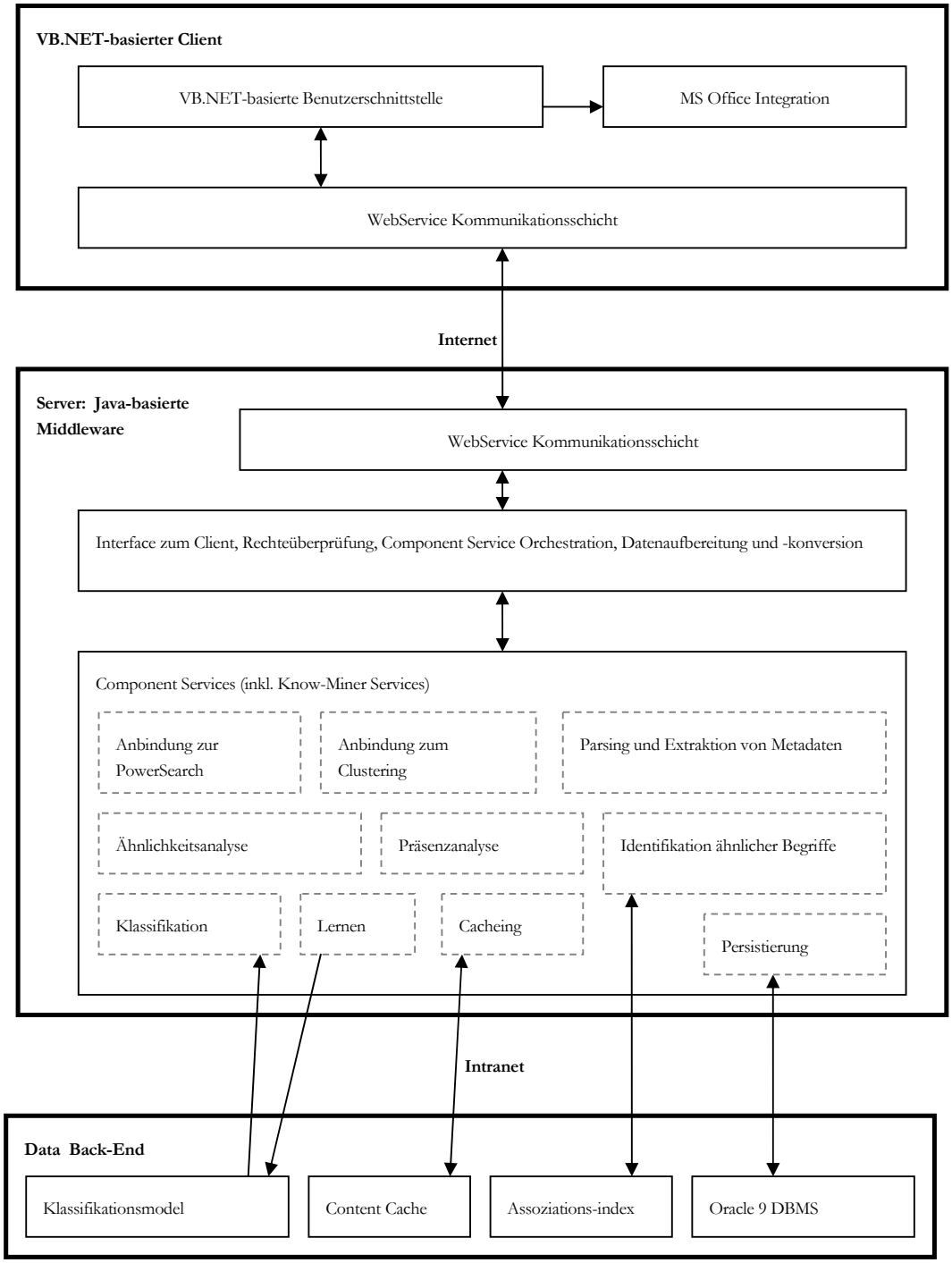


Abbildung 9-1: 3-tier Systemarchitektur der X-Trend Applikation

Der Prototyp basiert auf einer Service-basierten 3-tier Architektur, die sich aus folgenden Komponenten zusammengesetzt:

- Client: Eine in VB.NET entworfene Benutzerschnittstelle mit MS Office Integration.

- Server: Eine in Java entworfene, auf Apache Tomcat laufende Middleware Schicht in der die Logik und alle relevanten Algorithmen des Systems enthalten sind. Der Server selbst basiert auf einer Service-orientierten Architektur (SOA).
- Data Backend: Besteht aus einer Oracle 9 Datenbank sowie aus einigen begleitenden Ablagestrukturen (Klassifikationsmodel, Content-Cache, Assoziationsindex).

Die Kommunikation zwischen dem Client und dem Server erfolgt über eine wohldefinierte Web-Service-Schnittstelle. Die Implementierung der Web-Service Schnittstelle sorgte dabei auch für die Orchestrierung der einzelnen Services.

Wichtig im Zusammenhang mit der vorliegenden Arbeit ist die Integration der KnowMiner Dienste in die Xtrend Anwendung. Xtrend als Anwendung benötigt nicht nur KD-Dienste, sondern stellt auch anderen Dienste wie etwa die Thesaurusverwaltung bereit und bedient sich dabei der Oracle Datenbank. Zusätzlich konnten die KnowMiner-Dienste, im Speziellen jene zur Klassifikation, Informationsextraktion, Ähnlichkeitsanalyse und zum Finden von ähnlichen Begriffen, integriert werden.

Die Informationsextraktion kam beispielsweise bei der Analyse von über PowerSearch gefundenen Zeitungsartikeln zum Einsatz und extrahierte dabei Personen-, Orts- und Zeitbezüge, um die Analyse zu erleichtern. Die Analyseergebnisse speicherte die Oracle Datenbank.

Xtrend demonstriert die erfolgreiche Integration von KD-Diensten des KnowMiner-Frameworks in ein heterogenes Umfeld. Auch die Einbindung der APA Suchmaschine PowerSearch konnte aufgrund der wohldefinierten Schnittstellen unproblematisch durchgeführt werden.

9.1.1.3 Zusammenfassung und Lessons Learned

In der Middleware-Schicht befindet sich der Großteil der Systemlogik und alle relevante Algorithmen und Services aus dem KnowMiner-Framework. Die allgemeinen Vorteile, die sich aus einer solchen Architektur ergeben, sind:

- Verbessertes Change-Management wird durch Abkopplung des Clients vom Rest des Systems erreicht indem eine Schnittstelle zwischen dem Client and dem Server definiert und fast die gesamte Logik auf die anderen zwei Tiers verlagert wird. Dadurch wird die Client-Applikation wesentlich „leichter“ und ist von Änderungen an Algorithmen oder Datenbankschema i.A. nicht betroffen.
- Wiederverwendbarkeit der Methoden der Verarbeitung: Da die Logik nicht im Client enthalten ist, können die entsprechenden Funktionalitäten von verschiedenen Applikationen verwendet werden. Beispielsweise greift die Applikation NTrace ebenfalls auf Dienste von XTrend zu und umgekehrt.
- Erhöhte Leistungsfähigkeit
    - Da der Client nur die Ergebnisse einer Verarbeitung bekommt, müssen weniger Daten zum Client übertragen werden, was wiederum zu weniger Netzwerk-Traffic und in weiterer Folge, besonders bei beschränkter Bandbreite, zu besserer Leistung führt.
    - KD-Algorithmen benötigen i.A. relativ leistungsfähige Hardware, welche mittels Server bereitgestellt werden kann. Die Leistungsfähigkeit der Client Maschine hat daher nur marginalen Anteil an der Gesamtleistungsfähigkeit.

- Die durch PL-SQL und/oder komplexe SQL-Statements realisierten Operationen lassen sich oft in der Middleware durch intelligentes Zwischenspeichern rasch verarbeiten. Ausnahmen sind Locking-Mechanismen und Transaktionsabwicklung, die weiterhin am effizientesten von der Datenbank ausgeführt werden.
- Da die Middleware auf einer Service-orientierten Architektur aufgebaut ist, wäre es möglich, einzelne Component Services (siehe Abbildung 9-1) zu replizieren und auf verschiedenen Maschinen laufen zu lassen (PowerSearch und Clustering sind schon solche externe Dienste), wodurch sich ein n-Tier System ergeben würde. Prinzipiell erlaubt die Architektur der entworfenen Middleware Schicht eine n-Tier Konfiguration, dies wurde jedoch im Rahmen des Projektes nicht evaluiert.

Zur Umsetzung der Architektur wurden verschiedenste, sehr heterogene Technologien erfolgreich miteinander kombiniert und die KnowMiner Service Architektur erfolgreich eingebunden. Der Client wurde, aufgrund von notwendigen engen Integrationsmöglichkeiten in MS-Office Umgebungen, in VB.NET entworfen. Der Server (Middleware) basiert auf Java mit Apache Tomcat/AXIS als Web-Service, wobei sich die Wahl von Java aufgrund der Plattformunabhängigkeit und natürlich aufgrund des in Java umgesetzten KnowMiner-Frameworks ergab. Das Data Backend besteht im Wesentlichen aus einer Oracle 9i Datenbank und begleitenden proprietären Ablagestrukturen (Klassifikationsmodel, Content-Cache, Assoziationsindex).

Speziell im Bezug auf die KnowMiner Knowledge Disocvery Dienste konnten diese in verschiedenen Szenarien erfolgreich eingesetzt werden. Das *InformationExtraction* Service analysierte einerseits die von PowerSearch gelieferten Suchergebnisse und zerlegte andererseits die Texte beim Import in die Datenbank. Die Flexibilität der Architektur und die wohl überlegte Gestaltung der Schnittstellen ermöglicht dabei diese Mehrfachverwendung. Die Service Architektur bewährte sich auch im Multi-User Betrieb und damit verbunden bei der parallelen Abarbeitung von Aufgaben. Das Prozessierungsmodell der Services ermöglichte dabei dem Benutzer, die volle Kontrolle über das Service zu haben. Dauerte z.B. der Import von Dokumenten zu lange, so konnte der Prozess pausiert und bei Bedarf wieder aufgenommen werden; essentielle Funktionen für die benutzerorientierte Anwendung von SoA's.

Eine Anforderung in Xtrend war die Integration von Daten aus dem Filesystem sowie aus der bereits bestehenden Oracle Datenbank und der APA Powersearch. Unter Anwendung der flexiblen Importarchitektur war auch diese Anforderung gut erfüllbar. Es zeigten sich jedoch hinsichtlich der Interface-Definition für die Loader und FormatNormalizer sowie für den Zugriff auf GML Möglichkeiten zur Verbesserung. Vor allem das Java-XML Binding von GML muss weiter gekapselt werden.

Das im KnowMiner-Framework verwendete abstrakte Datenmodell erwies sich ebenfalls als hilfreich, da die Abbildung von Inhalten und Metadaten einfach möglich war. Vor allem bei der Analyse von ähnlichen Begriffen half die Definition von Metadaten, wie z.B. Zeit, um den Kontext richtig einzuschränken. So ist z.B. Bundeskanzler immer nur in einem gewissen Zeitraum ein Synonym für die Person, die den aktuellen Kanzler stellt.

Als problematisch erwiesen sich jedoch die Datenkonvertierungen zwischen Client und Middleware sowie der Import der APA Originaldaten in die Oracle 9i Datenbank. Einige Datentypen wurden clientseitig von .Net nicht unterstützt, wodurch es zu Abstürzen und, bedingt durch die Verwendung von Web-Services, langen

Fehlersuchzeiten kam. Auch das richtige, gezielte Locking von Entitäten erwies sich als schwierig, da es eine genaue Implementierung und die Kenntnis der von einem Service benötigten Daten voraussetzt.

Die Punkte in Bezug auf das KnowMiner-Framework können folgend zusammengefasst werden:

1. Gute Integrationsmöglichkeiten in heterogene Umgebungen
2. Wiederverwendbarkeit der verschiedenen Services
3. Bewährt im Multi-User Betrieb und bei der parallelen Abarbeitung von Methoden
4. Brauchbare Anbindung anderer Datenquellen
5. Typsystem hat sich zur Abbildung von Metadaten und Merkmalen bewährt
6. Als problematisch erwiesen sich die Datenkonvertierungen zwischen verschiedenen Servicearten (durch die Integration von Services in ein heterogenes System)
7. Locking teilweise zu komplex

9.2 MISTRAL

Das im Folgenden beschriebene Projekt MISTRAL[50] (Measurable intelligent and secure semantic extraction and retrieval of multimedia data), fokussiert auf die cross modale Analyse multimedialer Daten um diese semantisch anzureichern und damit einfacher suchbar zu machen. Teile des vorliegenden Textes zur Beschreibung von MISTRAL sind der Publikation [Tochtermann et. al. 2005] entnommen, an der der Autor mit beteiligt war. MISTRAL wird von der Österreichischen Forschungsförderungsgesellschaft (www.ffg.at) im Rahmen der Programmlinie FIT-IT unter der Projektnummer 809264/9338 gefördert und an der Umsetzung des Systems sind 6 Institute der TU Graz beteiligt, das Know-Center Graz, sowie zwei Firmen.

Ziele von MISTRAL sind die weitestgehend automatisierte semantische Annotation und Erschließung von multimedialen Datenbeständen unter Berücksichtigung von Beziehungen zwischen den verschiedenen Modalitäten wie Audio, Video, Bild und Text. Um dieses Ziel zu erreichen wurden, basierend auf einer Service-orientierten Architektur, folgende Kernfunktionalitäten entwickelt:

- Cross-modale Erkennung von Objekten innerhalb eines multimedialen Dokumentes. Extraktionsmöglichkeiten verschiedener Modalitäten sollen kombiniert werden, um die Zuverlässigkeit und Robustheit der Extraktion von Objekten wie z.B. Personen in multimedialen Dokumenten zu erhöhen. Beispielsweise können Personen anhand ihrer Gesichter oder ihrer Stimmen erkannt werden. Durch Kombination von Video- und Audioerkennung wird das Vertrauen in die Entscheidung erhöht.
- Cross-modale Erkennung von Objekten innerhalb eines multimedialen Dokumentenkorpus. Ziel ist die Modalitäten-übergreifende Erkennung von Konzepten in einer Menge von multimedialen Dokumenten.

[50] http://www.mistral-project.at

Beispielsweise können Folien einer Präsentation über Bildähnlichkeiten in einem Video gefunden werden, da die Darstellung einer Folie als Bild angesehen werden kann. Anhand des Texts der Folie ist nun eine Verschlagwortung des Videoteiles möglich.

- Anwendung domänenunabhängiger, logikbasierter Inferenzmechanismen zur Auflösung von Mehrdeutigkeiten und zum Anreichern extrahierter Konzepte.
  Hierbei wird von einer zugrunde liegenden formalen Beschreibung der Domäne ausgegangen, wie sie z.B. über Ontologien oder Wissensbasen möglich ist. Beispielsweise modelliert eine Ontologie für Sitzungen Konzepte wie Personen, Sitzungsleiter, Präsentation etc. Auf Basis der zuvor erwähnten Extraktionsmechanismen erfolgt etwa die Erkennung von Personen und deren Interaktionen, wobei von einer gewissen Fehlerrate ausgegangen werden muss. So kann z.B. mittels Audioanalyse erkannt werden, das Person A spricht und sich im Raum bewegt. Gleichzeitig könnte jedoch die Bildanalyse Person A als an einem Ort sitzend erkennen. Über Modellierung und Inferenzmechanismen kann dieser Konflikt erkannt und (semi)automatisch behoben werden.

Neben den funktionalen Aspekten liegt ein weiterer Hauptaspekt der Arbeiten auf dem einfachen und flexiblen Zusammenspiel der einzelnen Komponenten. Web-Services spielen hier eine zentrale Rolle und sollen den einfachen Austausch von unterschiedlichen Extraktions- und Validierungskomponenten ermöglichen. Während andere Systeme wie z.B. MARVEL[51] auf die Extraktion inhaltlicher Beschreibungen fokussieren, sollen in MISTRAL ebenfalls Interaktions-orientierte Konzepte extrahiert und zur Annotation verwendet werden.

Zur Analyse dieser Interkations-orientierten Konzepte lag das Hauptaugenmerk im MISTRAL Projekt auf der Aufzeichnung von Face to Face Meetings und der Analyse der erhaltenen Daten. Über Web-Kameras und Microfon-Array's zeichnet MISTRAL die Teilnehmer eines Meetings und deren Verhalten auf. Als zusätzliche Informationsquelle erfasst ein KeyLogger Interaktionen der Teilnehmer des Meetings mit einem digitalen Endgerät, im speziellen mit dem Präsentations-Laptop. Dadurch stehen neben den Modalitäten Audio und Video noch Daten wie die zu einem Zeitpunkt aktuelle Präsentation zu Verfügung.

Nach dieser kurzen Beschreibung des MISTRAL Projekts gibt der Rest des Abschnitts einen Überblick über die Einzelkomponenten und beschreibt anschließend die Architektur mit der Einbindung des KnowMiner-Frameworks im Detail. Die Diskussion der Lessons Learned aus der Integration des KnowMiner-Frameworks in die MISTRAL Gesamtarchitektur stellt den Abschluss des Abschnitts dar.

9.2.1 Systemüberblick

Das MISTRAL System ist in unterschiedliche Komponentengruppen nach den jeweiligen zu verarbeitenden Modalitäten einteilbar. Zur Strukturierung wird zwischen vertikalen und horizontalen Komponenten unterschieden. Die horizontalen Komponenten definieren einen klaren Ablauf zur Extraktion semantischer Annotationen aus Multimediadaten zuzüglich deren Persistierung. Vertikale Komponenten definieren Aspekte, welche keinen direkten Einfluss auf die Extraktion haben, jedoch für den anwendungsorientierten Einsatz des Systems notwendig sind. Zu den vertikalen Services zählen z.B. Benchmarking bzgl. Güte der Extraktion von Multimediadaten, User-Interface und Visualisierungskomponenten. Abbildung 9-2 zeigt die unterschiedlichen Informationsebenen und die entsprechenden Komponenten von MISTRAL.

[51] http://domino.research.ibm.com/comm/research_projects.nsf/pages/marvel.index.html

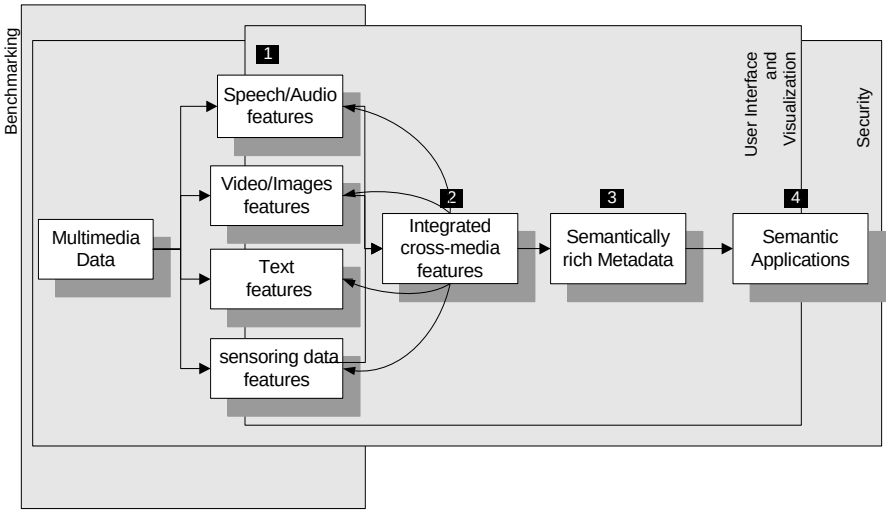


Abbildung 9-2: Überblick über die MISTRAL Dienste

9.2.1.1 Horizontale Komponenten

Die horizontalen Komponenten beinhalten die Kernfunktionalität von MISTRAL und sind für die semantische Extraktion sowie für die Suche nach semantisch annotierten Multimediadaten verantwortlich. Grundlegend wird zwischen (1) der Extraktion von uni-modalen Merkmalen und Konzepten, (2) der Integration von multi-modalen Konzepten, (3) der semantischen Anreicherung von den extrahierten Konzepten sowie (4) der Anwendung der extrahierten Konzepte im Bereich Suche und Retrieval unterschieden. Die Komponenten konzentrieren sich dabei auf die Analyse von Videodaten, d.h. es sind sowohl Komponenten unterschiedlicher Modalitäten, als auch eine zeitlich Komponente zu berücksichtigen.

9.2.1.2 Uni-modale Komponenten

Der uni-modale Funktionsblock setzt sich aus Video- u. Bildextraktion, Audioextraktion, Speech To Text, Textextraktion und der Extraktion von Konzepten aus verschiedenen Sensordaten zusammen (siehe 1 in Abbildung 9-2). Aus den verschiedenen Modalitäten erfolgt die Extraktion von Konzepten und Merkmalen, wobei zwischen Inhalts-orientierten Konzepten und Interaktions-orientierten Konzepten unterschieden wird. Inhalts-orientierte Konzepte beschreiben das in einer Szene adressierte Thema, also z.B. worüber gerade gesprochen wird. Interaktions-orienterte Konzepte beschreiben die Interaktion zwischen Personen und Objekten in einer Szene, also z.B. welche Personen miteinander interagieren oder mit welchen Objekten interagiert wird.

In MISTRAL ist die Extraktion Interaktions-orientierter Konzepte aus Video, Audio und Sensormodalitäten bzw. deren Modalitäten-übergreifende Kombination vorgesehen. Im Videobereich sind dies z.B. Techniken zur robusten Erkennung von Gesichtern und Objekten sowie deren Verfolgung innerhalb einer Szene [Grabner et. al 2006]. Zur Vervollständigung dieser interaktionsorientierten Konzepte stellt die Audio Komponente Möglichkeiten zur Erkennung der räumlichen Positionierung von Sprechern und Umgebungsgeräuschen mittels Beamforming bereit, sowie die Unterscheidung zwischen verschiedenen Geräuschquellen und der Trennung zwischen Sprache und anderen Geräuschen wie z.B. das Öffnen einer Tür [Képesi et. al. 2005]. Ergänzend zu den Audio- und Videomodalitäten erfolgt die Analyse von so

genannten Sensordaten bzw. Sensormodalitäten, welche eine Fülle an Möglichkeiten bereitstellen. Die Berücksichtigung dieser Modalität erfolgt in MISTRAL mittels der Aufzeichnung der Interaktion von Personen mit digitalen Endgeräten wie z.B. Laptops bei einer Präsentation. Durch Aufzeichnung von Maus- und Tastaturereignissen können Interaktionsmuster abgeleitet und zur weiteren Analyse zu Verfügung gestellt werden. Zusätzlich entsteht hier im Beispiel von Präsentationen einen Brücke zwischen Interaktionskonzepten und inhaltlichen Konzepten, da zusätzlich die Inhalte der Interaktion in digitaler Form vorliegen (z.B. Powerpoint Präsentationen).

Neben der Extraktion von Interaktionskonzepten ist auch die inhaltliche Analyse multimedialer Daten, im speziellen Video, ein wichtiges Kriterium zum Finden von relevanter multimedialer Information. Neben der bereits erwähnten Sensormodalität zur Extraktion von relevanten inhaltlichen Konzepten einer Szene kommen hier vor allem Speech to Text Techniken und Techniken aus dem Bereich Textanalyse zum Einsatz. Ausgehend von einem für die jeweilige Domäne relevanten Dokumentkorpus (z.B. Intranet, Diskussionsforen etc.) erfolgt die textuelle Analyse von relevanten inhaltlichen Konzepten mittels statistischer Textanalyseverfahren wie z.B. Concept Clustering. Zusätzlich ermöglichen Techniken aus dem Bereich der Informationsextraktion die automatische Erkennung von relevanten Akteuren (z.B. Personen, Firmen etc.). Während mittels Textextraktionsdienst der Domänenkontext aufgespannt wird, erfolgt die Erkennung von inhaltlichen Konzepten aus einer Kombination von Speech to Text und Textextraktionsdienst. Speech to Text transformiert als Sprache erkannte Audioteile in Text, welche anschließend in den entsprechenden Domänenkontext transformiert wird.

9.2.1.3 Cross-modale und Semantische Komponenten

Die cross-modalen Dienste ermöglichen die Extraktion neuer bzw. Bestätigung bereits extrahierter Konzepte auf Basis maschineller Lernverfahren. Grundlage für diese Komponenten sind die zuvor extrahierten uni-modalen Konzepte, die zeitlich ändernde Konzeptsequenzen unterschiedlicher Modalitäten darstellen. Grundlegend kann zwischen folgenden Aufgaben für die Extraktion multimodaler Konzepte unterschieden werden:

1. Erhöhung der Konfidenz für ein extrahiertes Konzept
   Die uni-modalen Komponenten extrahieren Konzepte mit einer gewissen Sicherheit, welche durch andere Modalitäten erhöht werden kann (z.B. Positionsermittlung einer Person über Audio und Video).

2. Zusammenfassen von Konzepten
   Durch Untersuchung der Korrelation zwischen einzelnen uni-modalen Konzepten und unter Berücksichtigung von zeitlichen Abhängigkeiten werden uni-modale Konzepte zu Gruppen zusammengefasst. Die Zusammenfassung von Konzepten erfolgt unter Anwendung von auf Sequenzen basierenden Ähnlichkeitsmaßen (z.B. längste gemeinsame Subsequenz) und durch die Anwendung verschiedener Clustering Techniken.

3. Finden von Mustern mit hohem Überraschungswert
   Neben dem Finden von Gemeinsamkeiten zwischen uni-modalen Konzepten ist auch die Identifikation von überraschenden Konstellationen zwischen uni-modalen Konzepten von Interesse. Dadurch sollen außerordentliche Ereignisse wie z.B. der Wechsel des Vortragenden erkannt werden.

4. Extraktion neuer Konzepte
   Basierend auf den unterschiedlichen Modalitäten und zeitlichen Änderungen der Konzepte sollen maschinelle Lernverfahren Regeln zur Identifikation neuer Konzepte finden. Ziel ist es z.B. über Sprecherposition bzw. Anzahl der Quellen und der Bildposition der beteiligten Personen zu erkennen, ob es sich um einen Dialog oder einen Monolog handelt.

Zur Einbindung von formalen Beschreibungen und zur Erhöhung der Semantik erfolgt die Anreicherung der extrahierten Konzepte unter Einsatz semantischer Technologien sowie einer Wissensbasis. Ausgangspunkt sind die über einen Zeitraum extrahierten Konzepte aus den uni-modalen Komponenten und die über multi-modale Korrelationen extrahierten Multimodalkonzepte. Ziel ist unter zu Hilfenahme der Wissensbasis und Anwendung von Inferenzmechanismen widersprüchliche Ereignisse zu erkennen und domänenabhängige Beziehungen zwischen Konzepten zu identifizieren. Beispielsweise könnte eine Person mittels Videoextraktion als sitzend identifiziert werden, während der Audio-Dienst eine Bewegung dieser Person im Raum meldet. Durch entsprechende Modellierung der Wissensbasis kann dieser Widerspruch erkannt und annotiert werden. Eine weitere Möglichkeit ist die multimodale Identifikation eines Monologes und die Interaktion einer Person mit dem Präsentationsnotebook, woraus sich für die Domäne von Sitzungen oder Konferenzen der Zusammenhang ableiten lässt, dass die gerade sprechende Person einen Vortrag hält. Durch die Einbindung von Wissensbasen in Form einer Linear Time Logic [Wotawa & Angerer 2006] können Widersprüche erkannt und der Beschreibung eines Meetings hinzugefügt werden.

9.2.1.4 Vertikale Komponenten

Vertikale Komponenten können als unterstützende und für verschiedene Anwendungsbereiche notwendige Komponenten aufgefasst werden, welche jedoch soweit nichts mit Extraktion oder Retrieval zu tun haben. MISTRAL unterscheidet drei solcher vertikalen Services, auf die hier nur kurz eingegangen wird.

- Sicherheits-Dienste behandeln Aspekte wie die Authentifizierung von Benutzern und die sichere und zuverlässige Kommunikation zwischen den verschiedenen Diensten
- Benchmarking erlaubt die Standardisierung und den Vergleich zwischen verschiedenen Algorithmen bzgl. semantischer Annotation. Dadurch soll eine Plattform geschaffen werden, die das Benchmarken von verschiedenen Algorithmen ermöglicht.
- Visualisierungsdienste stellen Möglichkeiten zur visuellen Darstellung komplexer Zusammenhänge zur Verfügung und können auf Basis der annotierten Daten z.B. Interaktionsnetzwerke visualisieren.

9.2.2 Systemarchitektur und Integration des KnowMiner-Frameworks

Nach der Vorstellung der einzelnen Komponenten des MISTRAL Systems gibt dieser Abschnitt einen Überblick über die Systemarchitektur und die Integration des KnowMiner-Frameworks. Da in MISTRAL komplexe Datenflüsse zwischen den einzelnen Komponenten zu berücksichtigen sind, erfolgt die Diskussion der Systemarchitektur anhand des Datenflussdiagramms in Abbildung 9-3.

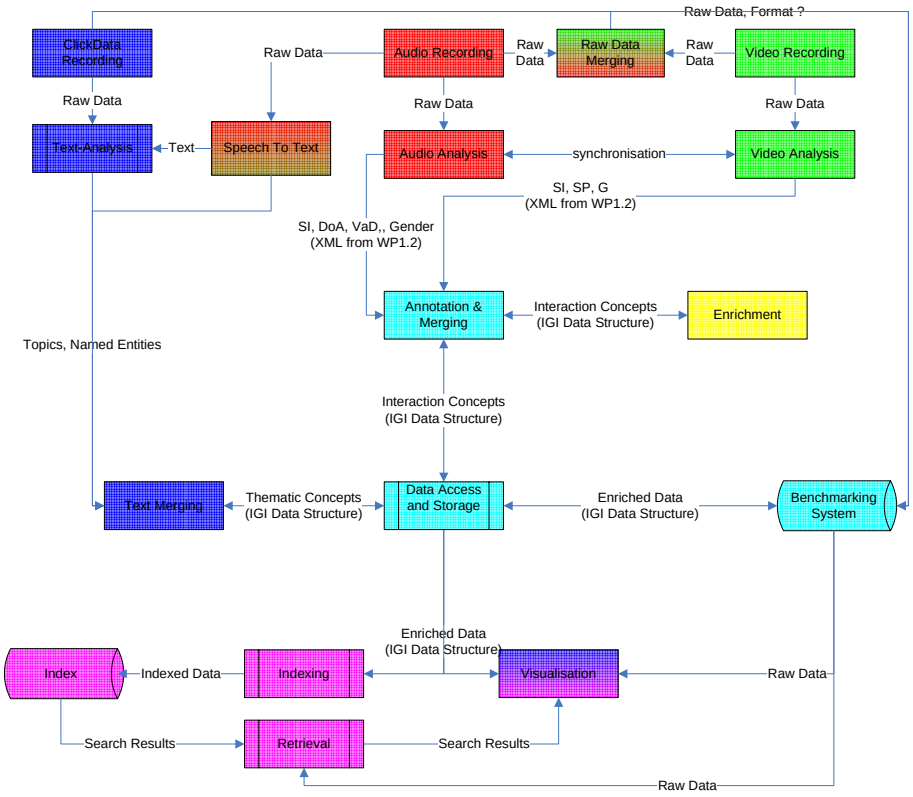


Abbildung 9-3: Datenfluss zwischen den MISTRAL Komponenten

Ausgehend von der Aufzeichnung eines Meetings erfolgt die Analyse der Rohdaten der unterschiedlichen Modalitäten (Text Analysis, Audio Analysis, Video Analysis). Aus der Analyse entstehen die Aufzeichnung beschreibende Interaktionsmerkmale und Themenmerkmale. Interaktionsmerkmale entstehen aus Audio und Videoanalyse. Im Speziellen identifiziert Audio die Identität des Sprechers (Speaker Indexing, SI), wann gesprochen wurde (Voice Activity Detection, VAD), wo sich der Sprecher befindet (Direction of Arrival, DoA) und welchen Geschlechts er ist (Gender); die Videoanalyse liefert die Identität von Personen im Bild (SI), deren Position und Bewegungen (SP), sowie offensichtliche Gesten einer Person (G).

Die identifizierten Interaktionskonzepte landen im Annotationsframework, wo die Transformation der Ausgangsdaten von Audio und Video in eine MPEG-7 ähnliche Datenstruktur erfolgt. Das Annotationsframework integriert ebenfalls die Multi-Modal Merging und Semantic Enrichment Komponenten zur Anreicherung der Daten. Semantic Enrichment liefert dafür widersprüchliche Konzepte und Multi-Modal Merging extrahiert Konzepte wie Monolog/Dialog. Die so aufbereiteten Daten können über eine Data Access und Storage Komponente im Benchmarking System abgelegt werden.

Über die Data Access und Storage Komponente erfolgt auch die Integration der aus den Klick-Daten aufgezeichneten Themenmerkmale. Die Klick-Daten ermöglichen den aktuellen Inhalt des Bildschirms, beispielsweise die Folie einer Präsentation, zu erfassen. Die Textanalyse-Komponente wertet die Inhalte aus und integriert die extrahierten Konzepten und Named Entities in die Datenstruktur. Für die mittels Speech to Text extrahierter Texte gilt das gleiche, wenngleich das in MISTRAL verwendete Speech to Text System teilweise Schlüsselwörter und Named Entities selbstständig extrahieren kann.

Auf den mit Interaktionsmerkmale und Themenmerkmale erschlossenen Meeting Daten setzen die verschiedenen MISTRAL Applikationen zum Retrieval und zur Visualisierung der Daten auf. Zusätzlich zu den semantisch erschlossenen Meeting Daten berücksichtigt die MISTRAL Applikation auch Dokumentkorpora, welche in Beziehung zu den Meetings stehen. Die Idee dahinter ist, dass es z.B. zu einem Projekt in dem verschiedenen Meetings statt finden, weiterführende Dokumente wie z.B. ein Anforderungsdokument gibt, die für den Benutzer von MISTRAL ebenfalls von Interesse sind. Hier kommen zusätzliche, rein auf der Modalität Text basierende Verfahren - wie beispielsweise Ähnlichkeitssuchen und Clustering - zum Einsatz.

Im MISTRAL System deckt das KnowMiner-Framework einerseits die Extraktion der Themenmerkmale aus den Klick-Daten und die Bereitstellung von Diensten zum Umgang mit den oben genannten Dokumentkorpora ab. Das KnowMiner-Framework bietet dabei die komplette Funktionalität, über das in Abschnitt 8.5.1 vorgestellte TopLevelService, als Web-Service, an. Während der Extraktion der Textmerkmale kommen dabei die unterschiedlichen Funktionen wie Clustering, Klassifikation, Retrieval etc. zum Einsatz. Im Zuge der Anwendung des TopLevleService's von MISTRAL erfolgte auch die Umsetzung des in Abschnitt 8.5.3 beschriebenen Service Frontend.

9.2.3 Zusammenfassung und Lessons Learned

Bei der Integration des KnowMiner-Frameworks ins Mistral Szenario kam die gesamte Funktionalität in Form eines Web-Service zum Einsatz. Bei der Erstellung dieses Web-Service lag das Hauptaugenmerk auf der Orchestrierung der Basis-Services und deren Kombination. Aus der Erstellung dieses Web-Service, sowie der Anwendung des KnowMiner-Frameworks innerhalb des MISTRAL-Projektes konnten die folgenden Erkenntnisse gezogen werden:

In Bezug auf nicht sequentielle Daten erwies sich die Strukturierung des Informationsraums als hinreichend. Problematisch zeigte sich jedoch die Integration von sequentiellen Daten in die Struktur des Informationsraumes. Die Abbildung solcher Strukturen konnte nicht effizient umgesetzt werden, da Relationen und deren programmatische Unterstützung zur Realisierung von Sequenzen fehlen. Durch hinzufügen einer einfach Zeitrelation wäre eine Abbildung zwar möglich, jedoch erweist sich die Verwendung als sehr komplex wenn die Unterstützung durch die ElementSet Klasse fehlt.

Bewährt hat sich auch die Einbindung von Open Source Bibliotheken bzw. die Möglichkeit dies einfach zu integrieren. Der Content Processor bietet hier mit den Pipeline Elementen eine guten Ansatzpunkt Annotationstools einzubinden und diese leicht verfügbar zu machen, wie es mit GATE gemacht wurde.

Im MISTRAL Projekt erwies sich die Integration von bereits vorhandenen KnowMiner-Services als einfach, wodurch der Fokus mehr in Richtung Implementierung neuer Services und der Verbesserung der bestehenden Services verlagert werden konnte. Die einfache Erweiterung der Services weißt auf eine gute funktionale Trennung hin. Zusätzlich erfolgte während dem MISTRAL Projekt ein Re-Design wichtiger Kernkomponente wie z.B. der GIR, die sich jedoch kaum auf die Schnittstellendefinition der Services auswirkte. Daraus kann auf eine gute Kapselung der einzelnen KnowMiner Schichten geschlossen werden und es weißt auf eine gute Austauschbarkeit einzelner Softwaremodule hin.

Die semantische Beschreibung in Form des GIR-Kontextes erwies sich bei Teilen der Basisdienste als schwierig und konnte nicht vollständig umgesetzt werden. Vor allem bei Struktur modifizierenden Diensten erwies sich eine Realisierung der Kontexte, aufgrund der Komplexität der Beschreibung der Struktur des Informationsraumes, als schwierig.

Zusammenfassend können folgende Kernpunkte heraus gestrichen werden:

1. Gute Erweiterbarkeit und Wiederverwertbarkeit der Services durch funktionale Kapselung und modulares Schichtenmodell

2. Gute Integrationsmöglichkeit für Open Source Bibliotheken

3. Erweiterung der Strukturierungsmöglichkeiten des Informationsraumes notwendig

4. Semantische Beschreibung in Form des GIR-Kontextes nur teilweise realisierbar

9.3 Sonstige Anwendungen unter Verwendung des KnowMiner-Frameworks

Neben den bisher erwähnten Großprojekten, in denen das KnowMiner-Framework zum Einsatz kam, entwickelten bzw. entwickeln weitere Projekte Applikationen unter Verwendung des KnowMiner-Frameworks. Dieser Abschnitt beleuchtet das aktuell in Umsetzung befindliche Projekt DYONIPOS, sowie eine bereits abgeschlossene Diplomarbeit zur semiautomatischen Erzeugung von Wissenslandkarten.

Die aus den Projekten abgeleiteten Erkenntnisse summiert der Abschluss dieses Abschnitts.

9.3.1 DYONIPOS - Dynamic Ontology based Integration Process Optimization

Das aktuelle am Know-Center durchgeführte FIT-IT Forschungsprojekt DYONIPOS beschäftigt sich mit der Optimierung von Geschäftsprozessen und der Bereitstellung relevanter Information im Kontext von Geschäftsprozessen. Um einen Eindruck über die Ausrichtung von DYONIPOS zu geben, wird im Folgenden der Text zur Projektbeschreibung, der unter Mithilfe des Autors erzeugt wurde, angegeben. Der Text erscheint im Laufe des Jahres bei der FFG.

„Zielsetzung von DYONIPOS ist die Entwicklung eines kontextsensitiven, intelligenten, hochflexiblen Systems, welches zu einer signifikanten Produktivitätssteigerung in wissensintensiven Unternehmen beiträgt. Basierend auf semantischen Technologien, Methoden des Knowledge Discovery und Verfahren wie der Wissensflussanalyse integriert DYONIPOS strukturierte, explizit abgebildete Geschäftsprozesse und dynamische, undokumentierte Wissensflüsse. Semantische Technologien ermöglichen dabei eine flexible und einheitliche Behandlung von strukturierten als auch unstrukturierten Geschäftsprozessen und Daten. DYONIPOS adressiert zwei entscheidende Arbeitsbereiche der Wissensarbeit in agilen Organisationen: Einerseits unterstützt DYONIPOS den einzelnen Prozessausführenden bei der effizienten, standardisierten

aber dennoch flexiblen Abarbeitung von Geschäftsprozessen gemäß seiner individuellen Arbeitsweise, andererseits den Prozessingenieur bei der Entwicklung standardisierter Prozesse, basierend auf tatsächlich von den unterschiedlichen Wissensarbeitern ausgeführten Geschäftsprozessinstanzen.

Als prozessorientiertes Ziel setzt sich DYONIPOS die Lösung des Dilemmas hinsichtlich genügend Flexibilität für Wissensarbeiter auf der einen Seite und der organisatorischen Anforderung nach Standardisierung und Kontrolle auf der anderen Seite. Die Herangehensweise motiviert die Entwicklung und Evolution von Arbeitsflüssen aus ad hoc Prozessen, ohne die notwendige, organisatorische Kontrolle aufzugeben. Ad hoc Prozesse bilden dabei das Bindeglied zwischen den Ausführenden von Geschäftsprozessen und dem Geschäftsprozessingenieur. Sie bestehen aus definierten Einsatzfaktoren (Inputs) und definierten Ergebnissen (Outputs), überlassen jedoch die Art und Weise der Abwicklung dem Wissensarbeiter und dem zugehörigen Kontext. Die Interaktionen zwischen Wissensarbeitern und der digitalen Arbeitsumgebung werden aufgezeichnet, analysiert und semantisch angereichert.

Technologisch orientiertes Ziel von DYONIPOS ist die Entwicklung fortgeschrittener und flexibler semantischer Technologien, welche in der Lage sind, die Vielzahl an technologischen Herausforderungen zu meistern, die in der Abwicklung von ad hoc Prozessen und ihrer Integration mit Standardprozessen auftreten. Dem Prozessanwender erleichtert DYONIPOS seine Aufgaben durch intelligentes Vorschlagen von Nachfolgeprozessen, durch intelligente Informationsbereitstellung und intelligente Informationsextraktion basierend auf dem aktuellen Kontext.

Der Prozessingenieur wird bei der semantischen Analyse von ad hoc Prozessen unterstützt, indem Methoden und Algorithmen zur automatischen Optimierung und Visualisierung bereitgestellt werden. Insgesamt entwickelt DYONIPOS neue Technologien, welche dynamische und flexible Wissensprozesse optimieren und dennoch Standardisierung und Kontrolle erlauben."

Vor allem die Informationsbereitstellung und -aufbereitung wird dabei vom KnowMiner-Framework übernommen, wozu Services bereitgestellt werden. Clientseitig erfolgt die Erfassung der Benutzeraktivitäten und die Erfassung der vom Benutzer betrachteten Ressourcen. Unter Anwendung von Regeln sowie Methoden aus dem Bereich Process Mining extrahiert der Client den Kontext des Benutzers in Form eines Graphen. Sowohl die aktuell bearbeiteten Inhalte als auch der Kontext übergibt der Client an das serverseitig laufende KnowMiner-Framework, welches zum Kontext und zum Inhalt passende Dokumente, Experten und andere Ressourcen dem Client liefert. Die so dem Benutzer „Just-in-Time" bereitgestellten Ressourcen sollen die tägliche Wissensarbeit effektiv unterstützen.

Aktuell ist im Projekt DYONIPOS das Planungsstadium abgeschlossen und es kommt zu ersten Implementierungen. Während serverseitige Algorithmen, vor allem im Bereich des Retrievals, neu zu entwickeln sind, erweist sich die Service- und Modulstruktur als hinreichend, um die benötigte Funktionalität abbilden zu können. Auch die aktuell laufende Integration mit von Projektpartnern bereitgestellten serverseitigen Funktionalitäten sowie die Datenanbindung erweisen sich als hinreichend einfach.

Zum aktuellen Projektstand erweist sich die Verwendung des KnowMiner-Frameworks als vorteilhaft, da der Integrationsaufwand der im Projekt vorhandenen Technologien deutlich reduziert werden kann.

9.3.2 Semiautomatische Erzeugung von Wissenslandkarten

Die letzte Anwendung integrierte das KnowMiner-Framework im Rahmen einer Diplomarbeit [Neidhart 2005], in der es um die semiautomatische Erstellung von Wissenslandkarten ging. Während MISTRAL und XTrend auf den vom KnowMiner-Framework bereitgestellten Services aufsetzten, arbeitete die Diplomarbeit direkt mit dem KnowMiner Core. Gründe dafür waren einerseits die Entwicklung neuer Algorithmen zum Lernen von Ähnlichkeiten zwischen Entitäten auf Basis von Benutzervorgaben und andererseits die gerade erst in Entwicklung befindliche Service-Architektur.

Die Grundidee der Arbeit spiegelt Abbildung 9-4 wider. Unter Anwendung von Clustering- und MDS-Techniken erzeugt die entwickelte Applikation eine zweidimensionale Wissenslandkarte, in der ähnliche Objekte räumlich näher zueinander zum Liegen kommen als unähnliche Objekte. Die Distanz in der Wissenslandkarte gibt also direkt Aufschluss über die Ähnlichkeit zweier Objekte (siehe Vektorraum ➔ 2D Raum in Abbildung 9-4, Mitte oben und rechts oben).

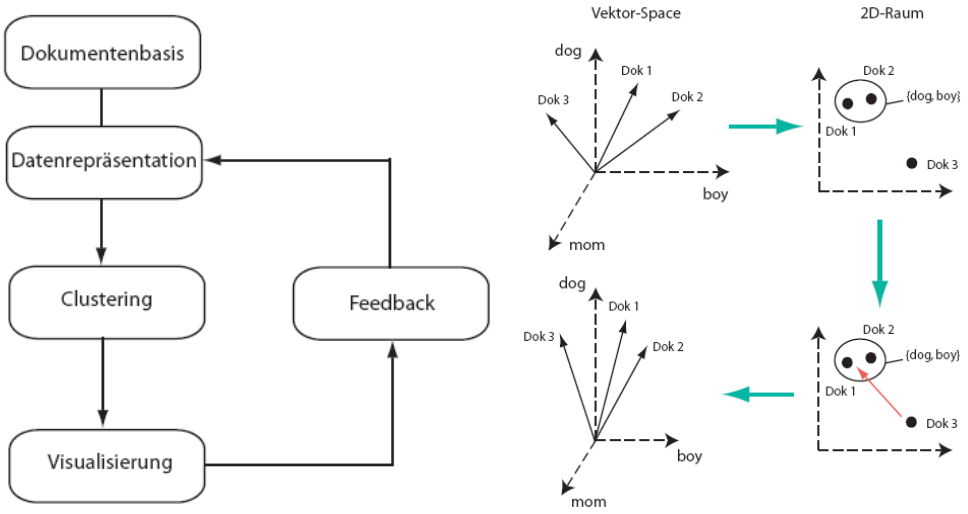


Abbildung 9-4: Überblick über den Feedback-Loop zur Erzeugung von Wissenslandkarten

Da eine solche Darstellung meist nicht den Erwartungen des Benutzers entspricht erlaubt die entwickelte Applikation dem Benutzer das Layout nach seinen Anforderungen anzupassen (Abbildung 9-4 rechts unten). Vor dem Hintergrund, dass Dokumenbestände meist hoch dynamisch sind und somit neue Dokumente ebenfalls in das Layout eingepasst werden müssen, erlaubt die Applikation nicht nur die Anpassung des Layouts, sondern errechnet gleichzeitig eine neue hochdimensionale Repräsentation, die die Änderungen des Benutzers berücksichtigt.

Bei der nächsten Berechnung der Wissenslandkarte soll durch Adaption der hochdimensionalen Repräsentation - und damit verbunden auch der Anpassung der Ähnlichkeiten zwischen den Objekten - die Visualisierung entstehen, die sich der Benutzer erwartet. Auch die Integration neuer Dokumente kann durch das implementierte Verfahren berücksichtigt werden. Die Arbeit zeigt, dass dies unter gewissen Voraussetzungen möglich ist. Für Details sei der Leser auf die Publikation [Granitzer et. al. 2006] verwiesen.

Die beschriebene Funktionalität wurde unter Anwendung des KnowMiner-Frameworks umgesetzt und demonstrierte dabei die Flexibilität der Repräsentation von Merkmalen und Ähnlichkeitsmaßen. Während in

der Entstehung der Arbeit neue Algorithmen entwickelt und integriert wurden, konnte auch auf bereits existierende Verfahren wie MDS einfach zurückgegriffen werden. Dennoch zeigte sich, dass Anwendungen auf Basis der Kernfunktionalität des KnowMiner-Frameworks komplex in ihrer Anwendung und zu abhängig von Änderungen im Kern sind. Parallel zu dieser Arbeit kam es zu solchen Änderungen und Erweiterungen am KnowMiner-Framework, wodurch Änderungen in der vorliegenden Applikation ebenfalls häufig durchzuführen waren. Auch der Einarbeitungsaufwand zur Benutzung der Kernfunktionalität erwies sich als zu hoch, was auf die Entwicklung höheren Abstraktionsebenen, wie z.B. der Service Architektur, hinwies.

9.3.3 Lessons Learned

Aus den aktuell auf Basis des KnowMiner-Frameworks laufenden Projekten und der abgeschlossenen Diplomarbeit können folgende Lessons Learned abgeleitet werden:

1. Die Integration des KnowMiner-Frameworks mit unterschiedlichen Applikationen und in unterschiedlichen Anwendungsszenarien erweist sich als einfach, d.h. sie ist unter geringem Aufwand durchführbar

2. Die Modularität der Dienste erweist sich als flexibel genug, um in unterschiedlichen Projekten einsetzbar zu sein

3. Die Abstraktion der internen Repräsentation erweist sich ebenfalls als flexibel und bis dato hinreichend

4. Algorithmen können einfach wieder verwendet und erweitert werden.

5. Der Einarbeitungsaufwand in Details des KnowMiner-Framework erweist sich als hoch

9.4 Zusammenfassung

Dieser Abschnitt stellte die unter Anwendung des KnowMiner-Frameworks durchgeführten Projekte vor. Die Projekte sind dabei in unterschiedlichen Anwendungsbereichen angesiedelt und haben unterschiedliche Zielsetzungen. Bei der Durchführung und in der Nachbetrachtung der Projekte ließen sich, bezogen auf das KnowMiner-Framework, wichtige Erkenntnisse hinsichtlich Eigenschaften, Anwendbarkeit und noch offenen Punkte des Frameworks ableiten.

Zum Abschluss des Kapitels sollen die in Bezug auf das KnowMiner-Framework abgeleiteten Erkenntnisse nochmals, wie folgt, zusammengefasst werden:

- Die gewählte Service Architektur erwies sich als modular und flexibel einsetzbar, sowohl in Bezug auf verschiedene Anwendungsszenarien als auch in Bezug auf die Integration mit verschiedenen anderen Applikationen. Zusätzlich zeigte sich eine gute Wiederverwertbarkeit der einzelnen Algorithmen und eine einfache Anbindung unterschiedlicher Datenquellen.

- Im Multi-User Betrieb bewährte sich sowohl die Service-Architektur, als auch die darunter liegende Worker-Architektur. Verschiedene Algorithmen konnten parallel am gleichen Datenbestand arbeiten

ohne dabei konkurrente Zugriffe zu verursachen. Das Locking von Entitätseigenschaften erwies sich dabei als für den Anwender komplex, wenn auch lösbar. Hierbei ist jedoch zu berücksichtigen, dass die semantische Beschreibung nicht vollständig erreicht werden konnte und dadurch verschiedene Preconditions inkl. der benötigten Locks nicht zur Verfügung standen.

- Die interne Abstraktion von Informationsraum und Eigenschaften der Entitäten erwies sich ebenfalls als gut anwendbar. Die Abstraktion der Operationen und der Merkmalstypen ermöglichte eine gute Trennung zwischen Algorithmen und Daten sowie eine einfache Anpassbarkeit hinsichtlich unterschiedlicher Ähnlichkeitsmaße.

- Die Einbindung von Open Source Bibliotheken erwies sich ebenfalls als hilfreich. Dadurch konnte zuverlässige Funktionalität bereitgestellt werden.

- Offen bleibt die semantische Beschreibung, die sich vor allem bei den Struktur-modifizierenden Diensten als äußerst komplex erwies. Die dadurch verbesserte Orchestrierung sollte die Anwendbarkeit der Services zusätzlich verbessern und die Fehleranfälligkeit reduzieren, was jedoch nur bei den einfachen Services umsetzbar war.

- In den Anwendungen zeigte sich auch die Notwendigkeit der flexibleren Erweiterung des Typsystems. Derzeit erfolgt die Typisierung programmatisch, ein Umstand der die Flexibilität einschränkt. Ein flexibleres, semantisch beschriebenes Typsystem sowohl auf Ebene der Eigenschaften als auch auf Ebene des Informationsraums sollte die Flexibilität erhöhen und vor allem die Datenkonvertierungen zwischen verschiedenen Servicearten verbessern.

Alles in allem erwies sich das KnowMiner-Framework als gut einsetzbar, wenngleich noch einige Punkte offen geblieben sind.

10 Zusammenfassung und offene Punkte

Ziel dieses letzten Abschnitts ist die Zusammenfassung der Arbeit, die Zusammenfassung der Ergebnisse und der abgeleiteten Erkenntnisse sowie der Vergleich des Erreichten zu den Zielen der Arbeit. Ergebnisse und Erkenntnisse leiten sich dabei einerseits aus dem vorherigen Kapitel und aus den in den Iterationszyklen abgeleiteten Ergebnissen/Erkenntnissen, wie in Kapitel 6 beschrieben, ab.

Auf Basis der gesammelten Ergebnisse erfolgt ein Vergleich zur Zielsetzung der Arbeit. Im Anschluss der aktuelle Stand des KnowMiner-Frameworks mit den in Abschnitt 5.1 eingeführten Frameworks verglichen wird, um dabei die Unterschiede heraus zu arbeiten. Das Kapitel schließt mit der Diskussion der gewonnenen Erkenntnisse, offenen Punkte und darauf aufbauend weiterführenden Arbeiten.

10.1 Ergebnisse und Vergleich mit der Zielsetzung

In Abschnitt 1.1 erfolgte die Zielsetzung der Arbeit als

„die Konzeption und Entwicklung eines integrativen und flexiblen Software Frameworks zur Umsetzung von Knowledge Discovery und Information Retrieval Diensten und die Möglichkeit der Einsetzbarkeit dieses Frameworks in unterschiedlichen Anwendungsgebieten."

wobei folgende Verfeinerungen des Gesamtziels beschrieben wurden:

1. *Unabhängigkeit von den zu verarbeitenden Datenquellen und der zu verarbeitenden Informationsart*
2. *Parallelisierbarkeit von Methoden und Algorithmen zur Unterstützung von Multi-User-Szenarien*
3. *Wiederverwendbarkeit bereits entwickelter Techniken in unterschiedlichen Anwendungsszenarien und auf unterschiedlichen Informationsmengen*
4. *Integration von Open Source - Bibliotheken*
5. *Einfache Erweiterbarkeit*

Auf Basis dieser Zielsetzung analysierten die Kapitel 2-5 die Anforderungen an das Framework, die in Kapitel 6 in den verschiedenen Entwicklungszyklen verfeinert und ergänzt wurden. Wie Kapitel 8 zeigt, erfüllt das KnowMiner-Framework die gestellten Anforderungen. Die in Kapitel 9 beschriebenen Anwendungen des Frameworks und die daraus abgeleiteten Erkenntnisse dienen nun zur Analyse der Realisierung der gestellten Zielsetzung.

Unabhängigkeit von den zu verarbeitenden Datenquellen und der zu verarbeitenden Informationsart: Die Anwendungen des KnowMiner-Frameworks zeigen ein breites Spektrum an unterschiedlichen Datenquellen und Informationsarten. Während Applikationen wie OnAIR rein auf Textdaten fokussierten, erlaubte MAGICK die Integration von Bild- und Audiodaten. Aktuell erfolgt die Erweiterung des KnowMiner-Frameworks in Richtung Graph basierter Informationen im Forschungsprojekt DYONIPOS. Während der Großteil der umgesetzten Anwendungen auf Text und Metadaten ausgelegt sind, ein Umstand der auch den in Kapitel 2 erarbeiteten Anforderungen entspricht, zeigen diese Beispiele doch die Erweiterbarkeit und Unabhängigkeit hinsichtlich der Informationsart.

Betrachtet man die verschiedenen verarbeitbaren Datenquellen so zeigt sich bzgl. der umgesetzten Anwendungen ein sehr heterogenes Bild. Ausgehend von als Dateien vorliegenden Dokumenten in onAIR und MAGICK, ermöglichte XTrend die Einbindung von Datenbanken als Informationsquelle und MISTRAL integriert Usage Data bei der Analyse der Klick-Daten. Die unterstützten Formate reichen dabei von einfachem Text über PDF und Word bis hin zu strukturierten, XML und RDF basierten Daten wie z.B. in der hierarchischen Textklassifikation, die den DMOZ Datensatz verwendet.

Angesichts der einfachen Erweiterbarkeit kann dieses erste Kriterium der Zielsetzung als erfüllt angesehen werden.

Parallelisierbarkeit von Methoden und Algorithmen zur Unterstützung von Multi-User-Szenarien: Sowohl auf Service Ebene als auch im Kern des Frameworks wird die Parallelisierbarkeit von Algorithmen und Methoden berücksichtigt. Während die parallele Abarbeitung grundlegend vom Algorithmus und dessen Implementierung abhängt, stellt das KnowMiner-Framework eine Umgebung zum konkurrenten Zugriff auf den gemeinsamen Informationsraum und zur Thread-sicheren Abwicklung von Verarbeitungsschritten bereit, wodurch die Implementierung von parallelen Algorithmen erleichtert wird.

Die Multi-User Fähigkeit und die parallele Abarbeitung von Workern zeigt sich in den verschiedenen Anwendungen wie z.B. XTrend, MISTRAL und OnAir, die jeweils Funktionalität als Web Service zur Verfügung stellen.

Wiederverwendbarkeit bereits entwickelter Techniken in unterschiedlichen Anwendungsszenarien und auf unterschiedlichen Informationsmengen: Die Fülle an unterschiedlichen Anwendungen zeigt deutlich die Wiederverwendbarkeit bereits entwickelter Techniken und die Anwendung auf unterschiedlichen Informationsmengen. So führt beispielsweise MAGICK Clustering und MDS auf Basis multimedialer Daten durch, während OnAIR und die semiautomatische Erzeugung von Wissenslandkarten die gleichen Methoden zum Clustering und Visualisieren von Textdaten anwendet. Teilweise erfolgte auch die Erweiterung und Adaption der verschiedenen Algorithmen.

Integration von Open Source – Bibliotheken: Mit der Integration von GATE, Lucene und der MTJ Bibliothek konnten erfolgreich verschiedene Open Source Bibliotheken eingebunden werden. Die Modularität des Frameworks erlaubt hierbei verschiedene Eingriffspunkte, je nach Art der Anwendung. Durch die Einbindung der Bibliotheken konnte auch eine breite Grundlage an stabilen Algorithmen geschaffen und als Services bereitgestellt werden.

Einfache Erweiterbarkeit: Der im KnowMiner-Framework verfügbare Funktionsumfang und die Anwendungen in den verschiedenen Szenarien sind ein Indikator für die relativ einfache Erweiterbarkeit des KnowMiner-Frameworks. Problematisch erweisen sich Erweiterungen im Kern des Frameworks, da hier detaillierte Kenntnisse über die Funktionsweise benötigt werden. Auch müssen hier teilweise Abstriche im Vergleich zur Entwicklung von hochspezialisierten Lösungen hingenommen werden. So ist die Entwicklung einer reinen Suchlösung unter Anwendung von Lucene, einfache Informations-/Dokumentbestände vorausgesetzt, einfacher zu bewerkstelligen als die Integration von Lucene in das KnowMiner-Framework und darauf aufbauend die Umsetzung der Anwendung. Zu berücksichtigen ist hier jedoch, das durch die Breite der Funktionalität des KnowMiner-Frameworks und durch den integrativen Aspekt in komplexen, heterogenen Umgebungen, in solchen Anwendungsszenarien deutliche Vorteile auf Seiten des KnowMiner-Frameworks liegen.

In Summe betrachtet wurde die Zielsetzung in allen Punkten erreicht, wobei noch – wie im Ausblick diskutiert - Raum zu Verbesserungen offen bleibt. Suma Summarum stellt das KnowMiner-Framework ein integratives und flexibles Software Framework zur Umsetzung von Knowledge Discovery und Information Retrieval Diensten dar. Die Anwendung in den verschiedenen oben illustrierten Projekten zeigt auch die Möglichkeit der Einsetzbarkeit dieses Frameworks in unterschiedlichen Anwendungsgebieten auf.

10.2 Vergleich zu existierenden Systemen

Um einen Vergleich zu ähnlichen Systemen in diesem Umfeld ziehen zu können, betrachtet der nächste Abschnitt die in Abschnitt 5.1 beschriebenen Frameworks UIMA, GATE, WEKA und ATLAS.

GATE: Ähnlich wie im KnowMiner-Framework, trennt GATE zwischen Daten und Algorithmen, wodurch eine datenunabhängige Algorithmenentwicklung und eine anwendungsspezifische Kombination möglich wird. Im Vergleich zum KnowMiner-Framework bietet GATE jedoch lediglich Funktionen zur Annotation von Dokumenten an und folgt dabei einem einfachen Pipelineprinzip, indem immer nur ein Dokument prozessiert wird. Auch unterstützt GATE keine multi-modalen Analysemethoden, wie dies bei UIMA der Fall ist. Im Vergleich zum KnowMiner-Framework sind die einzelnen GATE-Ressourcen auch nicht Multithreading fähig, wodurch es zu Schwierigkeiten bei der Entwicklung in multithreaded Umgebungen kommen kann. GATE bietet aber ausgezeichnete Technologie im Bereich NLP und erlaubt das einfache Erstellen von NLP Applikationen. Dieser Umstand wurde auch bei der Umsetzung des KnowMiner-Frameworks bedacht, woraus eine Integration von GATE in den KnowMiner Annotierungsteil erfolgte.

UIMA: UIMA verfolgt ähnliche Ziele wie das KnowMiner-Framework, wobei im KnowMiner-Framework eine stärkere Strukturierung hinsichtlich der verschiedenen KD-Methoden wie Clustering, Klassifikation sowie die Definition der Schnittstellen zu diesen erfolgt. UIMA unterstützt als Framework lediglich Prozessoren für einzelne und für mehrere Objekte, definiert aber nicht die Art der Prozessierung. Dadurch existieren bei UIMA auch keine definierten Mechanismen zur Unterstützung unterschiedlicher Strukturierungen des Informationsraumes. UIMA unterscheidet auch nicht zwischen Annotationen und Merkmalen, wie es im KnowMiner-Framework der Fall ist, bietet dafür aber eine bessere Anbindung hinsichtlich Web-Services und Distributed Processing, da diese Komponenten im KnowMiner-Framework erst im letzten Zyklus integriert wurden. Auch verfügt UIMA momentan über ein flexibleres Typsystem, welches in das KnowMiner-Framework jedoch ebenfalls mit geringem Aufwand integriert werden kann. Dafür bietet das KnowMiner-Framework die mächtigere Pipeline (UIMA unterstützt momentan nur lineare Pipes, wenngleich dies erweiterbar ist) und eine modularere Service Struktur.

WEKA: Im Vergleich zu WEKA erlaubt das KnowMiner-Framework eine flexiblere Anbindung an bestehende Datenquellen - eine zentrale Eigenschaft zur schnellen Umsetzung von KD-Anwendungen. Das KnowMiner-Framework bietet auch ein API an, um WEKA-Algorithmen einbinden zu können und das KnowMiner interne Format auf das WEKA-Format transformieren zu können. Im Vergleich zum KnowMiner-Framework und den anderen vorgestellten Frameworks bietet WEKA auch keine Möglichkeit, Merkmale semantisch zu beschreiben bzw. eigene Typsysteme zu verwenden. WEKA zielt hauptsächlich auf die schnelle und einfache Entwicklung und Evaluierung neuer Algorithmen und den Austausch von Standardtestdaten ab. Ein Grund für den hohen Beliebtheitsgrad in der wissenschaftlichen Gemeinschaft.

ATLAS: Während ATLAS viel Wert auf die semantische Annotierung legt, bietet es im Vergleich zum KnowMiner-Framework momentan kein Framework zur Entwicklung von KD-Applikationen und deckt mit

dem Fokus auf Annotationen auch nur einen kleinen Teilbereich im KD ab. Nichtsdestotrotz stellt die hochwertige semantische Annotation eine Möglichkeit der Erweiterung des KnowMiner-Frameworks dar, im speziellen der Annotationsfunktionalität.

10.3 Ausblick

Die mit dem KnowMiner-Framework umgesetzten Projekte und die daraus abgeleiteten Erkenntnisse erlauben die Identifikation einiger offener Punkte für die Weiterentwicklung des KnowMiner-Frameworks. Zum Abschluss der Arbeit gibt der vorliegende Abschnitt einen Überblick über die weiteren Forschungsansätze sowie einen Ausblick in Richtung der zu erwartenden zukünftigen Entwicklungen im Bereich des Knowledge Discovery.

Erster Ansatzpunkt für zukünftige Arbeiten ist die Erweiterbarkeit der semantischen Beschreibung der KnowMiner-Services und die damit verbundene Orchestrierung. Bei der programmatischen Beschreibung der Services in Form des *GIRContext*-Objektes (Abschnitt 8.5.2.2) zeigte sich, dass vor allem die Definition der verarbeitbaren Informationsstrukturen schwierig realisierbar ist. Weiterführende Arbeiten können untersuchen, inwieweit ontologische Ansätze hier Möglichkeiten zur Beschreibung bieten. Über Ontologien wäre auch eine hochdynamische Orchestrierung der Dienste realisierbar. Über Anwendung von Inferenzen kann die Prüfung der Ausführbarkeit und die automatische Komposition von Services erreicht werden. Aktuell fehlen auch noch Möglichkeiten zur Suche nach Services in Form einer Registrierung, was mittels der ontologischen Beschreibung einfach realisierbar ist. Die Lösung all dieser Punkte kann die Bereitstellung von Semantischen Web Services für den Bereich Knowledge Discovery ermöglichen.

Wie bereits oben erwähnt, ist die Erweiterung der programmatischen Typisierung von Entitäten, Relationen sowie Eigenschaften von Entitäten ein denkbarer Ansatzpunkt für weiterführende Arbeiten. Ein flexibleres, semantisch eindeutigeres Typsystem kann die Anwendung des KnowMiner-Frameworks und dessen Flexibilität noch weiter steigern. Die Typisierung unter Verwendung semantischer Beschreibungssprachen würde dabei auch den oben genannten Punkt der Orchestrierung begünstigen, wobei bei solchen Modellierungsansätzen immer von einem hohen Arbeitsaufwand auszugehen ist.

Hinsichtlich der Kernfunktionalität sind offene Punkte im Bereich der Repräsentation von Merkmalen hervor zu heben. Einerseits sollte hier eine engere Integration von Techniken aus dem Bereich der linearen Algebra erfolgen und andererseits ist die modulare Kapselung von Metriken hinsichtlich der Integration von Kernel-Methoden suboptimal. Zu prüfen ist, inwieweit Kernel-Methoden als zentrale Methodik für Merkmalen heran gezogen werden können. Vor allem, da Kernel-Methoden umfassende Ähnlichkeitsvergleiche, auch in Bezug auf unterschiedliche Repräsentationsformen wie zum Beispiel Graphen oder Vektoren, anbieten.

Ein weiterer wichtiger Punkt ist das verteilte Rechnen. Aktuell geht das KnowMiner-Framework immer von einer einzelnen, physikalischen Maschine zur Verarbeitung der Daten und Algorithmen aus. Gerade im Bereich Knowledge Discovery und verbunden mit der weiter steigenden Informationsmenge zeigt sich aber, dass verteiltes Rechnen essentiell für die Verarbeitung großer Informationsmengen und damit verbunden für

den Erfolg von Anwendungen ist. Auch hier existieren Open Source Bestrebungen wie das Hadoop[52] Projekt der Apache Software-Foundation, welches ein Framework zur Ausführung von Algorithmen auf Basis des MapReduce Konzepts von Google bereitstellt. Dadurch sind bis zu 600 Knoten parallel ansprechbar. Eine andere Möglichkeit wäre die Integration von GRID Technologie über zum Beispiel das Globus Toolkit[53]. Damit kann die Skalierbarkeit des aktuellen Frameworks erhöht und die nächste Größenordnung an verarbeitbaren Dokumentbeständen geschafft werden.

Das entwickelte KnowMiner-Framework bietet momentan einen großen, einfach integrierbaren Funktionsumfang. Die Integration der hier besprochenen Punkte würde weiters zur Steigerung der Qualität der Dienste über semantische Erschließung und zur Steigerung der Leistungsfähigkeit über die Verteilung von Algorithmen in Cluster-Systemen beitragen.

Abgesehen von den Erweiterungen des Frameworks an sich bietet das KnowMiner-Framework eine gute Basis für die Entwicklung neuer Wissenserschließungstechniken und kann als Fundament für die Entwicklung neuer Forschungsrichtungen zur semantischen Anreicherung und Strukturierung von Information angesehen werden.

Aktuell kristallisieren sich einige dieser möglichen Richtungen heraus. Beispielsweise ist eine starke Tendenz in Richtung der Bereitstellung von Diensten zur Verbesserung von wissensintensiven Arbeitsumgebungen zu beobachten. Eine Teilmenge dieser Dienste benötigt Wissenserschließungs- und Retrievalfunktionalität, welche vom KnowMiner-Framework bereitgestellt werden kann. Die einfachere Orchestrier- und Verwaltbarkeit von Diensten wird von Forschungstrends aufgegriffen, die in Richtung ontologische Beschreibung von Web-Services, so genannten Semantic Web-Services, gehen, welche die Anwendung und Verteilung von Diensten vereinfachen. Die vollständige semantische Beschreibung solcher Dienste ermöglicht die Erstellung intelligenter Agenten, die in dem Pool der zu Verfügung stehenden Services automatisch die notwendigen Services zur Erfüllung von komplexen Aufgaben finden und in der richtigen Reihenfolge anwenden. Semantische Web Services können also als Wegbereiter für Tim Berners Lee's Vision vom Semantic Web verstanden werden.

Neben der Bereitstellung von Diensten zeichnet sich auch eine enge Integration deduktiver Modellierungstechniken mit induktiven Techniken aus dem Bereich des Text Mining und Knowledge Discovery ab. Ziel dabei ist es, den Aufwand der Modellierung von Ontologien zu verringern sowie die Validierung von Ontologien zu verbessern. Umgekehrt können Ontologien zur Verbesserung der Wissenserschließungstechniken heran gezogen werden, wie dies z.B. beim Ontology Based Information Extraction der Fall ist. Betrachtet man dies als Regelkreis, so zeichnet sich eine enge Kombination, eine gegenseitige Befruchtung von Ontologien und Wissenserschließungstechniken und damit eine Erhöhung der Informationsqualität ab.

Ein wichtiges Kriterium für die Qualität der Wissenserschließung stellt die Berücksichtigung von unterschiedlichen Anwendungskontexten dar. Um Wissen in einer für den Benutzer verständlichen und der

[52] http://projects.apache.org/projects/hadoop.html

[53] http://www.globus.org/ogsa/

Domäne entsprechenden Art und Weise präsentieren zu können, muss der jeweilige Kontext mit berücksichtigt werden. Problematisch hierbei ist wiederum die Abbildung des Kontexts in eine für die Wissenserschließung verwendbare Form. Eine Möglichkeit hierfür besteht im Ausnützen der Prozesskontexts (wie z.B. in DYONIPOS) zur Steuerung der Ähnlichkeiten zwischen Objekten. Eine vom aktuellen Prozessschritt abhängige, kontext-sensitive Informationsbereitstellung wäre dadurch beispielsweise möglich. Die engere Integration von Wissenserschließungstechniken, die nicht mehr nur ausschließlich Experten zugänglich sind, bedarf einer Benutzer-zentrierten Herangehensweise und damit auch der Berücksichtigung unterschiedlicher Kontexte.

Während der Arbeit am KnowMiner-Framework kristallisierte sich zunehmend die Wichtigkeit der Beziehungen zwischen Entitäten und damit verbunden die Beziehung zwischen Informationen heraus. Auch für die Zukunft zeichnet sich ein starker Trend in Richtung der Analyse von Beziehungen, anstatt einzelner Informationsträger, heraus. Knowledge Relationship Discovery bezeichnet diesen Trend, wo nicht mehr das Entdecken von Mustern, sondern das Entdecken von Beziehungen zwischen einzelnen Wissensträgern im Vordergrund steht. Die Kombination von verschiedenen Wissensträgern über verschiedene Wissens- und Informationsräume hinweg birgt das Potential zur Erschließung von Informationsbeständen aus verschiedenen Blickwinkeln und unter Einbeziehung des Wissens des Benutzers in sich. Die Schaffung eines virtuellen Wissensraums, der verschiedene heterogene Wissens- und Informationsräume verbindet und zugänglich macht, stellt eine Lösung mit hohem Potential im Umgang mit der Informationsflut und möglicherweise die nächste Generation von Informationssystemen dar.

Literaturverzeichnis

[Ackerschott 2001] ACKERSCHOTT, Harald: Wissensmanagement fur Marketing und Vertrieb. 1. Gabler Verlag, Wiesbaden, 2001

[Aha et. al. 1991] Aha, D.; Kibler, D. & Albert, M. (1991), 'Instance-Based Learning Algorithms', Machine Learning 6, 37-66.

[Ahad & Yao 1993] Ahad, R. & Yao, B. (1993), 'RQL: A Recursive Query Language', IEEE Trans. Knowl. Data Eng. 5(3), 451-461.

[Andrews 2002] Andrews, K. (2002),'Visualising Information Structures: Aspects of Information Visualisation', Professorial Thesis, IICM, Graz University of Technology, Austria.

[Andrews et. al. 2002] Andrews, K.; Kienreich, W.; Sabol, V.; Becker, J.; Droschl, G.; Kappe, F.; Granitzer, M.; Auer, P. & Tochtermann, K. (2002), 'The InfoSky visual explorer: exploiting hierarchical structure and document similarities', Information Visualization 1(3/4), 166--181.

[Baeza-Yates et. al 1999] Baeza-Yates, R.A. & Ribeiro-Neto, B.A. (1999), *Modern Information Retrieval*, ACM Press / Addison-Wesley.

[Beretti et. al. 2004] Berretti, S., Del Bimbo, A., Pala, P., "A Graph Edit Distance Based on Node Merging", in Proceedings Image and Video Retrieval: Third International Conference, CIVR, Springer LNCS 3115, pp. 464-472, 2004

[Berkhin 2002] Berkhin, P. (2002),'Survey Of Clustering Data Mining Techniques', Technical report, Accrue Software, San Jose, CA.

[Berners-Lee 1998] Berners-Lee, T. (1998),'Semantic Web Road map.', http://www.w3.org/DesignIssues/Semantic.html.

[Berners-Lee et. al. 2001] Berners-Lee, T.; Hendler, J. & Lassila, O. (2001), 'The Semantic Web', *Scientific American* **284(5)**, 34-43.

[Bieber 2001] Bieber, G. & Carpenter, J. (2001),'Introduction to Service-Oriented Programming (Rev 2.1)', Technical report, http://www.openwings.org/download/specs/ServiceOrientedIntroduction.pdf.

[Bisson et. al. 2000] Bisson G., Nedellec C., and Canamero L. (2000). Designing clustering methods for ontology building- The Mo'K workbench. In Proceedings of the ECAI Ontology Learning Workshop, pages 13–19

[Boehm 1986] Boehm, B. (1986), 'A spiral model of software development and enhancement', SIGSOFT Softw. Eng. Notes 11, 14--24.

[Bontcheva et. al. 2003] Bontcheva K., Maynard D., Tablan V., and Cunningham H. (2003), GATE: A Unicode-based Infrastructure Supporting Multilingual Information Extraction. In Proceedings on Information Extraction for Slavonic and other Central and Eastern European Languages, Borovets, Bulgaria.

[Broder et. al. 1997] Broder, A.Z.; Glassman, S.C.; Manasse, M.S. & Zweig, G. (1997),Syntactic clustering of the Web, in 'Selected papers from the sixth international conference

on World Wide Web', Elsevier Science Publishers Ltd., Essex, UK, pp. 1157--1166.

[Broekstra et. al. 2002] Broekstra, J.; Kampman, A. & van Harmelen, F. (2002),Sesame: A Generic Architecture for Storing and Querying RDF and RDF Schema, *in* 'ISWC '02: Proceedings of the First International Semantic Web Conference on The Semantic Web', Springer-Verlag, London, UK, pp. 54--68.

[Brugger 1976] Brugger W. (1976), Philosophisches Wörterbuch, Herder, Freiburg

[Budzik et. al. 2001] Budzik, J.; Hammond, K.J. & Birnbaum, L. (2001), 'Information access in context.', *Knowl.-Based Syst.* **14**(1-2), 37-53.

[Bunke 2000] Bunke, H. (2000), 'Recent Developments in Graph Matching', 15th International Conference on Pattern Recognition 02, 2117.

[Bush 1945] Bush, V. (1945), 'As We May Think', *The Atlantic Monthly.*

[Campbell 2000] Campbell, I. (2000),'The ostensive model of developing information-needs', PhD thesis, University of Glasgow.

[Cannataro & Comito 2003] Cannataro, M. & Comito, C. (2003),A DataMining Ontology for Grid Programming, in 'In Workshop on Semantics in Peer-to-Peer and Grid Computing (in conj. with WWW2003)'.

[Carroll et. al. 2004] Carroll, J.J.; Dickinson, I.; Dollin, C.; Reynolds, D.; Seaborne, A. & Wilkinson, K. (2004),Jena: implementing the semantic web recommendations, *in* 'WWW Alt. '04: Proceedings of the 13th international World Wide Web conference on Alternate track papers & posters', ACM Press, New York, NY, USA, pp. 74--83.

[Cavnar et. al. 1994] Cavnar W. B. and Trenkle J. , N-Gram Statistics for Natural Language Understanding and Text Processing, IEEE Trans. on Pattern Analysis and Machine Intelligence, 1994

[Champion et. al. 2002] M. Champion, C. Ferris, E. Newcomer, and D. Orchard (2002), Web-Services Architecture W3C Working Draft, www.w3.org/TR/ws-arch/

[Chute 2006] Chute, C. (2006), 'Worldwide Digital Image 2005-2009 Forecast: The Image Capture and Share Bible', *IDC Forecasts.*

[Cimiano et. al. 2005] Cimiano P., Pivk A., Schmidt-Thieme L., Staab S., Learning Taxonomic Relations from Heterogeneous Sources of Evidence, In Ontology Learning from Text: Methods, Evaluation and Applications, pp. 59-73. IOS Press, 2005

[Cristianini et. al. 2002] Cristianini, N.; Shawe-Taylor, J. & Lodhi, H. (2002), 'Latent Semantic Kernels', J. Intell. Inf. Syst. 18(2-3), 127--152.

[Crysmann 2002] Crysmann, B.; Frank, A.; Kiefer, B.; Müller, S.; Neumann, G.; Piskorski, J.; Schäfer, U.; Siegel, M.; Uszkoreit, H.; Xu, F.; Becker, M. & Krieger, H. (2001),An integrated architecture for shallow and deep processing, *in* 'ACL '02: Proceedings of the 40th Annual Meeting on Association for Computational Linguistics', Association for Computational Linguistics, Morristown, NJ, USA, pp. 441--448.

[Cutrell et. al. 2006] Cutrell, E.; Dumais, S.T. & Teevan, J. (2006), 'Searching to eliminate personal information management.', *Commun. ACM* **49**(1), 58-64.

[Deerwester et. al. 1990] Deerwester, S.C.; Dumais, S.T.; Landauer, T.K.; Furnas, G.W. & Harshman, R.A. (1990), 'Indexing by Latent Semantic Analysis', Journal of the American Society of Information Science 41(6), 391-407.

[Dewey 1876] Dewey, M. (1876),'A Classification and Subject Index for Cataloguing and Arranging the Books and Pamphlets of a Library Dewey Decimal Classification', available at http://www.gutenberg.net/etext/12513.

[Dhillon et. al. 2002] Dhillon, I.; Mallela, S. & Kumar, R. (2002),'Enhanced Word Clustering for Hierarchical Text Classification', to be published.

[Duda et. al. 2000] Duda, R.O.; Hart, P.E. & Stork, D.G. (2001), Pattern Classification, Wiley-Interscience.

[Dumais et. al. 2004] Dumais, S.T.; Cutrell, E.; Sarin, R. & Horvitz, E. (2004), Implicit queries (IQ) for contextualized search., *in* 'SIGIR', pp. 594.

[Eppler et. al. 2002] Eppler, M., Mengis, J. (2002). The concept of information overload: A review of literature from organization science, marketing, accounting, MIS, and related disciplines. MCM Research Paper,HSG/MCM/01.

[Faloutsos et. al. 1995] Faloutsos, C. & Lin, K. (1995),FastMap: A Fast Algorithm for Indexing, Data-Mining and Visualization of Traditional and Multimedia Datasets, *in* Michael J. Carey & Donovan A. Schneider, ed.,'Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data', pp. 163--174.

[Fayyad et. al. 1996] Fayyad, U.M.; Piatetsky-Shapiro, G. & Smyth, P. (1996),From Data Mining to Knowledge Discovery: An Overview.'Advances in Knowledge Discovery and Data Mining', pp. 1-34.

[Feldman et. al. 1995] Feldman, R. & Dagan, I. (1995),Knowledge Discovery in Textual Databases (KDT), in 'Knowledge Discovery and Data Mining', pp. 112-117.

[Ferrucci & Lally 2004] Ferrucci, D. & Lally, A. (2004), 'Building an example application with the Unstructured Information Management Architecture', *IBM Systems Journal, Special Issue on Unstructured Information Management* 43, p. 455-476.

[Fodor 2002] Fodor, I.K. (2002),'A Survey of Dimension Reduction Techniques', Technical report, Lawrence Livermore National Lab., CA (US).

[Frawley et. al. 1991] Frawley, W.J.; Piatetsky-Shapiro, G. & Matheus, C.J. (1991), 'Knowledge Discovery in Databases: An Overview.', In G. Piatetsky-Shapiro and W. Frawley, editors, Knowledge Discovery in Databases. AAAI Press/The MIT Press 13, 1-30.

[Freund et. al. 1996] Freund, Y. & Schapire, R.E. (1996),Experiments with a New Boosting Algorithm, in 'International Conference on Machine Learning', pp. 148-156.

[Fuhr 2006] Norbert, F. (2006),'Information Retrieval SS 06', Skriptum zur Vorlesung. http://www.is.informatik.uni-duisburg.de/courses/dortmund/courses/ir/script/kapvorw.ps.gz

[Garg 1987] Garg, P.K. (1987),Abstraction mechanisms in hypertext, *in* 'HYPERTEXT '87: Proceeding of the ACM conference on Hypertext', ACM Press, New York, NY,

USA, pp. 375--395.

[Gärtner et. al. 2003] Gärtner, T. (2003), 'A survey of kernels for structured data', SIGKDD Explor. Newsl. 5(1), 49--58.

[Getoor & Diehl 2005] Getoor, L. & Diehl, C.P. (2005), 'Introduction to the special issue on link mining.', SIGKDD Explorations 7(2), 1-2.

[Gionis et. al. 1999] Gionis, A.; Indyk, P. & Motwani, R. (1999),Similarity Search in High Dimensions via Hashing, in 'VLDB '99: Proceedings of the 25th International Conference on Very Large Data Bases', Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, pp. 518--529.

[Grabner et. al 2006] Grabner, M.; Grabner, H. & Bischof, H. (2006),Real-Time Tracking with On-line Feature Selection, *in* 'Video Proc. in conjunction with IEEE Conf. on Computer Vision and Pattern Recognition'.

[Granitzer 2004a] Granitzer, M. (2004),'Hierarchical Text Classification using Methods from Machine Learning', Master's thesis, Graz University of Technology

[Granitzer 2004b] Granitzer M. (2004), 'OnAir II: Evaluierung Ähnlichkeitssuche' Abschlussbericht, Know-Center

[Granitzer 2006] Michael, G. (2006), Semantic Web - Wege zur vernetzten Wissensgesellschaft, Springer Verlag, Berlin, Germany, chapter Statistische Verfahren der Textanalyse.

[Granitzer & Auer 2005] Granitzer M., Auer P. Experiments with Hierarchical Text Classification, Proceedings of 9th IASTED International Conference onArtifical Intelligence, ACTA Press, Benidorm, Spain, 2005, IASTED

[Granitzer et. al. 2006] Granitzer, M.; Neidhart, T. & Lux, M. (2006),Learning Term Spaces based on Visual Feedback, *in* 'TAKMA Workshop Procceedings of 17th International Conference on Database and Expert Systems Applications (DEXA 06), IEEE Computer Society, Krakaw, Polen'.

[Grishman et. al. 1996] Grishman, R. and Sundheim B., Message Understanding Conference - 6: A Brief History., Proceedings of COLING'96, pp.466-471,1996

[Haase et. al. 2004] Haase, P.; Broekstra, J.; Eberhart, A. & Volz, R. (2004),A comparison of RDF query languages, in 'Proceedings of the Third International Semantic Web Conference, Hiroshima, Japan, 2004.'.

[Hatcher et. al. 2004] Hatcher, E. & Gospodnetic, O. (2004), Lucene in Action (In Action series), Manning Publications.

[Hearst 1992] M.A. Hearst. Automatic acquisition of hyponyms from large text corpora. In Proceedings of the 14th International Conference on Computational Linguistics, pages 539–545, 1992.

[Hearst 1999] Hearst, M.A. (1999),Untangling text data mining, in 'Proceedings of the 37th annual meeting of the Association for Computational Linguistics on Computational Linguistics', Association for Computational Linguistics, Morristown, NJ, USA, pp. 3--10.

[Heer et. al. 2001] Heer, J. & Chi, E. (2001),Identification of Web User Traffic Composition using Multi-Modal Clustering and Information Scent, in 'Proceedings of the Workshop on Web Mining, SIAM Conference on Data Mining, pp. 51-58'.

[Heinrich et. al. 2000] Heinrich, E. & Maurer, H. (2000), 'Active Documents: Concept, Implementation and Applications', *j-jucs* **6**(12), 1197--1202.

[Hofmann 1999] Hofmann, T. (1999),Probabilistic Latent Semantic Indexing, *in* 'Proceedings of the 22nd Annual ACM Conference on Research and Development in Information Retrieval', pp. 50-57.

[Hornick 2004] Hornick, M.F. (2004),'Java Data Mining (JSR-73): Status and Overview"KDD-2004 Workshop onData Mining Standards, Services and Platforms (DM-SSP 04), KDD-2004The Tenth ACM SIGKDD International Conference onKnowledge Discovery and Data Mining'.

[IBM 2004] IBM (2004),'Unstructured Information MAnagement Architecture (UIMA), SDK User'S Guide and Reference', Documentation.

[Joachims 1998] Joachims, T. (1998),Text categorization with support vector machines: learning with many relevant features, in 'Proceedings of ECML-98, 10th European Conference on Machine Learning', Springer Verlag, Heidelberg, DE, , pp. 137--142.

[JSR 247, 2004] Hornick, M.F. (2004),'JSR 247: Data Mining 2.0', http://www.jcp.org/en/jsr/detail?id=247.

[JSR 73, 2000] Hornick, M.F. (00),'JSR 73: Data Mining API', http://www.jcp.org/en/jsr/detail?id=73.

[Juffinger 2005] Juffinger, A. (2005), 'Focused crawling in the context of digital libraries', Master's thesis, Knowledge Management Institute, University of Technology Graz

[Kappe et. al. 2002] Frank Kappe , G.D. (2002),InfoSky: Eine neue Technologie zur Erforschung großer, hierarchischer Wissensräume., in 'KnowTech 2002, 4. Konferenz zum Einsatz von Knowledge Management in Wirtschaft und Verwaltung, München,Germany'.

[Karypis et. al. 2000] Karypis, G. & Han, E. (2000),'Concept indexing: A fast dimensionality reduction algorithm with applications to document retrieval and categorization', Technical report, University of Minnesota.

[Kaufman et. al. 1990] Kaufman, L. & Rousseeuw, P.J. (1990), Finding Groups in Data: An Introduction to Cluster Analysis., John Wiley.

[Kearns & Vazirani 1994] Kearns, M.J. & Vazirani, U.V. (1994), *An introduction to computational learning theory*, MIT Press, Cambridge, MA, USA.

[Keim 2005] Keim, D. (2005),'Scaling Visual Analytics to Very Large Data Sets', Workshop on Visual Analytics, Darmstadt, Germany.

[Képesi et. al. 2005] Képesi, M, Weruaga, L, Schofield, E: Detailed Multidimensional Analysis of our Acoustical Environment. Accepted for Forum Acusticum. September 2005,

Budapest.

[Kienreich et. al. 2005] Kienreich, W.; Sabol, V.; Granitzer, M.; Klieber, W.; Lux, M. & Sarka, W. (2005), A Visual Query Interface for a Very Large Newspaper Article Repository., in 'DEXA Workshops', pp. 415-419.

[Klieber 2006a] Werner Klieber (2006), Roadmap KnowMiner, Technischer Report Know-Center

[Klieber 2006b] Werner Klieber (2006), KnowMiner Service & Pipeline Dokumentation, Technischer Report Know-Center

[Klieber et. al. 2006] Klieber, W.; Sabol, V. & Granitzer, M. (2006),KnowMiner: Ein Service orientiertes Knowledge Discovery Framework, *in* 'Proceedings der Berliner XML-Tage, Berlin, Germany'.

[Kokkelink & Schwänzl 2002] Kokkelink, S. & Schwänzl, R. (2002),'Expressing Qualified Dublin Core in RDF / XML', Technical report, Dublin Core Metadata Initiative.

[Koller et. al. 1997] Koller, D. & Sahami, M. (1997),Hierarchically classifying documents using very few words, in 'Proceedings of ICML-97, 14th International Conference on Machine Learning', Morgan Kaufmann Publishers, San Francisco, US, Nashville, US, pp. 170--178.

[Kosch 2003] Kosch, H. (2003), *Distributed Multimedia Database Technologies supported by MPEG-7 and MPEG-21*, CRC Press,

[Kroeze et. al. 2003] Kroeze, J.H.; Matthee, M.C. & Bothma, T.J.D. (2003),Differentiating data- and text-mining terminology, in 'SAICSIT '03: Proceedings of the 2003 annual research conference of the South African institute of computer scientists and information technologists on Enablement through technology', South African Institute for Computer Scientists and Information Technologists, , pp. 93--101.

[Laprun 2002] C.Laprun, J. (2002),A Practical developer Introduction to ATLAS, in 'Proceedings of the Third International Conference on Language Resources and Evaluation, European Language Resources Association,Paris'.

[Lassila & Swick 1998] Lassila, O. & Swick, R. (1998),'Resource Description Framework (RDF) model and syntax specification', http://www.w3.org/ TR/WD-rdf-syntax

[Lesk 1995] Lesk, M. (1995),'The Seven Ages of Information Retrieval.', Proceedings of the Conference for the 50th anniversary of As We May Think, 12-14,. Available at the time of this writing as http://www.lesk.com/mlesk/ages/ages.html.

[Losee 1997] Losee, R.M. (1997), 'A Discipline Independent Definition of Information', *Journal of the American Society for Information Science* **Vol. 48 No. 3**, 254-269.

[Lux 2004] Lux, M. (2004),'Magick - Ein Werkzeug für Cross-Media Clustering und Visualisierung', Master's thesis, Knowledge Management Institute, University of Technology Graz.

[Lux et. al. 2006] Lux, M.; Tochtermann, K. & Granitzer, M. (2006),Retrieval basierend auf Semantischen Metadaten, *in* 'Tagungsband der Multikonferenz Wirtschaftsinformatik 2006, Passau (D)'.

[Lyman 2003] Lyman, P. and Varian, H.R. (2003),'How Much Information', Retrieved from http://www.sims.berkeley.edu/how-much-info-2003.

[Maedche & Staab 2001] Maedche, A. & Staab, S. (2001), 'Ontology Learning for the Semantic Web', *IEEE Intelligent Systems* **16**(2), 72--79.

[Manning et. al. 1999] Manning C. D. and Schütze H. (1999), Foundations of Statistical Natural Language Processing, The {MIT} Press, Cambridge, Massachusetts

[Maurer et. al. 2002] Maurer, H., Tochtermann, K., „On a New Powerful Model for Knowledge Management and its Applications", Journal of Universal Computer Science, Vol. 8, Num. 1, pp. 85-96, 2002

[Mayfield 2002] Mayfield, J. (2002),Ontologies and text retrieval, in , Cambridge University Press, New York, NY, USA, pp. 71—75.

[McCray et. al. 2001] McCray, A.T. & Gallagher, M.E. (2001), 'Principles for digital library development', Commun. ACM 44(5), 48--54.

[Meyer 1992] Meyer, B. (1992), 'Applying "Design by Contract".', IEEE Computer 25(10), 40-51.

[Meyer 1997] Meyer, B. (1997), Object Oriented Software Construction, Prentice Hall.

[Mitchell 1997] Mitchell, T. M. (1997). *Machine Learning*. Boston: McGraw-Hill.

[Mitterholzer & Horvath 2005] Mitterholzer, M. & Horvath, R. (2005),The APA-OnlineManager – Professional Semantic Retrieval At Austrias Largest Database Host, *in* Proceedings of SEMANTICS 2005 Conference, Vienna, Austria'.

[Mülller et. al. 2001] Müller, K.; Mika, S.; Rätsch, G.; Tsuda, K. & Schölkopf, B. (2001), 'An Introduction to Kernel-Based Learning Algorithms', IEEE Transactions on Neural Networks 12(2), 181-201.

[Neidhart 2005] Neidhart, T. (2005),'Semiautomatische Erstellung von Wissenslandkarten mittelsKnowledge Mining Techniken', Master's thesis, Graz University of Technology.

[O'Neill et. al. 2003] O'Neill, E.T.; Lavoie, B.F. & Bennett, R. (2003), 'Trends in the Evolution of the Public Web: 1998 - 2002', *D-LIB* **9**.

[O'Reilly 2005] O'Reilly, T. (2005), 'What Is Web 2.0, Design Patterns and Business Models for the Next Generation of Software', *Oreillynet.com, Sept. 30. 2005*. http://www.oreillynet.com/pub/a/oreilly/tim/news/2005/09/30/what-is-web-20.html

[Ogden & Richards 1923] C.K. Ogden, I.A. Richards (1923), 'The Meaning of Meaning: A Study of the Influence of Language Upon Thought and of the Science of Symbolism.', *8th ed. 1923. Reprint New York: Harcourt Brace Jovanovich.*

[Papazoglou & Georgakopoulos 2003] Papazoglou, M.P. & Georgakopoulos, D. (2003), 'Introduction', Communications of the ACM 46(10), 24--28.

[Perna et. al. 2004] Perna, J. & Spector, A.Z. (2004), 'Introduction.', IBM Systems Journal 43(3).

[Piatetsky-Shapiro 1991] Piatetsky-Shapiro, G. (1991), 'Knowledge Discovery in Real Databases: A Report

on the IJCAI-89 Workshop', *AI Magazine* 11(5), 68–70.

[Platt 2005] Platt, J. (2005),FastMap, MetricMap, and Landmark MDS are all Nystrom Algorithms, *in* Robert G. Cowell & Zoubin Ghahramani, ed.,'aistats05', Society for Artificial Intelligence and Statistics, , pp. 261-268.

[PMML 2004] Raspl, S. (2004),'PMML Version 3.0 - Overview and Status"KDD-2004 Workshop onData Mining Standards, Services and Platforms (DM-SSP 04), KDD-2004The Tenth ACM SIGKDD International Conference onKnowledge Discovery and Data Mining'.

[Potts 2003] Potts, M. (2003) *'Find Bind and Execute: Requirements for Web Service Lookup and Discovery.* 'www.talkingblocks.com/resources.htm#, accessed January 2003.

[Rhodes 2000] Rhodes, B.J. & Maes, P. (2000), 'Just-in-time information retrieval agents', IBM Syst. J. 39(3-4), 685--704.

[Rijsbergen 1979] van Rijsbergen, C.J. (1979), Information retrieval, Butterworths, London.

[Rijsbergen 2004] van Rijsbergen, C.J. (2004), The Geometry of Information Retrieval, Cambridge University Press, New York, NY, USA.

[Rittberger 2004] Rittberger, M. (2004). Informationsqualität. In: Grundlagen der praktischen Information und Dokumentation. Handbuch zur Einführung in die Informationswissenschaft und -praxis. Kuhlen, R., Seeger, T., und Strauch, D. (eds.); München: Saur, Kap. B 17, 315-321

[Rose et. al. 2002] Rose, T.; Stevenson, M. & Whitehead, M. (2002),The Reuters Corpus Volume 1 - from Yesterday's News to Tomorrow's Language Resources , in 'Proceedings of the Third Internaional Conference on Language Resources and Evaluation'.

[Sabol et. al. 2001] Vedran, S. (2001),'Visualisation Island', Master's thesis, IICM, Graz University of Technology.

[Sabol et. al. 2002] Vedran Sabol, W.K.G. (2002),Applications of a Lightweight, Web-Based Retrieval, Clustering, and Visualisation Framework, in 'Lecture Notes in Computer Science', Springer

[Sabol, Klieber, Granitzer 2006] Sabol, V.,; Klieber, W.,; Granitzer M., (2006) KnowMiner JavaDoc, Technischer Report Know-Center

[Salton 1968] Salton, G. (1968),'Automatic Information Organization and Retrieval', McGraw-Hill, New York.

[Salton et. al. 1988] Salton G. and Buckley C. (1988), Term-weighting approaches in automatic text retrieval, Information Processing and Management: an International Journal, Vol. 24, pp. 513-523

[Sauermann 2005] Sauermann, L. (2005),The Semantic Desktop - a Basis for Personal Knowledge Management, *in* Hermann Maurer; Cristian Calude; Arto Salomaa & Klaus Tochtermann, ed.,'Proceedings of the I-KNOW 2005. 5th International Conference on Knowledge Management'.

[Schapire et. al. 2000] Schapire, R.E. & Singer, Y. (2000), 'BoosTexter: A Boosting-based System for Text

Categorization', Machine Learning 39(2/3), 135-168.

[Scharl 2004] Scharl, A. & Bauer, C. (2004), 'Mining Large Samples of Web-Based Corpora', *Knowledge-Based Systems* **17(5-6)**, 229-233.

[Schmitt 2002] Schmitt, I. (2002), 'Retrieval in Multimedia-Datenbanksystemen.', Datenbank-Spektrum 4, 28-35.

[Sebastiani 2002] Sebastiani, F. (2002), 'Machine learning in automated text categorization', ACM Computing Surveys (CSUR) 34(1), 1--47.

[Shannon 1951] Shannon, C. E., Prediction and entropy of printed english., Bell Systems Technical Journal, 1951, pp. 50-64

[Shneiderman 1996] Shneiderman, B. (1996),The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations, in 'VL '96: Proceedings of the 1996 IEEE Symposium on Visual Languages', IEEE Computer Society, Washington, DC, USA, pp. 336.

[Soffer 2001] Pnina Soffer, B.G. (2001), 'Modelling Off-the-Shelf Information Systems Requirements: An Ontological Approach', Requirements Engineering Volume 6, Issue 3„ Pages 183 - 199,.

[Staab et. al. 2001] Staab, S.; Studer, R.; Schnurr, H. & Sure, Y. (2001), 'Knowledge Processes and Ontologies', *IEEE Intelligent Systems* **16**(1), 26--34.

[Stanfill et. al. 1986] Stanfill, C. & Waltz, D. (1986), 'Toward memory-based reasoning', Communications of the ACM 29, 1213-1228.

[Stein et. al. 2004] Stein B. and Meyer zu Eissen S.. Topic Identification: Framework and Application. In the Proceedings of the 4th International Conference on Knowledge Management (I-KNOW 2004), Graz, Austria, June 30-July 02, 2004.

[Tang et. al. 2005] Tang, B.; Shepherd, M.; Milios, E. & Heywood, M. (2005),Comparing and Combining Dimension Reduction Techniques for Efficient Text Clustering, in 'International Workshop on Feature Selection for Data Mining - Interfacing Machine Learning and Statistics in conjunction with 2005 SIAM International Conference on Data Mining, Newport Beach, California.'.

[Thomas et. al. 2001] Thomas, J.; Cowley, P.; Kuchar, O.; Nowell, L.; Thomson, J. & Wong, P.C. (2012), 'Discovering Knowledge Through Visual Analysis', Journal of Universal Computer Science 7(6), 517--529.

[Tochtermann 2006] Tochtermann, K. (2006), Der Service-basierte Knowledge Desktop, in wissensmanagement - Das Magazin für Führungskräfte, 6/2006

[Tochtermann et. al. 2002] Klaus Tochtermann, V.S. (2002),Intelligent Maps and Information Landscapes: Two new Approaches to support Search and Retrieval of Environmental Information Objects, in 'Proceedings of the International Symposium on Environmental Informatics, Vienna Austria'.

[Tochtermann et. al. 2005l] Tochtermann, K.; Granitzer, M. & Sabol, V. (2005),MISTRAL: Cross-Media Techniques for Extracting Semantics from Multimedia and their Application, *in* 'Proceedings of SEMANTICS 2005 Conference, Vienna, Austria'.

[Tochtermann et. al. 2006] Tochtermann, K.; Reisinger, D.; Granitzer, M. & Lindstaedt, S. (2006),Integrating Ad Hoc Processes and Standard Processes in Public Administrations, *in* 'Proceedings of the OCG eGovernment Conference, Linz (Austria), (to appear).'.

[Wal 2005] Wal, T.V. (2005),'Folksonomy Explanations :: Off the Top :: vanderwal.net', http://www.vanderwal.net/random/entrysel.php?blog=1622

[Weiss et. al. 2005] Weiss, S.M., Indurkhya, N., Zhang, T., Damerau, Text Mining - Predictive Methods for Analyzing Unstructured Information, F. 2005, XII, 236 p. 79 illus., ISBN: 0-387-95433-3

[Wilson et. al. 1997] Wilson, D. & Martinez, T. (1997), 'Improved Heterogeneous Distance Functions', Journal of Artificial Intelligence Research 6, 1-34.

[Witten et. al. 2005] Witten, I.H. & Frank, E. (2005), Data mining: practical machine learning tools and techniques with Java implementations, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[Wotawa & Angerer 2006] Wotawa, F. & Angerer, B. (2006),Retaining Consistency in Temporal Knowledge Bases, *in* '19th International Conference on Industrial, Engineering & Other Applications of Applied Intelligent Systems (IEA/AIE'06), Annecy, France'.

[Xu et. al. 2005] Xu, R. & Wunsch, D. (2005), 'Survey of clustering algorithms', *Neural Networks, IEEE Transactions on* 16(3), 645--678.

[Zamir et. al. 1998] Zamir, O. & Etzioni, O. (1998),Web document clustering: a feasibility demonstration, in 'SIGIR '98: Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval', ACM Press, New York, NY, USA, pp. 46--54.

[Zechner 2006] Zechner, M. (2006), Anwendung von RDF Triple Stores in der Wissenserschließung, Bakkelaureatsarbeit Technische Universität Graz

[Zipf 1949] Zipf G. K., Human Behaviour and the Principle of Least-Effort, Addison-Wesley, Cambridge MA, 1949